



Karnataka Digital
Economy Mission



K-VLSI

DIGITAL SYSTEM DESIGN USING VERILOG

NAME: DHANAPAL K

KVLSI ID: KVLSI2501061

Under the guidance of :

Shashi Kant Sharma

INDEX

Sl No.	CONTENTS	Page No.
1.	Logic Gates	01 – 02
2.	AND gate using NAND gates	03
3.	OR gate using NAND gates	04
4.	NOT gate using NAND gates	05
5.	Buffer using NAND gates	06
6.	NOR gate using NAND gates	07
7.	XOR gate using NAND gates	08 – 09
8.	XNOR gate using NAND gates	10 – 11
9.	Realization of all logic gates using NAND gates	12 – 13
10.	AND gate using NOR gates	14
11.	OR gate using NOR gates	15
12.	NOT gate using NOR gates	16
13.	Buffer using NOR gates	17
14.	NAND gate using NOR gates	18
15.	XOR gate using NOR gates	19 – 20
16.	XNOR gate using NOR gates	21 – 22
17.	Realization of all logic gates using NOR gates	23 – 24
18.	Realization of inverter using all possible gates	25 – 26
19.	Demorgan's Law	27 – 28
20.	Half Adder	29
21.	Half Adder (Behavioural Modeling Style)	30
22.	Half Adder using Half Subtractor	31
23.	Full Adder	32 – 33
24.	Full Adder (Behavioural Modeling Style)	34 – 35
25.	Full Adder using Half Adders	36 – 37
26.	Full Adder using Half Subtractors	38 – 39
27.	Half Subtractor	40
28.	Half Subtractor using Half Adder	41
29.	Full Subtractor	42 – 43
30.	Full Subtractor using Half Adders	44 – 45
31.	Full Subtractor using Half Subtractors	46 – 47
32.	4 bit ripple carry adder using full adders	48 – 49
33.	4 bit Carry Look Ahead Adder	50 – 51
34.	2 bit Multiplier	52 – 53
35.	Binary to Gray Code converter	54 – 55
36.	Gray to Binary Code converter	56 – 57
37.	Square Number Generator (2bits)	58 – 59
38.	Even and Odd number Detector	60 – 61
39.	Prime Number Detector	62 – 63
40.	Palindrome Number Detector	64 – 65

INDEX

Sl No.	CONTENTS	Page No.
41.	Palindrome number using ternary operator	66 – 67
42.	Magnitude Comparator	68 – 69
43.	Magnitude Comparator using ternary operator	70 – 71
44.	2x1 MUX	72
45.	2x1 MUX (Behavioral Modeling Style)	73
46.	MUX 2x1 using ternary operator	74 – 75
47.	2x1 MUX (in vector form)	76
48.	2x1 MUX using Tristate buffer	77
49.	MUX 4x1 (in vectors)	78 – 80
50.	MUX 4x1 using ternary operator	81 – 83
51.	4x1 MUX using 2x1 MUX	84 – 85
52.	8x1 MUX using 4x1 MUX and 2x1 MUX in vectors	86 – 87
53.	8x1 MUX using 2x1 MUX only	88 – 89
54.	3x1 MUX	90 – 91
55.	5x1 MUX	92 – 93
56.	10x1 MUX	94 – 95
57.	3x1 MUX using 2x1 MUX only	96 – 97
58.	5x1 MUX using 2x1 MUX only	98 – 99
59.	10x1 MUX using 2x1 MUX only	100 – 101
60.	3 input XNOR gate using 2x1 MUX	102
61.	1x2 De-MUX	103 - 104
62.	1x4 De-MUX	105 – 106
63.	1x4 De-MUX using 1x2 De-MUX	107 – 108
64.	Normal 4:2 Encoder	109 – 110
65.	Priority 4:2 Encoder	111 – 112
66.	Priority 8:3 Encoder	113 – 114
67.	Priority 8:3 Encoder using 4:2 Encoder	115 – 116
68.	1:2 Decoder	117
69.	2:4 Decoder	118
70.	2:4 Decoder using 1:2 Decoders	119
71.	3:8 Decoder using 2:4 and 1:2 Decoders	120 – 121
72.	4:16 Decoder using 2:4 Decoder	122 – 123
73.	BCD to 7-Segment Display Decoder Module	124 – 125
74.	ALU	126 – 127
75.	Generic Adder	128 – 129
76.	Generic MUX	130 – 131
77.	D flip-flop	132 – 133
78.	T flip-flop	134 – 135
79.	SR flip-flop	136 – 137
80.	JK flip-flop	138 – 139

INDEX

Sl No.	CONTENTS	Page No.
81.	D latch using 2x1 MUX	140 – 141
82.	D flip-flop using 2x1 MUX	142 – 143
83.	D flip-flop using T flip-flop	144 – 145
84.	D flip-flop using SR flip-flop	146 – 147
85.	D flip-flop using JK flip-flop	148 -149
86.	T flip-flop using D flip-flop	150 -151
87.	T flip-flop using SR flip-flop	152 – 153
88.	T flip-flop using JK flip-flop	154 – 155
89.	SR flip-flop using D flip-flop	156 – 157
90.	SR flip-flop using T flip-flop	158 – 159
91.	SR flip-flop using JK flip-flop	160 – 161
92.	JK flip-flop using D flip-flop	162 – 163
93.	JK flip-flop using T flip-flop	164 – 165
94.	JK flip-flop using SR flip-flop	166 – 167
95.	Serial-In Serial-Out	168 – 169
96.	Serial-In Parallel-Out	170 – 171
97.	Parallel-In Parallel-Out	172 – 173
98.	Parallel-In Serial-Out	174 – 175
99.	Universal Shift Register	176 – 177
100.	16 bit LFSR	178 – 179
101.	3 bit up counter	180 – 181
102.	3 bit down counter	182 – 183
103.	3 bit up-down counter	184 – 185
104.	3 bit gradual increment-decrement counter	186 – 187
105.	4 bit Ring Counter	188 – 189
106.	4 bit Johnson Counter	190 – 191
107.	MOD-10 Up counter	192 – 193
108.	MOD-6 Down counter	194 – 195
109.	State Skipping Counter	196 – 197
110.	Time period division by 2	198
111.	Time period division by 4	199
112.	Time period division by 5	200
113.	Time period division by 9	201
114.	Generic N-bit Clock Divider	202 – 203
115.	Mealy Overlapping (for sequence 101)	204 – 205
116.	Mealy Non-Overlapping (for sequence 101)	206 – 207
117.	Moore Overlapping (for sequence 101)	208 – 209
118.	Moore Non-Overlapping (for sequence 101)	210 – 211
119.	Up Down Counter using FSM approach	212 – 213
120.	State Skipping Counter using FSM approach (0,2,5,7)	214 – 215

INDEX

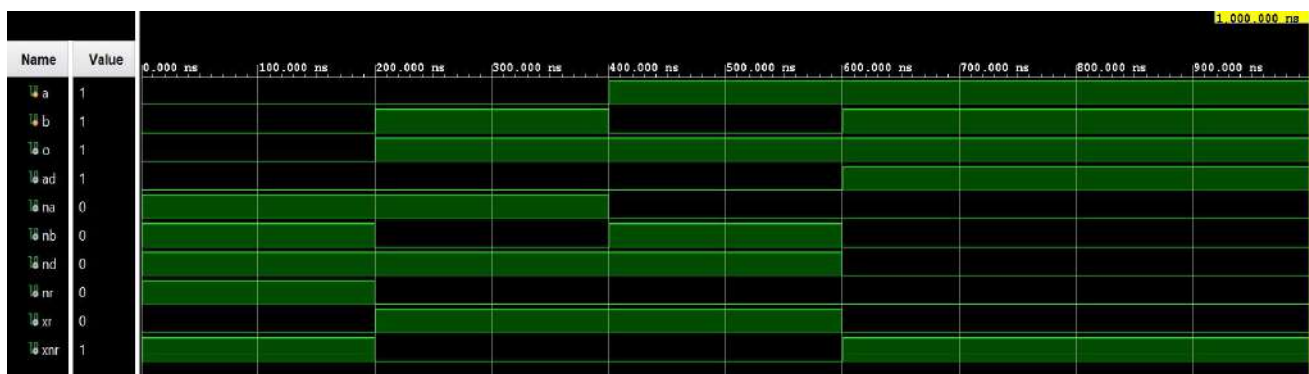
Sl No.	CONTENTS	Page No.
121.	Traffic Light Controller (3-state FSM: RED→GREEN→YELLOW)	216 – 217
122.	Lift Controller (2-Floor Lift: Floor 0 and 1)	218 – 219
123.	Vending Machine (Dispense item when 2 coins are inserted)	220 – 221
124.	Washing Machine (Simple 4-step FSM: Idle, Wash, Rinse, Dry)	222 – 223
125.	Digital Clock	224 – 225

VERILOG PROGRAMS

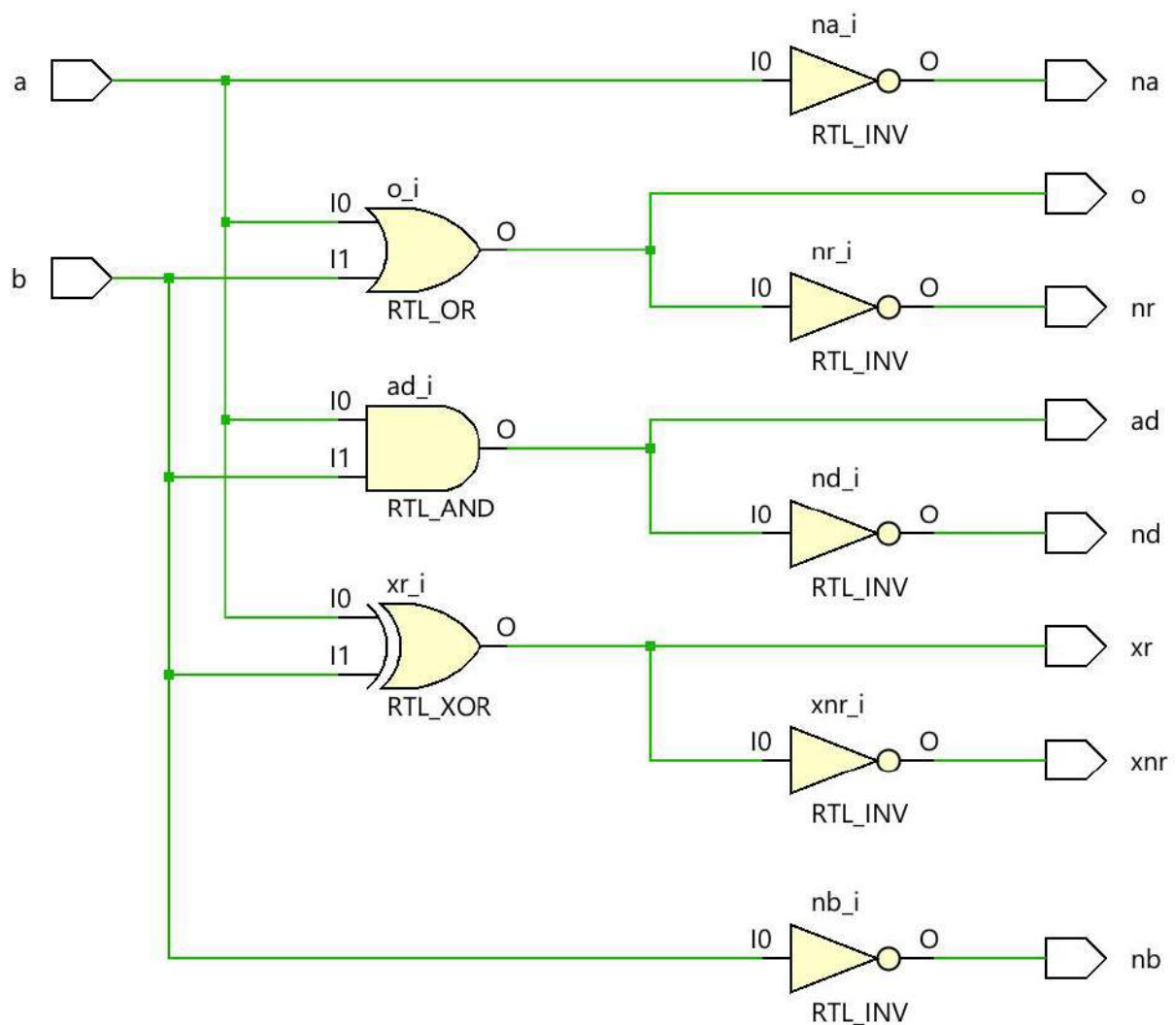
1. Logic Gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style: module logic_gates(a,b,o,ad,na,nb,nd,nr,xr,xnr); input a,b; output o,ad,na,nb,nd,nr,xr,xnr; assign o=a b; assign ad=a&b; assign na=~a; assign nb=~b; assign nr=~(a b); assign nd=~(a&b); assign xr=a^b; assign xnr=~(a^b); endmodule //Gate-level Modeling Style: module logic_gates_g(a,b,o,ad,na,nb,nd,nr,xr,xnr); input a,b; output o,ad,na,nb,nd,nr,xr,xnr; or o1(o,a,b); and a1(ad,a,b); not n1(na,a); not n2(nb,b); nor n01(nr,a,b); nand na1(nd,a,b); xor x1(xr,a,b); xnor xn1(xnr,a,b); endmodule</pre>	<pre>module logic_gates_tb; reg a,b; wire o,ad,na,nb,nd,nr,xr,xnr; logic_gates l1(a,b,o,ad,na,nb,nd,nr,xr,xnr); initial begin a=1'b0; b=1'b0; #200 a=1'b0; b=1'b1; #200 a=1'b1; b=1'b0; #200 a=1'b1; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, o=%b, ad=%b, na=%b, nb=%b, nd=%b, nr=%b, xr=%b, xnr=%b", \$time,a,b,o,ad,na,nb,nd,nr,xr,xnr); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,o,ad,na,nb,nd,nr,xr,xnr); end endmodule</pre>
● Output	
<pre>Simtime=0, a=0, b=0, o=0, ad=0, na=1, nb=1, nd=1, nr=1, xr=0, xnr=1 Simtime=200000, a=0, b=1, o=1, ad=0, na=1, nb=0, nd=1, nr=0, xr=1, xnr=0 Simtime=400000, a=1, b=0, o=1, ad=0, na=0, nb=1, nd=1, nr=0, xr=1, xnr=0 Simtime=600000, a=1, b=1, o=1, ad=1, na=0, nb=0, nd=0, nr=0, xr=0, xnr=1</pre>	

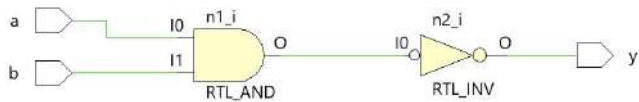
● Waveform



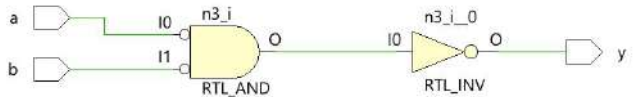
● Schematic



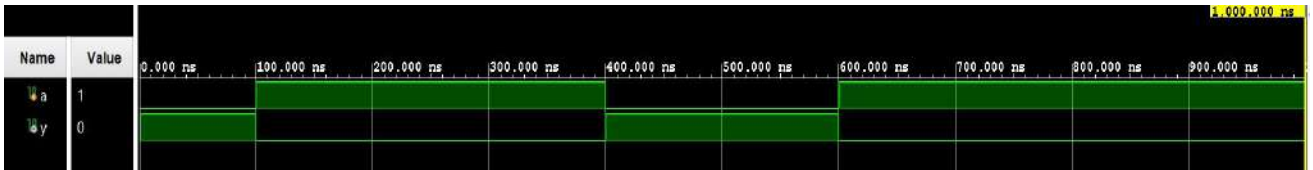
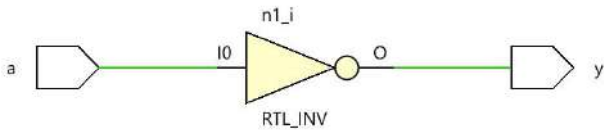
2. AND gate using NAND gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module nand_as_and(a,b,y); input a,b; output y; wire w1; assign w1=~(a&b); assign y=~(w1&1); endmodule //Gate-level Modeling Style module nand_as_and(a,b,y); input a,b; output y; wire w1; nand n1(w1,a,b); nand n2(y,w1,1); endmodule</pre>	<pre>module nand_as_and_tb; reg a,b; wire y; nand_as_and uut(a,b,y); initial begin a=1'b0; b=1'b0; #200 a=1'b0; b=1'b1; #200 a=1'b1; b=1'b0; #200 a=1'b1; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, y=%b", \$time,a,b,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,y); end endmodule</pre>
● Waveform 	
● Output <pre>Simtime=0, a=0, b=0, y=0 Simtime=200000, a=0, b=1, y=0 Simtime=400000, a=1, b=0, y=0 Simtime=600000, a=1, b=1, y=1</pre>	● Schematic 

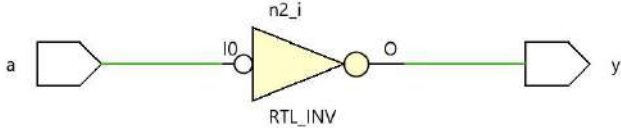
3. OR gate using NAND gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module nand_as_or(a,b,y); input a,b; output y; wire w1,w2; assign w1=~(a&1); assign w2=~(b&1); assign y=~(w1&w2); endmodule //Gate-level Modeling Style module nand_as_or(a,b,y); input a,b; output y; wire w1,w2; nand n1(w1,a,1); nand n2(w2,b,1); nand n3(y,w1,w2); endmodule</pre>	<pre>module nand_as_or_tb; reg a,b; wire y; nand_as_or uut(a,b,y); initial begin a=1'b0; b=1'b0; #200 a=1'b0; b=1'b1; #200 a=1'b1; b=1'b0; #200 a=1'b1; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, y=%b", \$time,a,b,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,y); end endmodule</pre>
● Waveform 	
● Output <pre>Simtime=0, a=0, b=0, y=0 Simtime=200000, a=0, b=1, y=1 Simtime=400000, a=1, b=0, y=1 Simtime=600000, a=1, b=1, y=1</pre>	● Schematic 

4. NOT gate using NAND gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module nand_as_not(a,y); input a; output y; assign y=~(a&1); endmodule //Gate-level Modeling Style module nand_as_not(a,y); input a; output y; nand n1(y,a,1); endmodule</pre>	<pre>module nand_as_not_tb; reg a; wire y; nand_as_not uut(a,y); initial begin a=1'b0; #100 a=1'b1; #100 a=1'b1; #100 a=1'b1; #100 a=1'b0; #100 a=1'b0; #100 a=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, y=%b", \$time,a,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,y); end endmodule</pre>
● Waveform	
	
● Output	● Schematic
<pre>Simtime=0, a=0, y=1 Simtime=100000, a=1, y=0 Simtime=400000, a=0, y=1 Simtime=600000, a=1, y=0</pre>	

5. Buffer using NAND gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module nand_as_buffer(a,y); input a; output y; wire w1; assign w1=~(a&1); assign y=~(w1&1); endmodule //Gate-level Modeling Style module nand_as_buffer(a,y); input a; output y; wire w1; nand n1(w1,a,1); nand n2(y,w1,1); endmodule</pre>	<pre>module nand_as_buffer_tb; reg a; wire y; nand_as_buffer uut(a,y); initial begin a=1'b0; #100 a=1'b1; #100 a=1'b1; #100 a=1'b1; #100 a=1'b0; #100 a=1'b0; #100 a=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, y=%b", \$time,a,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,y); end endmodule</pre>
● Waveform	
	
● Output	● Schematic
<pre>Simtime=0, a=0, y=0 Simtime=100000, a=1, y=1 Simtime=400000, a=0, y=0 Simtime=600000, a=1, y=1</pre>	

6. NOR gate using NAND gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module nand_as_nor(a,b,y); input a,b; output y; wire w1,w2,w3; assign w1=~(a&1); assign w2=~(b&1); assign w3=~(w1&w2); assign y=~(1&w3); endmodule //Gate-level Modeling Style module nand_as_nor(a,b,y); input a,b; output y; wire w1,w2,w3; nand n1(w1,a,1); nand n2(w2,b,1); nand n3(w3,w2,w1); nand n4(y,w3,1); endmodule</pre>	<pre>module nand_as_nor_tb; reg a,b; wire y; nand_as_nor uut(a,b,y); initial begin a=1'b0; b=1'b0; #200 a=1'b0; b=1'b1; #200 a=1'b1; b=1'b0; #200 a=1'b1; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, y=%b", \$time,a,b,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,y); end endmodule</pre>
● Waveform	
	
● Output	● Schematic
<pre>Simtime=0, a=0, b=0, y=1 Simtime=200000, a=0, b=1, y=0 Simtime=400000, a=1, b=0, y=0 Simtime=600000, a=1, b=1, y=0</pre>	

7. XOR gate using NAND gates

● Design Code

```
//Dataflow Modeling Style

module xor_using_nand(a,b,y);
    input a,b;
    output y;
    wire w1,w2,w3;
    assign w1=~(a&b);
    assign w2=~(a&w1);
    assign w3=~(w1&b);
    assign y=~(w2&w3);
endmodule

//Gate-level Modeling Style

module xor_using_nand_gate(a,b,y);
    input a,b;
    output y;
    wire w1,w2,w3;
    nand n1(w1,a,b);
    nand n2(w2,a,w1);
    nand n3(w3,w1,b);
    nand n4(y,w1,w2);
endmodule
```

● Testbench Code

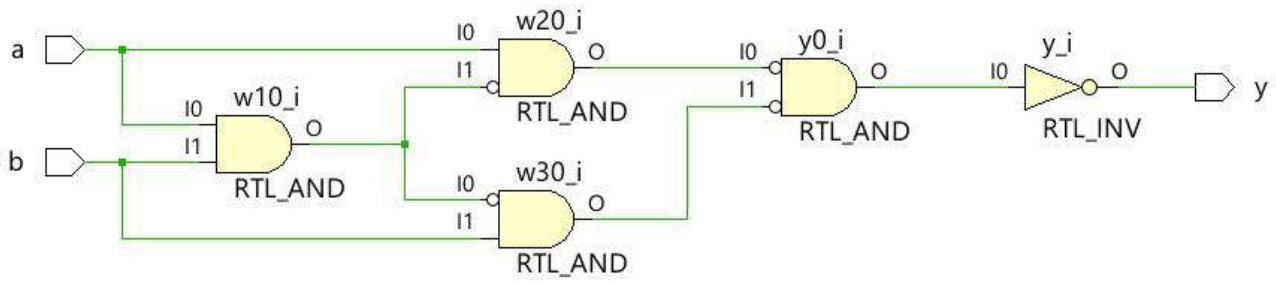
```
module xor_using_nand_tb;
    reg a,b;
    wire y;
    xor_using_nand m1(a,b,y);
    initial
        begin
            a=1'b0; b=1'b0;
            #100 a=1'b0; b=1'b1;
            #100 a=1'b1; b=1'b0;
            #100 a=1'b1; b=1'b1;
            #100 a=1'b0; b=1'b1;
        end
    initial
        begin
            $monitor("Simtime=%0t, a=%b, b=%b, y=%b",
$time, a,b,y);
        end
    initial
        begin
            $dumpfile("dump.vcd");
            $dumpvars(0,a,b,y);
        end
endmodule
```

● Waveform

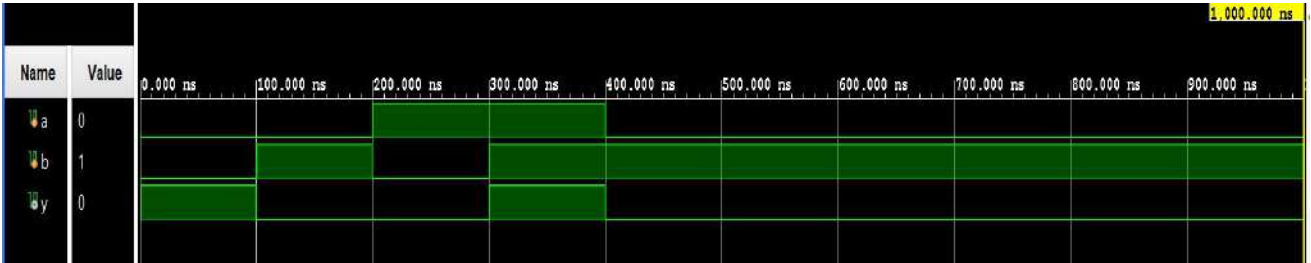
● Output

```
Simtime=0, a=0, b=0, y=0
Simtime=100000, a=0, b=1, y=1
Simtime=200000, a=1, b=0, y=1
Simtime=300000, a=1, b=1, y=0
Simtime=400000, a=0, b=1, y=1
```

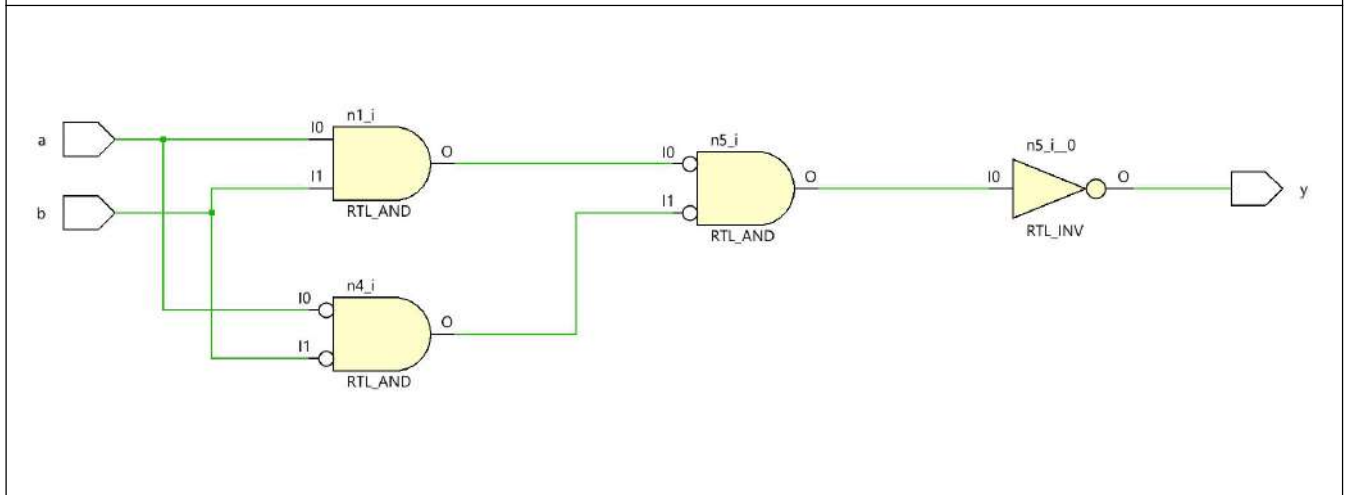
- Schematic



8. XNOR gate using NAND gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module nand_as_xnor(a,b,y); input a,b; output y; wire w1,w2,w3,w4; assign w1=~(a&b); assign w2=~(a&1); assign w3=~(1&b); assign w4=~(w2&w3); assign y=~(w1&w4); endmodule //Gate-level Modeling Style module nand_as_xnor(a,b,y); input a,b; output y; wire w1,w2,w3,w4; nand n1(w1,a,b); nand n2(w2,a,1); nand n3(w3,1,b); nand n4(w4,w2,w3); nand n5(y,w1,w4); endmodule</pre>	<pre>module nand_as_xnor_tb; reg a,b; wire y; nand_as_xnor m1(a,b,y); initial begin a=1'b0; b=1'b0; #100 a=1'b0; b=1'b1; #100 a=1'b1; b=1'b0; #100 a=1'b1; b=1'b1; #100 a=1'b0; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, y=%b", \$time, a,b,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,y); end endmodule</pre>
● Waveform	
	
● Output	
<pre>Simtime=0, a=0, b=0, y=1 Simtime=100000, a=0, b=1, y=0 Simtime=200000, a=1, b=0, y=0 Simtime=300000, a=1, b=1, y=1 Simtime=400000, a=0, b=1, y=0</pre>	

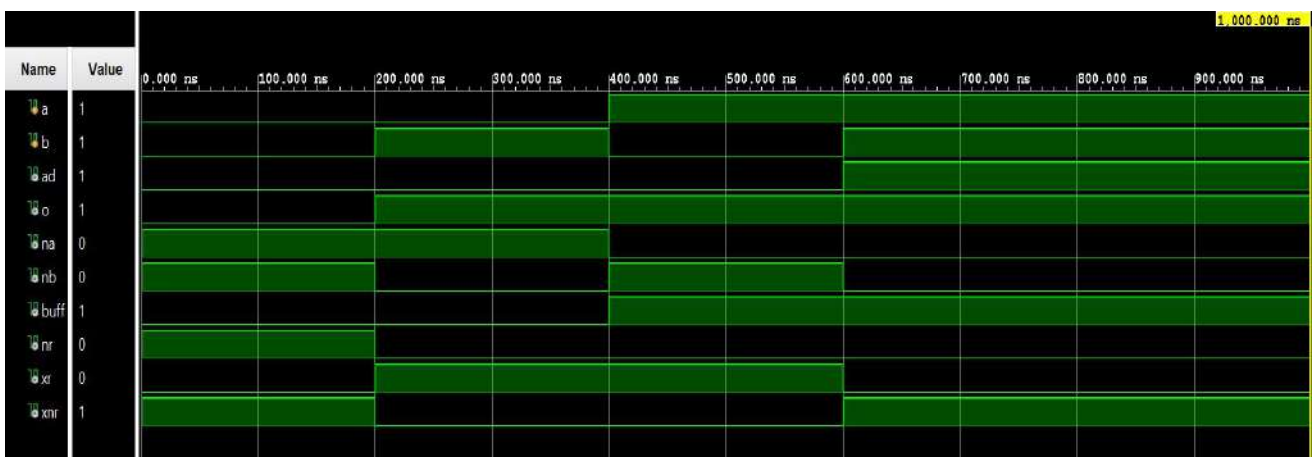
- Schematic



9. Realization of all logic gates using NAND gates

● Design Code	● Testbench Code
<pre> module universal_nand_gates(a,b,ad,o,na,nb,buff, nr,xr,xnr); input a,b; output ad,o,na,nb,buff,nr,xr,xnr; wire w1,w2,w3,w4,w5,w6,w7; //NOT gate nand n1(na,a,1); nand n2(nb,b,1); //AND gate nand n3(w1,a,b); nand n4(ad,w1,1); //OR gate nand n5(o,na,nb); //BUFFER nand n6(w2,a,1); nand n7(buff,w2,1); //NOR gate nand n8(w3,na,nb); nand n9(nr,w3,1); //XOR gate nand n10(w4,a,b); nand n11(w5,a,w4); nand n12(w6,w4,b); nand n13(xr,w5,w6); //XNOR gate nand n14(w7,a,b); nand n15(xnr,o,w7); endmodule </pre>	<pre> module universal_nand_gates_tb; reg a,b; wire ad,o,na,nb,buff,nr,xr,xnr; universal_nand_gates unand1(a,b,ad,o,na,nb,buff,nr,xr,xnr); initial begin a=1'b0; b=1'b0; #200 a=1'b0; b=1'b1; #200 a=1'b1; b=1'b0; #200 a=1'b1; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, ad=%b, o=%b, na=%b, nb=%b, buff=%b, nr=%b, xr=%b, xnr=%b", \$time,a,b,ad,o,na,nb,buff,nr,xr,xnr); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,ad,o,na,nb,buff,nr,xr,xnr); end endmodule </pre>

● Waveform



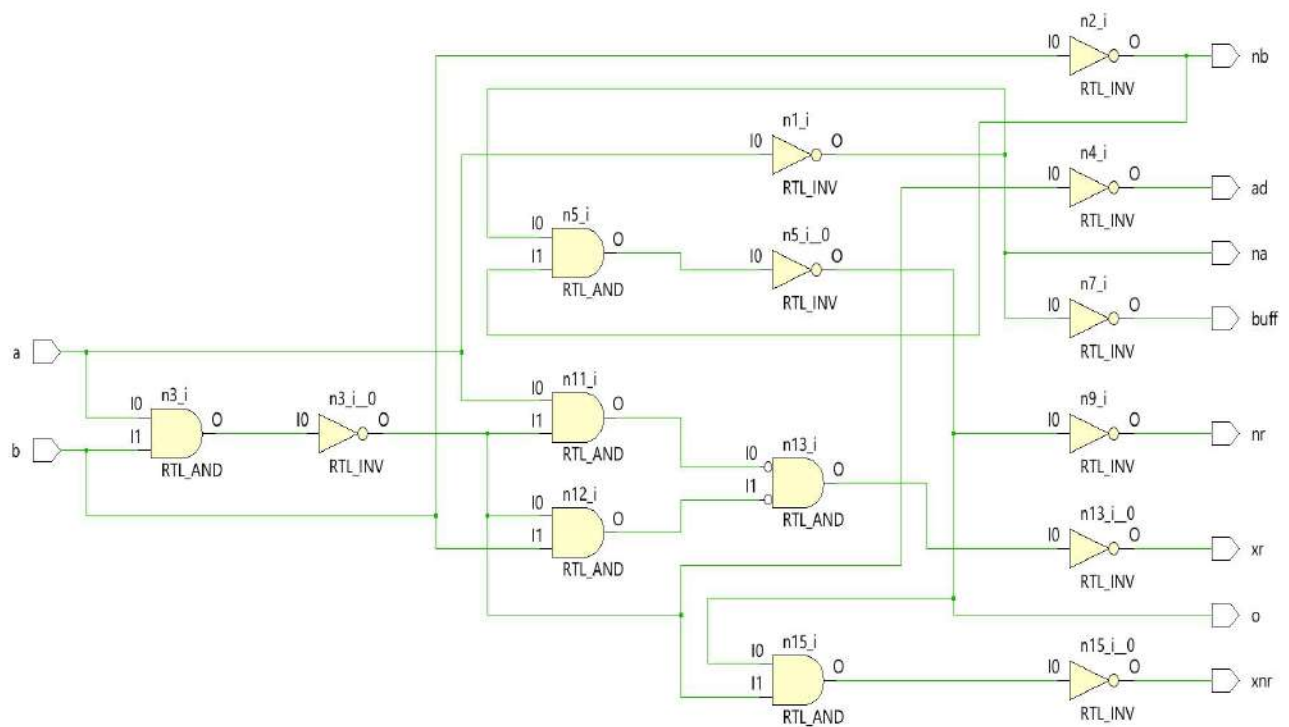
● Output

```

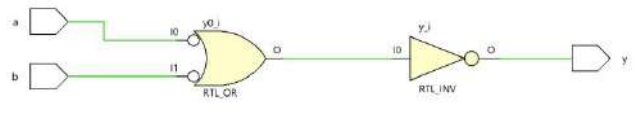
Simtime=0, a=0, b=0, ad=0, o=0, na=1, nb=1, buff=0, nr=1, xr=0, xnr=1
Simtime=200000, a=0, b=1, ad=0, o=1, na=1, nb=0, buff=0, nr=0, xr=1, xnr=0
Simtime=400000, a=1, b=0, ad=0, o=1, na=0, nb=1, buff=1, nr=0, xr=1, xnr=0
Simtime=600000, a=1, b=1, ad=1, o=1, na=0, nb=0, buff=1, nr=0, xr=0, xnr=1

```

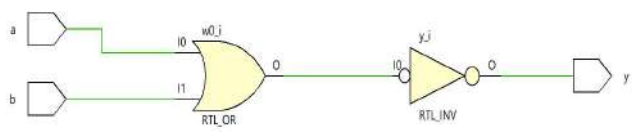
● Schematic



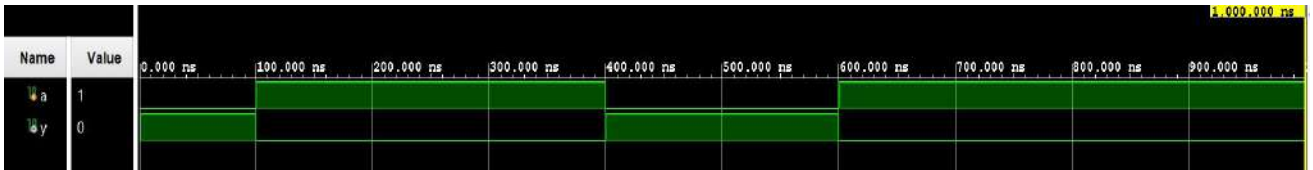
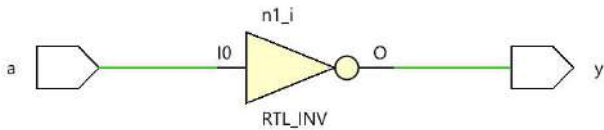
10. AND gate using NOR gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module nor_as_and(a,b,y); input a,b; output y; wire w1, w2; assign w1=~(a 0); assign w2=~(b 0); assign y=~(w1 w2); endmodule //Gate-level Modeling Style module nor_as_and(a,b,y); input a,b; output y; wire w1,w2; nor n1(w1,a,0); nor n2(w2,b,0); nor n3(y,w1,w2); endmodule</pre>	<pre>module nor_as_and_tb; reg a,b; wire y; nor_as_and uut(a,b,y); initial begin a=1'b0; b=1'b0; #200 a=1'b0; b=1'b1; #200 a=1'b1; b=1'b0; #200 a=1'b1; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, y=%b", \$time,a,b,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,y); end endmodule</pre>
● Waveform 	
● Output <pre>Simtime=0, a=0, b=0, y=0 Simtime=200000, a=0, b=1, y=0 Simtime=400000, a=1, b=0, y=0 Simtime=600000, a=1, b=1, y=1</pre>	● Schematic 

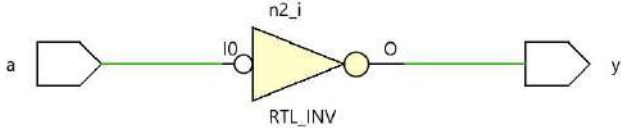
11. OR gate using NOR gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module nor_as_or(a,b,y); input a,b; output y; wire w; assign w=~(a b); assign y=~(w 0); endmodule //Gate-level Modeling Style module nor_as_or(a,b,y); input a,b; output y; wire w; nor n1(w,a,b); nor n2(y,w,0); endmodule</pre>	<pre>module nor_as_or_tb; reg a,b; wire y; nor_as_or uut(a,b,y); initial begin a=1'b0; b=1'b0; #200 a=1'b0; b=1'b1; #200 a=1'b1; b=1'b0; #200 a=1'b1; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, y=%b", \$time,a,b,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,y); end endmodule</pre>
● Waveform	
	
● Output	● Schematic
<pre>Simtime=0, a=0, b=0, y=0 Simtime=200000, a=0, b=1, y=1 Simtime=400000, a=1, b=0, y=1 Simtime=600000, a=1, b=1, y=1</pre>	

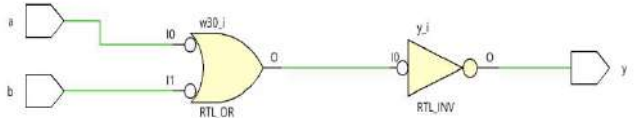
12. NOT gate using NOR gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module nor_as_not(a,y); input a; output y; assign y=~(a 0); endmodule //Gate-level Modeling Style module nor_as_not(a,y); input a; output y; nor n1(y,a,1); endmodule</pre>	<pre>module nor_as_not_tb; reg a; wire y; nor_as_not uut(a,y); initial begin a=1'b0; #100 a=1'b1; #100 a=1'b1; #100 a=1'b1; #100 a=1'b0; #100 a=1'b0; #100 a=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, y=%b", \$time,a,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,y); end endmodule</pre>
● Waveform	
	
● Output	● Schematic
<pre>Simtime=0, a=0, y=1 Simtime=100000, a=1, y=0 Simtime=400000, a=0, y=1 Simtime=600000, a=1, y=0</pre>	

13. Buffer using NOR gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module nor_as_buffer(a,y); input a; output y; wire w; assign w=~(a 0); assign y=~(w 0); endmodule //Gate-level Modeling Style module nor_as_buffer(a,y); input a; output y; wire w; nor n1(w,a,0); nor n2(y,w,0); endmodule</pre>	<pre>module nor_as_buffer_tb; reg a; wire y; nor_as_buffer uut(a,y); initial begin a=1'b0; #100 a=1'b1; #100 a=1'b1; #100 a=1'b1; #100 a=1'b0; #100 a=1'b0; #100 a=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, y=%b", \$time,a,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,y); end endmodule</pre>
● Waveform	
	
● Output	● Schematic
<pre>Simtime=0, a=0, y=0 Simtime=100000, a=1, y=1 Simtime=400000, a=0, y=0 Simtime=600000, a=1, y=1</pre>	

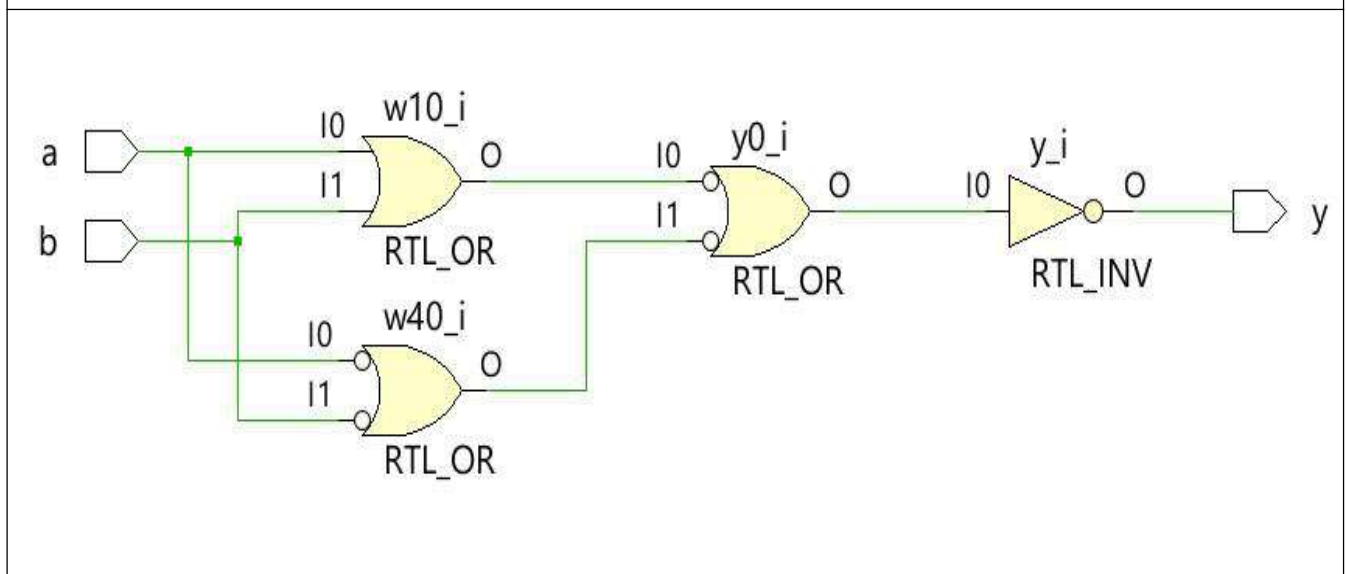
14. NAND gate using NOR gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module nor_as_nand(a,b,y); input a,b; output y; wire w1,w2,w3; assign w1=~(a 0); assign w2=~(b 0); assign w3=~(w1 w2); assign y=~(w3 0); endmodule //Gate-level Modeling Style module nor_as_nand(a,b,y); input a,b; output y; wire w1,w2,w3; nor n1(w1,a,0); nor n2(w2,b,0); nor n3(w3,w2,w1); nor n4(y,w3,0); endmodule</pre>	<pre>module nor_as_nand_tb; reg a,b; wire y; nor_as_nand uut(a,b,y); initial begin a=1'b0; b=1'b0; #200 a=1'b0; b=1'b1; #200 a=1'b1; b=1'b0; #200 a=1'b1; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, y=%b", \$time,a,b,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,y); end endmodule</pre>
● Waveform 	
● Output <pre>Simtime=0, a=0, b=0, y=1 Simtime=200000, a=0, b=1, y=1 Simtime=400000, a=1, b=0, y=1 Simtime=600000, a=1, b=1, y=0</pre>	● Schematic 

15. XOR gate using NAND gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module nor_as_xor(a,b,y); input a,b; output y; wire w1,w2,w3,w4; assign w1=~(a b); assign w2=~(a 0); assign w3=~(b 0); assign w4=~(w2 w3); assign y=~(w1 w4); endmodule //Gate-level Modeling Style module nand_as_xnor(a,b,y); input a,b; output y; wire w1,w2,w3,w4; nor n1(w1,a,b); nor n2(w2,a,0); nor n3(w3,b,0); nor n4(w4,w2,w3); nor n5(y,w1,w4); endmodule</pre>	<pre>module nor_as_xor_tb; reg a,b; wire y; nor_as_xor m1(a,b,y); initial begin a=1'b0; b=1'b0; #100 a=1'b0; b=1'b1; #100 a=1'b1; b=1'b0; #100 a=1'b1; b=1'b1; #100 a=1'b0; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, y=%b", \$time, a,b,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,y); end endmodule</pre>
● Waveform	
	
● Output	
<pre>Simtime=0, a=0, b=0, y=0 Simtime=100000, a=0, b=1, y=1 Simtime=200000, a=1, b=0, y=1 Simtime=300000, a=1, b=1, y=0 Simtime=400000, a=0, b=1, y=1</pre>	

- Schematic



16. XNOR gate using NOR gates

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module nor_as_xnor(a,b,y); input a,b; output y; wire w1,w2,w3; assign w1=~(a b); assign w2=~(a w1); assign w3=~(w1 b); assign y=~(w2 w3); endmodule //Gate-level Modeling Style module nor_as_xnor(a,b,y); input a,b; output y; wire w1,w2,w3; nor n1(w1,a,b); nor n2(w2,a,w1); nor n3(w3,w1,b); nor n4(y,w1,w2); endmodule</pre>	<pre>module nor_as_xnor_tb; reg a,b; wire y; nor_as_xnor m1(a,b,y); initial begin a=1'b0; b=1'b0; #100 a=1'b0; b=1'b1; #100 a=1'b1; b=1'b0; #100 a=1'b1; b=1'b1; #100 a=1'b0; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, y=%b", \$time, a,b,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,y); end endmodule</pre>

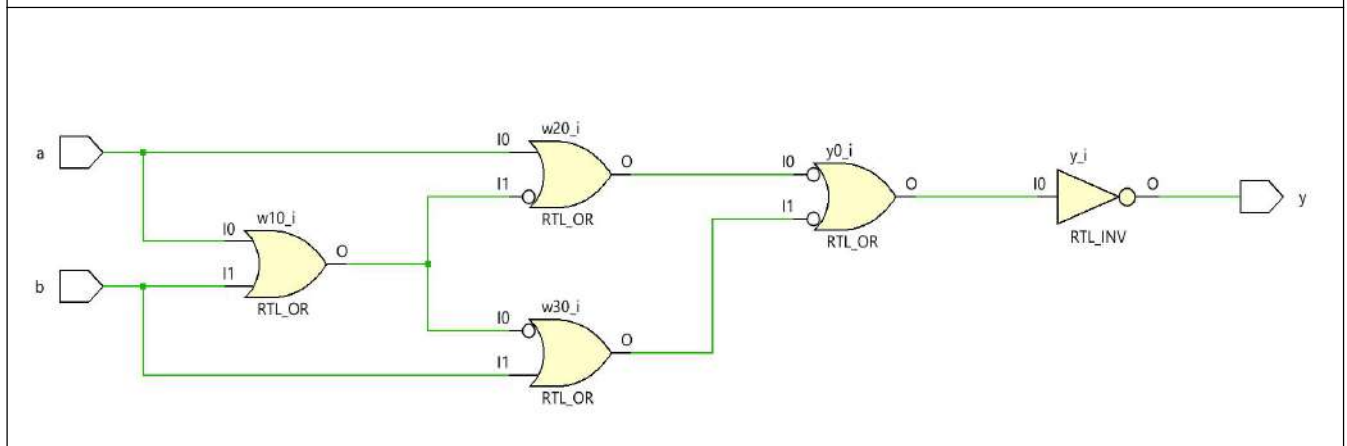
● Waveform



● Output

```
Simtime=0, a=0, b=0, y=1
Simtime=100000, a=0, b=1, y=0
Simtime=200000, a=1, b=0, y=0
Simtime=300000, a=1, b=1, y=1
Simtime=400000, a=0, b=1, y=0
```

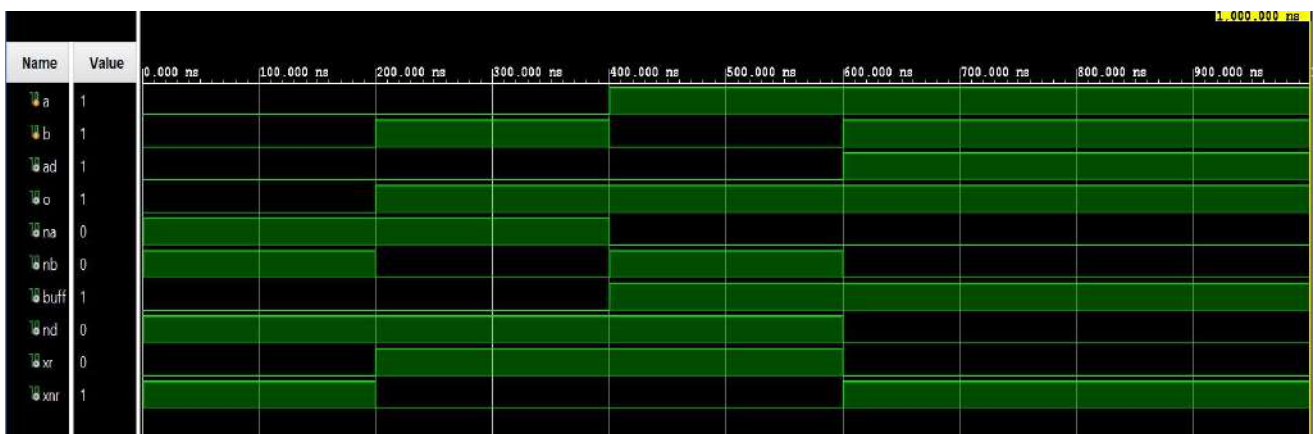
- Schematic



17. Realization of all logic gates using NOR gates

● Design Code	● Testbench Code
<pre> module universal_nor_gates(a,b,ad,o,na,nb,buff,n d,xr,xnr); input a,b; output ad,o,na,nb,buff,nd,xr,xnr; wire w1,w2,w3,w4,w5,w6,w7; //NOT gate nor n1(na,a,0); nor n2(nb,b,0); //AND gate nor n3(ad,na,nb); //OR gate nor n4(w1,a,b); nor n5(o,w1,0); //BUFFER nor n6(w2,a,0); nor n7(buff,w2,0); //NAND gate nor n8(w3,na,nb); nor n9(nd,w3,0); //XOR gate nor n10(w4,a,b); nor n11(xr,ad,w4); //XNOR gate nor n12(w5,a,b); nor n13(w6,a,w5); nor n14(w7,w5,b); nor n15(xnr,w7,w6); endmodule </pre>	<pre> module universal_nor_gates_tb; reg a,b; wire ad,o,na,nb,buff,nd,xr,xnr; universal_nor_gates unand1(a,b,ad,o,na,nb,buff,nd,xr,xnr); initial begin a=1'b0; b=1'b0; #200 a=1'b0; b=1'b1; #200 a=1'b1; b=1'b0; #200 a=1'b1; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, ad=%b, o=%b, na=%b, nb=%b, buff=%b, nd=%b, xr=%b, xnr=%b", \$time,a,b,ad,o,na,nb,buff,nd,xr,xnr); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,ad,o,na,nb,buff,nd,xr,xnr); end endmodule </pre>

● Waveform



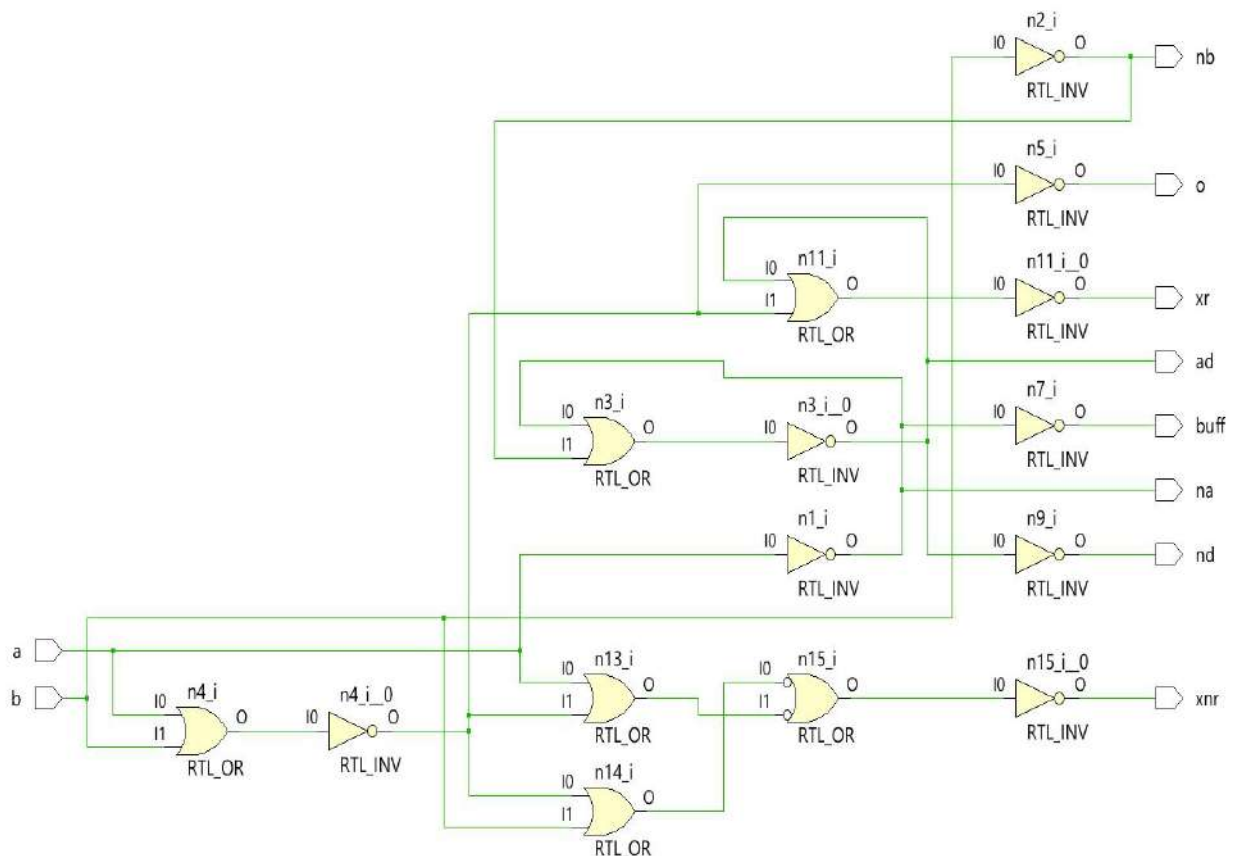
● Output

```

Simtime=0, a=0, b=0, ad=0, o=0, na=1, nb=1, buff=0, nd=1, xr=0, xnr=1
Simtime=200000, a=0, b=1, ad=0, o=1, na=1, nb=0, buff=0, nd=1, xr=1, xnr=0
Simtime=400000, a=1, b=0, ad=0, o=1, na=0, nb=1, buff=1, nd=1, xr=1, xnr=0
Simtime=600000, a=1, b=1, ad=1, o=1, na=0, nb=0, buff=1, nd=0, xr=0, xnr=1

```

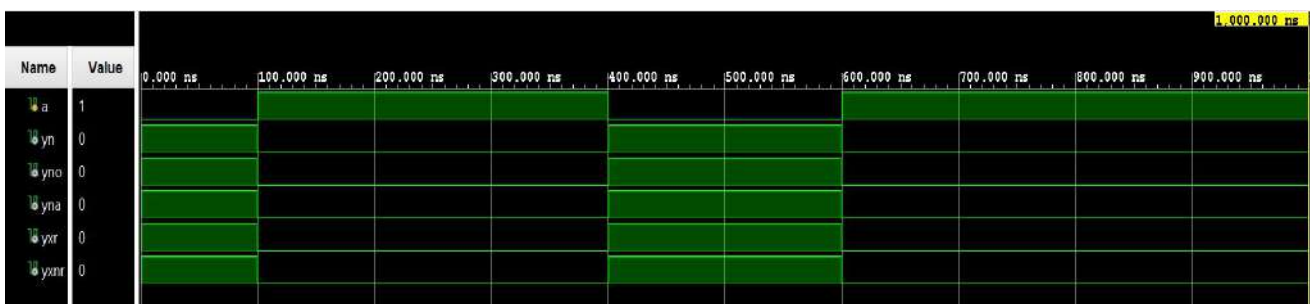
● Schematic



18. Realization of inverter using all possible gates

● Design Code	● Testbench Code
<pre> module all_gates_as_inverter(a,yn,yno,yna,yxr,yxnr); input a; output yn,yno,yna,yxr,yxnr; //or o1(yo,a,1); //and a1(ya,a,0); not n1(yn,a); nor nr1(yno,a,0); nand nd1(yna,a,1); xor xr1(yxr,a,1); xnor xnr1(yxnr,a,0); endmodule </pre>	<pre> module all_gates_as_inverter_tb; reg a; wire yn,yno,yna,yxr,yxnr; all_gates_as_inverter uut(a,yn,yno,yna,yxr,yxnr); initial begin a=1'b0; #100 a=1'b1; #100 a=1'b1; #100 a=1'b1; #100 a=1'b0; #100 a=1'b0; #100 a=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, yn=%b, yno=%b, yna=%b, yxr=%b, yxnr=%b", \$time,a,yn,yno,yna,yxr,yxnr); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,yn,yno,yna,yxr,yxnr); end endmodule </pre>

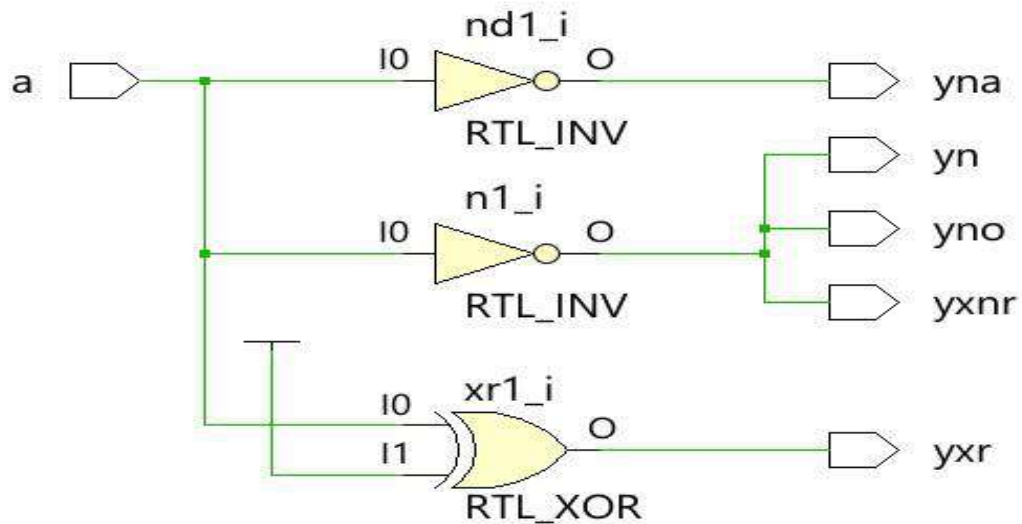
● Waveform



- Output

```
Simtime=0, a=0, yn=1, yno=1, yna=1, yxr=1, yxnr=1  
Simtime=100000, a=1, yn=0, yno=0, yna=0, yxr=0, yxnr=0  
Simtime=400000, a=0, yn=1, yno=1, yna=1, yxr=1, yxnr=1  
Simtime=600000, a=1, yn=0, yno=0, yna=0, yxr=0, yxnr=0
```

- Schematic



19. Demorgan's Law

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module demorgan_law(a,b,y1,y2,y3,y4); input a,b; output y1,y2,y3,y4; //1st law: ~(a b)=(~a)&(~b) assign y1=~(a b); assign y2=~(a)&(~(b)); //2nd law: ~(a&b)=(~a) (~b) assign y3=~(a&b); assign y4=~(a) (~(b)); endmodule</pre>	<pre>module demorgan_law_tb; reg a,b; wire y1,y2,y3,y4; demorgan_law dut(a,b,y1,y2,y3,y4); initial begin a=1'b0; b=1'b0; #200 a=1'b0; b=1'b1; #200 a=1'b1; b=1'b0; #200 a=1'b1; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, y1=%b, y2=%b, y3=%b, y4=%b", \$time,a,b,y1,y2,y3,y4); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,y1,y2,y3,y4); end endmodule</pre>

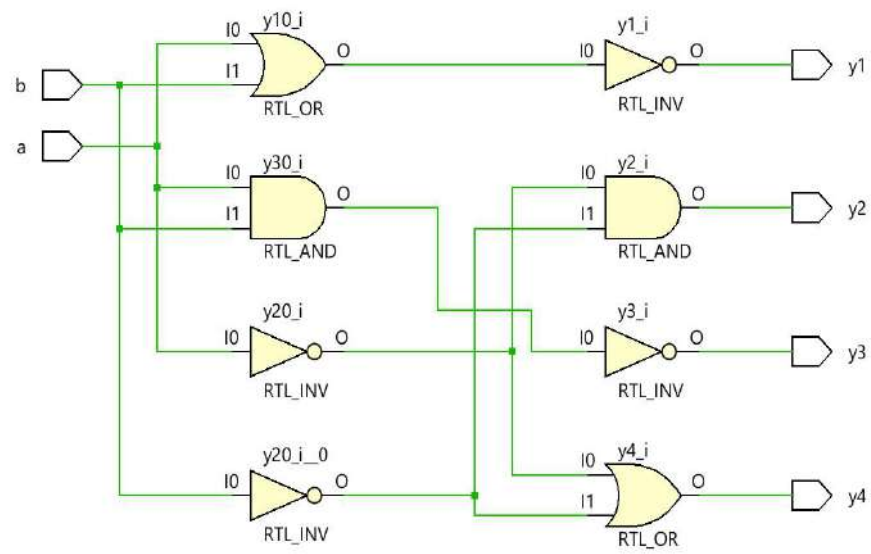
● Waveform



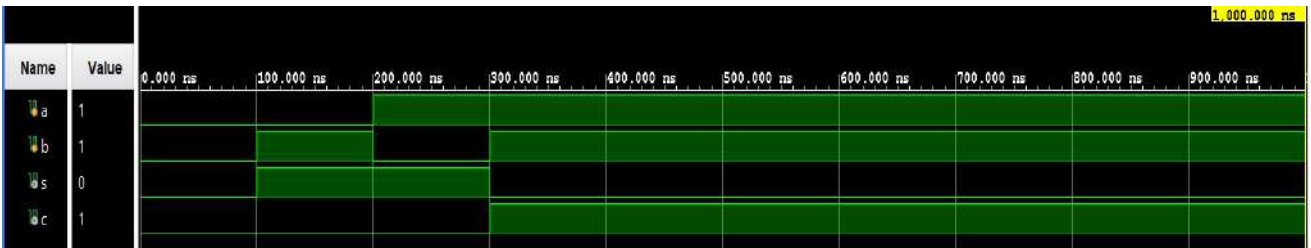
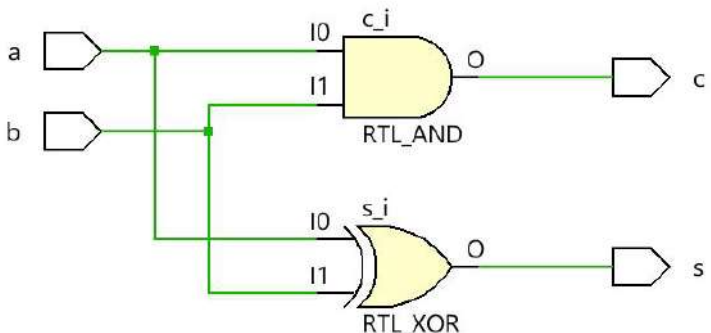
Output

```
Simtime=0, a=0, b=0, y1=1, y2=1, y3=1, y4=1
Simtime=200000, a=0, b=1, y1=0, y2=0, y3=1, y4=1
Simtime=400000, a=1, b=0, y1=0, y2=0, y3=1, y4=1
Simtime=600000, a=1, b=1, y1=0, y2=0, y3=0, y4=0
```


- Schematic



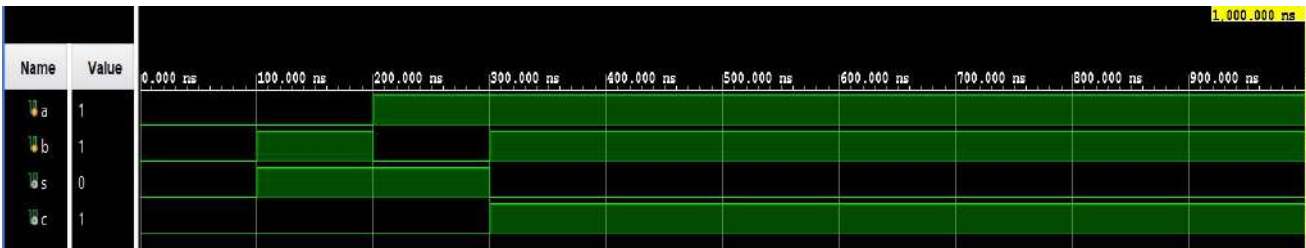
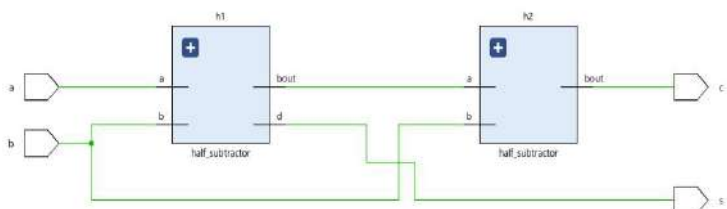
20. Half Adder

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module half_adder(a,b,s,c); input a,b; output s,c; assign s=a^b; assign c=a&b; endmodule //Gate-level Modeling Style module half_adder_gate(a,b,s,c); input a,b; output s,c; xor x1(s,a,b); and a1(c,a,b); endmodule</pre>	<pre>module half_adder_tb; reg a,b; wire s,c; half_adder ha1(a,b,s,c); initial begin a=1'b0; b=1'b0; #100 a=1'b0; b=1'b1; #100 a=1'b1; b=1'b0; #100 a=1'b1; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, s=%b, c=%b", \$time,a,b,s,c); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,s,c); end endmodule</pre>
● Waveform	
	
● Output	● Schematic
<pre>Simtime=0, a=0, b=0, s=0, c=0 Simtime=100000, a=0, b=1, s=1, c=0 Simtime=200000, a=1, b=0, s=1, c=0 Simtime=300000, a=1, b=1, s=0, c=1</pre>	

21. Half Adder (Behavioural Modeling Style)

● Design Code	● Testbench Code
<pre>//Behavioral Modeling Style module HA_beh(a,b,s,c); input a,b; output reg s,c; always@(a,b) begin s=a^b; c=a&b; end endmodule</pre>	<pre>module HA_beh_tb; reg a,b; wire s,c; HA_beh ha1(a,b,s,c); initial begin a=1'b0; b=1'b0; #100 a=1'b0; b=1'b1; #100 a=1'b1; b=1'b0; #100 a=1'b1; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, s=%b, c=%b", \$time,a,b,s,c); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,s,c); end endmodule</pre>
● Waveform	
● Output	● Schematic
<pre>Simtime=0, a=0, b=0, s=0, c=0 Simtime=100000, a=0, b=1, s=1, c=0 Simtime=200000, a=1, b=0, s=1, c=0 Simtime=300000, a=1, b=1, s=0, c=1</pre>	

22. Half Adder using Half Subtractor

● Design Code	● Testbench Code
<pre> `include "half_subtractor.v" module ha_using_hs(a,b,s,c); input a,b; output s,c; wire w1, w2; half_subtractor h1(a,b,s,w1); half_subtractor h2(w1,b,w2,c); endmodule </pre>	<pre> module ha_using_hs_tb; reg a,b; wire s,c; ha_using_hs ha1(a,b,s,c); initial begin a=1'b0; b=1'b0; #100 a=1'b0; b=1'b1; #100 a=1'b1; b=1'b0; #100 a=1'b1; b=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, s=%b, c=%b", \$time,a,b,s,c); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,s,c); end endmodule </pre>
● Waveform	
	
● Output	● Schematic
<pre> Simtime=0, a=0, b=0, s=0, c=0 Simtime=100000, a=0, b=1, s=1, c=0 Simtime=200000, a=1, b=0, s=1, c=0 Simtime=300000, a=1, b=1, s=0, c=1 </pre>	

23. Full Adder

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module full_adder(a,b,c,s,cout); input a,b,c; output s,cout; assign s=a^b^c; assign cout=(a&b) (b&c) (a&c); endmodule //Gate-level Modeling Style module full_adder_gate(a,b,c,s,cout); input a,b,c; output s,cout; wire w1,w2,w3; xor x1(s,a,b,c); and a1(w1,a,b); and a2(w2,b,c); and a3(w3,a,c); or o1(cout,w1,w2,w3); endmodule</pre>	<pre>module full_adder_tb; reg a,b,c; wire s,cout; full_adder fa1(a,b,c,s,cout); initial begin a=1'b0; b=1'b0; c=1'b0; #100 a=1'b0; b=1'b0; c=1'b1; #100 a=1'b0; b=1'b1; c=1'b0; #100 a=1'b0; b=1'b1; c=1'b1; #100 a=1'b1; b=1'b0; c=1'b0; #100 a=1'b1; b=1'b0; c=1'b1; #100 a=1'b1; b=1'b1; c=1'b0; #100 a=1'b1; b=1'b1; c=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, c=%b, s=%b, cout=%b", \$time, a,b,c,s,cout); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,c,s,cout); end endmodule</pre>

● Waveform



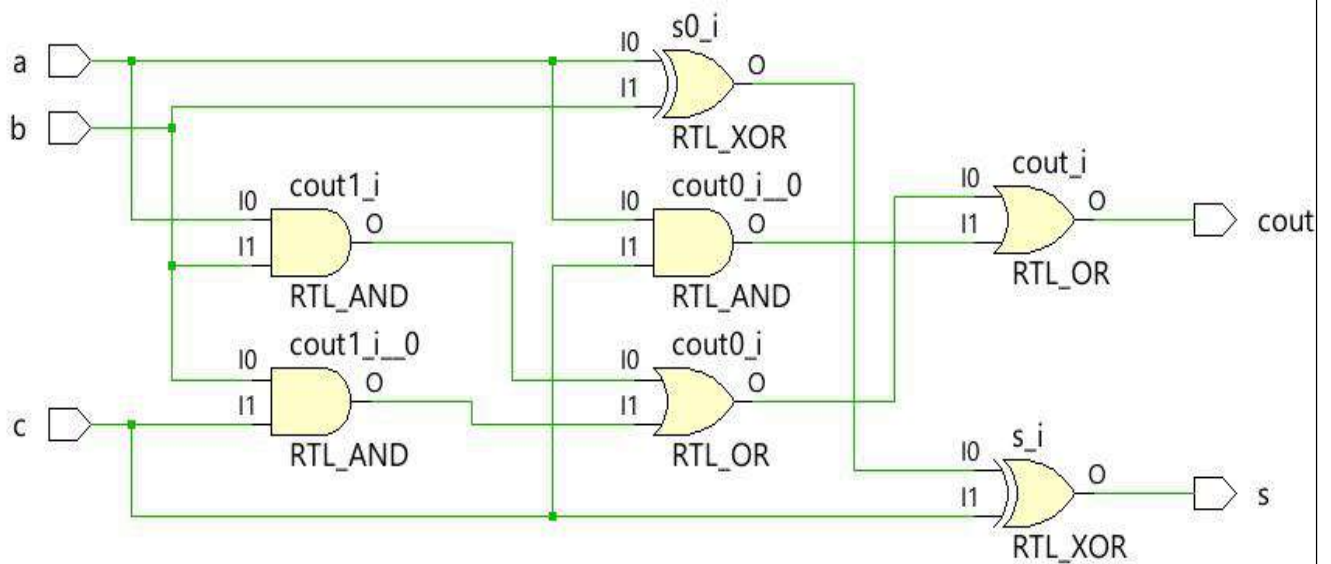
Output

```

Simtime=0, a=0, b=0, c=0, s=0, cout=0
Simtime=100000, a=0, b=0, c=1, s=1, cout=0
Simtime=200000, a=0, b=1, c=0, s=1, cout=0
Simtime=300000, a=0, b=1, c=1, s=0, cout=1
Simtime=400000, a=1, b=0, c=0, s=1, cout=0
Simtime=500000, a=1, b=0, c=1, s=0, cout=1
Simtime=600000, a=1, b=1, c=0, s=0, cout=1
Simtime=700000, a=1, b=1, c=1, s=1, cout=1

```

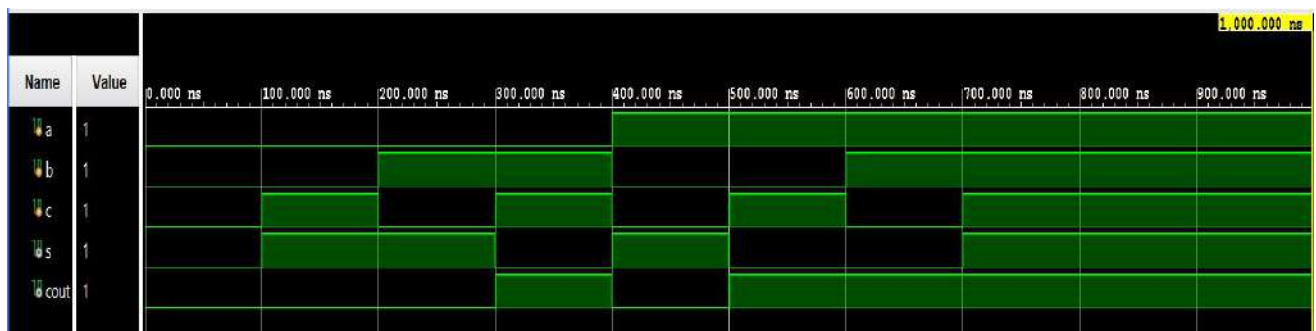
● Schematic



24. Full Adder (Behavioural Modeling Style)

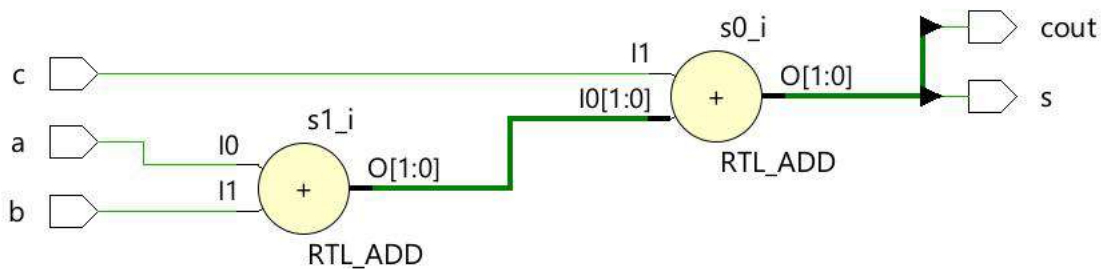
● Design Code	● Testbench Code
<pre>//Behavioral Modeling Style module FA_beh(a,b,c,s,cout); input a,b,c; output reg s,cout; always@(a,b,c) {cout,s}=a+b+c; //performs s=a^b^c; cout=(a&b) (b&c) (a&c) endmodule</pre>	<pre>module FA_beh_tb; reg a,b,c; wire s,cout; FA_beh fa1(a,b,c,s,cout); initial begin a=1'b0; b=1'b0; c=1'b0; #100 a=1'b0; b=1'b0; c=1'b1; #100 a=1'b0; b=1'b1; c=1'b0; #100 a=1'b0; b=1'b1; c=1'b1; #100 a=1'b1; b=1'b0; c=1'b0; #100 a=1'b1; b=1'b0; c=1'b1; #100 a=1'b1; b=1'b1; c=1'b0; #100 a=1'b1; b=1'b1; c=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, c=%b, s=%b, cout=%b", \$time, a,b,c,s,cout); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,c,s,cout); end endmodule</pre>

● Waveform



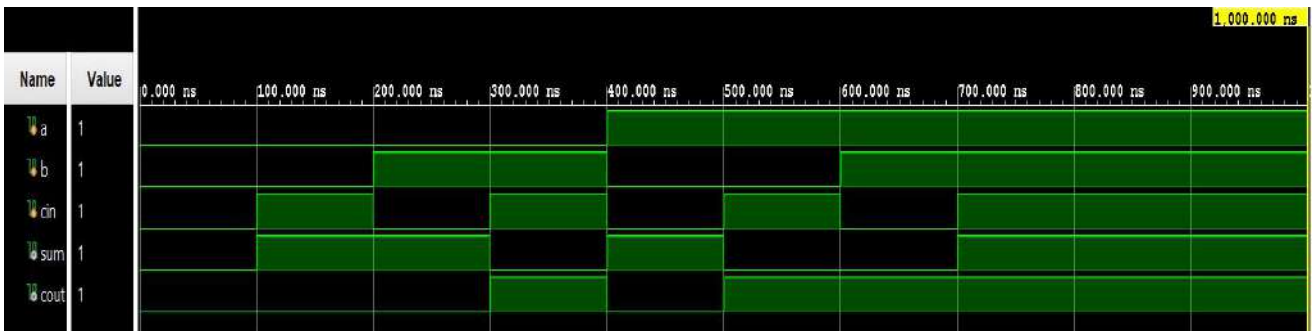
Output

```
Simtime=0, a=0, b=0, c=0, s=0, cout=0
Simtime=100000, a=0, b=0, c=1, s=1, cout=0
Simtime=200000, a=0, b=1, c=0, s=1, cout=0
Simtime=300000, a=0, b=1, c=1, s=0, cout=1
Simtime=400000, a=1, b=0, c=0, s=1, cout=0
Simtime=500000, a=1, b=0, c=1, s=0, cout=1
Simtime=600000, a=1, b=1, c=0, s=0, cout=1
Simtime=700000, a=1, b=1, c=1, s=1, cout=1
```

● Schematic

NOTE: The above obtained schematic is due to “+” assigned in the design code.

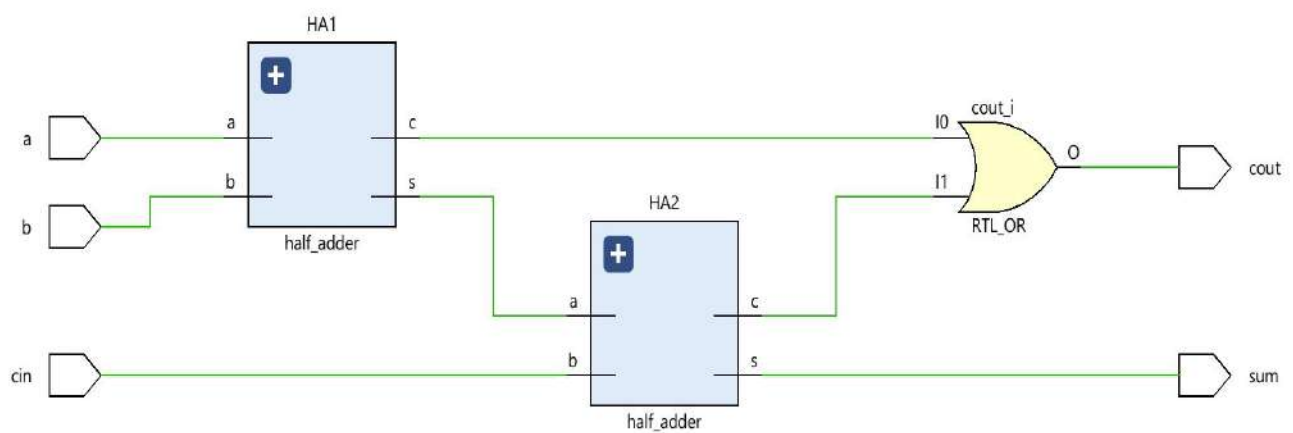
25. Full Adder using Half Adders

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style `include "half_adder.v" module full_adder_using_half_adder (a,b,cin,sum,cout); input a, b, cin; output sum, cout; wire s1, c1, c2; half_adder HA1 (a,b,s1,c1); half_adder HA2 (s1,cin,sum,c2); assign cout = c1 c2; endmodule</pre>	<pre>module full_adder_using_half_adder_tb; reg a,b,cin; wire sum,cout; full_adder_using_half_adder fa1(a,b,cin,sum,cout); initial begin a=1'b0; b=1'b0; cin=1'b0; #100 a=1'b0; b=1'b0; cin=1'b1; #100 a=1'b0; b=1'b1; cin=1'b0; #100 a=1'b0; b=1'b1; cin=1'b1; #100 a=1'b1; b=1'b0; cin=1'b0; #100 a=1'b1; b=1'b0; cin=1'b1; #100 a=1'b1; b=1'b1; cin=1'b0; #100 a=1'b1; b=1'b1; cin=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, cin=%b, sum=%b, cout=%b", \$time, a,b,cin,sum,cout); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,cin,sum,cout); end endmodule</pre>
● Waveform	
 The waveform shows the timing of signals for a Full Adder using Half Adders. The signals are: a, b, cin (inputs) and sum, cout (outputs). The time scale ranges from 0.000 ns to 1.000 ns. The inputs a, b, and cin are all high (1) throughout the entire time period. The output sum is high (1) throughout the entire time period. The output cout is high (1) throughout the entire time period.	

Output

Simtime=0, a=0, b=0, cin=0, sum=0, cout=0
Simtime=100000, a=0, b=0, cin=1, sum=1, cout=0
Simtime=200000, a=0, b=1, cin=0, sum=1, cout=0
Simtime=300000, a=0, b=1, cin=1, sum=0, cout=1
Simtime=400000, a=1, b=0, cin=0, sum=1, cout=0
Simtime=500000, a=1, b=0, cin=1, sum=0, cout=1
Simtime=600000, a=1, b=1, cin=0, sum=0, cout=1
Simtime=700000, a=1, b=1, cin=1, sum=1, cout=1

● Schematic



26. Full Adder using Half Subtractors

● Design Code	● Testbench Code
<pre>`include "half_subtractor.v" module full_adder_using_half_subtractor (a, b, cin, sum, cout); input a, b, cin; output sum, cout; wire s1, b1, b2; half_subtractor HS1 (a, b, s1, b1); half_subtractor HS2 (s1, cin, sum, b2); assign cout = (a & b) (cin & (a ^ b)); // correct carry logic Or wire na, nb, ncin; wire t1, t2, t3; assign na = ~a; assign nb = ~b; assign ncin = ~cin; assign t1 = a & b; assign t2 = b & cin; assign t3 = a & cin; assign cout = t1 t2 t3; endmodule</pre>	<pre>module full_adder_using_half_subtractor_tb; reg a,b,cin; wire sum,cout; full_adder_using_half_subtractor fa1(a,b,cin,sum,cout); initial begin a=1'b0; b=1'b0; cin=1'b0; #100 a=1'b0; b=1'b0; cin=1'b1; #100 a=1'b0; b=1'b1; cin=1'b0; #100 a=1'b0; b=1'b1; cin=1'b1; #100 a=1'b1; b=1'b0; cin=1'b0; #100 a=1'b1; b=1'b0; cin=1'b1; #100 a=1'b1; b=1'b1; cin=1'b0; #100 a=1'b1; b=1'b1; cin=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, cin=%b, sum=%b, cout=%b", \$time, a,b,cin,sum,cout); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,cin,sum,cout); end endmodule</pre>

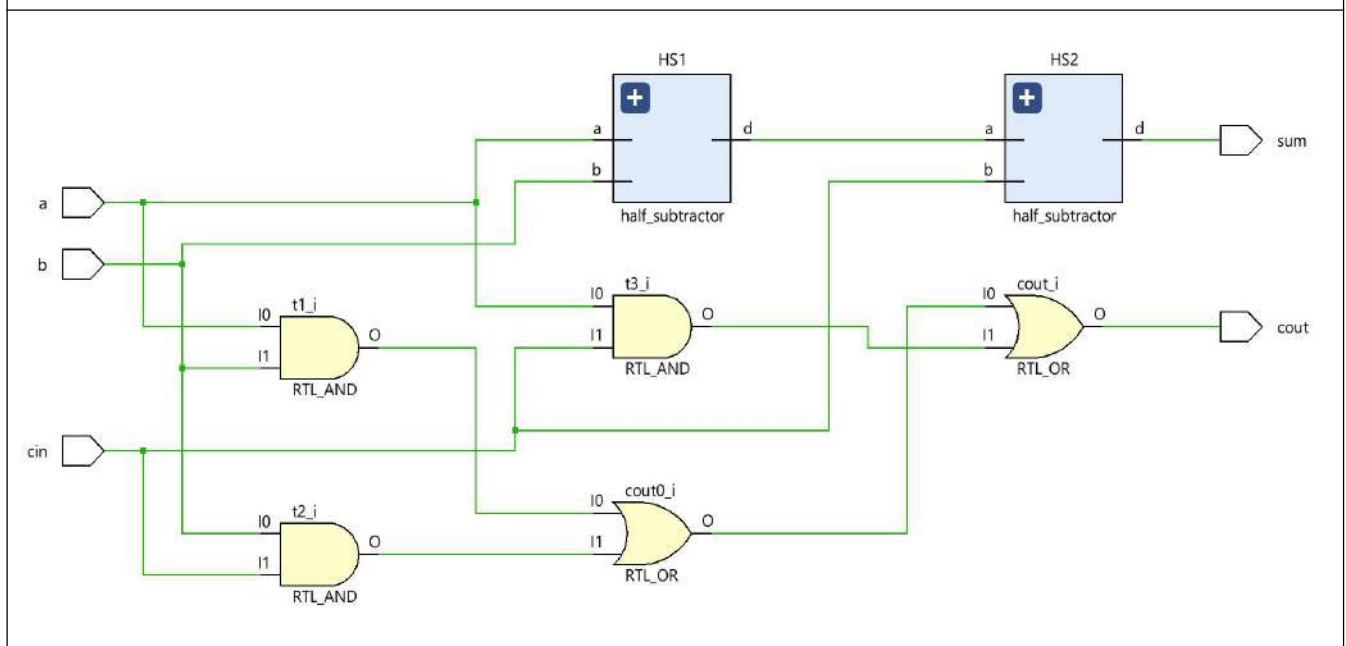
● Waveform

		1,000.000 ns										
Name	Value	0.000 ns	100.000 ns	200.000 ns	300.000 ns	400.000 ns	500.000 ns	600.000 ns	700.000 ns	800.000 ns	900.000 ns	
a	1											
b	1											
cin	1											
sum	1											
cout	1											

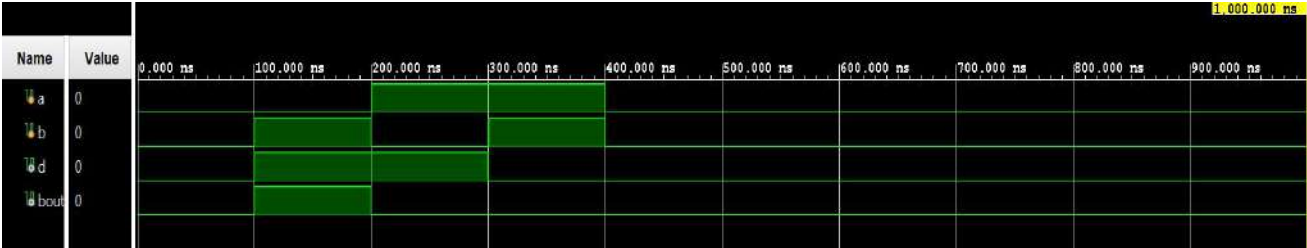
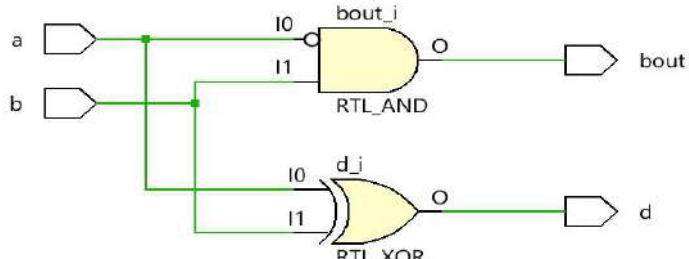
Output

Simtime=0, a=0, b=0, cin=0, sum=0, cout=0
Simtime=100000, a=0, b=0, cin=1, sum=1, cout=0
Simtime=200000, a=0, b=1, cin=0, sum=1, cout=0
Simtime=300000, a=0, b=1, cin=1, sum=0, cout=1
Simtime=400000, a=1, b=0, cin=0, sum=1, cout=0
Simtime=500000, a=1, b=0, cin=1, sum=0, cout=1
Simtime=600000, a=1, b=1, cin=0, sum=0, cout=1
Simtime=700000, a=1, b=1, cin=1, sum=1, cout=1

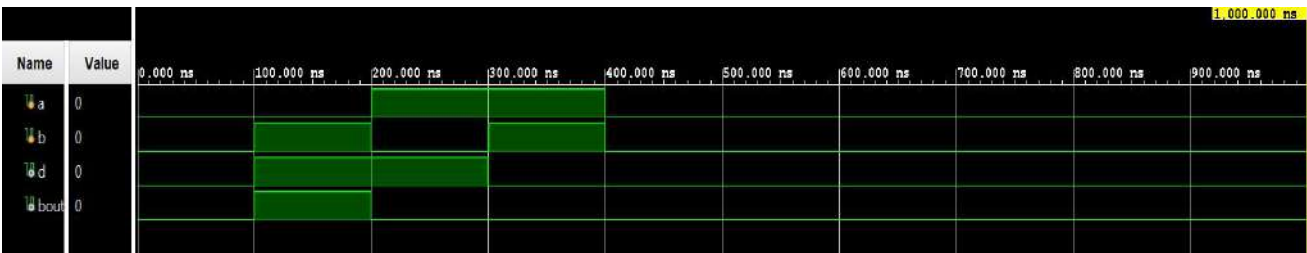
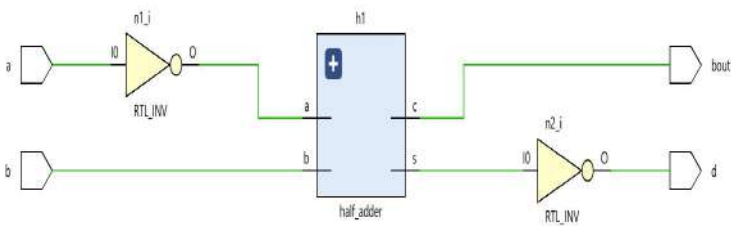
● Schematic



27. Half Subtractor

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module half_subtractor(a,b,d,bout); input a,b; output d,bout; assign d=a^b; assign bout=((~a)&b); endmodule //Gate-level Modeling Style module half_subtractor_gate(a,b,d,bout); input a,b; output d,bout; wire w1; not n1(w1,a); xor x1(d,a,b); and a1(bout,w1,b); endmodule</pre>	<pre>module half_subtractor_tb; reg a,b; wire d,bout; half_subtractor hs_d1(a,b,d,bout); initial begin a=1'b0; b=1'b0; #100 a=1'b0; b=1'b1; #100 a=1'b1; b=1'b0; #100 a=1'b1; b=1'b1; #100 a=1'b0; b=1'b0; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, d=%b, bout=%b", \$time, a,b,d,bout); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,d,bout); end endmodule</pre>
● Waveform	
	
● Output	● Schematic
<pre>Simtime=0, a=0, b=0, d=0, bout=0 Simtime=100000, a=0, b=1, d=1, bout=1 Simtime=200000, a=1, b=0, d=1, bout=0 Simtime=300000, a=1, b=1, d=0, bout=0 Simtime=400000, a=0, b=0, d=0, bout=0</pre>	

28. Half Subtractor using Half Adder

● Design Code	● Testbench Code
<pre> `include "half_adder.v" module hs_using_ha(a,b,d,bout); input a,b; output d,bout; wire w1, w2; not n1(w1,a); half_adder h1(w1,b,w2,bout); not n2(d,w2); endmodule </pre>	<pre> module hs_using_ha_tb; reg a,b; wire d,bout; hs_using_ha hs_d1(a,b,d,bout); initial begin a=1'b0; b=1'b0; #100 a=1'b0; b=1'b1; #100 a=1'b1; b=1'b0; #100 a=1'b1; b=1'b1; #100 a=1'b0; b=1'b0; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, d=%b, bout=%b", \$time, a,b,d,bout); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,d,bout); end endmodule </pre>
● Waveform	
	
● Output	● Schematic
<pre> Simtime=0, a=0, b=0, d=0, bout=0 Simtime=100000, a=0, b=1, d=1, bout=1 Simtime=200000, a=1, b=0, d=1, bout=0 Simtime=300000, a=1, b=1, d=0, bout=0 Simtime=400000, a=0, b=0, d=0, bout=0 </pre>	

29. Full Subtractor

● Design Code	● Testbench Code												
<pre>//Dataflow Modeling Style module full_subtractor(a,b,c,d,bout); input a,b,c; output d,bout; assign d=a^b^c; assign bout=((~a)&b) (b&c) ((~a)&c); endmodule //Gate-level Modeling Style module full_subtractor_gate(a,b,c,d,bout); input a,b,c; output d,bout; wire w1,w2,w3,w4; not n1(w1,a); xor x1(d,a,b,c); and a1(w2,w1,b); and a2(w3,b,c); and a3(w4,w1,c); or o1(bout,w2,w3,w4); endmodule</pre>	<pre>module full_subtractor_tb; reg a,b,c; wire d,bout; full_subtractor fs_d1(a,b,c,d,bout); initial begin a=1'b0; b=1'b0; c=1'b0; #100 a=1'b0; b=1'b0; c=1'b1; #100 a=1'b0; b=1'b1; c=1'b0; #100 a=1'b0; b=1'b1; c=1'b1; #100 a=1'b1; b=1'b0; c=1'b0; #100 a=1'b1; b=1'b0; c=1'b1; #100 a=1'b1; b=1'b1; c=1'b0; #100 a=1'b1; b=1'b1; c=1'b1; #100 a=1'b0; b=1'b0; c=1'b0; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b,c=%b, d=%b, bout=%b", \$time, a,b,c,d,bout); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,c,d,bout); end endmodule</pre>												
● Waveform													
 <table border="1"> <thead> <tr> <th>Name</th> <th>Value</th> </tr> </thead> <tbody> <tr> <td>a</td> <td>0</td> </tr> <tr> <td>b</td> <td>0</td> </tr> <tr> <td>c</td> <td>0</td> </tr> <tr> <td>d</td> <td>0</td> </tr> <tr> <td>bout</td> <td>0</td> </tr> </tbody> </table>		Name	Value	a	0	b	0	c	0	d	0	bout	0
Name	Value												
a	0												
b	0												
c	0												
d	0												
bout	0												

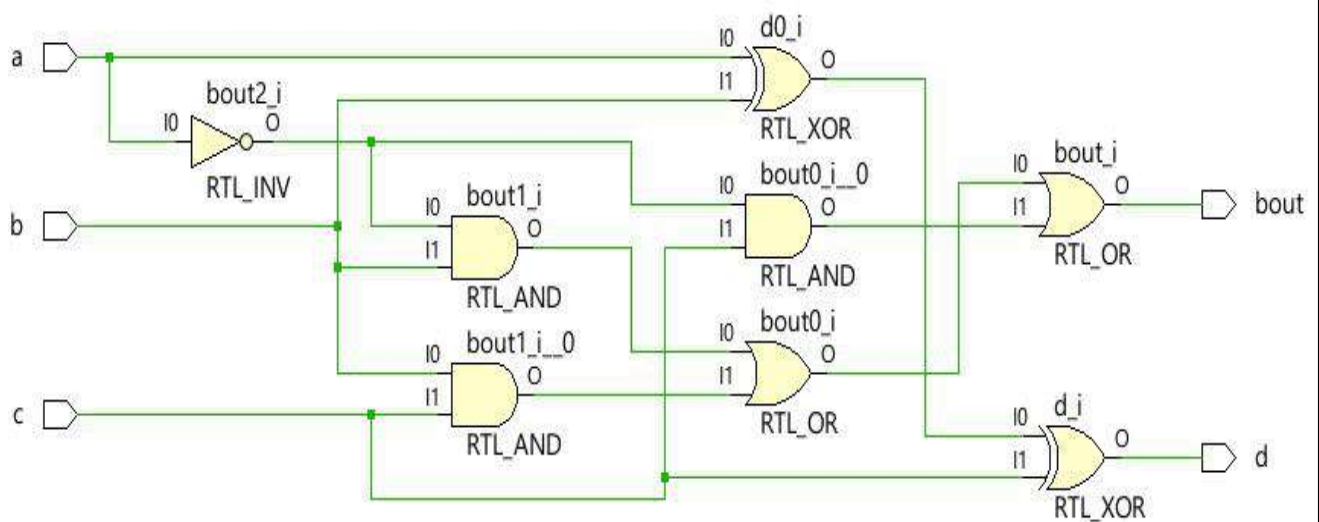
● Output

```

Simtime=0, a=0, b=0,c=0, d=0, bout=0
Simtime=100000, a=0, b=0,c=1, d=1, bout=1
Simtime=200000, a=0, b=1,c=0, d=1, bout=1
Simtime=300000, a=0, b=1,c=1, d=0, bout=1
Simtime=400000, a=1, b=0,c=0, d=1, bout=0
Simtime=500000, a=1, b=0,c=1, d=0, bout=0
Simtime=600000, a=1, b=1,c=0, d=0, bout=0
Simtime=700000, a=1, b=1,c=1, d=1, bout=1
Simtime=800000, a=0, b=0,c=0, d=0, bout=0

```

● Schematic



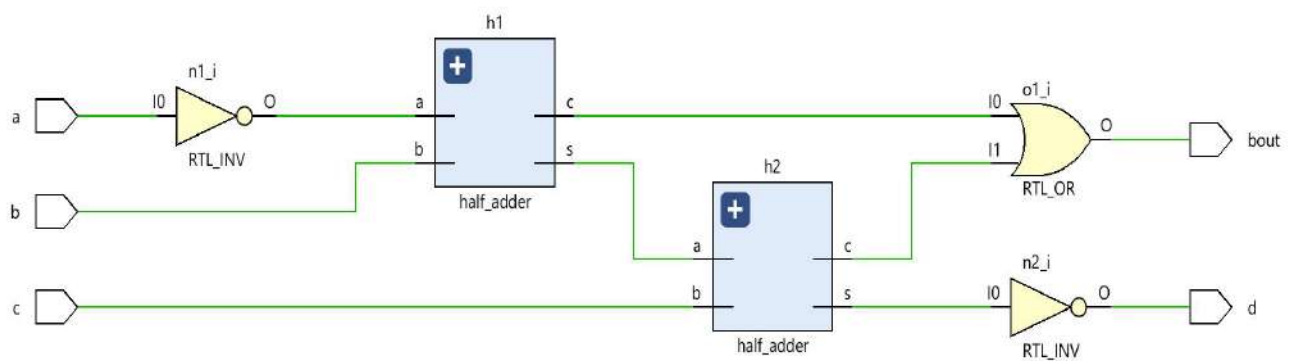
30. Full Subtractor using Half Adders

● Design Code	● Testbench Code												
<pre> `include "half_adder.v" module fs_using_ha(a,b,c,d,bout); input a,b,c; output d,bout; wire w1,w2,w3,w4,w5; not n1(w1,a); half_adder h1(w1,b,w2,w3); half_adder h2(w2,c,w4,w5); or o1(bout,w3,w5); not n2(d,w4); endmodule </pre>	<pre> module fs_using_ha_tb; reg a,b,c; wire d,bout; fs_using_ha fs_d1(a,b,c,d,bout); initial begin a=1'b0; b=1'b0; c=1'b0; #100 a=1'b0; b=1'b0; c=1'b1; #100 a=1'b0; b=1'b1; c=1'b0; #100 a=1'b0; b=1'b1; c=1'b1; #100 a=1'b1; b=1'b0; c=1'b0; #100 a=1'b1; b=1'b0; c=1'b1; #100 a=1'b1; b=1'b1; c=1'b0; #100 a=1'b1; b=1'b1; c=1'b1; #100 a=1'b0; b=1'b0; c=1'b0; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b,c=%b, d=%b, bout=%b", \$time, a,b,c,d,bout); end initial begin \$dumpfile("dump.vcd"); \$dumppvars(0,a,b,c,d,bout); end endmodule </pre>												
● Waveform													
 <table border="1"> <thead> <tr> <th>Name</th><th>Value</th></tr> </thead> <tbody> <tr> <td>a</td><td>0</td></tr> <tr> <td>b</td><td>0</td></tr> <tr> <td>c</td><td>0</td></tr> <tr> <td>d</td><td>0</td></tr> <tr> <td>bout</td><td>0</td></tr> </tbody> </table>		Name	Value	a	0	b	0	c	0	d	0	bout	0
Name	Value												
a	0												
b	0												
c	0												
d	0												
bout	0												

● Output

```
Simtime=0, a=0, b=0, c=0, d=0, bout=0
Simtime=100000, a=0, b=0, c=1, d=1, bout=1
Simtime=200000, a=0, b=1, c=0, d=1, bout=1
Simtime=300000, a=0, b=1, c=1, d=0, bout=1
Simtime=400000, a=1, b=0, c=0, d=1, bout=0
Simtime=500000, a=1, b=0, c=1, d=0, bout=0
Simtime=600000, a=1, b=1, c=0, d=0, bout=0
Simtime=700000, a=1, b=1, c=1, d=1, bout=1
Simtime=800000, a=0, b=0, c=0, d=0, bout=0
```

● Schematic



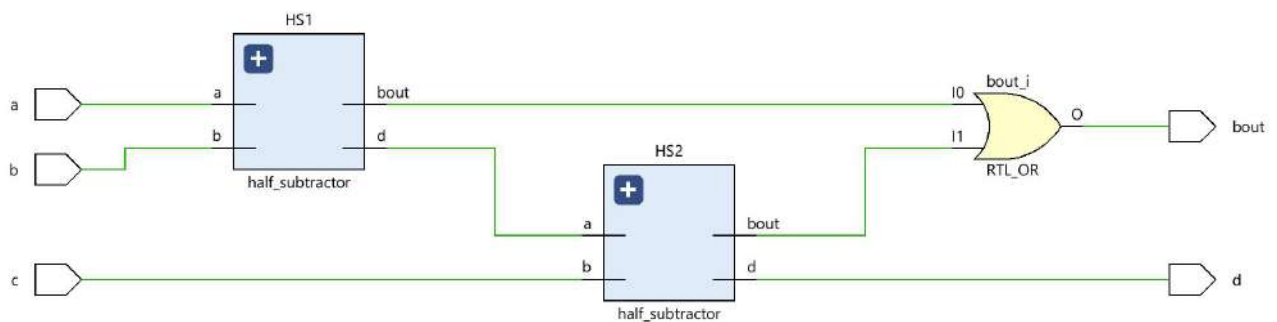
31. Full Subtractor using Half Subtractors

● Design Code	● Testbench Code
<pre> `include "half_subtractor.v" module fs_using_hs(a,b,c,d,bout); input a, b, c; output d, bout; wire d1, b1, b2; half_subtractor HS1 (a,b,d1,b1); half_subtractor HS2 (d1,c,d,b2); assign bout = b1 b2; endmodule </pre>	<pre> module fs_using_hs_tb; reg a,b,c; wire d,bout; fs_using_hs fs_d1(a,b,c,d,bout); initial begin a=1'b0; b=1'b0; c=1'b0; #100 a=1'b0; b=1'b0; c=1'b1; #100 a=1'b0; b=1'b1; c=1'b0; #100 a=1'b0; b=1'b1; c=1'b1; #100 a=1'b1; b=1'b0; c=1'b0; #100 a=1'b1; b=1'b0; c=1'b1; #100 a=1'b1; b=1'b1; c=1'b0; #100 a=1'b1; b=1'b1; c=1'b1; #100 a=1'b0; b=1'b1; c=1'b0; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b,c=%b, d=%b, bout=%b", \$time, a,b,c,d,bout); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,c,d,bout); end endmodule </pre>
● Waveform	
	

● Output

```
Simtime=0, a=0, b=0, c=0, d=0, bout=0
Simtime=100000, a=0, b=0, c=1, d=1, bout=1
Simtime=200000, a=0, b=1, c=0, d=1, bout=1
Simtime=300000, a=0, b=1, c=1, d=0, bout=1
Simtime=400000, a=1, b=0, c=0, d=1, bout=0
Simtime=500000, a=1, b=0, c=1, d=0, bout=0
Simtime=600000, a=1, b=1, c=0, d=0, bout=0
Simtime=700000, a=1, b=1, c=1, d=1, bout=1
Simtime=800000, a=0, b=0, c=0, d=0, bout=0
```

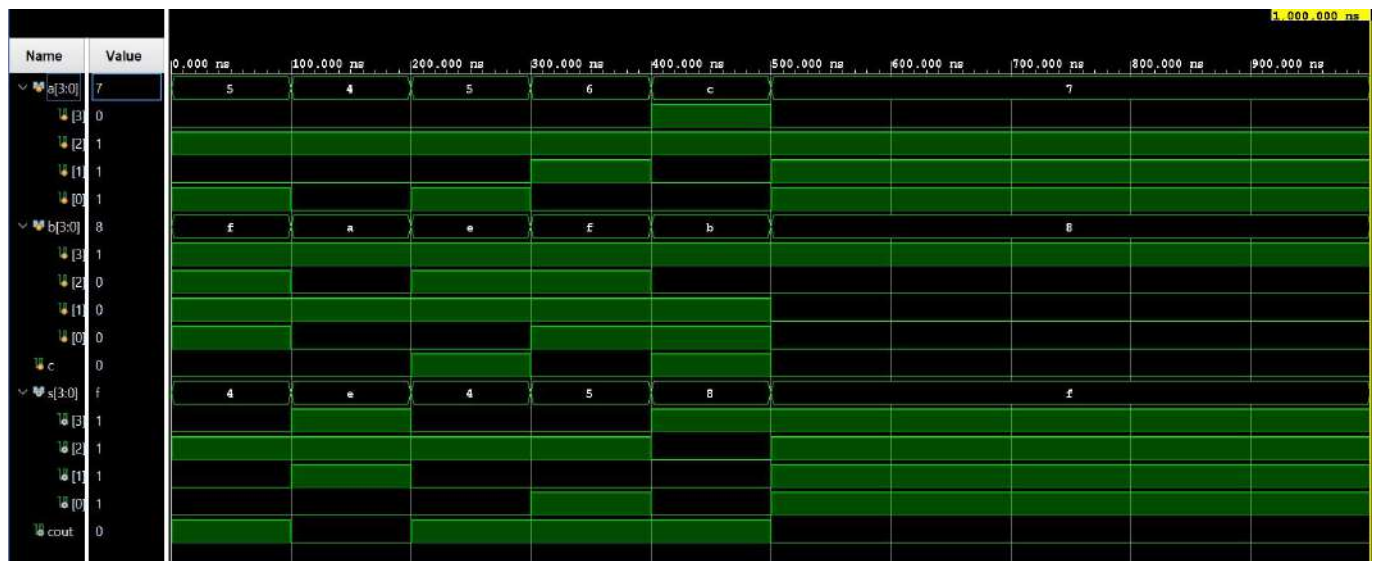
● Schematic



32. 4 bit ripple carry adder using full adders

● Design code of Half Adder	● Design code of Full Adder using half adder
<pre> module Adder(a,b,s,c); input a,b; output s,c; assign s=a^b; assign c=a&b; endmodule </pre>	<pre> `include "Adder.v" module FA(a,b,c,s,cout); input a,b,c; output s,cout; wire w1,w2,w3; Adder h1(a,b,w1,w2); Adder h2(w1,c,s,w3); assign cout=w2 w3; endmodule </pre>
● Design code of 4 bit adder using Full Adder	● Testbench
<pre> `include "FA.v" module Adder_4_bits(a,b,c,s,cout); input [3:0]a,b; input c; output [3:0]s; output cout; wire [2:0]w; FA F1(a[0], b[0], c, s[0], w[0]); FA F2(a[1], b[1], w[0], s[1], w[1]); FA F3(a[2], b[2], w[1], s[2], w[2]); FA F4(a[3], b[3], w[2], s[3], cout); endmodule </pre>	<pre> module Adder_4_bits_tb; reg [3:0]a,b; reg c; wire [3:0]s; wire cout; Adder_4_bits b1(a,b,c,s,cout); initial begin a=4'b0101; b=4'b1111; c=1'b0; #100 a=4'b0100; b=4'b1010; c=1'b0; #100 a=4'b0101; b=4'b1110; c=1'b1; #100 a=4'b0110; b=4'b1111; c=1'b0; #100 a=4'b1100; b=4'b1011; c=1'b1; #100 a=4'b0111; b=4'b1000; c=1'b0; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b,c=%b, s=%b, cout=%b", \$time, a,b,c,s,cout); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,c,s,cout); end endmodule </pre>

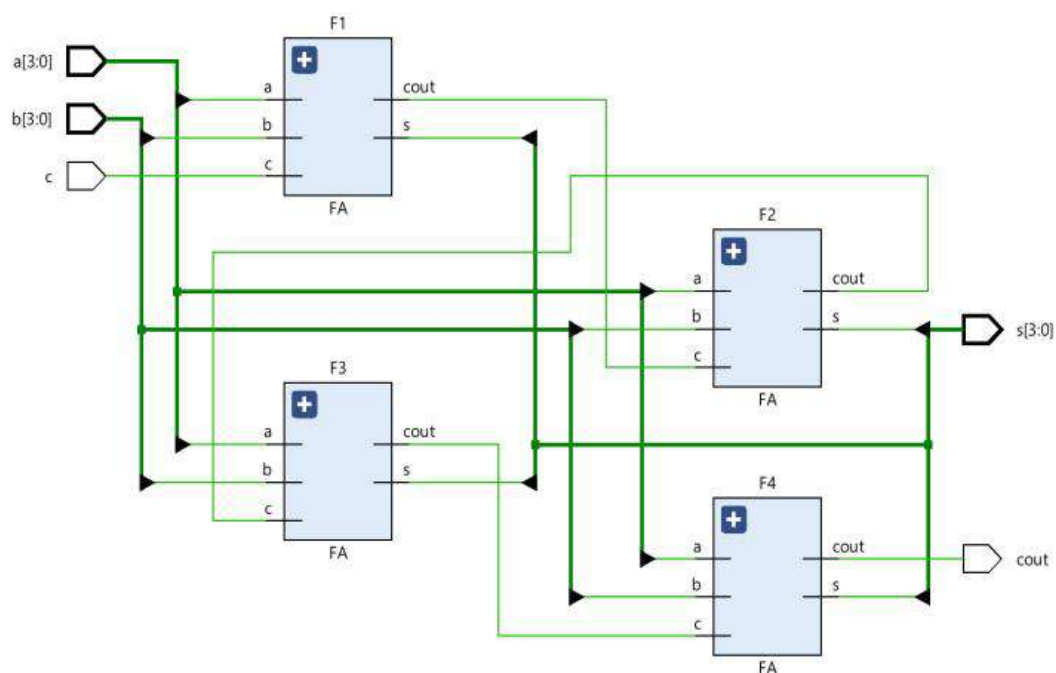
● Waveform



● Output

```
# run 1000ns
Simtime=0, a=0101, b=1111, c=0, s=0100, cout=1
Simtime=100000, a=0100, b=1010, c=0, s=1110, cout=0
Simtime=200000, a=0101, b=1110, c=1, s=0100, cout=1
Simtime=300000, a=0110, b=1111, c=0, s=0101, cout=1
Simtime=400000, a=1100, b=1011, c=1, s=1000, cout=1
Simtime=500000, a=0111, b=1000, c=0, s=1111, cout=0
```

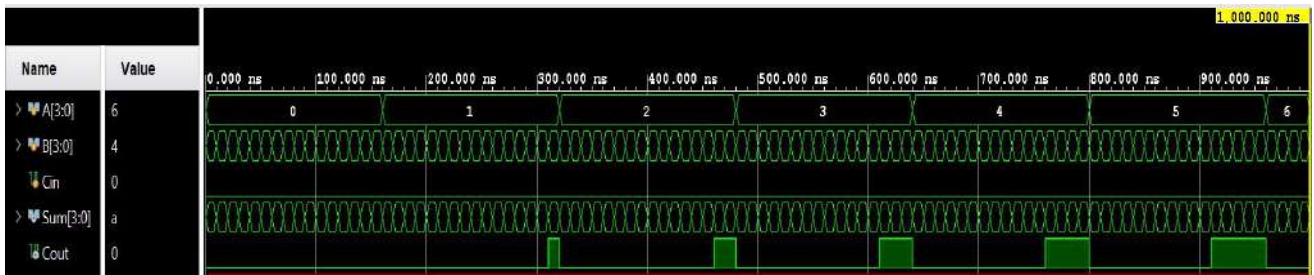
● Schematic



33. 4 bit Carry Look Ahead Adder

● Design Code	● Testbench Code
<pre> module carry_lookahead_adder(A,B,Cin,Sum,Cout); input [3:0] A, B; input Cin; output [3:0] Sum; output Cout; wire [3:0] G, P; // Generate and Propagate wire [4:0] C; // Carry assign C[0] = Cin; // Generate and Propagate assign G = A & B; // Generate assign P = A ^ B; // Propagate // Carry Look-Ahead Logic assign C[1] = G[0] (P[0] & C[0]); assign C[2] = G[1] (P[1] & C[1]); assign C[3] = G[2] (P[2] & C[2]); assign C[4] = G[3] (P[3] & C[3]); // Sum Bits assign Sum = P ^ C[3:0]; // Sum[i] = P[i] ⊕ C[i] // Final Carry-out assign Cout = C[4]; endmodule </pre>	<pre> module carry_lookahead_adder_tb; reg [3:0] A, B; reg Cin; wire [3:0] Sum; wire Cout; // Loop variable integer i; // DUT instantiation carry_lookahead_adder cla(A,B,Cin,Sum,Cout); // Apply all 512 combinations using for loop initial begin for (i = 0; i < 512; i = i + 1) begin {Cin, A, B} = i[8:0]; #10; end end // Monitor output on value change initial begin \$monitor("Time=%0t A=%b B=%b Cin=%b Sum=%b Cout=%b", \$time, A, B, Cin, Sum, Cout); end // Waveform dump initial begin \$dumpfile("cla_dump.vcd"); \$dumpvars(0, carry_lookahead_adder_tb); end endmodule </pre>

● Waveform



● Output

Time=0 | A=0000 B=0000 Cin=0 | Sum=0000 Cout=0

Time=10000 | A=0000 B=0001 Cin=0 | Sum=0001 Cout=0

Time=20000 | A=0000 B=0010 Cin=0 | Sum=0010 Cout=0

Time=30000 | A=0000 B=0011 Cin=0 | Sum=0011 Cout=0

Time=40000 | A=0000 B=0100 Cin=0 | Sum=0100 Cout=0

Time=50000 | A=0000 B=0101 Cin=0 | Sum=0101 Cout=0

Time=60000 | A=0000 B=0110 Cin=0 | Sum=0110 Cout=0

Time=70000 | A=0000 B=0111 Cin=0 | Sum=0111 Cout=0

Time=80000 | A=0000 B=1000 Cin=0 | Sum=1000 Cout=0

Time=90000 | A=0000 B=1001 Cin=0 | Sum=1001 Cout=0

Time=100000 | A=0000 B=1010 Cin=0 | Sum=1010 Cout=0

Time=110000 | A=0000 B=1011 Cin=0 | Sum=1011 Cout=0

Time=120000 | A=0000 B=1100 Cin=0 | Sum=1100 Cout=0

Time=130000 | A=0000 B=1101 Cin=0 | Sum=1101 Cout=0

Time=140000 | A=0000 B=1110 Cin=0 | Sum=1110 Cout=0

Time=150000 | A=0000 B=1111 Cin=0 | Sum=1111 Cout=0

Time=160000 | A=0001 B=0000 Cin=0 | Sum=0001 Cout=0

Time=170000 | A=0001 B=0001 Cin=0 | Sum=0010 Cout=0

Time=180000 | A=0001 B=0010 Cin=0 | Sum=0011 Cout=0

Time=190000 | A=0001 B=0011 Cin=0 | Sum=0100 Cout=0

Time=200000 | A=0001 B=0100 Cin=0 | Sum=0101 Cout=0

Time=210000 | A=0001 B=0101 Cin=0 | Sum=0110 Cout=0

Time=220000 | A=0001 B=0110 Cin=0 | Sum=0111 Cout=0

Time=230000 | A=0001 B=0111 Cin=0 | Sum=1000 Cout=0

Time=240000 | A=0001 B=1000 Cin=0 | Sum=1001 Cout=0

Time=250000 | A=0001 B=1001 Cin=0 | Sum=1010 Cout=0

Time=260000 | A=0001 B=1010 Cin=0 | Sum=1011 Cout=0

Time=270000 | A=0001 B=1011 Cin=0 | Sum=1100 Cout=0

Time=280000 | A=0001 B=1100 Cin=0 | Sum=1101 Cout=0

Time=290000 | A=0001 B=1101 Cin=0 | Sum=1110 Cout=0

Time=300000 | A=0001 B=1110 Cin=0 | Sum=1111 Cout=0

Time=310000 | A=0001 B=1111 Cin=0 | Sum=0000 Cout=1

Time=320000 | A=0010 B=0000 Cin=0 | Sum=0010 Cout=0

Time=330000 | A=0010 B=0001 Cin=0 | Sum=0011 Cout=0

Time=340000 | A=0010 B=0010 Cin=0 | Sum=0100 Cout=0

Time=350000 | A=0010 B=0011 Cin=0 | Sum=0101 Cout=0

Time=360000 | A=0010 B=0100 Cin=0 | Sum=0110 Cout=0

Time=370000 | A=0010 B=0101 Cin=0 | Sum=0111 Cout=0

Time=380000 | A=0010 B=0110 Cin=0 | Sum=1000 Cout=0

Time=390000 | A=0010 B=0111 Cin=0 | Sum=1001 Cout=0

Time=400000 | A=0010 B=1000 Cin=0 | Sum=1010 Cout=0

Time=410000 | A=0010 B=1001 Cin=0 | Sum=1011 Cout=0

Time=420000 | A=0010 B=1010 Cin=0 | Sum=1100 Cout=0

Time=430000 | A=0010 B=1011 Cin=0 | Sum=1101 Cout=0

Time=440000 | A=0010 B=1100 Cin=0 | Sum=1110 Cout=0

Time=450000 | A=0010 B=1101 Cin=0 | Sum=1111 Cout=0

Time=460000 | A=0010 B=1110 Cin=0 | Sum=0000 Cout=1

Time=470000 | A=0010 B=1111 Cin=0 | Sum=0001 Cout=1

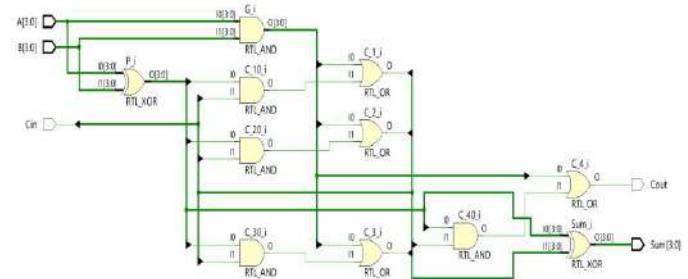
Time=480000 | A=0011 B=0000 Cin=0 | Sum=0011 Cout=0

Time=490000 | A=0011 B=0001 Cin=0 | Sum=0100 Cout=0

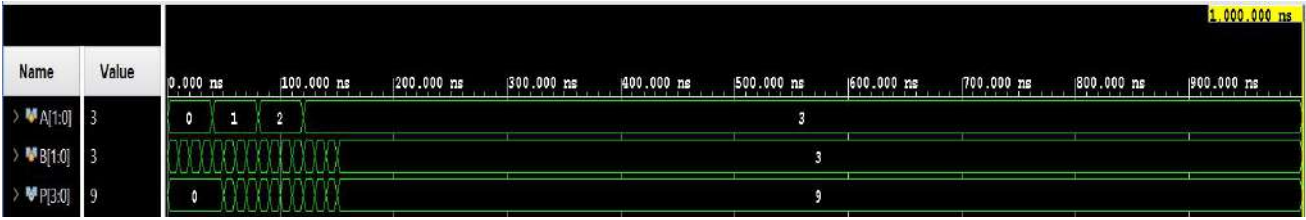
Time=500000 | A=0011 B=0010 Cin=0 | Sum=0101 Cout=0

Time=510000 | A=0011 B=0011 Cin=0 | Sum=0110 Cout=0

● Schematic



34. 2 bit Multiplier

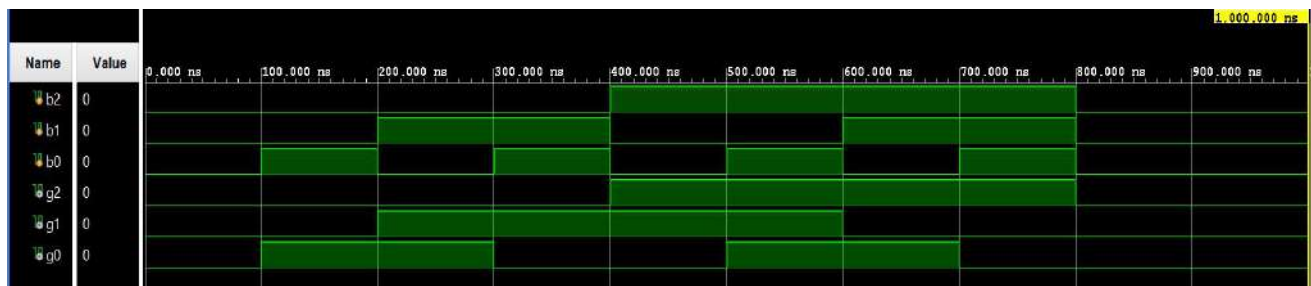
● Design Code	● Testbench Code								
<pre> module multiplier(A,B,P); input [1:0] A, B; output [3:0] P; wire a0b0, a1b0, a0b1, a1b1; wire s1, c1, s2, c2; // Partial products assign a0b0 = A[0] & B[0]; // LSB assign a1b0 = A[1] & B[0]; assign a0b1 = A[0] & B[1]; assign a1b1 = A[1] & B[1]; // First bit of product assign P[0] = a0b0; // Second bit: a1b0 ^ a0b1 assign s1 = a1b0 ^ a0b1; assign c1 = a1b0 & a0b1; assign P[1] = s1; // Third bit: a1b1 ^ c1 assign s2 = a1b1 ^ c1; assign c2 = a1b1 & c1; assign P[2] = s2; // Fourth bit (MSB): carry out from previous assign P[3] = c2; endmodule </pre>	<pre> module multiplier_tb; reg [1:0] A, B; wire [3:0] P; // Instantiate the multiplier module multiplier uut (A,B,P); // Loop variable integer i; // Apply all combinations of inputs (4 × 4 = 16) initial begin for (i = 0; i < 16; i = i + 1) begin {A, B} = i[3:0]; // Assign 2-bit A and B #10; end end // Display simulation output initial begin \$monitor("Time=%0t A=%b B=%b Product=%b (%0d)", \$time, A, B, P, P); end // VCD waveform generation initial begin \$dumpfile("multiplier.vcd"); \$dumpvars(0, multiplier_tb); end endmodule </pre>								
● Waveform									
 <table border="1"> <thead> <tr> <th>Name</th> <th>Value</th> </tr> </thead> <tbody> <tr> <td>A[1:0]</td> <td>3</td> </tr> <tr> <td>B[1:0]</td> <td>3</td> </tr> <tr> <td>P[3:0]</td> <td>9</td> </tr> </tbody> </table>		Name	Value	A[1:0]	3	B[1:0]	3	P[3:0]	9
Name	Value								
A[1:0]	3								
B[1:0]	3								
P[3:0]	9								

Time=0	A=00 B=00	Product=0000 (0)
Time=10000	A=00 B=01	Product=0000 (0)
Time=20000	A=00 B=10	Product=0000 (0)
Time=30000	A=00 B=11	Product=0000 (0)
Time=40000	A=01 B=00	Product=0000 (0)
Time=50000	A=01 B=01	Product=0001 (1)
Time=60000	A=01 B=10	Product=0010 (2)
Time=70000	A=01 B=11	Product=0011 (3)
Time=80000	A=10 B=00	Product=0000 (0)
Time=90000	A=10 B=01	Product=0010 (2)
Time=100000	A=10 B=10	Product=0100 (4)
Time=110000	A=10 B=11	Product=0110 (6)
Time=120000	A=11 B=00	Product=0000 (0)
Time=130000	A=11 B=01	Product=0011 (3)
Time=140000	A=11 B=10	Product=0110 (6)
Time=150000	A=11 B=11	Product=1001 (9)

35. Binary to Gray Code converter

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module b2g_converter(b2,b1,b0,g2,g1,g0); input b0,b1,b2; output g2,g1,g0; assign g2=b2; assign g1=b2^b1; assign g0=b1^b0; endmodule //Gate-level Modeling Style module b2g_converter_gate(b2,b1,b0,g2,g1,g0); input b2,b1,b0; output g2,g1,g0; buf b(g2,b2); xor x1(g1,b2,b1); xor x2(g0,b1,b0); endmodule</pre>	<pre>module b2g_converter_tb; reg b2,b1,b0; wire g2,g1,g0; b2g_converter d(b2,b1,b0,g2,g1,g0); initial begin b2=1'b0; b1=1'b0; b0=1'b0; #100 b2=1'b0; b1=1'b0; b0=1'b1; #100 b2=1'b0; b1=1'b1; b0=1'b0; #100 b2=1'b0; b1=1'b1; b0=1'b1; #100 b2=1'b1; b1=1'b0; b0=1'b0; #100 b2=1'b1; b1=1'b0; b0=1'b1; #100 b2=1'b1; b1=1'b1; b0=1'b0; #100 b2=1'b1; b1=1'b1; b0=1'b1; #100 b2=1'b0; b1=1'b0; b0=1'b0; end initial begin \$monitor("Simtime=%0t, b2=%b, b1=%b, b0=%b, g2=%b, g1=%b g0=%b", \$time, b2,b1,b0,g2,g1,g0); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,b2,b1,b0,g2,g1,g0); end endmodule</pre>

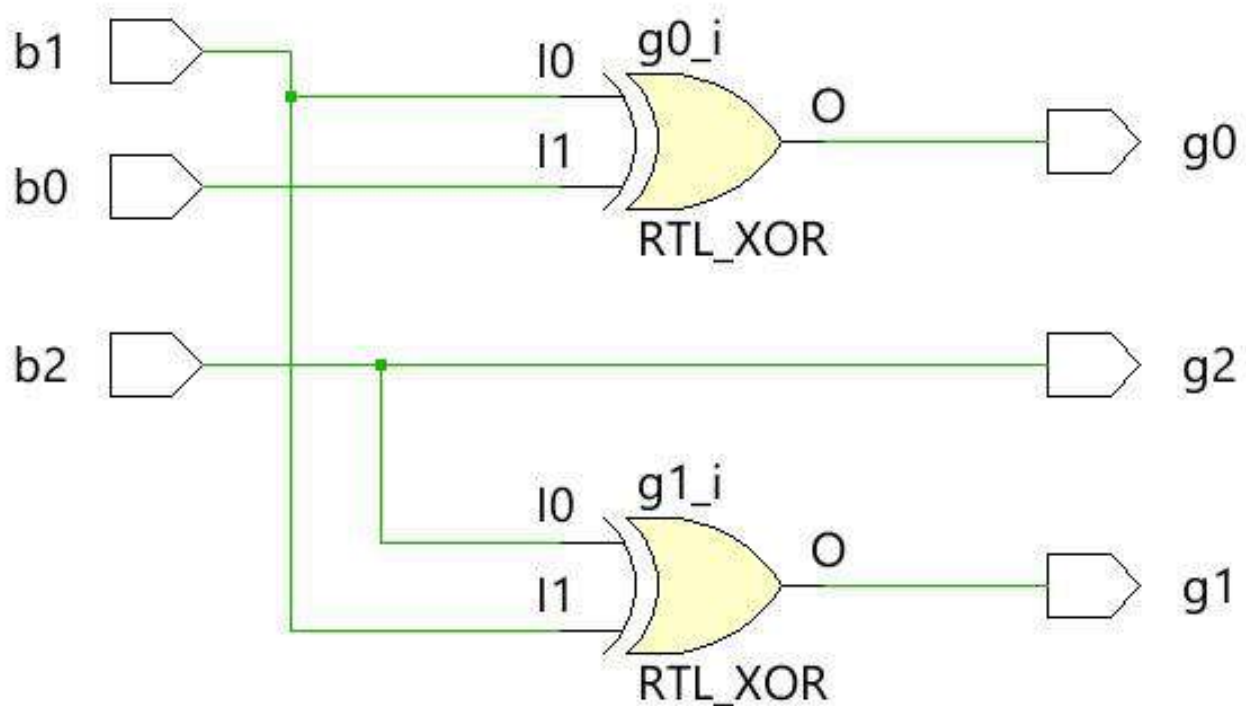
● Waveform



- Output

```
Simtime=0, b2=0, b1=0, b0=0, g2=0, g1=0 g0=0
Simtime=100000, b2=0, b1=0, b0=1, g2=0, g1=0 g0=1
Simtime=200000, b2=0, b1=1, b0=0, g2=0, g1=1 g0=1
Simtime=300000, b2=0, b1=1, b0=1, g2=0, g1=1 g0=0
Simtime=400000, b2=1, b1=0, b0=0, g2=1, g1=1 g0=0
Simtime=500000, b2=1, b1=0, b0=1, g2=1, g1=1 g0=1
Simtime=600000, b2=1, b1=1, b0=0, g2=1, g1=0 g0=1
Simtime=700000, b2=1, b1=1, b0=1, g2=1, g1=0 g0=0
Simtime=800000, b2=0, b1=0, b0=0, g2=0, g1=0 g0=0
```

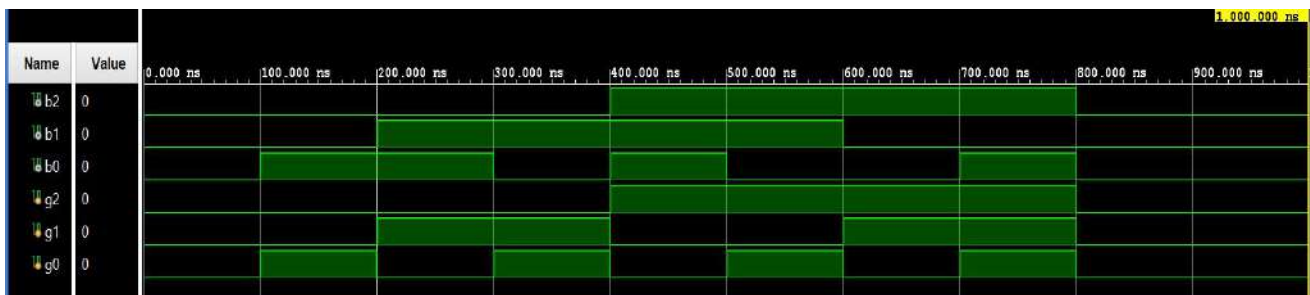
- Schematic



36. Gray to Binary Code converter

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module g2b_converter(g2,g1,g0,b2,b1,b0); input g2,g1,g0; output b2,b1,b0; assign b2=g2; assign b1=g2^g1; assign b0=g2^g1^g0; endmodule //Gate-level Modeling Style module g2b_converter_gate(g2,g1,g0,b2,b1,b0); input g2,g1,g0; output b2,b1,b0; buf b(b2,g2); xor x1(b1,g2,g1); xor x2(b0,g2,g1,g0); endmodule</pre>	<pre>module g2b_converter_tb; wire b2,b1,b0; reg g2,g1,g0; g2b_converter d(g2,g1,g0,b2,b1,b0); initial begin g2=1'b0; g1=1'b0; g0=1'b0; #100 g2=1'b0; g1=1'b0; g0=1'b1; #100 g2=1'b0; g1=1'b1; g0=1'b0; #100 g2=1'b0; g1=1'b1; g0=1'b1; #100 g2=1'b1; g1=1'b0; g0=1'b0; #100 g2=1'b1; g1=1'b0; g0=1'b1; #100 g2=1'b1; g1=1'b1; g0=1'b0; #100 g2=1'b1; g1=1'b1; g0=1'b1; #100 g2=1'b0; g1=1'b0; g0=1'b0; end initial begin \$monitor("Simtime=%0t, g2=%b, g1=%b, g0=%b, b2=%b, b1=%b b0=%b", \$time, g2,g1,g0,b2,b1,b0); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,g2,g1,g0,b2,b1,b0); end endmodule</pre>

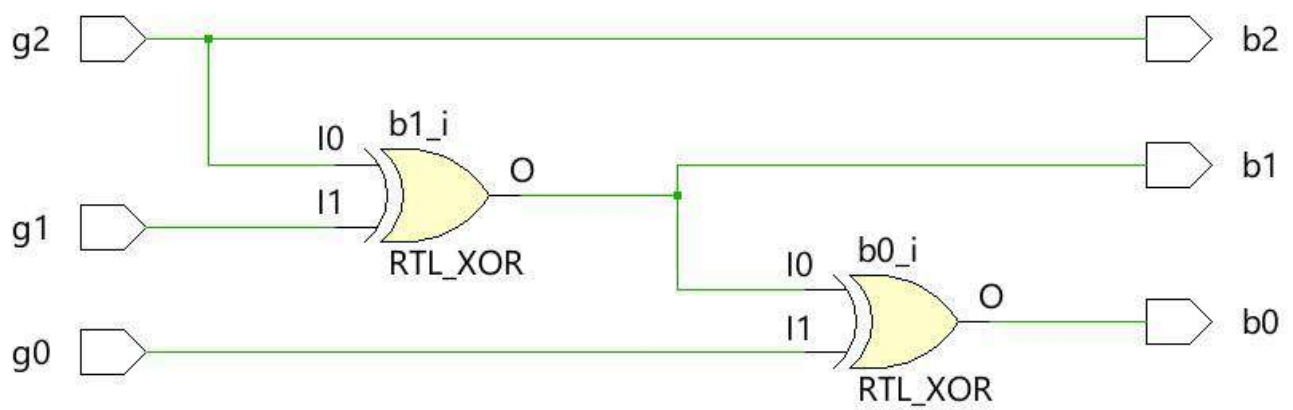
● Waveform



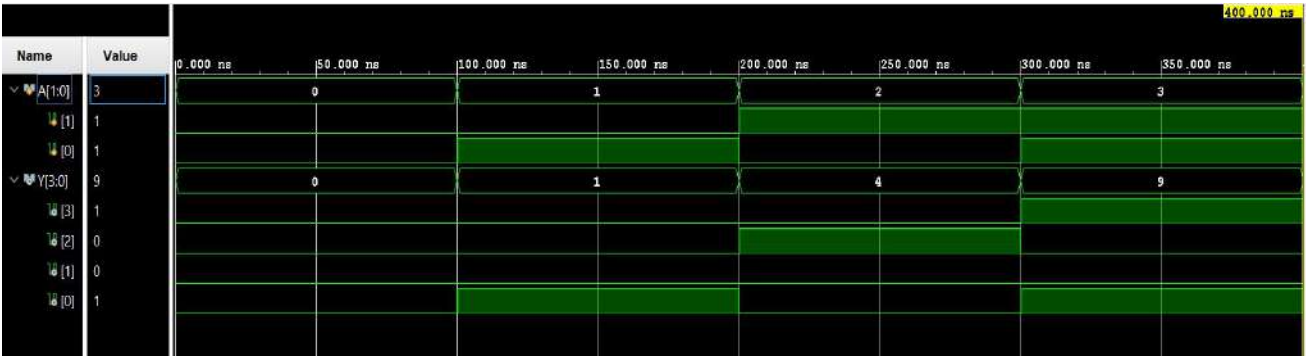
● Output

```
Simtime=0, g2=0, g1=0, g0=0, b2=0, b1=0 b0=0  
Simtime=100000, g2=0, g1=0, g0=1, b2=0, b1=0 b0=1  
Simtime=200000, g2=0, g1=1, g0=0, b2=0, b1=1 b0=1  
Simtime=300000, g2=0, g1=1, g0=1, b2=0, b1=1 b0=0  
Simtime=400000, g2=1, g1=0, g0=0, b2=1, b1=1 b0=1  
Simtime=500000, g2=1, g1=0, g0=1, b2=1, b1=1 b0=0  
Simtime=600000, g2=1, g1=1, g0=0, b2=1, b1=0 b0=0  
Simtime=700000, g2=1, g1=1, g0=1, b2=1, b1=0 b0=1  
Simtime=800000, g2=0, g1=0, g0=0, b2=0, b1=0 b0=0
```

● Schematic



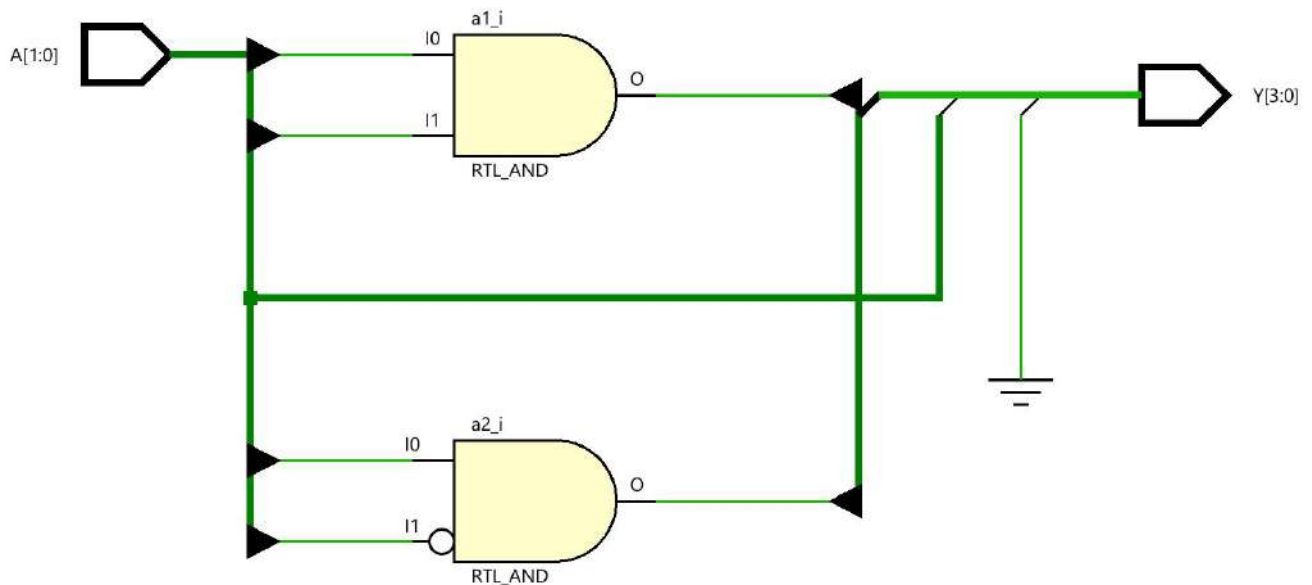
37. Square Number Generator (2 bits)

● Design Code	● Testbench Code
<pre>//Gate-level Modeling Style module square_generator(A,Y); input [1:0]A; output [3:0]Y; wire w; not n1(w,A[0]); buf b1(Y[0],A[0]); buf b2(Y[1],0); and a1(Y[3],A[1],A[0]); and a2(Y[2],A[1],w); endmodule //Behavioural Modeling Style module n_to_n2_generator (n, n2); input [1:0] n; output reg [3:0] n2; always @(n) begin if (n == 2'd0) n2 = 4'd0; else if (n == 2'd1) n2 = 4'd1; else if (n == 2'd2) n2 = 4'd4; else (n == 2'd3) n2 = 4'd9; end endmodule</pre>	<pre>module square_generator_tb; reg [1:0] A; wire [3:0] Y; square_generator uut (A,Y); initial begin \$monitor("Time = %0t A = %b (%d) Y = %b (%0d)", \$time, A, A, Y, Y); A = 2'b00; #100 A = 2'b01; #100 A = 2'b10; #100 A = 2'b11; #100; \$finish; initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,A,Y); end endmodule</pre>
● Waveform	
 The waveform shows the input A [1:0] and output Y [3:0] over time. The input A changes from 00 to 01 to 10 to 11 at 50ns intervals. The output Y changes from 0000 to 0001 to 0100 to 1001 at the same intervals. The diagram is titled '400.000 ns'.	

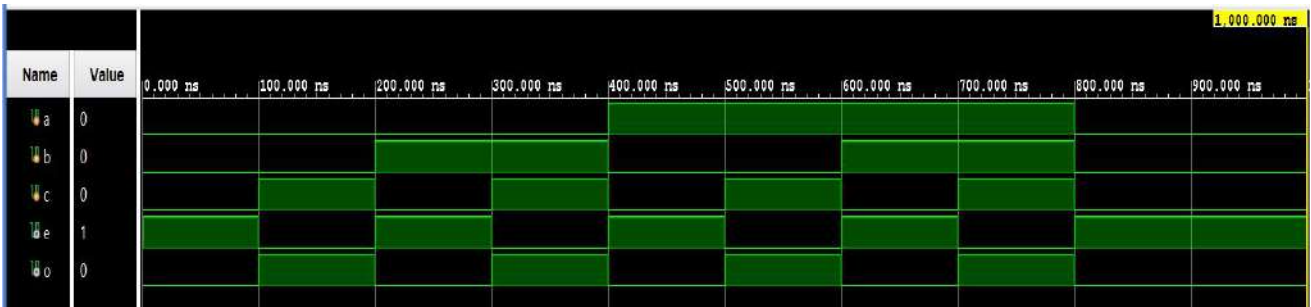
- **Output**

Time = 0 | A = 00 (0) | Y = 0000 (0)
Time = 100000 | A = 01 (1) | Y = 0001 (1)
Time = 200000 | A = 10 (2) | Y = 0100 (4)
Time = 300000 | A = 11 (3) | Y = 1001 (9)

- **Schematic**



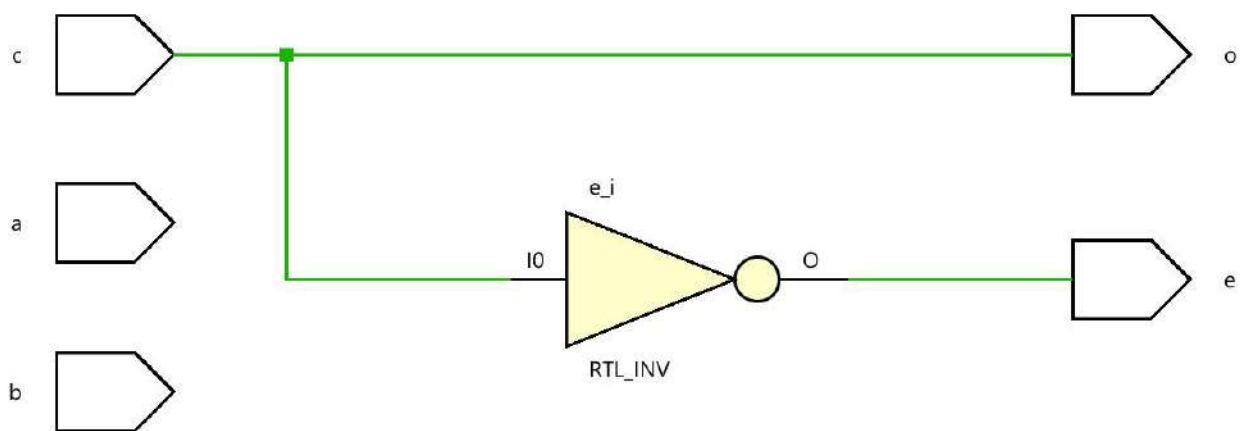
38. Even and Odd number Detector

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module even_odd_detector_data(a,b,c,e,o); input a,b,c; output e,o; assign e=~c; assign o=c; endmodule //Gate-level Modeling Style module even_odd_detector_gate(a,b,c,e,o); input a,b,c; output e,o; not n1(e,c); buf b1(o,c); endmodule</pre>	<pre>module even_odd_detector_data_tb; reg a,b,c; wire e,o; even_odd_detector_data eo_d1(a,b,c,e,o); initial begin a=1'b0; b=1'b0; c=1'b0; #100 a=1'b0; b=1'b0; c=1'b1; #100 a=1'b0; b=1'b1; c=1'b0; #100 a=1'b0; b=1'b1; c=1'b1; #100 a=1'b1; b=1'b0; c=1'b0; #100 a=1'b1; b=1'b0; c=1'b1; #100 a=1'b1; b=1'b1; c=1'b0; #100 a=1'b1; b=1'b1; c=1'b1; #100 a=1'b0; b=1'b0; c=1'b0; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, c=%b, e=%b, o=%b", \$time, a,b,c,e,o); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,c,e,o); end endmodule</pre>
● Waveform	
	

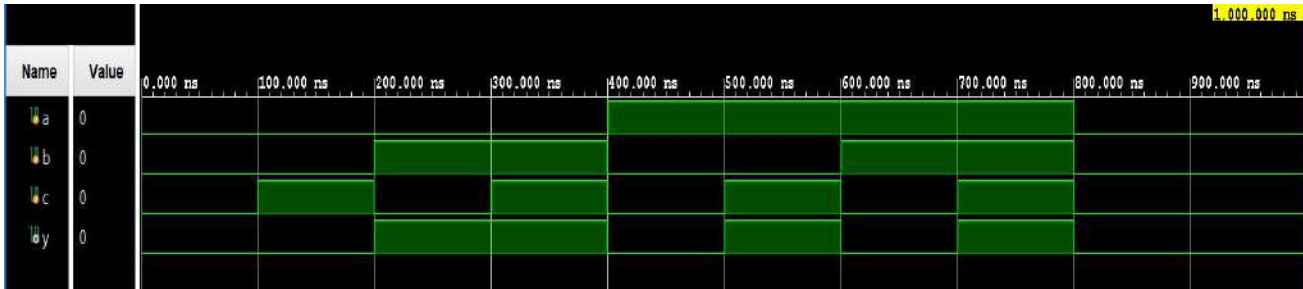
● Output

```
Simtime=0, a=0, b=0, c=0, e=1, o=0
Simtime=100000, a=0, b=0, c=1, e=0, o=1
Simtime=200000, a=0, b=1, c=0, e=1, o=0
Simtime=300000, a=0, b=1, c=1, e=0, o=1
Simtime=400000, a=1, b=0, c=0, e=1, o=0
Simtime=500000, a=1, b=0, c=1, e=0, o=1
Simtime=600000, a=1, b=1, c=0, e=1, o=0
Simtime=700000, a=1, b=1, c=1, e=0, o=1
Simtime=800000, a=0, b=0, c=0, e=1, o=0
```

● Schematic



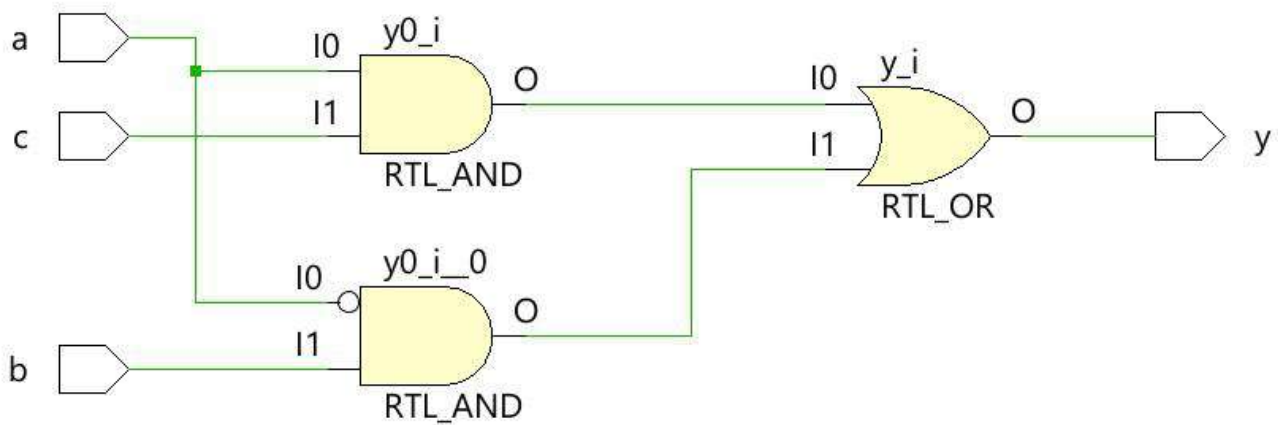
39. Prime Number Detector

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module prime_number_detector(a,b,c,y); input a,b,c; output y; assign y=(a&c) ((~a)&b); endmodule //Gate-level Modeling Style module prime_number_detector_gate(a,b,c,y); input a,b,c; output y; wire w1,w2,w3; and a1(w2,a,c); not n1(w1,a); and a2(w3,w1,b); or o1(y,w2,w3); endmodule</pre>	<pre>module prime_number_detector_tb; reg a,b,c; wire y; prime_number_detector d1(a,b,c,y); initial begin a=1'b0; b=1'b0; c=1'b0; #100 a=1'b0; b=1'b0; c=1'b1; #100 a=1'b0; b=1'b1; c=1'b0; #100 a=1'b0; b=1'b1; c=1'b1; #100 a=1'b1; b=1'b0; c=1'b0; #100 a=1'b1; b=1'b0; c=1'b1; #100 a=1'b1; b=1'b1; c=1'b0; #100 a=1'b1; b=1'b1; c=1'b1; #100 a=1'b0; b=1'b0; c=1'b0; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, c=%b, y=%b", \$time, a,b,c,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,c,y); end endmodule</pre>
● Waveform	
	

● Output

```
Simtime=0, a=0, b=0, c=0, y=0
Simtime=100000, a=0, b=0, c=1, y=0
Simtime=200000, a=0, b=1, c=0, y=1
Simtime=300000, a=0, b=1, c=1, y=1
Simtime=400000, a=1, b=0, c=0, y=0
Simtime=500000, a=1, b=0, c=1, y=1
Simtime=600000, a=1, b=1, c=0, y=0
Simtime=700000, a=1, b=1, c=1, y=1
Simtime=800000, a=0, b=0, c=0, y=0
```

● Schematic



40. Palindrome Number Detector

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module palindrome_number_detector(a,b,c,y); input a,b,c; output y; assign y=~(a^c); endmodule //Gate-level Modeling Style module palindrome_number_detector_gate(a,b,c,y); input a,b,c; output y; xnor x1(y,a,c); endmodule</pre>	<pre>module palindrome_number_detector_tb; reg a,b,c; wire y; palindrome_number_detector d1(a,b,c,y); initial begin a=1'b0; b=1'b0; c=1'b0; #100 a=1'b0; b=1'b0; c=1'b1; #100 a=1'b0; b=1'b1; c=1'b0; #100 a=1'b0; b=1'b1; c=1'b1; #100 a=1'b1; b=1'b0; c=1'b0; #100 a=1'b1; b=1'b0; c=1'b1; #100 a=1'b1; b=1'b1; c=1'b0; #100 a=1'b1; b=1'b1; c=1'b1; #100 a=1'b0; b=1'b0; c=1'b0; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, c=%b, y=%b", \$time, a,b,c,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,c,y); end endmodule</pre>

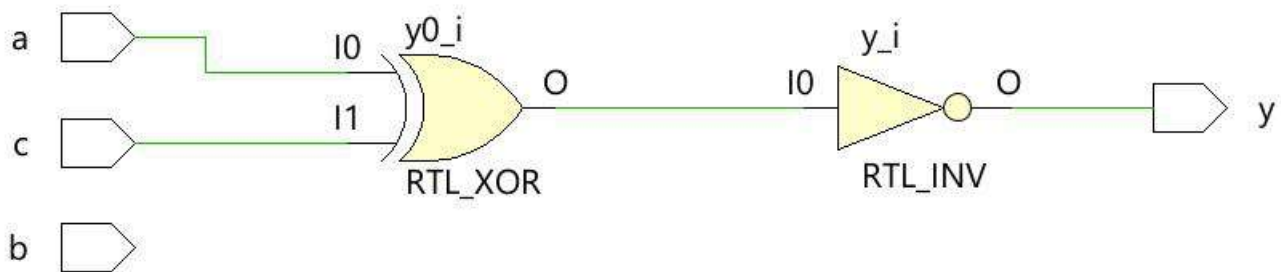
● Waveform



● Output

```
Simtime=0, a=0, b=0, c=0, y=1
Simtime=100000, a=0, b=0, c=1, y=0
Simtime=200000, a=0, b=1, c=0, y=1
Simtime=300000, a=0, b=1, c=1, y=0
Simtime=400000, a=1, b=0, c=0, y=0
Simtime=500000, a=1, b=0, c=1, y=1
Simtime=600000, a=1, b=1, c=0, y=0
Simtime=700000, a=1, b=1, c=1, y=1
Simtime=800000, a=0, b=0, c=0, y=1
```

● Schematic



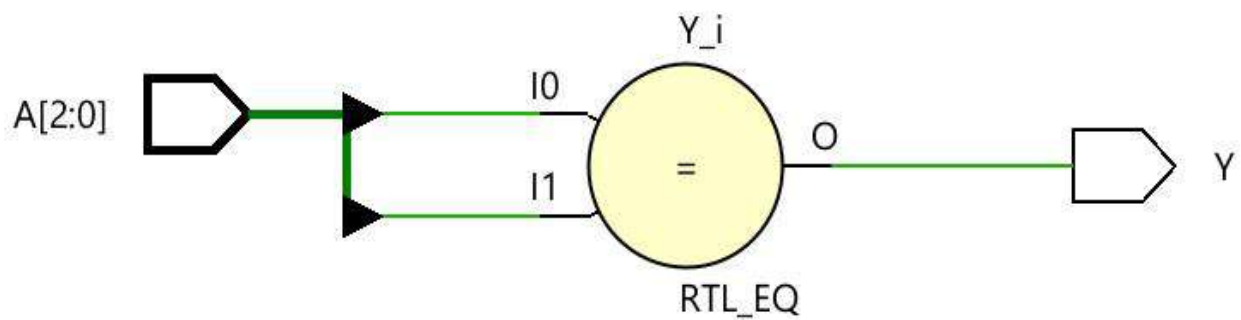
41. Palindrome number using ternary operator

● Design Code	● Testbench Code
<pre> module palindrome_ternary(A,Y); input [2:0]A; output Y; assign Y=(A[0]==A[2])?1'b1:1'b0; endmodule </pre>	<pre> module palindrome_ternary_tb; reg [2:0]A; wire Y; palindrome_ternary dut(.A(A),.Y(Y));//positional port mapping initial begin A=3'b000; #100 A=3'b001; #100 A=3'b010; #100 A=3'b011; #100 A=3'b100; #100 A=3'b101; #100 A=3'b110; #100 A=3'b111; #100 A=3'b000; end initial begin \$monitor("Simtime=%0t,A=%b,Y=%b", \$time,A,Y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,A,Y); end endmodule </pre>
● Waveform	

- **Output**

```
Simtime=0,A=000,Y=1
Simtime=100000,A=001,Y=0
Simtime=200000,A=010,Y=1
Simtime=300000,A=011,Y=0
Simtime=400000,A=100,Y=0
Simtime=500000,A=101,Y=1
Simtime=600000,A=110,Y=0
Simtime=700000,A=111,Y=1
Simtime=800000,A=000,Y=1
```

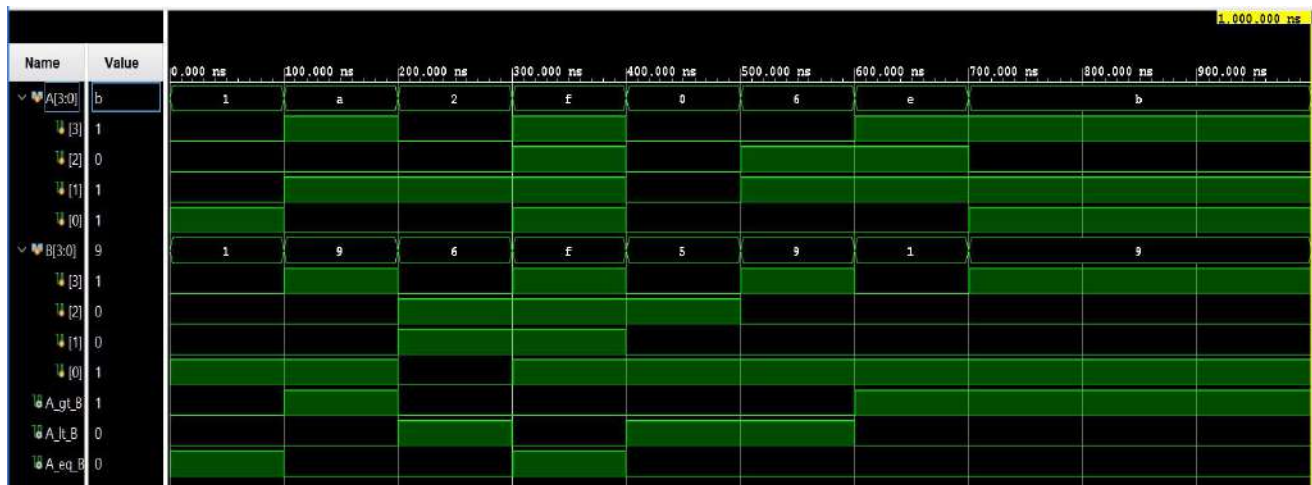
- **Schematic**



42. Magnitude Comparator

● Design Code	● Testbench Code
<pre> module magnitude_comparator(A,B,A_gt_B,A_lt_B,A_eq_B); input [3:0] A; input [3:0] B; output A_gt_B; //A>B output A_lt_B; //A<B output A_eq_B; //A=B wire [3:0] x; assign x[0] = ~(A[0] ^ B[0]); assign x[1] = ~(A[1] ^ B[1]); assign x[2] = ~(A[2] ^ B[2]); assign x[3] = ~(A[3] ^ B[3]); assign A_gt_B = (A[3] & ~B[3]) (x[3] & A[2] & ~B[2]) (x[3] & x[2] & A[1] & ~B[1]) (x[3] & x[2] & x[1] & A[0] & ~B[0]); assign A_lt_B = (~A[3] & B[3]) (x[3] & ~A[2] & B[2]) (x[3] & x[2] & ~A[1] & B[1]) (x[3] & x[2] & x[1] & ~A[0] & B[0]); assign A_eq_B = x[3] & x[2] & x[1] & x[0]; endmodule </pre>	<pre> module magnitude_comparator_tb; reg [3:0] A, B; wire A_gt_B, A_lt_B, A_eq_B; magnitude_comparator dut(A,B,A_gt_B,A_lt_B,A_eq_B); initial begin A = 4'b0001; B = 4'b0001; #100 A = 4'b1010; B = 4'b1001; #100 A = 4'b0010; B = 4'b0110; #100 A = 4'b1111; B = 4'b1111; #100 A = 4'b0000; B = 4'b0101; #100 A = 4'b0110; B = 4'b1001; #100 A = 4'b1110; B = 4'b0001; #100 A = 4'b1011; B = 4'b1001; end initial begin \$monitor("Simtime=%0t, A=%b, B=%b, A_gt_B=%b, A_lt_B=%b, A_eq_B=%b", \$time, A, B, A_gt_B, A_lt_B, A_eq_B); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0, A, B, A_gt_B, A_lt_B, A_eq_B); end endmodule </pre>

● Waveform



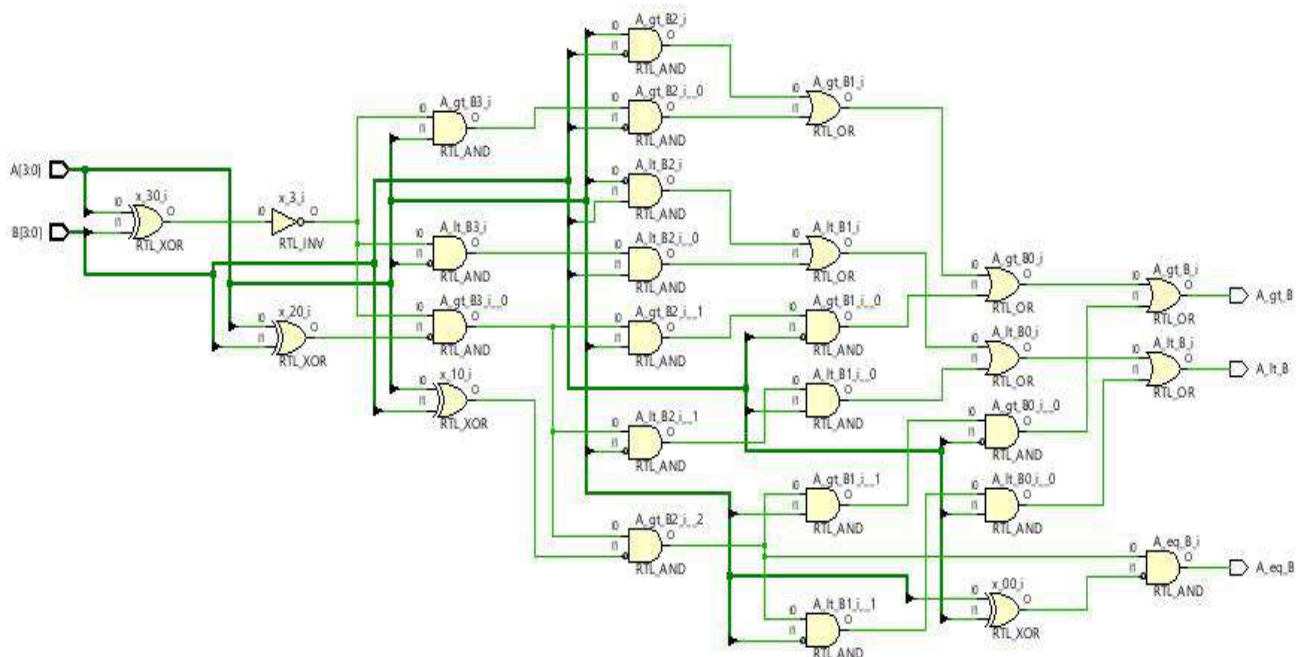
● Output

```

Simtime=0, A=0001, B=0001, A_gt_B=0, A_lt_B=0, A_eq_B=1
Simtime=100000, A=1010, B=1001, A_gt_B=1, A_lt_B=0, A_eq_B=0
Simtime=200000, A=0010, B=0110, A_gt_B=0, A_lt_B=1, A_eq_B=0
Simtime=300000, A=1111, B=1111, A_gt_B=0, A_lt_B=0, A_eq_B=1
Simtime=400000, A=0000, B=0101, A_gt_B=0, A_lt_B=1, A_eq_B=0
Simtime=500000, A=0110, B=1001, A_gt_B=0, A_lt_B=1, A_eq_B=0
Simtime=600000, A=1110, B=0001, A_gt_B=1, A_lt_B=0, A_eq_B=0
Simtime=700000, A=1011, B=1001, A_gt_B=1, A_lt_B=0, A_eq_B=0

```

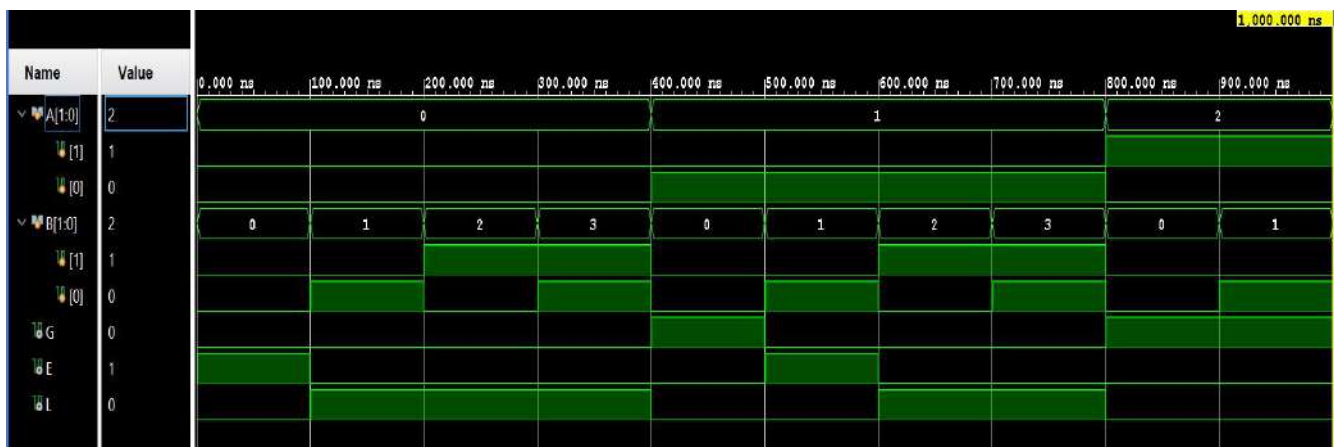
● Schematic



43. Magnitude Comparator using ternary operator

● Design Code	● Testbench Code
<pre> module magnitude_comparator_ternary(A,B,G,E,L); input [1:0]A,B; output G,E,L; assign G=(A>B)?1'b1:1'b0; assign E=(A==B)?1'b1:1'b0; assign L=(A<B)?1'b1:1'b0; endmodule </pre>	<pre> module magnitude_comparator_ternary_tb; reg [1:0]A,B; wire G,E,L; magnitude_comparator_ternary dut(.A(A),.B(B),.G(G),.E(E),.L(L)); initial begin {A,B}=4'b0000; #100 {A,B}=4'b0001; #100 {A,B}=4'b0010; #100 {A,B}=4'b0011; #100 {A,B}=4'b0100; #100 {A,B}=4'b0101; #100 {A,B}=4'b0110; #100 {A,B}=4'b0111; #100 {A,B}=4'b1000; #100 {A,B}=4'b1001; #100 {A,B}=4'b1010; #100 {A,B}=4'b1011; #100 {A,B}=4'b1100; #100 {A,B}=4'b1101; #100 {A,B}=4'b1110; #100 {A,B}=4'b1111; end initial begin \$monitor("Simtime=%0t,A=%b,B=%b,G=%b,E=%b,L =%b", \$time,A,B,G,E,L); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,A,B,G,E,L); end endmodule </pre>

● Waveform



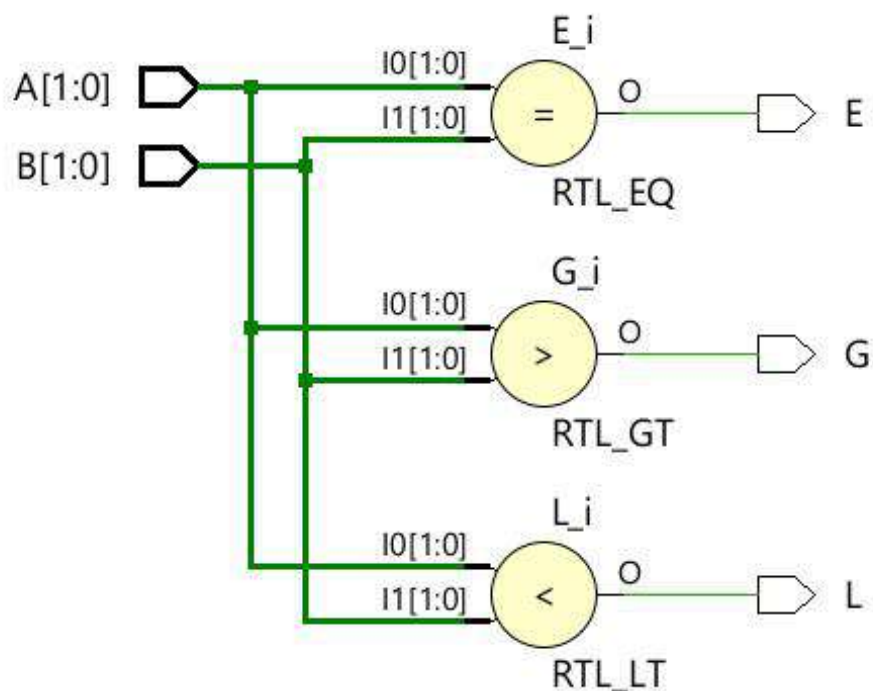
● Output

```

Simtime=0,A=00,B=00,G=0,E=1,L=0
Simtime=100000,A=00,B=01,G=0,E=0,L=1
Simtime=200000,A=00,B=10,G=0,E=0,L=1
Simtime=300000,A=00,B=11,G=0,E=0,L=1
Simtime=400000,A=01,B=00,G=1,E=0,L=0
Simtime=500000,A=01,B=01,G=0,E=1,L=0
Simtime=600000,A=01,B=10,G=0,E=0,L=1
Simtime=700000,A=01,B=11,G=0,E=0,L=1
Simtime=800000,A=10,B=00,G=1,E=0,L=0
Simtime=900000,A=10,B=01,G=1,E=0,L=0
Simtime=1000000,A=10,B=10,G=0,E=1,L=0

```

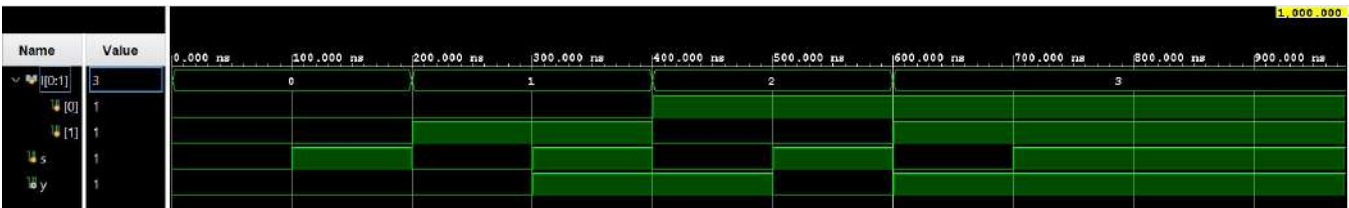
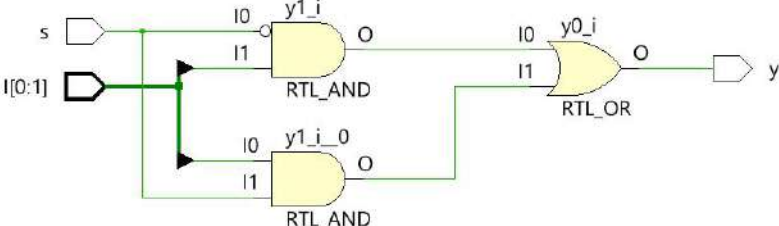
● Schematic



44. 2x1 MUX

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module mux_2_1(a,b,s,y); input a,b,s; output y; assign y=((~s)&a) (s&b); endmodule //Gate-level Modeling Style module mux_2_1(a,b,s,y); input a,b,s; output y; wire w1,w2,w3; not n1(w1,s); and a1(w2,w1,s); and a2(w3,s,b); or o1(y,w2,w3); endmodule</pre>	<pre>module mux_2_1_tb; reg a,b,s; wire y; mux_2_1 m1(a,b,s,y); initial begin a=1'b0; b=1'b0; s=1'b0; #100 a=1'b0; b=1'b0; s=1'b1; #100 a=1'b0; b=1'b1; s=1'b0; #100 a=1'b0; b=1'b1; s=1'b1; #100 a=1'b1; b=1'b0; s=1'b0; #100 a=1'b1; b=1'b0; s=1'b1; #100 a=1'b1; b=1'b1; s=1'b0; #100 a=1'b1; b=1'b1; s=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, s=%b, y=%b", \$time, a,b,s,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,s,y); end endmodule</pre>
● Waveform	
	
● OUTPUT	● Schematic <pre>Simtime=0, a=0, b=0, s=0, y=0 Simtime=100000, a=0, b=0, s=1, y=0 Simtime=200000, a=0, b=1, s=0, y=0 Simtime=300000, a=0, b=1, s=1, y=1 Simtime=400000, a=1, b=0, s=0, y=1 Simtime=500000, a=1, b=0, s=1, y=0 Simtime=600000, a=1, b=1, s=0, y=1 Simtime=700000, a=1, b=1, s=1, y=1</pre>

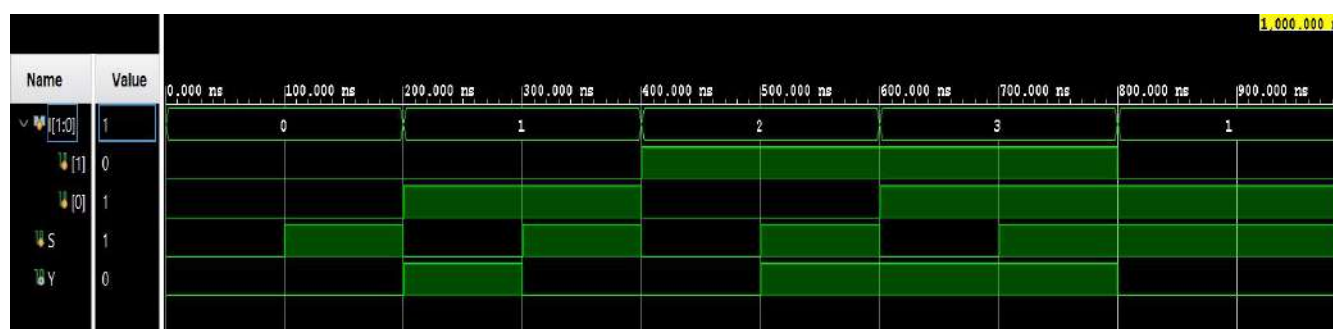
45. 2x1 MUX (Behavioral Modeling Style)

● Design Code	● Testbench Code
<pre>//Behavioral Modeling Style module mux_2_1_beh(I,s,y); input [0:1]I; input s; output reg y; always@(I,s,y) begin y=((~s)&I[0]) (I[1]&s); end endmodule</pre>	<pre>module mux_2_1_beh_tb; reg [0:1]I; reg s; wire y; mux_2_1_beh m1(I,s,y); initial begin I[0]=1'b0; I[1]=1'b0; s=1'b0; #100 I[0]=1'b0; I[1]=1'b0; s=1'b1; #100 I[0]=1'b0; I[1]=1'b1; s=1'b0; #100 I[0]=1'b0; I[1]=1'b1; s=1'b1; #100 I[0]=1'b1; I[1]=1'b0; s=1'b0; #100 I[0]=1'b1; I[1]=1'b0; s=1'b1; #100 I[0]=1'b1; I[1]=1'b1; s=1'b0; #100 I[0]=1'b1; I[1]=1'b1; s=1'b1; end initial begin \$monitor("Simtime=%0t, I[0]=%b, I[1]=%b, s=%b, y=%b", \$time, I[0],I[1],s,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,I[0],I[1],s,y); end endmodule</pre>
● Waveform	
	
● OUTPUT	● Schematic
<pre>Simtime=0, I[0]=0, I[1]=0, s=0, y=0 Simtime=100000, I[0]=0, I[1]=0, s=1, y=0 Simtime=200000, I[0]=0, I[1]=1, s=0, y=0 Simtime=300000, I[0]=0, I[1]=1, s=1, y=1 Simtime=400000, I[0]=1, I[1]=0, s=0, y=1 Simtime=500000, I[0]=1, I[1]=0, s=1, y=0 Simtime=600000, I[0]=1, I[1]=1, s=0, y=1 Simtime=700000, I[0]=1, I[1]=1, s=1, y=1</pre>	

46. MUX 2x1 using ternary operator

● Design Code	● Testbench Code
<pre> module mux_2_1_ternary(I,S,Y); input [1:0]I; input S; output Y; assign Y=(S)?I[1]:I[0]; endmodule </pre>	<pre> module mux_2_1_ternary_tb; reg [1:0]I; reg S; wire Y; mux_2_1_ternary dut(.I(I),.S(S),.Y(Y)); initial begin {I,S}=3'b000; #100 {I,S}=3'b001; #100 {I,S}=3'b010; #100 {I,S}=3'b011; #100 {I,S}=3'b100; #100 {I,S}=3'b101; #100 {I,S}=3'b110; #100 {I,S}=3'b111; #100 {I,S}=3'b011; end initial begin \$monitor("Simtime=%0t,I[0]=%b,I[1]=%b,S=%b,Y=%b", \$time,I[0],I[1],S,Y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,I,S,Y); end endmodule </pre>

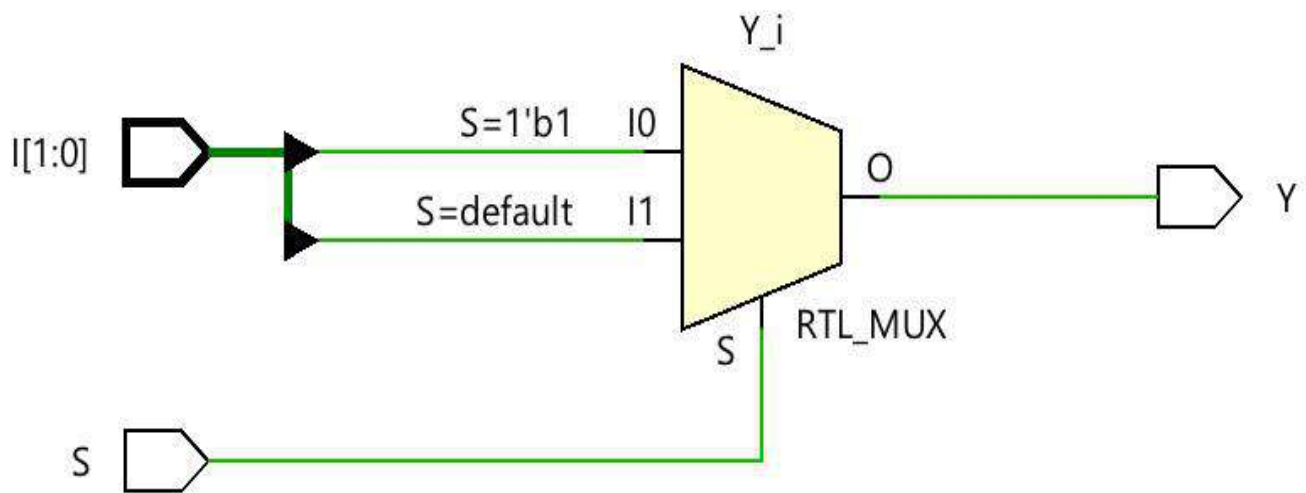
● Waveform



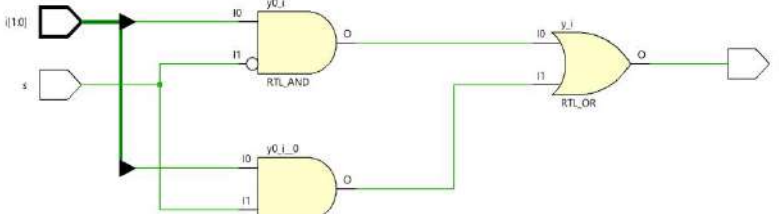
● Output

```
Simtime=0,I[0]=0,I[1]=0,S=0,Y=0  
Simtime=100000,I[0]=0,I[1]=0,S=1,Y=0  
Simtime=200000,I[0]=1,I[1]=0,S=0,Y=1  
Simtime=300000,I[0]=1,I[1]=0,S=1,Y=0  
Simtime=400000,I[0]=0,I[1]=1,S=0,Y=0  
Simtime=500000,I[0]=0,I[1]=1,S=1,Y=1  
Simtime=600000,I[0]=1,I[1]=1,S=0,Y=1  
Simtime=700000,I[0]=1,I[1]=1,S=1,Y=1  
Simtime=800000,I[0]=1,I[1]=0,S=1,Y=0
```

● Schematic



47. 2x1 MUX (in vector form)

● Design Code	● Testbench Code
<pre> module mux_2_1(i,s,y); input [1:0]i; input s; output y; assign y=(i[0]&(~s)) (i[1]&s); endmodule </pre>	<pre> module mux_2_1_tb; reg [1:0]i; reg s; wire y; mux_2_1 m1(i,s,y); initial begin i=2'b00; s=1'b0; #100 i=2'b00; s=1'b1; #100 i=2'b01; s=1'b0; #100 i=2'b01; s=1'b1; #100 i=2'b10; s=1'b0; #100 i=2'b10; s=1'b1; #100 i=2'b11; s=1'b0; #100 i=2'b11; s=1'b1; end initial begin \$monitor("Simtime=%0t, i=%b, s=%b, y=%b", \$time, i,s,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,i,s,y); end endmodule </pre>
● Waveform	
	
● OUTPUT	● Schematic
Simtime=0, i=00, s=0, y=0 Simtime=100000, i=00, s=1, y=0 Simtime=200000, i=01, s=0, y=1 Simtime=300000, i=01, s=1, y=0 Simtime=400000, i=10, s=0, y=0 Simtime=500000, i=10, s=1, y=1 Simtime=600000, i=11, s=0, y=1 Simtime=700000, i=11, s=1, y=1	

48. 2x1 MUX using Tristate buffer

● Design Code	● Testbench Code
<pre> module mux_2_1_tristate(a,b,s,y); input a,b,s; output y; bufif0 b1(y,a,s); bufif1 b2(y,b,s); endmodule </pre>	<pre> module mux_2_1_tb; reg a,b,s; wire y; mux_2_1_tristate m1(a,b,s,y); initial begin a=1'b0; b=1'b0; s=1'b0; #100 a=1'b0; b=1'b0; s=1'b1; #100 a=1'b0; b=1'b1; s=1'b0; #100 a=1'b0; b=1'b1; s=1'b1; #100 a=1'b1; b=1'b0; s=1'b0; #100 a=1'b1; b=1'b0; s=1'b1; #100 a=1'b1; b=1'b1; s=1'b0; #100 a=1'b1; b=1'b1; s=1'b1; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, s=%b, y=%b", \$time, a,b,s,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,s,y); end endmodule </pre>
● Waveform	
● Output	● Schematic
<pre> Simtime=0, a=0, b=0, s=0, y=0 Simtime=100000, a=0, b=0, s=1, y=0 Simtime=200000, a=0, b=1, s=0, y=0 Simtime=300000, a=0, b=1, s=1, y=1 Simtime=400000, a=1, b=0, s=0, y=1 Simtime=500000, a=1, b=0, s=1, y=0 Simtime=600000, a=1, b=1, s=0, y=1 Simtime=700000, a=1, b=1, s=1, y=1 </pre>	

49. MUX 4x1 (in vectors)

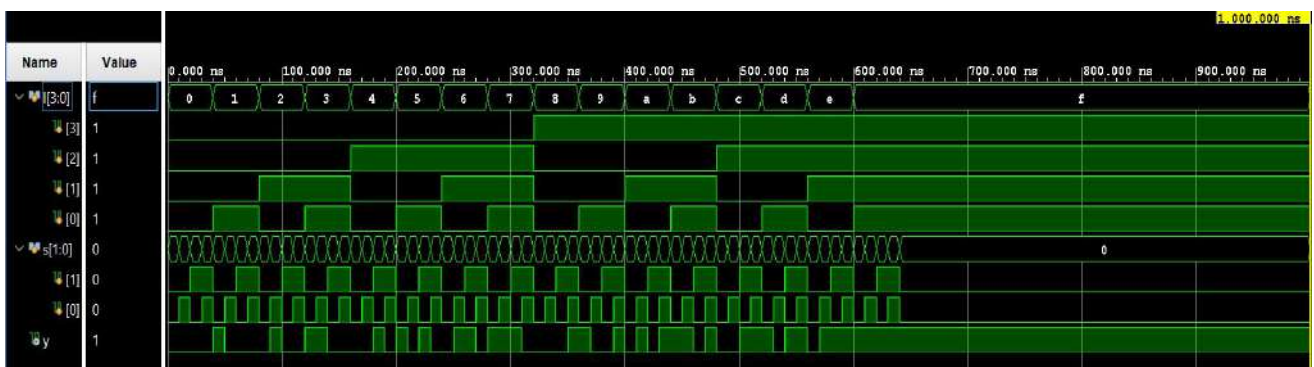
● Design Code	● Testbench Code
<pre> module mux_4_1_vec(I,s,y); input [3:0]I; input [1:0]s; output y; assign y=(((~s[1])&(~s[0])&I[0]) ((~s[1])&(s[0]) &I[1]) ((s[1])&(~s[0])&I[2]) ((s[1])&(s[0]) &I[3])); endmodule </pre>	<pre> module mux_4_1_vec_tb; reg [3:0]I; reg [1:0]s; wire y; mux_4_1_vec m1(I,s,y); initial begin I=4'b0000; s=2'b00; #10 I=4'b0000; s=2'b01; #10 I=4'b0000; s=2'b10; #10 I=4'b0000; s=2'b11; #10 I=4'b0001; s=2'b00; #10 I=4'b0001; s=2'b01; #10 I=4'b0001; s=2'b10; #10 I=4'b0001; s=2'b11; #10 I=4'b0010; s=2'b00; #10 I=4'b0010; s=2'b01; #10 I=4'b0010; s=2'b10; #10 I=4'b0010; s=2'b11; #10 I=4'b0011; s=2'b00; #10 I=4'b0011; s=2'b01; #10 I=4'b0011; s=2'b10; #10 I=4'b0011; s=2'b11; #10 I=4'b0100; s=2'b00; #10 I=4'b0100; s=2'b01; #10 I=4'b0100; s=2'b10; #10 I=4'b0100; s=2'b11; #10 I=4'b0101; s=2'b00; #10 I=4'b0101; s=2'b01; #10 I=4'b0101; s=2'b10; #10 I=4'b0101; s=2'b11; #10 I=4'b0110; s=2'b00; #10 I=4'b0110; s=2'b01; #10 I=4'b0110; s=2'b10; #10 I=4'b0110; s=2'b11; #10 I=4'b0111; s=2'b00; #10 I=4'b0111; s=2'b01; #10 I=4'b0111; s=2'b10; #10 I=4'b0111; s=2'b11; #10 I=4'b1000; s=2'b00; #10 I=4'b1000; s=2'b01; #10 I=4'b1000; s=2'b10; #10 I=4'b1000; s=2'b11; #10 I=4'b1001; s=2'b00; #10 I=4'b1001; s=2'b01; #10 I=4'b1001; s=2'b10; #10 I=4'b1001; s=2'b11; </pre>

```

#10 I=4'b1010; s=2'b00;
#10 I=4'b1010; s=2'b01;
#10 I=4'b1010; s=2'b10;
#10 I=4'b1010; s=2'b11;
#10 I=4'b1011; s=2'b00;
#10 I=4'b1011; s=2'b01;
#10 I=4'b1011; s=2'b10;
#10 I=4'b1011; s=2'b11;
#10 I=4'b1100; s=2'b00;
#10 I=4'b1100; s=2'b01;
#10 I=4'b1100; s=2'b10;
#10 I=4'b1100; s=2'b11;
#10 I=4'b1101; s=2'b00;
#10 I=4'b1101; s=2'b01;
#10 I=4'b1101; s=2'b10;
#10 I=4'b1101; s=2'b11;
#10 I=4'b1110; s=2'b00;
#10 I=4'b1110; s=2'b01;
#10 I=4'b1110; s=2'b10;
#10 I=4'b1110; s=2'b11;
#10 I=4'b1111; s=2'b00;
#10 I=4'b1111; s=2'b01;
#10 I=4'b1111; s=2'b10;
#10 I=4'b1111; s=2'b11;
#10 I=4'b1111; s=2'b00;
end
initial
begin
    $monitor("Simtime=%0t, I=%b, s=%b, y=%b",
$time, I,s,y);
end
initial
begin
    $dumpfile("dump.vcd");
    $dumpvars(0,I,s,y);
end
endmodule

```

● Waveform



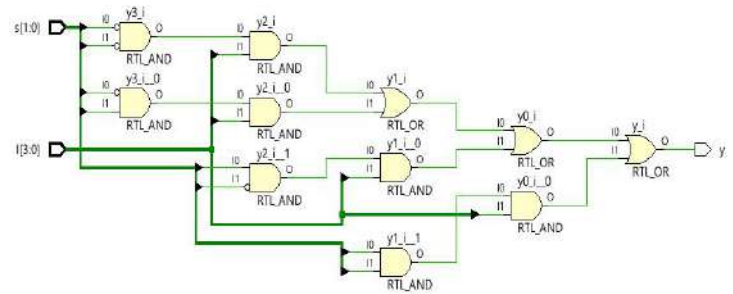
● Output

```

Simtime=0, I=0000, s=00, y=0
Simtime=10000, I=0000, s=01, y=0
Simtime=20000, I=0000, s=10, y=0
Simtime=30000, I=0000, s=11, y=0
Simtime=40000, I=0001, s=00, y=1
Simtime=50000, I=0001, s=01, y=0
Simtime=60000, I=0001, s=10, y=0
Simtime=70000, I=0001, s=11, y=0
Simtime=80000, I=0010, s=00, y=0
Simtime=90000, I=0010, s=01, y=1
Simtime=100000, I=0010, s=10, y=0
Simtime=110000, I=0010, s=11, y=0
Simtime=120000, I=0011, s=00, y=1
Simtime=130000, I=0011, s=01, y=1
Simtime=140000, I=0011, s=10, y=0
Simtime=150000, I=0011, s=11, y=0
Simtime=160000, I=0100, s=00, y=0
Simtime=170000, I=0100, s=01, y=0
Simtime=180000, I=0100, s=10, y=1
Simtime=190000, I=0100, s=11, y=0
Simtime=200000, I=0101, s=00, y=1
Simtime=210000, I=0101, s=01, y=0
Simtime=220000, I=0101, s=10, y=1
Simtime=230000, I=0101, s=11, y=0
Simtime=240000, I=0110, s=00, y=0
Simtime=250000, I=0110, s=01, y=1
Simtime=260000, I=0110, s=10, y=1
Simtime=270000, I=0110, s=11, y=0
Simtime=280000, I=0111, s=00, y=1
Simtime=290000, I=0111, s=01, y=1
Simtime=300000, I=0111, s=10, y=1
Simtime=310000, I=0111, s=11, y=0
Simtime=320000, I=1000, s=00, y=0
Simtime=330000, I=1000, s=01, y=0
Simtime=340000, I=1000, s=10, y=0
Simtime=350000, I=1000, s=11, y=1
Simtime=360000, I=1001, s=00, y=1
Simtime=370000, I=1001, s=01, y=0
Simtime=380000, I=1001, s=10, y=0
Simtime=390000, I=1001, s=11, y=1
Simtime=400000, I=1010, s=00, y=0
Simtime=410000, I=1010, s=01, y=1
Simtime=420000, I=1010, s=10, y=0
Simtime=430000, I=1010, s=11, y=1
Simtime=440000, I=1011, s=00, y=1
Simtime=450000, I=1011, s=01, y=1
Simtime=460000, I=1011, s=10, y=0
Simtime=470000, I=1011, s=11, y=1
Simtime=480000, I=1100, s=00, y=0
Simtime=490000, I=1100, s=01, y=0

```

● Schematic



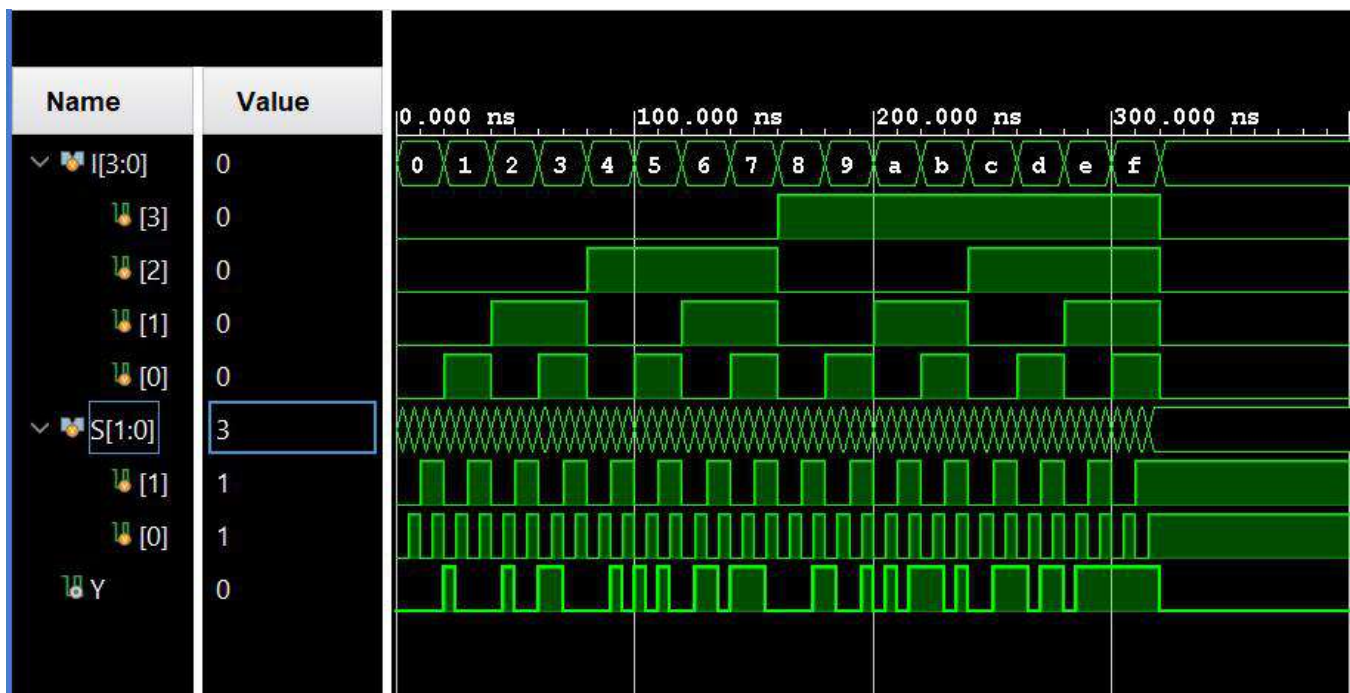
50. MUX 4x1 using ternary operator

● Design Code	● Testbench Code
<pre> module mux_4_1_ternary(I,S,Y); input [3:0]I; input [1:0]S; output Y; assign Y=(S[1])? ((S[0])?I[3]:I[2]): ((S[0])?I[1]:I[0]); endmodule </pre>	<pre> module mux_4_1_ternary_tb; reg [3:0]I; reg [1:0]S; wire Y; mux_4_1_ternary dut(.I(I),.S(S),.Y(Y)); initial begin {I,S}=6'b000000; #5 {I,S}=6'b000001; #5 {I,S}=6'b000010; #5 {I,S}=6'b000011; #5 {I,S}=6'b000100; #5 {I,S}=6'b000101; #5 {I,S}=6'b000110; #5 {I,S}=6'b000111; #5 {I,S}=6'b001000; #5 {I,S}=6'b001001; #5 {I,S}=6'b001010; #5 {I,S}=6'b001011; #5 {I,S}=6'b001100; #5 {I,S}=6'b001101; #5 {I,S}=6'b001110; #5 {I,S}=6'b001111; #5 {I,S}=6'b010000; #5 {I,S}=6'b010001; #5 {I,S}=6'b010010; #5 {I,S}=6'b010011; #5 {I,S}=6'b010100; #5 {I,S}=6'b010101; #5 {I,S}=6'b010110; #5 {I,S}=6'b010111; #5 {I,S}=6'b011000; #5 {I,S}=6'b011001; #5 {I,S}=6'b011010; #5 {I,S}=6'b011011; #5 {I,S}=6'b011100; #5 {I,S}=6'b011101; #5 {I,S}=6'b011110; #5 {I,S}=6'b011111; #5 {I,S}=6'b100000; #5 {I,S}=6'b100001; #5 {I,S}=6'b100010; #5 {I,S}=6'b100011; #5 {I,S}=6'b100100; #5 {I,S}=6'b100101; #5 {I,S}=6'b100110; #5 {I,S}=6'b100111; </pre>

```
#5 {I,S}=6'b101000;
#5 {I,S}=6'b101001;
#5 {I,S}=6'b101010;
#5 {I,S}=6'b101011;
#5 {I,S}=6'b101100;
#5 {I,S}=6'b101101;
#5 {I,S}=6'b101110;
#5 {I,S}=6'b101111;
#5 {I,S}=6'b110000;
#5 {I,S}=6'b110001;
#5 {I,S}=6'b110010;
#5 {I,S}=6'b110011;
#5 {I,S}=6'b110100;
#5 {I,S}=6'b110101;
#5 {I,S}=6'b110110;
#5 {I,S}=6'b110111;
#5 {I,S}=6'b111000;
#5 {I,S}=6'b111001;
#5 {I,S}=6'b111010;
#5 {I,S}=6'b111011;
#5 {I,S}=6'b111100;
#5 {I,S}=6'b111101;
#5 {I,S}=6'b111110;
#5 {I,S}=6'b111111;
#5 {I,S}=6'b000011;
end
initial begin

$monitor("Simtime=%0t,I=%b,S=%b,Y=%b",$time,I,S,
Y);
end
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,I,S,Y);
end
endmodule
```


● Waveform



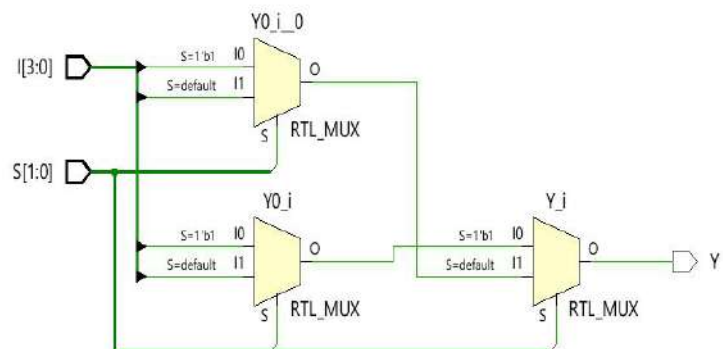
● Output

```

Simtime=0, I=0000, S=00, Y=0
Simtime=5000, I=0000, S=01, Y=0
Simtime=10000, I=0000, S=10, Y=0
Simtime=15000, I=0000, S=11, Y=0
Simtime=20000, I=0001, S=00, Y=1
Simtime=25000, I=0001, S=01, Y=0
Simtime=30000, I=0001, S=10, Y=0
Simtime=35000, I=0001, S=11, Y=0
Simtime=40000, I=0010, S=00, Y=0
Simtime=45000, I=0010, S=01, Y=1
Simtime=50000, I=0010, S=10, Y=0
Simtime=55000, I=0010, S=11, Y=0
Simtime=60000, I=0011, S=00, Y=1
Simtime=65000, I=0011, S=01, Y=1
Simtime=70000, I=0011, S=10, Y=0
Simtime=75000, I=0011, S=11, Y=0
Simtime=80000, I=0100, S=00, Y=0
Simtime=85000, I=0100, S=01, Y=0
Simtime=90000, I=0100, S=10, Y=1
Simtime=95000, I=0100, S=11, Y=0
Simtime=100000, I=0101, S=00, Y=1
Simtime=105000, I=0101, S=01, Y=0
Simtime=110000, I=0101, S=10, Y=1
Simtime=115000, I=0101, S=11, Y=0

```

● Schematic



51. 4x1 MUX using 2x1 MUX

● Design code of 2x1 MUX	● Design code of 4x1 MUX	● Testbench
<pre> module mux_2_1(a,b,s,y); input a,b,s; output y; assign y=((~s)&a) (s&b); endmodule </pre>	<pre> //Structural Modeling Style `include "mux_2_1.v" module mux_4_1_using_2_1(a,b,c,d,s1,s0,y); input a,b,c,d,s1,s0; output y; wire w1,w2; mux_2_1 m1(a,b,s0,w1); mux_2_1 m2(c,d,s0,w2); mux_2_1 m3(w1,w2,s1,y); endmodule </pre>	<pre> module mux_4_1_using_2_1_tb; reg a, b, c, d,s1, s0; wire y; // Declare loop variable outside of initial block integer i; mux_4_1_using_2_1 m4_1 (a, b, c, d, s1, s0, y); // Apply all combinations of inputs using for loop initial begin for (i = 0; i < 64; i = i + 1) begin {s1, s0, a, b, c, d} = i[5:0]; #10; end end initial begin \$monitor("Time=%0t s1=%b s0=%b a=%b b=%b c=%b d=%b y=%b", \$time, s1, s0, a, b, c, d, y); end initial begin \$dumppfile("dump.vcd"); \$dumppvars(0, mux_4_1_using_2_1_tb); end endmodule </pre>

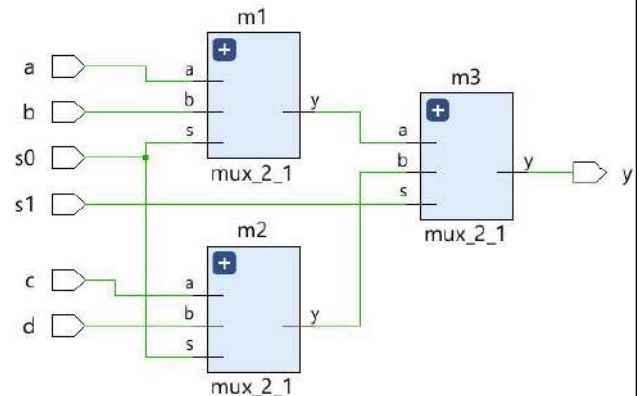
● Waveform



● Output

Time=0	s1=0	s0=0	a=0	b=0	c=0	d=0	y=0
Time=10000	s1=0	s0=0	a=0	b=0	c=0	d=1	y=0
Time=20000	s1=0	s0=0	a=0	b=0	c=1	d=0	y=0
Time=30000	s1=0	s0=0	a=0	b=0	c=1	d=1	y=0
Time=40000	s1=0	s0=0	a=0	b=1	c=0	d=0	y=0
Time=50000	s1=0	s0=0	a=0	b=1	c=0	d=1	y=0
Time=60000	s1=0	s0=0	a=0	b=1	c=1	d=0	y=0
Time=70000	s1=0	s0=0	a=0	b=1	c=1	d=1	y=0
Time=80000	s1=0	s0=0	a=1	b=0	c=0	d=0	y=1
Time=90000	s1=0	s0=0	a=1	b=0	c=0	d=1	y=1
Time=100000	s1=0	s0=0	a=1	b=0	c=1	d=0	y=1
Time=110000	s1=0	s0=0	a=1	b=0	c=1	d=1	y=1
Time=120000	s1=0	s0=0	a=1	b=1	c=0	d=0	y=1
Time=130000	s1=0	s0=0	a=1	b=1	c=0	d=1	y=1
Time=140000	s1=0	s0=0	a=1	b=1	c=1	d=0	y=1
Time=150000	s1=0	s0=0	a=1	b=1	c=1	d=1	y=1
Time=160000	s1=0	s0=1	a=0	b=0	c=0	d=0	y=0
Time=170000	s1=0	s0=1	a=0	b=0	c=0	d=1	y=0
Time=180000	s1=0	s0=1	a=0	b=0	c=1	d=0	y=0
Time=190000	s1=0	s0=1	a=0	b=0	c=1	d=1	y=0
Time=200000	s1=0	s0=1	a=0	b=1	c=0	d=0	y=1
Time=210000	s1=0	s0=1	a=0	b=1	c=0	d=1	y=1
Time=220000	s1=0	s0=1	a=0	b=1	c=1	d=0	y=1
Time=230000	s1=0	s0=1	a=0	b=1	c=1	d=1	y=1
Time=240000	s1=0	s0=1	a=1	b=0	c=0	d=0	y=0
Time=250000	s1=0	s0=1	a=1	b=0	c=0	d=1	y=0
Time=260000	s1=0	s0=1	a=1	b=0	c=1	d=0	y=0
Time=270000	s1=0	s0=1	a=1	b=0	c=1	d=1	y=0
Time=280000	s1=0	s0=1	a=1	b=1	c=0	d=0	y=1
Time=290000	s1=0	s0=1	a=1	b=1	c=0	d=1	y=1
Time=300000	s1=0	s0=1	a=1	b=1	c=1	d=0	y=1
Time=310000	s1=0	s0=1	a=1	b=1	c=1	d=1	y=1
Time=320000	s1=1	s0=0	a=0	b=0	c=0	d=0	y=0
Time=330000	s1=1	s0=0	a=0	b=0	c=0	d=1	y=0
Time=340000	s1=1	s0=0	a=0	b=0	c=1	d=0	y=1
Time=350000	s1=1	s0=0	a=0	b=0	c=1	d=1	y=1
Time=360000	s1=1	s0=0	a=0	b=1	c=0	d=0	y=0
Time=370000	s1=1	s0=0	a=0	b=1	c=0	d=1	y=0
Time=380000	s1=1	s0=0	a=0	b=1	c=1	d=0	y=1
Time=390000	s1=1	s0=0	a=0	b=1	c=1	d=1	y=1
Time=400000	s1=1	s0=0	a=1	b=0	c=0	d=0	y=0
Time=410000	s1=1	s0=0	a=1	b=0	c=0	d=1	y=0
Time=420000	s1=1	s0=0	a=1	b=0	c=1	d=0	y=1
Time=430000	s1=1	s0=0	a=1	b=0	c=1	d=1	y=1
Time=440000	s1=1	s0=0	a=1	b=1	c=0	d=0	y=0
Time=450000	s1=1	s0=0	a=1	b=1	c=0	d=1	y=0
Time=460000	s1=1	s0=0	a=1	b=1	c=1	d=0	y=1
Time=470000	s1=1	s0=0	a=1	b=1	c=1	d=1	y=1
Time=480000	s1=1	s0=1	a=0	b=0	c=0	d=0	y=0
Time=490000	s1=1	s0=1	a=0	b=0	c=0	d=1	y=1
Time=500000	s1=1	s0=1	a=0	b=0	c=1	d=0	y=0
Time=510000	s1=1	s0=1	a=0	b=0	c=1	d=1	y=1
Time=520000	s1=1	s0=1	a=0	b=1	c=0	d=0	y=0
Time=530000	s1=1	s0=1	a=0	b=1	c=0	d=1	y=1
Time=540000	s1=1	s0=1	a=0	b=1	c=1	d=0	y=0
Time=550000	s1=1	s0=1	a=0	b=1	c=1	d=1	y=1
Time=560000	s1=1	s0=1	a=1	b=0	c=0	d=0	y=0
Time=570000	s1=1	s0=1	a=1	b=0	c=0	d=1	y=1
Time=580000	s1=1	s0=1	a=1	b=0	c=1	d=0	y=0
Time=590000	s1=1	s0=1	a=1	b=0	c=1	d=1	y=1
Time=600000	s1=1	s0=1	a=1	b=1	c=0	d=0	y=0
Time=610000	s1=1	s0=1	a=1	b=1	c=0	d=1	y=1
Time=620000	s1=1	s0=1	a=1	b=1	c=1	d=0	y=0
Time=630000	s1=1	s0=1	a=1	b=1	c=1	d=1	y=1

● Schematic



52. 8x1 MUX using 4x1 MUX and 2x1 MUX in vectors

● Design code of 2x1 MUX	● Design code of 4x1 MUX
<pre> module vector_mux_2_1(i,s,y); input [1:0]i; input s; output y; assign y=(i[0]&(~s)) (i[1]&s); endmodule </pre>	<pre> `include "vector_mux_2_1.v" module vector_mux_4_1(i,s,y); input [3:0]i; input [1:0]s; output y; wire [1:0]w; vector_mux_2_1 m1(i[1:0],s[0],w[0]); vector_mux_2_1 m2(i[3:2],s[0],w[1]); vector_mux_2_1 m3(w,s[1],y); endmodule </pre>
● Design code of 8x1 MUX	● Testbench
<pre> //Structural Modeling Style `include "vector_mux_4_1.v" `include "vector_mux_2_1.v" module vector_mux_8_1(i,s,y); input [7:0]i; input [2:0]s; output y; wire [1:0]w; vector_mux_4_1 m1(i[3:0],s[1:0],w[0]); vector_mux_4_1 m2(i[7:4],s[1:0],w[1]); vector_mux_2_1 m3(w,s[2],y); endmodule </pre>	<pre> module vector_mux_8_1_tb; reg [7:0] i; reg [2:0] s; wire y; // Declare loop variable integer j; vector_mux_8_1 uut (i, s, y); // Apply all combinations of select lines with fixed input patterns initial begin i = 8'b10101010; for (j = 0; j < 8; j = j + 1) begin s = j[2:0]; #10; end i = 8'b11001100; for (j = 0; j < 8; j = j + 1) begin s = j[2:0]; #10; end i = 8'b00010000; for (j = 0; j < 8; j = j + 1) begin s = j[2:0]; #100; end end initial begin \$monitor("Time=%0t s=%b i=%b y=%b", \$time, s, i, y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0, vector_mux_8_1_tb); end endmodule </pre>

● Waveform



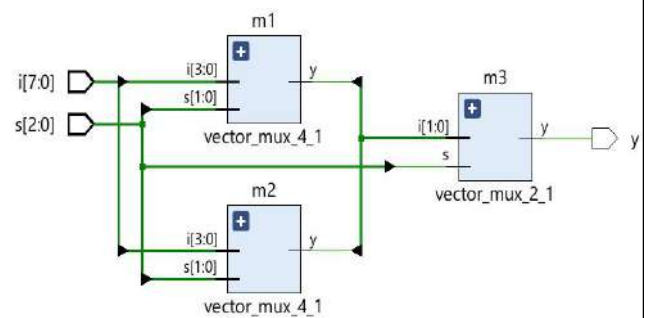
● Output

```

Time=0 | s=000 | i=10101010 | y=0
Time=10000 | s=001 | i=10101010 | y=1
Time=20000 | s=010 | i=10101010 | y=0
Time=30000 | s=011 | i=10101010 | y=1
Time=40000 | s=100 | i=10101010 | y=0
Time=50000 | s=101 | i=10101010 | y=1
Time=60000 | s=110 | i=10101010 | y=0
Time=70000 | s=111 | i=10101010 | y=1
Time=80000 | s=000 | i=11001100 | y=0
Time=90000 | s=001 | i=11001100 | y=0
Time=100000 | s=010 | i=11001100 | y=1
Time=110000 | s=011 | i=11001100 | y=1
Time=120000 | s=100 | i=11001100 | y=0
Time=130000 | s=101 | i=11001100 | y=0
Time=140000 | s=110 | i=11001100 | y=1
Time=150000 | s=111 | i=11001100 | y=1
Time=160000 | s=000 | i=00010000 | y=0
Time=260000 | s=001 | i=00010000 | y=0
Time=360000 | s=010 | i=00010000 | y=0
Time=460000 | s=011 | i=00010000 | y=0
Time=560000 | s=100 | i=00010000 | y=1
Time=660000 | s=101 | i=00010000 | y=0
Time=760000 | s=110 | i=00010000 | y=0

```

● Schematic



53. 8x1 MUX using 2x1 MUX only

● Design code of 2x1 MUX	● Design code of 4x1 MUX	● Testbench
<pre> module vector_mux_2_1(i,s,y); input [1:0]i; input s; output y; assign y=(i[0]&(~s)) (i[1]&s); endmodule </pre>	<pre> //Structural Modeling Style `include "vector_mux_2_1.v" module mux_8_1_using_mux_2_1(i,s,y); input [7:0]i; input [2:0]s; output y; wire [5:0]w; vector_mux_2_1 m1(i[1:0],s[0],w[5]); vector_mux_2_1 m2(i[3:2],s[0],w[4]); vector_mux_2_1 m3(i[5:4],s[0],w[3]); vector_mux_2_1 m4(i[7:6],s[0],w[2]); vector_mux_2_1 m5(w[5:4],s[1],w[1]); vector_mux_2_1 m6(w[3:2],s[1],w[0]); vector_mux_2_1 m7(w[1:0],s[2],y); endmodule </pre>	<pre> module mux_8_1_using_mux_2_1_tb; reg [7:0] i; reg [2:0] s; wire y; // Declare loop variable integer j; mux_8_1_using_mux_2_1 uut (i, s, y); // Apply all combinations of select lines with fixed input patterns initial begin i = 8'b10101010; for (j = 0; j < 8; j = j + 1) begin s = j[2:0]; #10; end i = 8'b11001100; for (j = 0; j < 8; j = j + 1) begin s = j[2:0]; #10; end i = 8'b00010000; for (j = 0; j < 8; j = j + 1) begin s = j[2:0]; #100; end end initial begin \$monitor("Time=%0t s=%b i=%b y=%b", \$time, s, i, y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0, vector_mux_8_1_tb); end endmodule </pre>

● Waveform



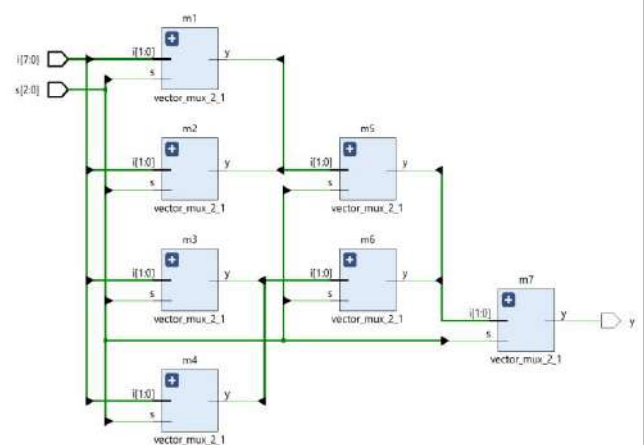
● Output

```

Time=0 | s=000 | i=10101010 | y=0
Time=10000 | s=001 | i=10101010 | y=1
Time=20000 | s=010 | i=10101010 | y=0
Time=30000 | s=011 | i=10101010 | y=1
Time=40000 | s=100 | i=10101010 | y=0
Time=50000 | s=101 | i=10101010 | y=1
Time=60000 | s=110 | i=10101010 | y=0
Time=70000 | s=111 | i=10101010 | y=1
Time=80000 | s=000 | i=11001100 | y=1
Time=90000 | s=001 | i=11001100 | y=1
Time=100000 | s=010 | i=11001100 | y=0
Time=110000 | s=011 | i=11001100 | y=0
Time=120000 | s=100 | i=11001100 | y=1
Time=130000 | s=101 | i=11001100 | y=1
Time=140000 | s=110 | i=11001100 | y=0
Time=150000 | s=111 | i=11001100 | y=0
Time=160000 | s=000 | i=00010000 | y=0
Time=260000 | s=001 | i=00010000 | y=0
Time=360000 | s=010 | i=00010000 | y=1
Time=460000 | s=011 | i=00010000 | y=0
Time=560000 | s=100 | i=00010000 | y=0
Time=660000 | s=101 | i=00010000 | y=0
Time=760000 | s=110 | i=00010000 | y=0
Time=860000 | s=111 | i=00010000 | y=0

```

● Schematic



54. 3x1 MUX

● Design Code

//Behavioural Modeling Style

```
module mux3x1_behavioral (sel,in,y);
input [1:0] sel;
input [2:0] in;
output reg y;
always @(sel,in)
begin
case (sel)
2'b00: y = in[0];
2'b01: y = in[1];
2'b10: y = in[2];
default: y = 1'b0;
endcase
end
endmodule
```

//Dataflow Modeling Style

```
module mux3x1_dataflow (
input [1:0] sel,
input [2:0] in,
output y
);
assign y = (sel == 2'b00) ? in[0] :
(sel == 2'b01) ? in[1] :
(sel == 2'b10) ? in[2] : 1'b0;
endmodule
```

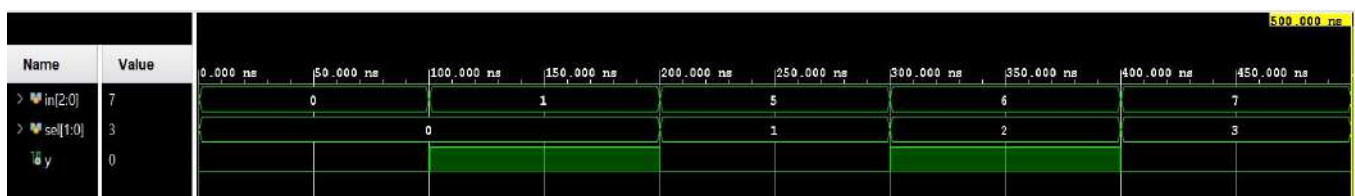
● Testbench Code

```
module mux_3x1_tb;
reg [2:0] in;
reg [1:0] sel;
wire y;

mux3x1_behavioral uut (sel,in,y);
initial
begin
$display("sel in | y");
$monitor("%b %b | %b", sel, in, y);

in = 3'b000; sel = 2'b00;
#100 in = 3'b001; sel = 2'b00;
#100 in = 3'b101; sel = 2'b01;
#100 in = 3'b110; sel = 2'b10;
#100 in = 3'b111; sel = 2'b11;
#100;
$finish;
end
endmodule
```

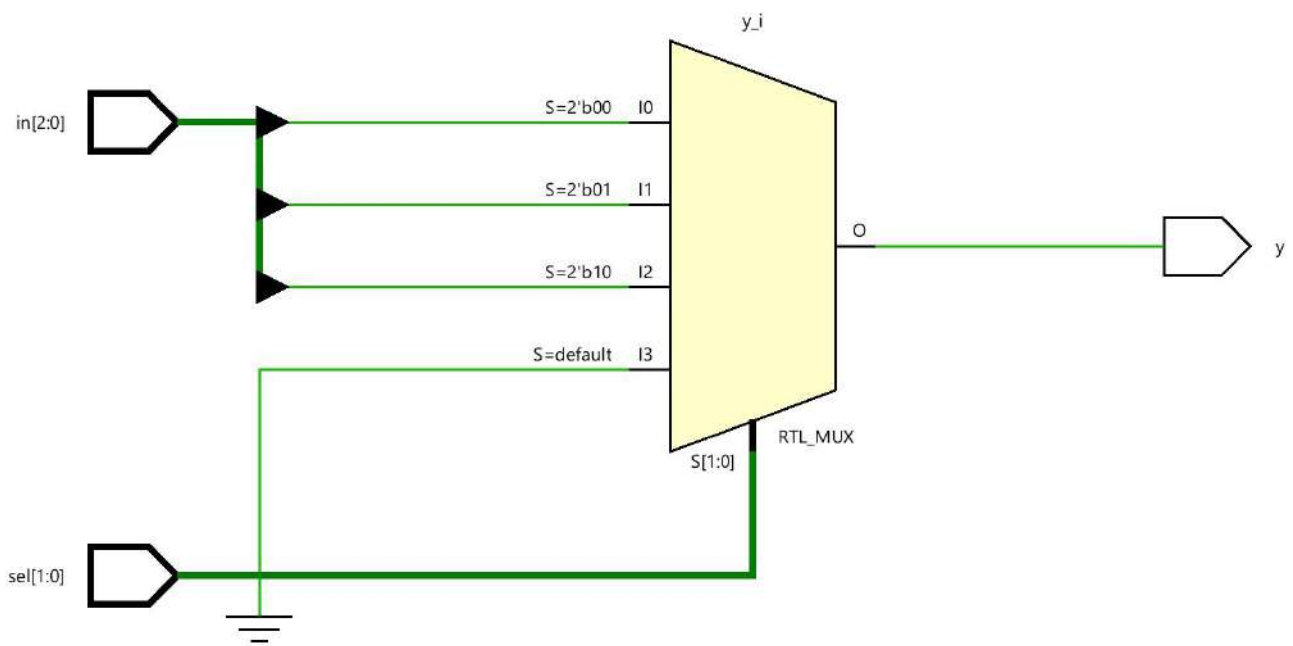
● Waveform



● OUTPUT

```
sel  in  | y
00  000 | 0
00  001 | 1
01  101 | 0
10  110 | 1
11  111 | 0
```

- Schematic



55. 5x1 MUX

● Design Code

//Behavioural Modeling Style

```
module mux5x1(in, sel, out);
  input [4:0] in;
  input [2:0] sel;
  output reg out;
  always @(in,sel)
  begin
    case (sel)
      3'b000: out = in[0];
      3'b001: out = in[1];
      3'b010: out = in[2];
      3'b011: out = in[3];
      3'b100: out = in[4];
      default: out = 1'b0;
    endcase
  end
endmodule
```

//Dataflow Modeling Style

```
module mux5x1_ternary (
  input [4:0] in,
  input [2:0] sel,
  output out
);
  assign out = (sel == 3'b000) ? in[0] :
    (sel == 3'b001) ? in[1] :
    (sel == 3'b010) ? in[2] :
    (sel == 3'b011) ? in[3] :
    (sel == 3'b100) ? in[4] : 1'b0;
endmodule
```

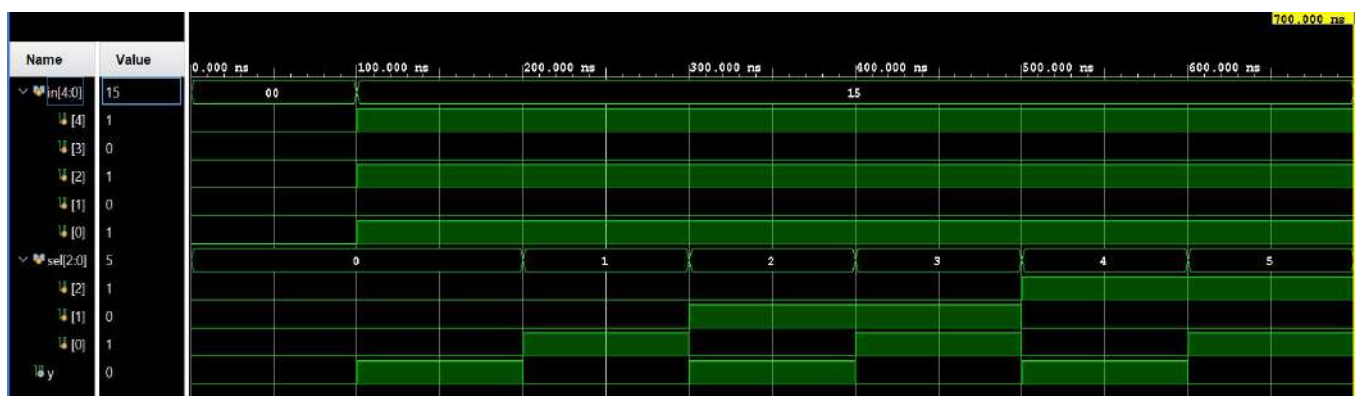
● Testbench Code

```
module mux5x1_tb;
  reg [4:0] in;
  reg [2:0] sel;
  wire y;

  mux5x1 uut (in,sel,y);
  initial
  begin
    $display("sel in | y");
    $monitor("%b %b | %b", sel, in, y);

    in = 5'b00000; sel = 3'b000;
    #100 in = 5'b10101; sel = 3'b000;
    #100 in = 5'b10101; sel = 3'b001;
    #100 in = 5'b10101; sel = 3'b010;
    #100 in = 5'b10101; sel = 3'b011;
    #100 in = 5'b10101; sel = 3'b100;
    #100 in = 5'b10101; sel = 3'b101;
    #100 $finish;
  end
endmodule
```

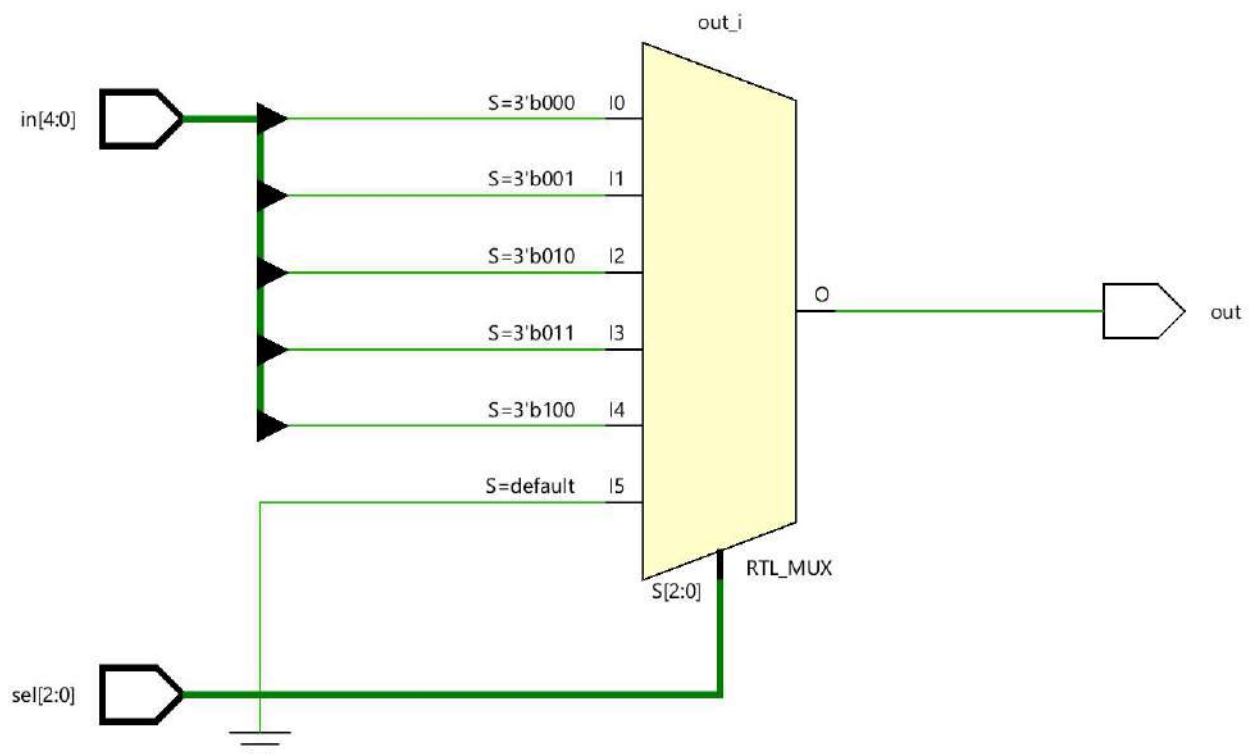
● Waveform



● OUTPUT

sel	in	y
000	00000	0
000	10101	1
001	10101	0
010	10101	1
011	10101	0
100	10101	1
101	10101	0

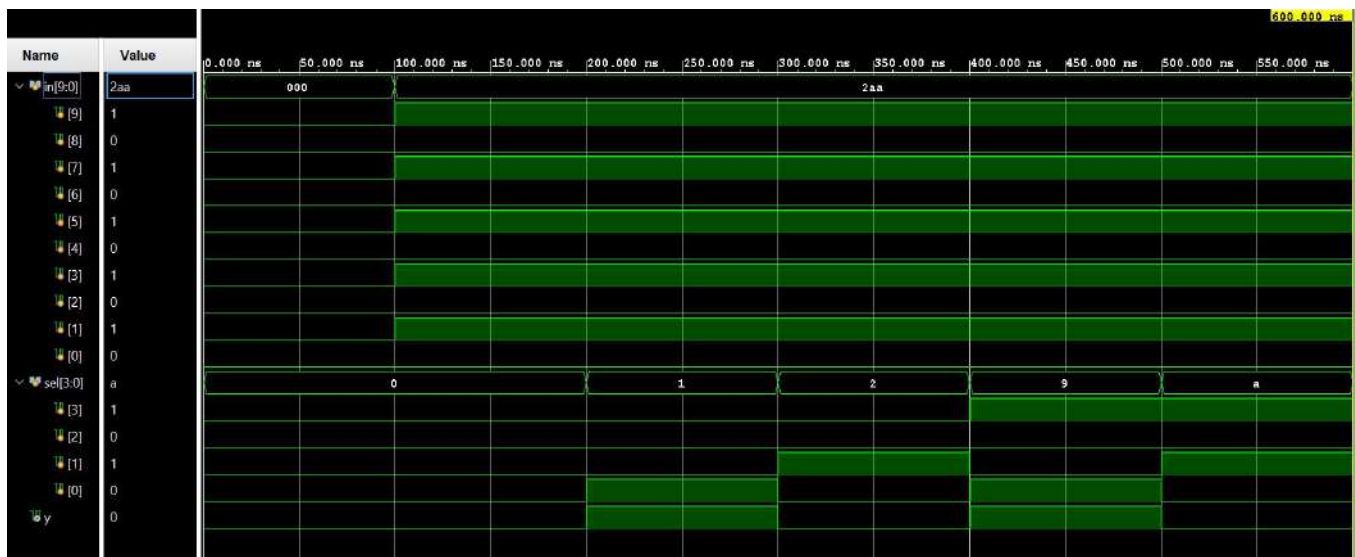
● Schematic



56. 10x1 MUX

● Design Code	● Testbench Code
<pre> //Behavioural Modeling Style module mux10x1(in,sel,out); input [9:0] in; input [3:0] sel; output reg out; always @(in,sel) begin case (sel) 4'd0: out = in[0]; 4'd1: out = in[1]; 4'd2: out = in[2]; 4'd3: out = in[3]; 4'd4: out = in[4]; 4'd5: out = in[5]; 4'd6: out = in[6]; 4'd7: out = in[7]; 4'd8: out = in[8]; 4'd9: out = in[9]; default: out = 1'b0; endcase end endmodule //Dataflow Modeling Style module mux5x1_ternary (input [4:0] in, input [2:0] sel, output out); assign out = (sel == 3'b000) ? in[0] : (sel == 3'b001) ? in[1] : (sel == 3'b010) ? in[2] : (sel == 3'b011) ? in[3] : (sel == 3'b100) ? in[4] : 1'b0; endmodule </pre>	<pre> module mux10x1_tb; reg [9:0] in; reg [3:0] sel; wire y; mux10x1 uut (in,sel,y); initial begin \$display("sel in y"); \$monitor("%b %b %b", sel, in, y); in = 10'b0000000000; sel = 4'b0000; #100 in = 10'b1010101010; sel = 4'b0000; #100 in = 10'b1010101010; sel = 4'b0001; #100 in = 10'b1010101010; sel = 4'b0010; #100 in = 10'b1010101010; sel = 4'b1001; #100 in = 10'b1010101010; sel = 4'b1010; #100 \$finish; end endmodule </pre>

● Waveform



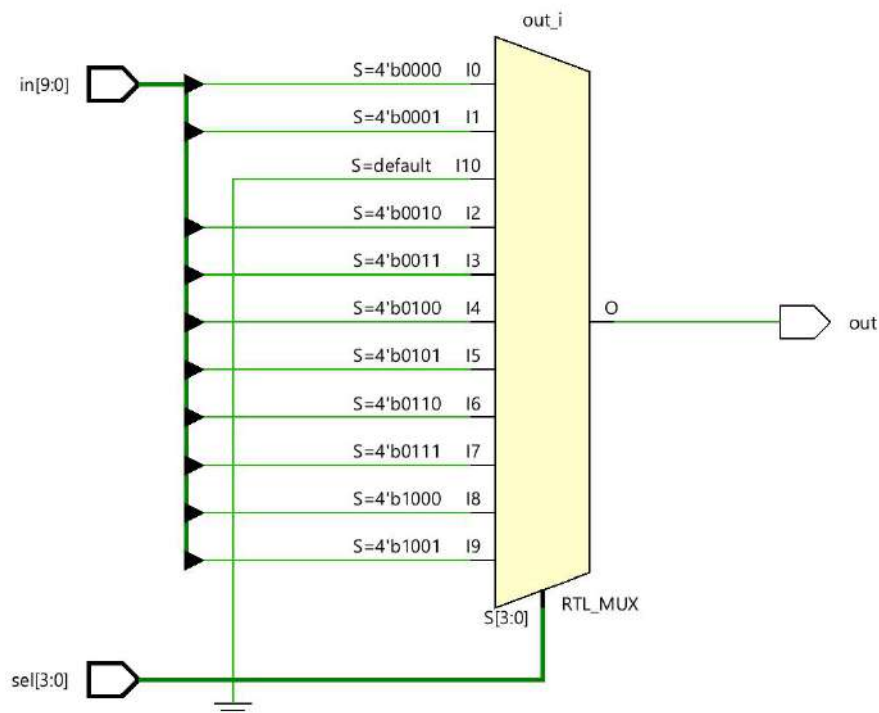
● OUTPUT

```

sel in | y
0000 0000000000 | 0
0000 1010101010 | 0
0001 1010101010 | 1
0010 1010101010 | 0
1001 1010101010 | 1
1010 1010101010 | 0

```

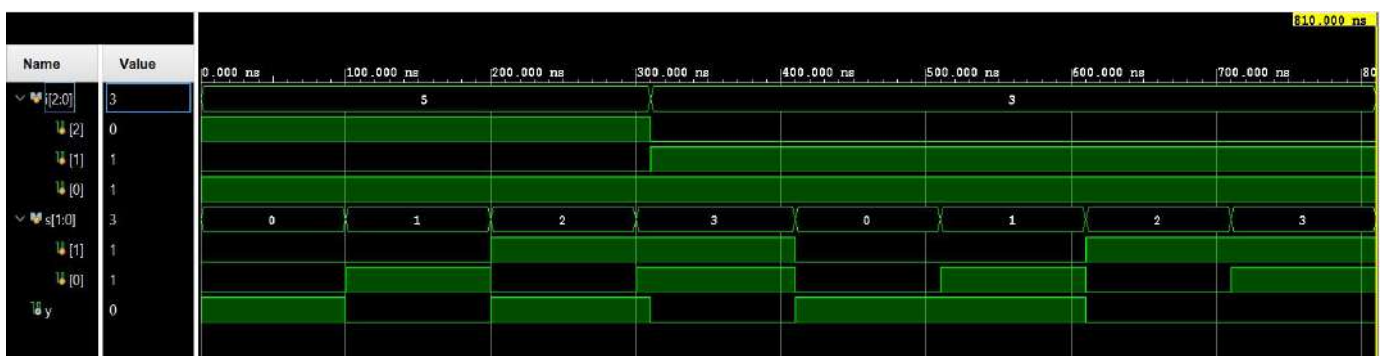
● Schematic



57. 3x1 MUX using 2x1 MUX only

● Design code of 2x1 MUX	● Design code of 3x1 MUX	● Testbench
<pre> module mux_2_1(i,s,y); input [1:0]i; input s; output y; assign y=(i[0]&(~s)) (i[1]&s); endmodule </pre>	<pre> //Structural Modeling Style `include "mux_2_1.v" module mux_3_1_structural(i, s, y); input [2:0] i; // 3 input lines input [1:0] s; // 2-bit select line (only 0-2 valid) output y; wire w; mux_2_1 m1(i[1:0], s[0], w); mux_2_1 m2({i[2], w}, s[1], y); endmodule </pre>	<pre> module mux_3_1_structural_tb; reg [2:0] i; reg [1:0] s; wire y; // Instantiate DUT mux_3_1_structural dut (i,s,y); initial begin \$display("Time i s y"); \$monitor("%0t %b %b %b", \$time, i, s, y); // Test with input pattern 3'b101 i = 3'b101; s = 2'b00; #100 s = 2'b01; #100 s = 2'b10; #100 s = 2'b11; // Test with input pattern 3'b011 #10 i = 3'b011; #100 s = 2'b00; #100 s = 2'b01; #100 s = 2'b10; #100 s = 2'b11; #100 \$finish; end endmodule </pre>

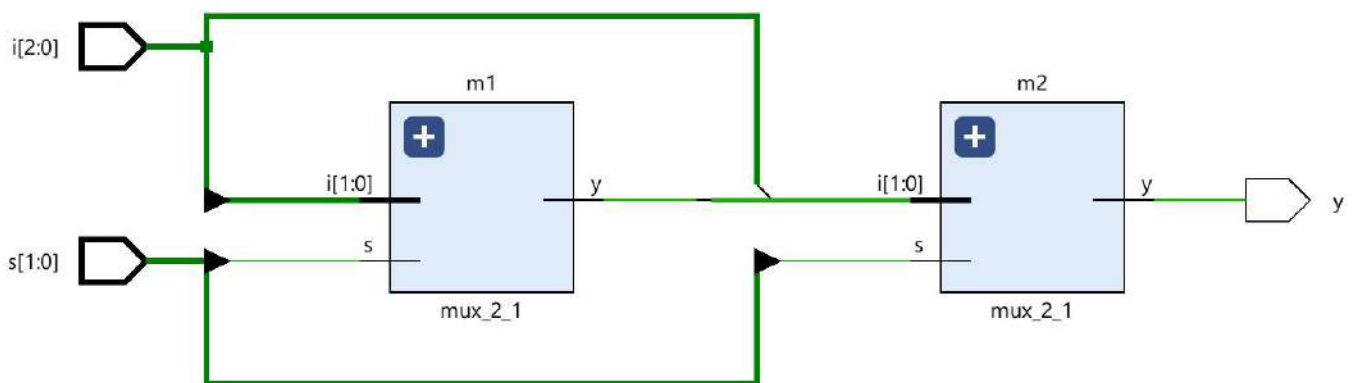
● Waveform



- Output

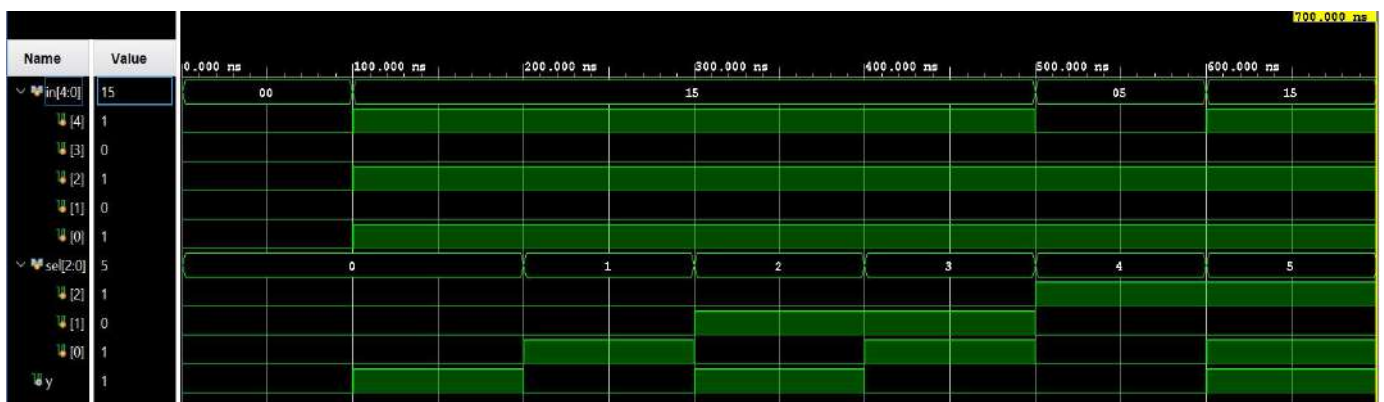
Time i s y
0 101 00 1
100000 101 01 0
200000 101 10 1
300000 101 11 1
310000 011 11 0
410000 011 00 1
510000 011 01 1
610000 011 10 0
710000 011 11 0

- Schematic



58. 5x1 MUX using 2x1 MUX only

● Design code of 2x1 MUX	● Design code of 5x1 MUX	● Testbench
<pre> module mux_2_1(i,s,y); input [1:0]i; input s; output y; assign y=(i[0]&(~s)) (i[1]&s); endmodule </pre>	<pre> //Structural Modeling Style `include "mux_2_1.v" module mux_5_1_structural(i,s,y); input [4:0] i; // 5 input lines input [2:0] s; // 3-bit select line (only 0-4 valid) output y; wire w1,w2,w3; mux_2_1 m1(i[1:0],s[0],w1); mux_2_1 m2(i[3:2],s[0],w2); mux_2_1 m3({w2,w1},s[1],w3); mux_2_1 m4({i[4],w3},s[2],y); endmodule </pre>	<pre> module mux_5_1_structural_tb; reg [4:0] in; reg [2:0] sel; wire y; mux_5_1_structural uut (in,sel,y); initial begin \$display("sel in y"); \$monitor("%b %b %b", sel, in, y); in = 5'b00000; sel = 3'b000; #100 in = 5'b10101; sel = 3'b000; #100 in = 5'b10101; sel = 3'b001; #100 in = 5'b10101; sel = 3'b010; #100 in = 5'b10101; sel = 3'b011; #100 in = 5'b00101; sel = 3'b100; #100 in = 5'b10101; sel = 3'b101; #100 \$finish; end endmodule </pre>

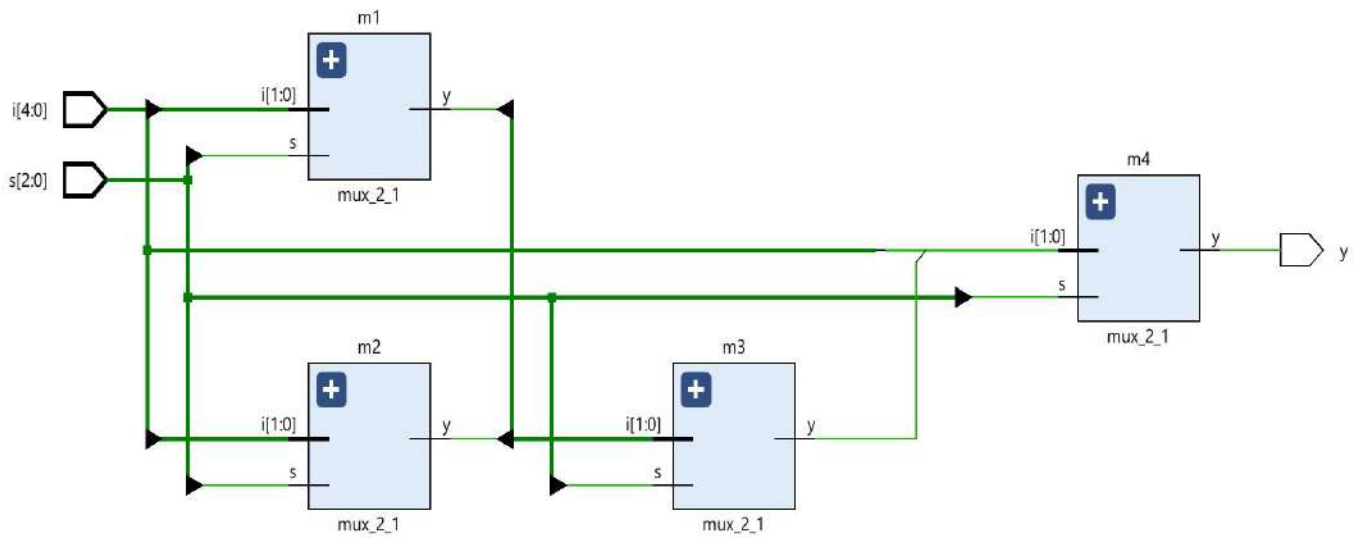
● Waveform**● Output**

```

sel  in  | y
000 00000 | 0
000 10101 | 1
001 10101 | 0
010 10101 | 1
011 10101 | 0
100 00101 | 0
101 10101 | 1

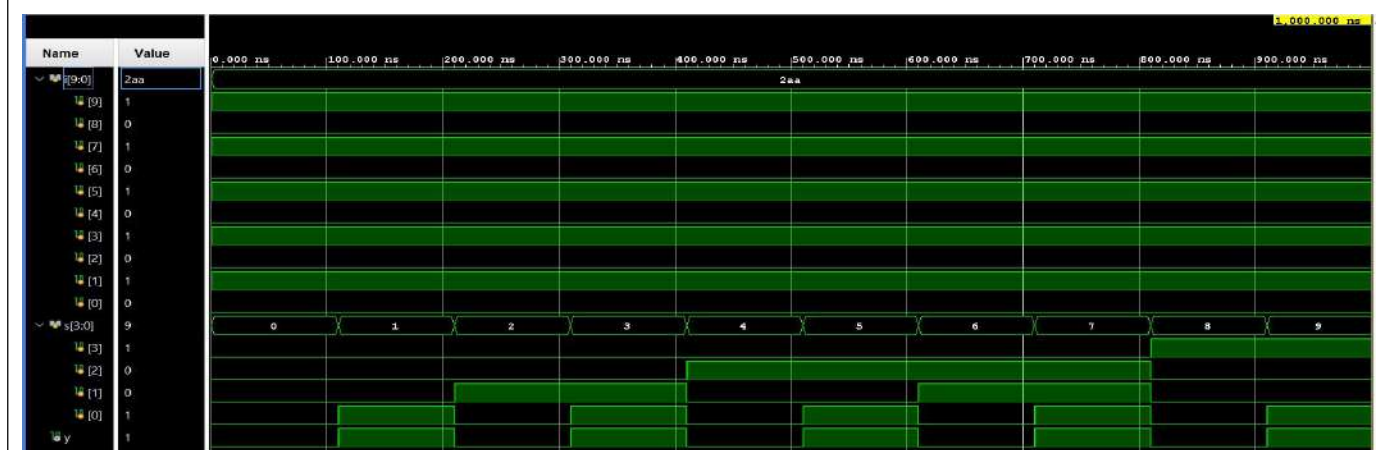
```

- Schematic



59. 10x1 MUX using 2x1 MUX only

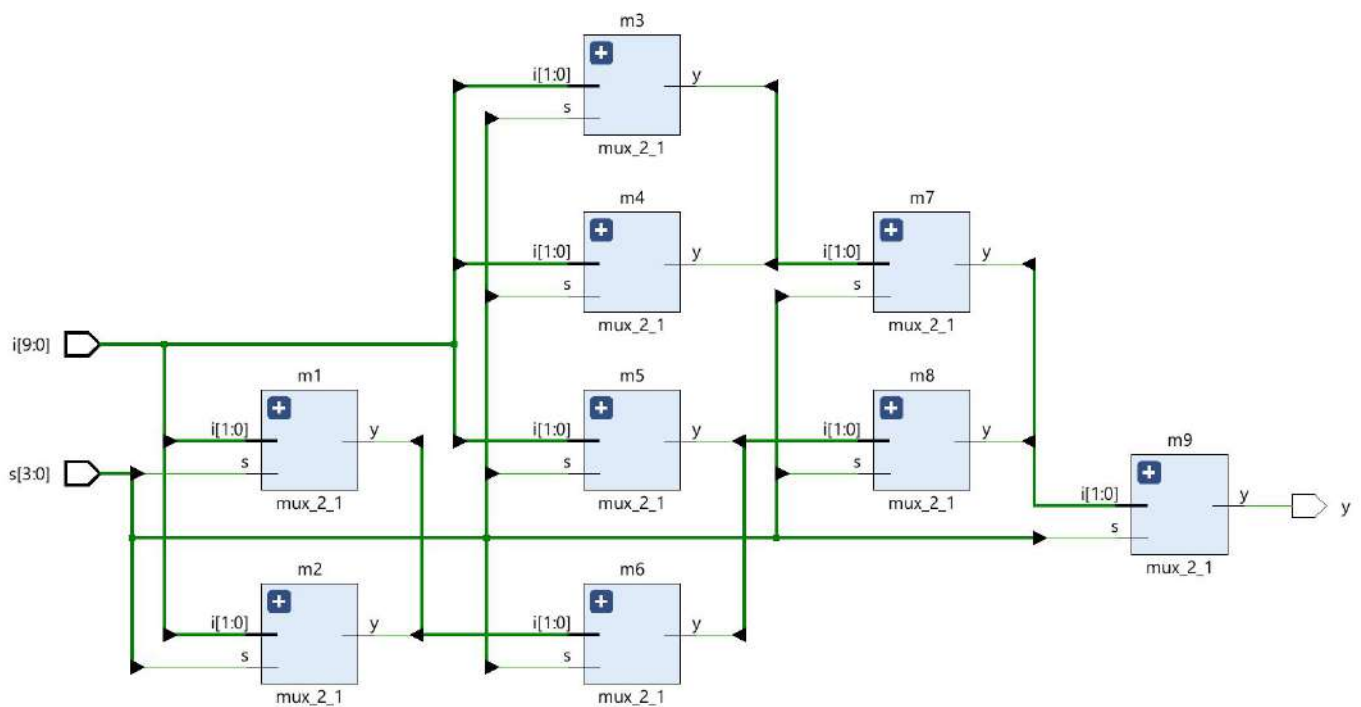
● Design code of 2x1 MUX	● Design code of 10x1 MUX	● Testbench
<pre> module mux_2_1(i,s,y); input [1:0]i; input s; output y; assign y=(i[0]&(~s)) (i[1]&s); endmodule </pre>	<pre> //Structural Modeling Style `include "mux_2_1.v" module mux_10_1_structural(i, s, y); input [9:0] i; // 10 inputs input [3:0] s; // 4-bit select (to represent 0-9) output y; wire w1, w2, w3, w4, w5,w6, w7, w8, w9; mux_2_1 m1(i[1:0], s[0], w1); mux_2_1 m2(i[3:2], s[0], w2); mux_2_1 m3(i[5:4], s[0], w3); mux_2_1 m4(i[7:6], s[0], w4); mux_2_1 m5(i[9:8], s[0], w5); mux_2_1 m6({w2, w1}, s[1], w6); mux_2_1 m7({w4, w3}, s[1], w7); mux_2_1 m8({w5, w6}, s[2], w8); mux_2_1 m9({w7, w8}, s[3], y); endmodule </pre>	<pre> module mux_10_1_structural_tb; reg [9:0] i; reg [3:0] s; wire y; // Instantiate the DUT mux_10_1_structural dut (i,s,y); initial begin \$display("Time\t i\t s\t y"); \$monitor("%0t\t %b\t %b\t %b", \$time, i, s, y); // Input pattern i = 10'b1010101010; s = 4'd0; #10; #100 s = 4'd1; #100 s = 4'd2; #100 s = 4'd3; #100 s = 4'd4; #100 s = 4'd5; #100 s = 4'd6; #100 s = 4'd7; #100 s = 4'd8; #100 s = 4'd9; #100 \$finish; end endmodule </pre>

● Waveform

● Output

Time	i	s	y
0	1010101010	0000	0
110000	1010101010	0001	1
210000	1010101010	0010	0
310000	1010101010	0011	1
410000	1010101010	0100	0
510000	1010101010	0101	1
610000	1010101010	0110	0
710000	1010101010	0111	1
810000	1010101010	1000	0
910000	1010101010	1001	1

● Schematic



60. 3 input XNOR gate using 2x1 MUX

● Design code of 2x1 mux	● Design code of XNOR gate	● Testbench
<pre> module v_2_1mux(i,s,y); input [0:1]i; input s; output y; assign y=(i[0]&(~s)) (i[1]&s); endmodule </pre>	<pre> `include "v_2_1_mux.v" module xnor_gate_using_mux_2_1(a,b,c,y); input a,b,c; output y; wire [0:1]w; v_2_1mux m1({~c,c},b,w[0]); v_2_1mux m2({c,~c},b,w[1]); v_2_1mux m3(w,a,y); endmodule </pre>	<pre> module xnor_gate_using_mux_2_1_tb; reg a,b,c; wire y; xnor_gate_using_mux_2_1 dut(a,b,c,y); initial begin a=1'b0; b=1'b0; c=1'b0; #100 a=1'b0; b=1'b0; c=1'b1; #100 a=1'b0; b=1'b1; c=1'b0; #100 a=1'b0; b=1'b1; c=1'b1; #100 a=1'b1; b=1'b0; c=1'b0; #100 a=1'b1; b=1'b0; c=1'b1; #100 a=1'b1; b=1'b1; c=1'b0; #100 a=1'b1; b=1'b1; c=1'b1; #100 a=1'b0; b=1'b1; c=1'b0; end initial begin \$monitor("Simtime=%0t, a=%b, b=%b, c=%b, y=%b", \$time,a,b,c,y); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,a,b,c,y); end endmodule </pre>

● Waveform



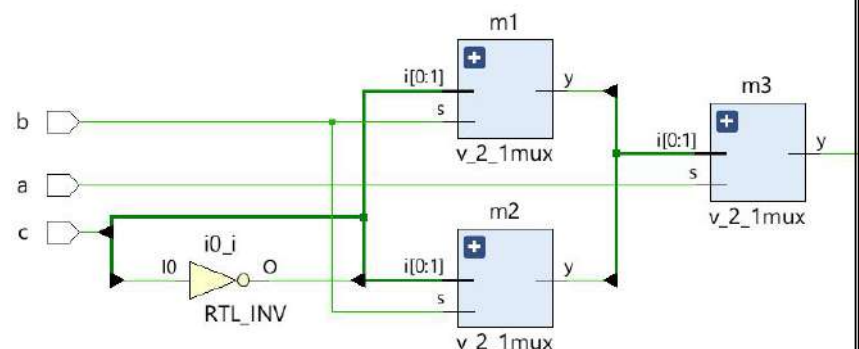
● Output

```

Simtime=0, a=0, b=0, c=0, y=1
Simtime=100000, a=0, b=0, c=1, y=0
Simtime=200000, a=0, b=1, c=0, y=0
Simtime=300000, a=0, b=1, c=1, y=1
Simtime=400000, a=1, b=0, c=0, y=0
Simtime=500000, a=1, b=0, c=1, y=1
Simtime=600000, a=1, b=1, c=0, y=1
Simtime=700000, a=1, b=1, c=1, y=0
Simtime=800000, a=0, b=1, c=0, y=0

```

● Schematic



61. 1x2 De-MUX

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module demux_1x2(din, sel, y0, y1); input wire din, sel; output wire y0, y1; assign y0 = (~sel) & din; assign y1 = sel & din; endmodule //Behavioural Modeling Style i. Using if-else module demux_1x2_behavioral_if(din, sel, y0,y1); input din, sel; output reg y0, y1; always @(din, sel) begin if (sel == 0) begin y0 = din; y1 = 0; end else begin y0 = 0; y1 = din; end end endmodule ii. Using case statement module demux_1x2_behavioral_case(din, sel, y0, y1); input din, sel; output reg y0, y1; always @(din, sel) begin case (sel)</pre>	<pre>module demux_1x2_tb; reg din; reg sel; wire y0, y1; // Instantiate the design demux_1x2 uut (din, sel, y0, y1); initial begin \$display("Time\tSel\tDin\tY0\tY1"); \$monitor("%0t\t%b\t%b\t%b\t%b", \$time, sel, din, y0, y1); // Test all combinations din = 0; sel = 0; #100 din = 0; sel = 1; #100 din = 1; sel = 0; #100 din = 1; sel = 1; #100 \$finish; end endmodule</pre>

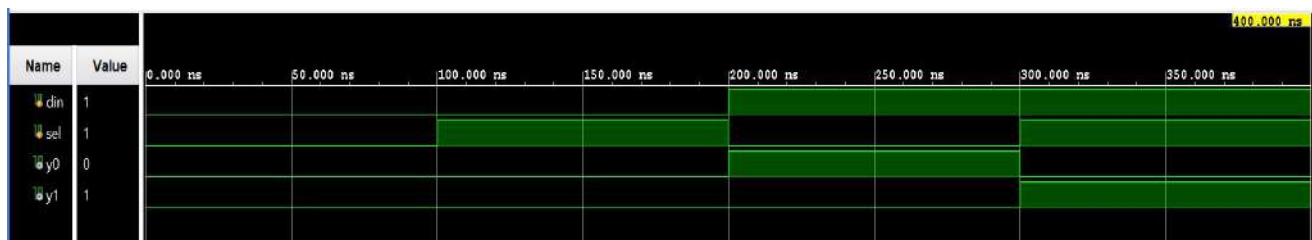
```

1'b0:
  begin
    y0 = din;
    y1 = 0;
  end
1'b1:
  begin
    y0 = 0;
    y1 = din;
  end
endcase
end

endmodule

```

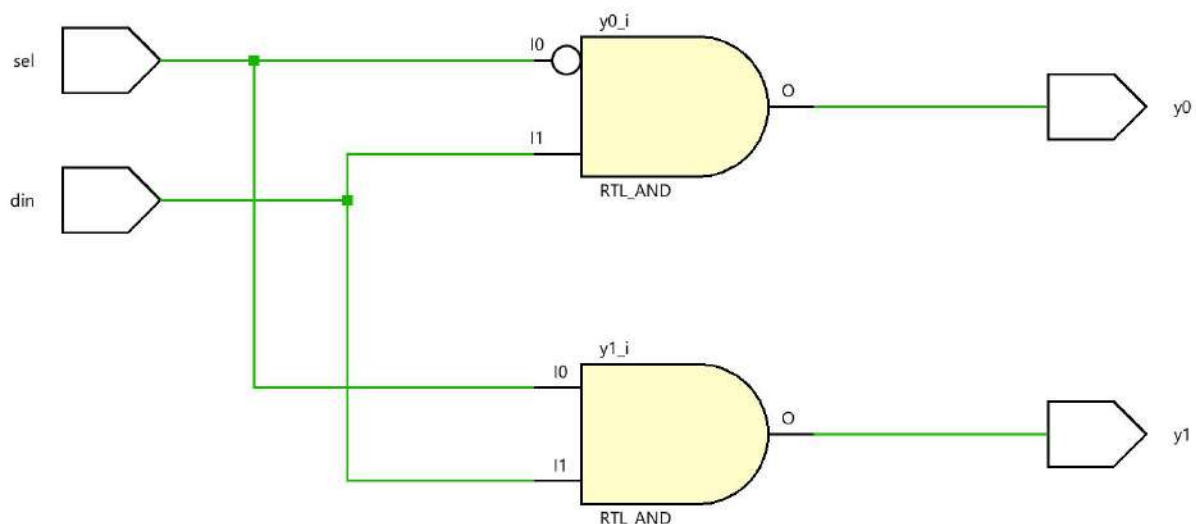
● Waveform



● Output

Time	Sel	Din	Y0	Y1
0	0	0	0	0
100000	1	0	0	0
200000	0	1	1	0
300000	1	1	0	1

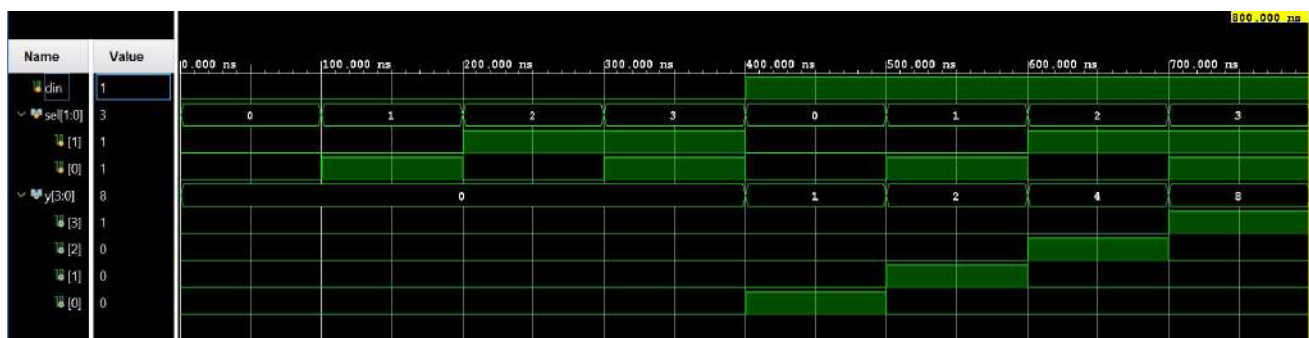
● Schematic



62. 1x4 De-MUX

● Design Code	● Testbench Code
<pre> //Dataflow Modeling Style module demux_1_4(din, sel, y); input din; input [1:0]sel; output [3:0]y; assign y[0] = (~sel[1]) & (~sel[0]) & din; assign y[1] = (~sel[1]) & (sel[0]) & din; assign y[2] = (sel[1]) & (~sel[0]) & din; assign y[3] = (sel[1]) & (sel[0]) & din; endmodule //Behavioural Modeling Style module demux_1x4_behavioral (din, sel, y); input din; input wire [1:0] sel; output reg [3:0]y; always @(din,sel) begin y[0] = 0; y[1] = 0; y[2] = 0; y[3] = 0; case (sel) 2'b00: y[0] = din; 2'b01: y[1] = din; 2'b10: y[2] = din; 2'b11: y[3] = din; endcase end endmodule //Using Ternary Operator module demux_1x4_ternary (din, sel, y); input wire din; input wire [1:0] sel, output wire y; assign y[0] = (~sel[1] & ~sel[0]) ? din : 1'b0; assign y[1] = (~sel[1] & sel[0]) ? din : 1'b0; assign y[2] = (sel[1] & ~sel[0]) ? din : 1'b0; assign y[3] = (sel[1] & sel[0]) ? din : 1'b0; endmodule </pre>	<pre> module demux_1_4_tb; reg din; reg [1:0] sel; wire [3:0] y; // Instantiate the design demux_1_4 uut (din, sel, y); initial begin \$display("Time\tSel\tDin\tY"); \$monitor("%0t\t%02b\t%b\t%04b\t", \$time, sel, din, y); // Test all combinations of sel with din = 0 and din = 1 din = 0; sel = 2'b00; #100 din = 0; sel = 2'b01; #100 din = 0; sel = 2'b10; #100 din = 0; sel = 2'b11; #100 din = 1; sel = 2'b00; #100 din = 1; sel = 2'b01; #100 din = 1; sel = 2'b10; #100 din = 1; sel = 2'b11; #100 \$finish; end endmodule </pre>

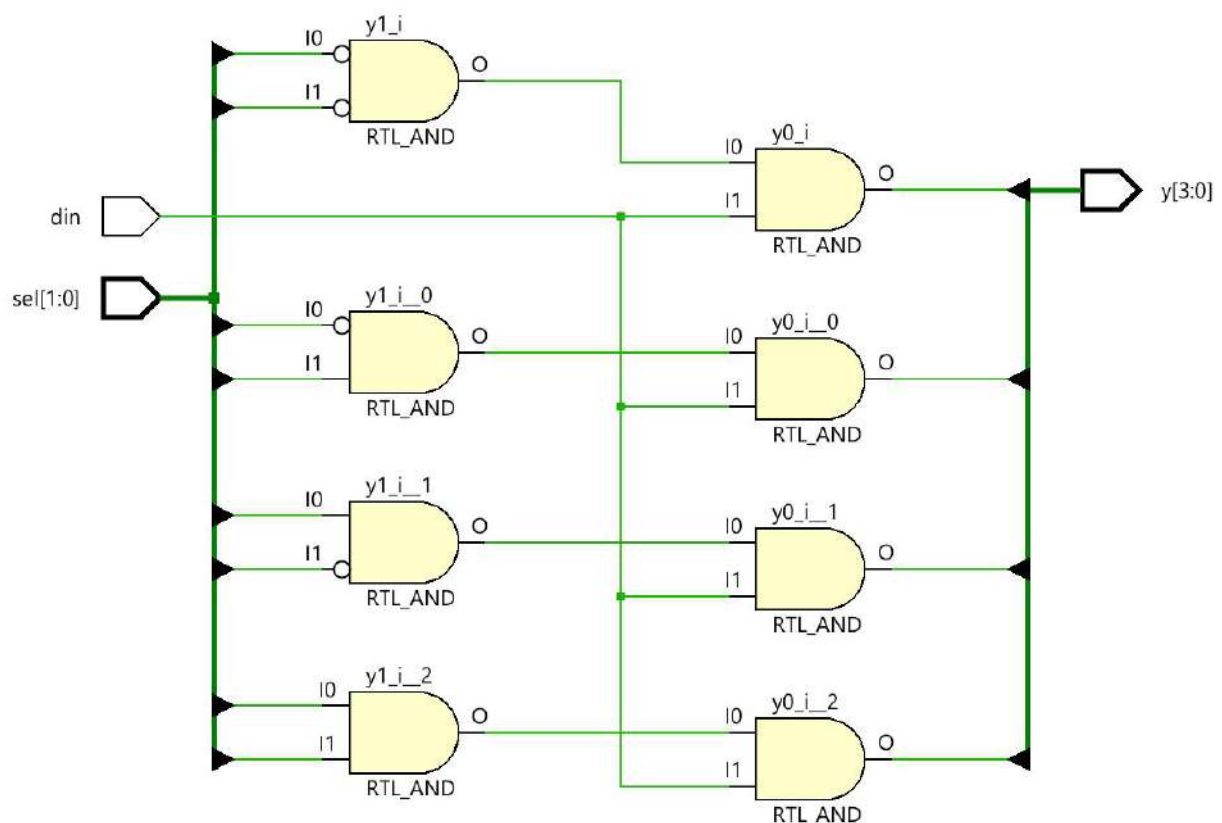
● Waveform



● Output

Time	Sel	Din	Y
0	00	0	0000
100000	01	0	0000
200000	10	0	0000
300000	11	0	0000
400000	00	1	0001
500000	01	1	0010
600000	10	1	0100
700000	11	1	1000

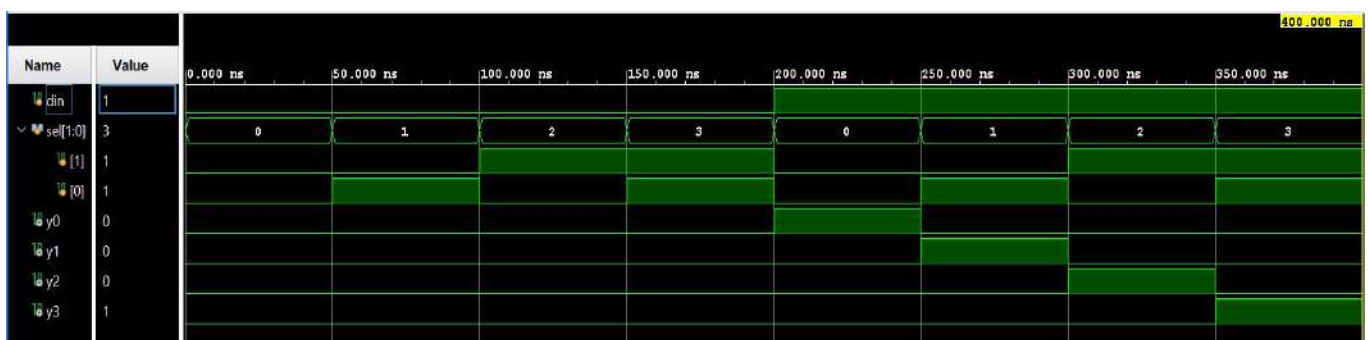
● Schematic



63. 1x4 De-MUX using 1x2 De-MUX

● Design code of 1x2 De-MUX	● Design code of 2x4 De-MUX	● Testbench
<pre> module demux_1x2(din, sel, y0, y1); input wire din, sel; output wire y0, y1; assign y0 = (~sel) & din; assign y1 = sel & din; endmodule </pre>	<pre> `include "demux_1x2.v" module demux_1x4_using_1x2(din, sel, y0, y1, y2, y3); input wire din; input wire [1:0] sel; output wire y0, y1, y2, y3; wire w0, w1; // First stage: split based on sel[1] demux_1x2 d1 (din, sel[1], w0, w1); // Second stage: each w0 and w1 go into another demux split by sel[0] demux_1x2 d2 (w0, sel[0], y0, y1); demux_1x2 d3 (w1, sel[0], y2, y3); endmodule </pre>	<pre> module demux_1x4_using_1x2_tb; reg din; reg [1:0] sel; wire y0, y1, y2, y3; // Instantiate the design demux_1x4_using_1x2 uut (din, sel, y0, y1, y2, y3); initial begin \$display("Time\tSel\tDin\tY0\tY1\tY 2\tY3"); \$monitor("%0t\t%b\t%b\t%b\t%b\t% b\t%b", \$time, sel, din, y0, y1, y2, y3); // Test all combinations din = 0; sel = 2'b00; #50; din = 0; sel = 2'b01; #50; din = 0; sel = 2'b10; #50; din = 0; sel = 2'b11; #50; din = 1; sel = 2'b00; #50; din = 1; sel = 2'b01; #50; din = 1; sel = 2'b10; #50; din = 1; sel = 2'b11; #50; \$finish; end endmodule </pre>

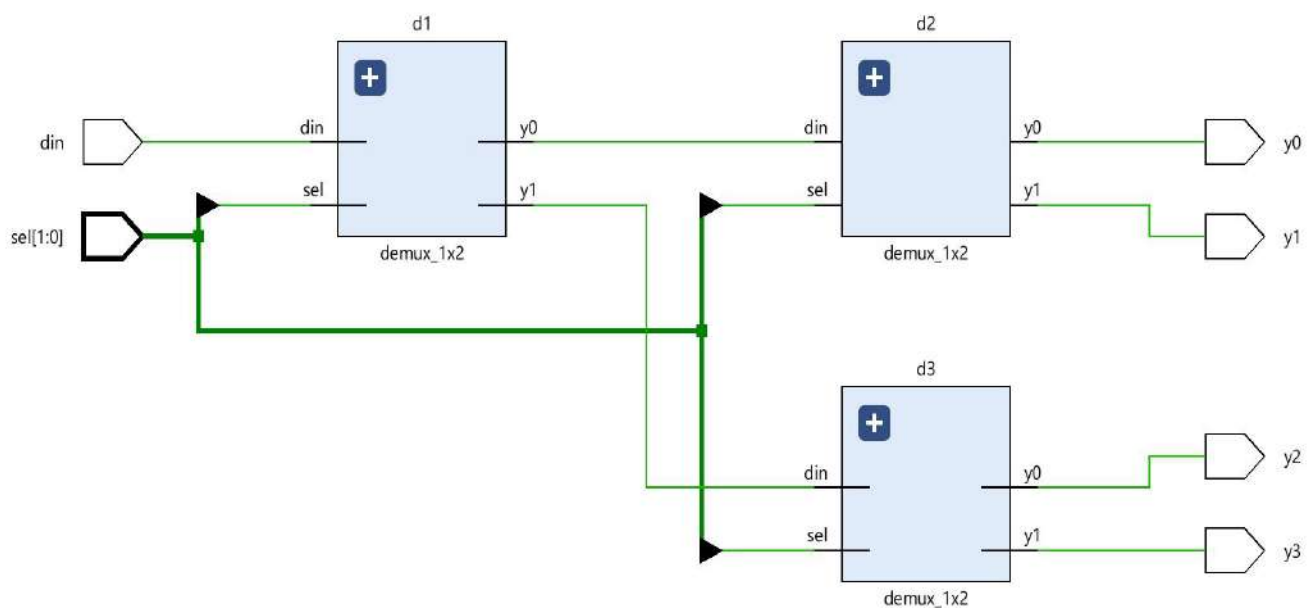
● Waveform



● Output

Time	Sel	Din	Y0	Y1	Y2	Y3
0	00	0	0	0	0	0
50000	01	0	0	0	0	0
100000	10	0	0	0	0	0
150000	11	0	0	0	0	0
200000	00	1	1	0	0	0
250000	01	1	0	1	0	0
300000	10	1	0	0	1	0
350000	11	1	0	0	0	1

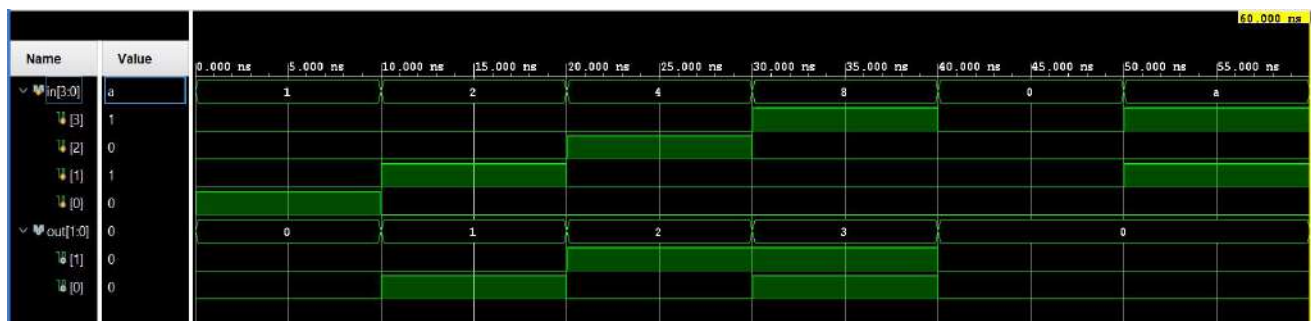
● Schematic



64. Normal 4:2 Encoder

● Design Code	● Testbench Code
<pre> //Dataflow Modeling Style module encoder_4to2_dataflow (in, out); input wire [3:0] in; output wire [1:0] out; assign out[1] = in[2] in[3]; assign out[0] = in[1] in[3]; endmodule //Behavioural Modeling Style i. Using if-else module normal_encoder_4to2(in, out); input wire [3:0] in; output reg [1:0] out; always @(in) begin if (in == 4'b0001) out = 2'b00; else if (in == 4'b0010) out = 2'b01; else if (in == 4'b0100) out = 2'b10; else if (in == 4'b1000) out = 2'b11; else out = 2'b00; // undefined or default end endmodule ii. Using case statement module encoder_4to2_case (in, out); input wire [3:0] in; output reg [1:0] out; always @(in) begin case (in) 4'b0001: out = 2'b00; 4'b0010: out = 2'b01; 4'b0100: out = 2'b10; 4'b1000: out = 2'b11; default: out = 2'b00; // optional handling endcase end endmodule </pre>	<pre> module normal_encoder_4to2_tb; reg [3:0] in; wire [1:0] out; // Change the module name based on the encoder you're testing normal_encoder_4to2 uut (in, out); initial begin \$display("Time\tInput\tOutput"); \$monitor("%0t\t%b\t%b", \$time, in, out); in = 4'b0001; #10; in = 4'b0010; #10; in = 4'b0100; #10; in = 4'b1000; #10; in = 4'b0000; #10; in = 4'b1010; #10; // Invalid: multiple inputs \$finish; end endmodule </pre>

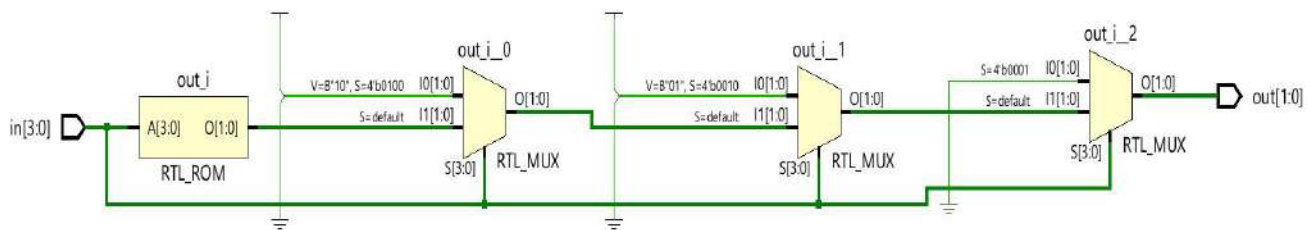
● Waveform



● Output

Time	Input	Output
0	0001	00
10000	0010	01
20000	0100	10
30000	1000	11
40000	0000	00
50000	1010	00

● Schematic



65. Priority 4:2 Encoder

● Design Code	● Testbench Code
<pre>//Behavioural Modeling Style Using case statement module priority_encoder_4to2(I,Y); input wire [3:0] I; output reg [1:0] Y; always @(I) begin casex (I) 4'b1xxx: Y = 2'b11; 4'b01xx: Y = 2'b10; 4'b001x: Y = 2'b01; 4'b0001: Y = 2'b00; default: Y = 2'b00; // No input active endcase end endmodule //Dataflow Modeling Style Using ternary operator module priority_encoder_4to2_dataflow (I,Y); input wire [3:0] I; output wire [1:0] Y; assign Y = (I[3]) ? 2'b11 : (I[2]) ? 2'b10 : (I[1]) ? 2'b01 : (I[0]) ? 2'b00 : 2'b00; endmodule</pre>	<pre>module priority_encoder_4to2_tb; reg [3:0] I; wire [1:0] Y; priority_encoder_4to2 dut(I,Y); initial begin \$display("Time\tInput\tOutput"); \$monitor("%0t\t%b\t%b", \$time, I, Y); I = 4'b0000; #10; I = 4'b0001; #10; I = 4'b0010; #10; I = 4'b0100; #10; I = 4'b1000; #10; I = 4'b1100; #10; I = 4'b0011; #10; I = 4'b1010; #10; #10 \$finish; end endmodule</pre>

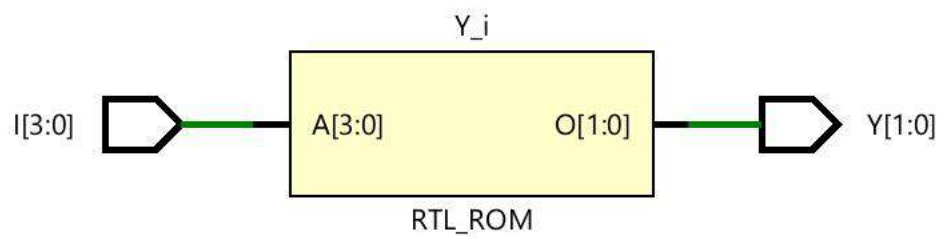
● Waveform



- **Output**

Time	Input	Output
0	0000	00
10000	0001	00
20000	0010	01
30000	0100	10
40000	1000	11
50000	1100	11
60000	0011	01
70000	1010	11

- **Schematic**



66. Priority 8:3 Encoder

● Design Code	● Testbench Code
<pre> //Dataflow Modeling Style Using ternary operator module priority_encoder_8to3(I,Y); input wire [7:0] I; output wire [2:0] Y; assign Y = (I[7]) ? 3'b111 : (I[6]) ? 3'b110 : (I[5]) ? 3'b101 : (I[4]) ? 3'b100 : (I[3]) ? 3'b011 : (I[2]) ? 3'b010 : (I[1]) ? 3'b001 : (I[0]) ? 3'b000 : 3'b000; endmodule //Behavioural Modeling Style Using case statement module priority_encoder_8to3_behav (I,Y); input wire [7:0] I; output reg [2:0] Y; always @(I) begin casex (I) 8'b1xxxxxxx: Y = 3'b111; 8'b01xxxxxx: Y = 3'b110; 8'b001xxxxx: Y = 3'b101; 8'b0001xxxx: Y = 3'b100; 8'b00001xxx: Y = 3'b011; 8'b000001xx: Y = 3'b010; 8'b0000001x: Y = 3'b001; 8'b00000001: Y = 3'b000; default: Y = 3'b000; endcase end endmodule </pre>	<pre> module priority_encoder_8to3_tb; reg [7:0] I; wire [2:0] Y; priority_encoder_8to3 dut(I,Y); initial begin \$display("Time\tInput\tOutput"); \$monitor("%0t\t%b\t%b", \$time, I, Y); I = 8'b00000000; #10; I = 8'b00000001; #10; I = 8'b00000010; #10; I = 8'b00000100; #10; I = 8'b00000110; #10; I = 8'b00001000; #10; I = 8'b00001111; #10; I = 8'b00010000; #10; I = 8'b00100000; #10; I = 8'b01000000; #10; I = 8'b01000101; #10; I = 8'b10000000; #10; I = 8'b11000000; #10; #10 \$finish; end endmodule </pre>

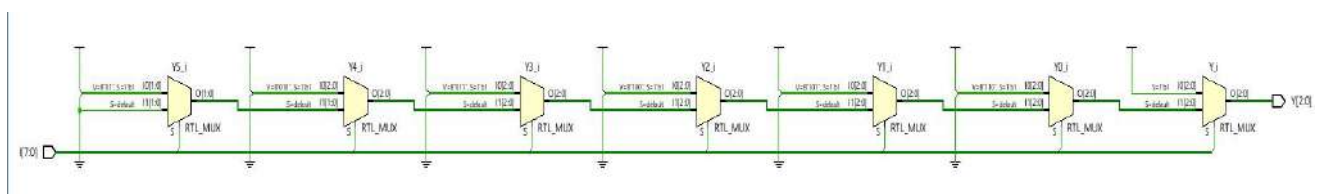
● Waveform



● Output

Time	Input	Output
0	00000000	000
10000	00000001	000
20000	00000010	001
30000	00000100	010
40000	00000110	010
50000	00001000	011
60000	00001111	011
70000	00010000	100
80000	00100000	101
90000	01000000	110
100000	01000101	110
110000	10000000	111
120000	11000000	111

● Schematic



67. Priority 8:3 Encoder using 4:2 Encoder

● Design Code	● Testbench Code
<pre> module priority_encoder_4to2 (input wire [3:0] I, output wire [1:0] Y, output wire valid); assign valid = I; assign Y = (I[3]) ? 2'b11 : (I[2]) ? 2'b10 : (I[1]) ? 2'b01 : (I[0]) ? 2'b00 : 2'b00; endmodule module priority_encoder_8to3_using_4to2(I, Y, valid); input wire [7:0] I; output wire [2:0] Y; output wire valid; wire [1:0] Y_hi, Y_lo; wire valid_hi, valid_lo; priority_encoder_4to2 U1 (I[7:4], Y_hi, valid_hi); priority_encoder_4to2 U2 (I[3:0], Y_lo, valid_lo); assign Y = valid_hi ? {1'b1, Y_hi} : {1'b0, Y_lo}; assign valid = valid_hi valid_lo; endmodule </pre>	<pre> module priority_encoder_8to3_using_4to2_tb; reg [7:0] I; wire [2:0] Y; wire valid; priority_encoder_8to3_using_4to2 dut(I, Y, valid); initial begin \$display("Time\tInput\tOutput\tValid"); \$monitor("%0t\t%b\t%b\t%b", \$time, I, Y, valid); I = 8'b00000000; #10; I = 8'b00000001; #10; I = 8'b00000010; #10; I = 8'b00000100; #10; I = 8'b00000110; #10; I = 8'b00001000; #10; I = 8'b00001111; #10; I = 8'b00010000; #10; I = 8'b00100000; #10; I = 8'b01000000; #10; I = 8'b01000101; #10; I = 8'b10000000; #10; I = 8'b11000000; #10; #10 \$finish; end endmodule </pre>

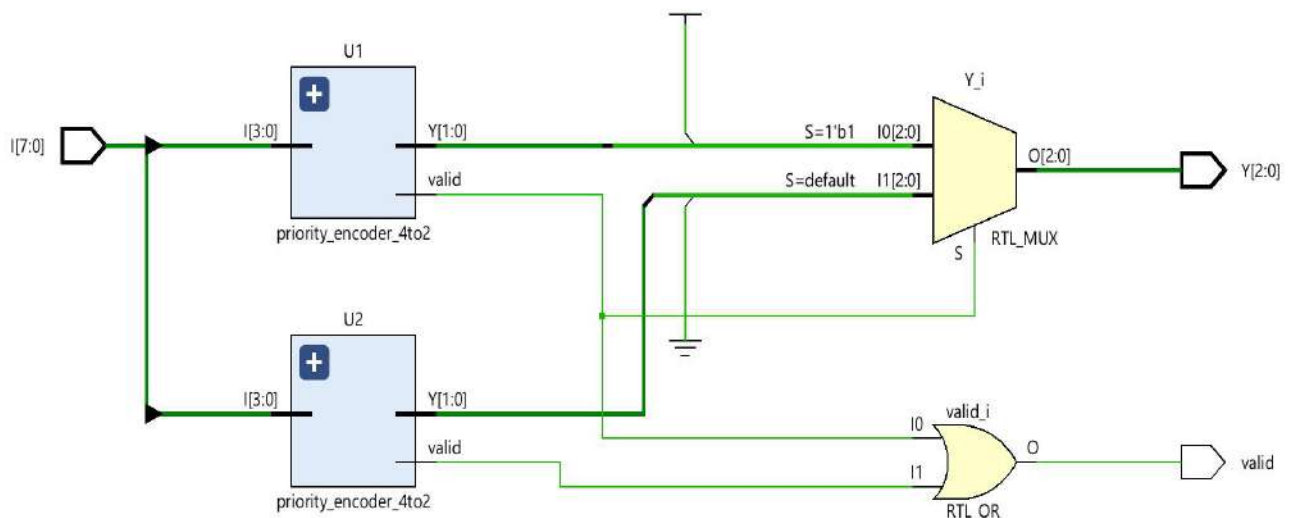
● Waveform



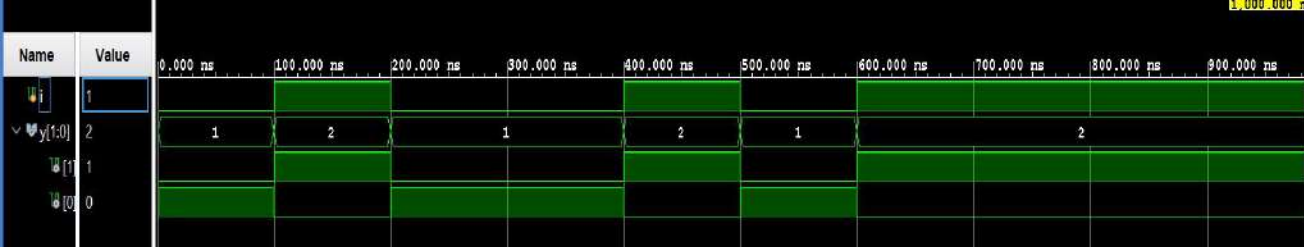
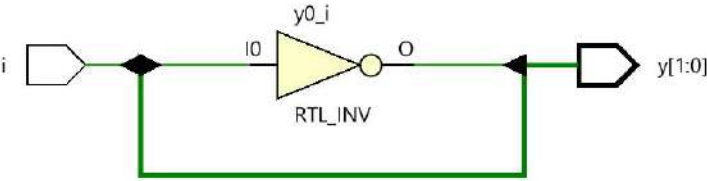
● Output

Time	Input	Output	Valid
0	00000000	000	0
10000	00000001	000	1
20000	00000010	001	1
30000	00000100	010	1
40000	00000110	010	1
50000	00001000	011	1
60000	00001111	011	1
70000	00010000	100	1
80000	00100000	101	1
90000	01000000	110	1
100000	01000101	110	1
110000	10000000	111	1
120000	11000000	111	1

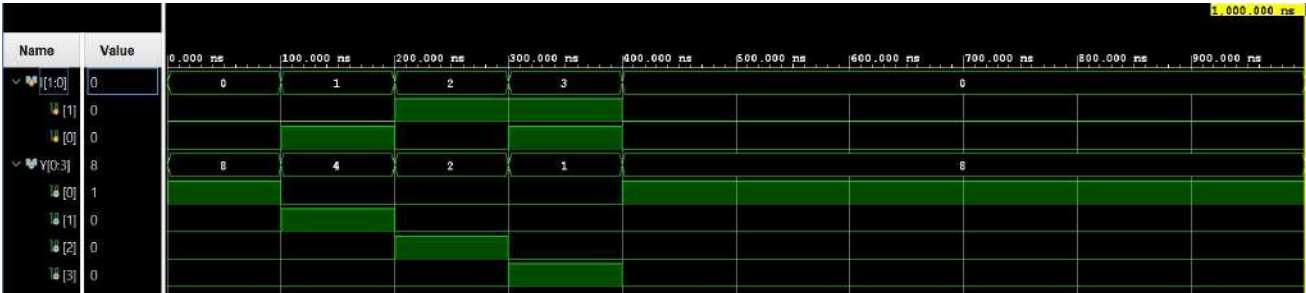
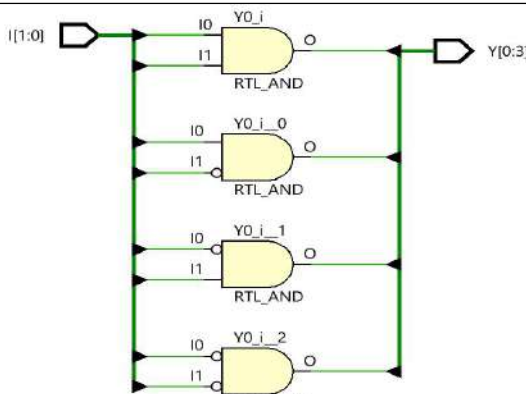
● Schematic



68. 1:2 Decoder

● Design Code	● Testbench Code
<pre>//Dataflow Modeling Style module dec_1_2(i,y); input i; output [1:0]y; assign y[0]=(~i); assign y[1]=(i); endmodule</pre>	<pre>module dec_1_2_tb; reg i; wire [1:0] y; dec_1_2 dut(i, y); initial begin i = 1'b0; #100 i = 1'b1; #100 i = 1'b0; #100 i = 1'b0; #100 i = 1'b1; #100 i = 1'b0; #100 i = 1'b1; #100 i = 1'b1; end initial begin \$monitor("Time = %0t i = %b y = %b", \$time, i, y); end initial begin \$dumpfile("dec_1_2.vcd"); \$dumpvars(0, dec_1_2_tb); end endmodule</pre>
● Waveform	
	
● Output <pre># run 1000ns Time = 0 i = 0 y = 01 Time = 100000 i = 1 y = 10 Time = 200000 i = 0 y = 01 Time = 400000 i = 1 y = 10 Time = 500000 i = 0 y = 01 Time = 600000 i = 1 y = 10</pre>	● Schematic 

69. 2:4 Decoder

● Design Code	● Testbench Code
<pre>//Behavioral Modeling Style module dec_2_4_beh(I,Y); input [1:0]I; output reg [0:3]Y; always@(I) begin Y[0]=((~I[1])&(~I[0])); Y[1]=((~I[1])&(I[0])); Y[2]=((I[1])&(~I[0])); Y[3]=((I[1])&(I[0])); end endmodule</pre>	<pre>module dec_2_4_beh_tb; reg [1:0] I; wire [0:3] Y; dec_2_4_beh dut(I,Y); initial begin I = 2'b00; #100 I = 2'b01; #100 I = 2'b10; #100 I = 2'b11; #100 I = 2'b00; end initial begin \$monitor("Time = %0t I = %b Y = %b", \$time, I, Y); end initial begin \$dumpfile("dec_2_4_beh.vcd"); \$dumpvars(0, dec_2_4_beh_tb); end endmodule</pre>
● Waveform	
	
● Output	● Schematic
<pre>Time = 0 I = 00 Y = 1000 Time = 100000 I = 01 Y = 0100 Time = 200000 I = 10 Y = 0010 Time = 300000 I = 11 Y = 0001 Time = 400000 I = 00 Y = 1000</pre>	

70. 2:4 Decoder using 1:2 Decoders

● Design code of 1:2 Decoder	● Design code of 2:4 Decoder	● Testbench
<pre> module dec_1_2(i,y); input i; output [1:0]y; assign y[0]=(~i); assign y[1]=(i); endmodule </pre>	<pre> `include "dec_1_2.v" module dec_2_4 (i,y); input [1:0] i; output [3:0] y; wire [1:0] w; wire [1:0] y_upper, y_lower; // First level: decode i[1] dec_1_2 d1(i[1], w); // Second level: decode i[0] with enable from w dec_1_2 d2(i[0], y_upper); // Will be used when w[1] = 1 dec_1_2 d3(i[0], y_lower); // Will be used when w[0] = 1 // Output masking with enables assign y[3] = w[1] & y_upper[1]; assign y[2] = w[1] & y_upper[0]; assign y[1] = w[0] & y_lower[1]; assign y[0] = w[0] & y_lower[0]; endmodule </pre>	<pre> module dec_2_4_tb; reg [1:0] i; wire [3:0] y; dec_2_4 dut(i, y); initial begin i = 2'b00; #100 i = 2'b01; #100 i = 2'b10; #100 i = 2'b11; #100 i = 2'b00; end initial begin \$monitor("Time = %0t i = %b y = %b", \$time, i, y); end initial begin \$dumpfile("dec_2_4.vcd"); \$dumpvars(0, dec_2_4_tb); end endmodule </pre>

- **Waveform**

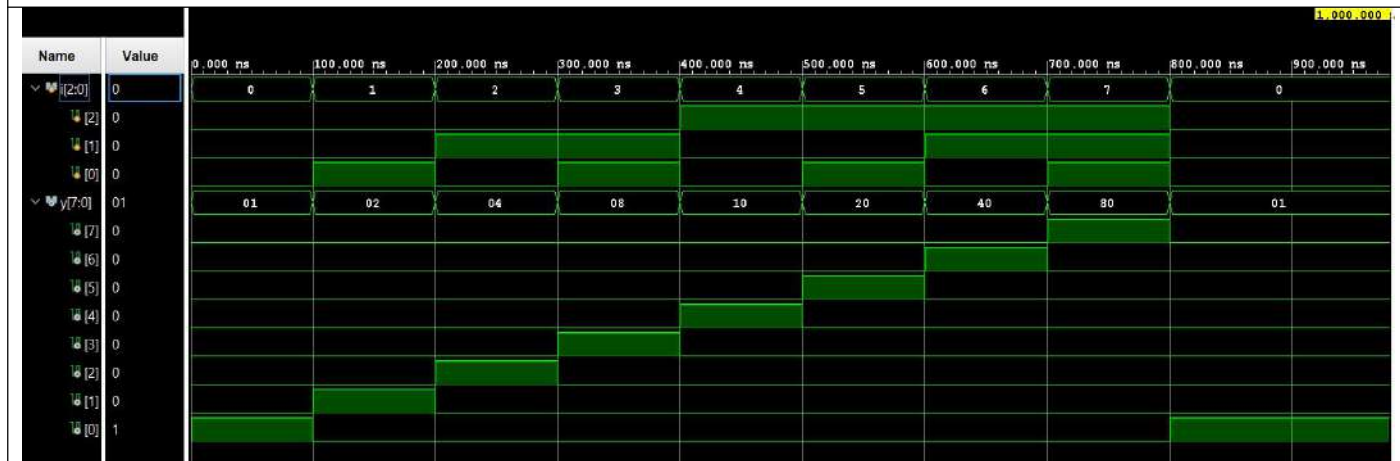


● Output <pre># run 1000ns Time = 0 i = 00 y = 0001 Time = 100000 i = 01 y = 0010 Time = 200000 i = 10 y = 0100 Time = 300000 i = 11 y = 1000 Time = 400000 i = 00 y = 0001</pre>	● Schematic
---	---------------------------

71. 3:8 Decoder using 2:4 and 1:2 Decoders

● Design code of 1:2 Decoder	● Design code of 2:4 Decoder
<pre> module dec_1_2(i,y); input i; output [1:0]y; assign y[0]=(~i); assign y[1]=(i); endmodule </pre>	<pre> `include "dec_1_2.v" module dec_2_4 (i,y); input [1:0] i; output [3:0] y; wire [1:0] w; wire [1:0] y_upper, y_lower; // First level: decode i[1] dec_1_2 d1(i[1], w); // Second level: decode i[0] with enable from w dec_1_2 d2(i[0], y_upper); // Will be used when w[1] = 1 dec_1_2 d3(i[0], y_lower); // Will be used when w[0] = 1 // Output masking with enables assign y[3] = w[1] & y_upper[1]; assign y[2] = w[1] & y_upper[0]; assign y[1] = w[0] & y_lower[1]; assign y[0] = w[0] & y_lower[0]; endmodule </pre>
● Design code of 3:8 Decoder	● Testbench
<pre> module dec_3_8(i,y); input [2:0] i; output [7:0] y; wire [1:0] w; // Output of 1:2 decoder (enable signals) wire [3:0] l; // Lower 4 outputs (from first 2:4 decoder) wire [3:0] u; // Upper 4 outputs (from second 2:4 decoder) // Decode MSB (i[2]) dec_1_2 d1(i[2], w); // i[2] --> w[1:0] // Decode LSBs (i[1:0]) with two 2:4 decoders dec_2_4 d2(i[1:0], l); // used when w[0] = 1 dec_2_4 d3(i[1:0], u); // used when w[1] = 1 assign y[0] = w[0] & l[0]; assign y[1] = w[0] & l[1]; assign y[2] = w[0] & l[2]; assign y[3] = w[0] & l[3]; assign y[4] = w[1] & u[0]; assign y[5] = w[1] & u[1]; assign y[6] = w[1] & u[2]; assign y[7] = w[1] & u[3]; endmodule </pre>	<pre> module dec_3_8_tb; reg [2:0] i; wire [7:0] y; // Instantiate the decoder module dec_3_8 dut(i, y); initial begin i = 3'b000; #100 i = 3'b001; #100 i = 3'b010; #100 i = 3'b011; #100 i = 3'b100; #100 i = 3'b101; #100 i = 3'b110; #100 i = 3'b111; #100 i = 3'b000; end initial begin \$monitor("Time = %0t i = %b y = %b", \$time, i, y); end initial begin \$dumpfile("dec_3_8.vcd"); \$dumpvars(0, dec_3_8); end endmodule </pre>

● Waveform



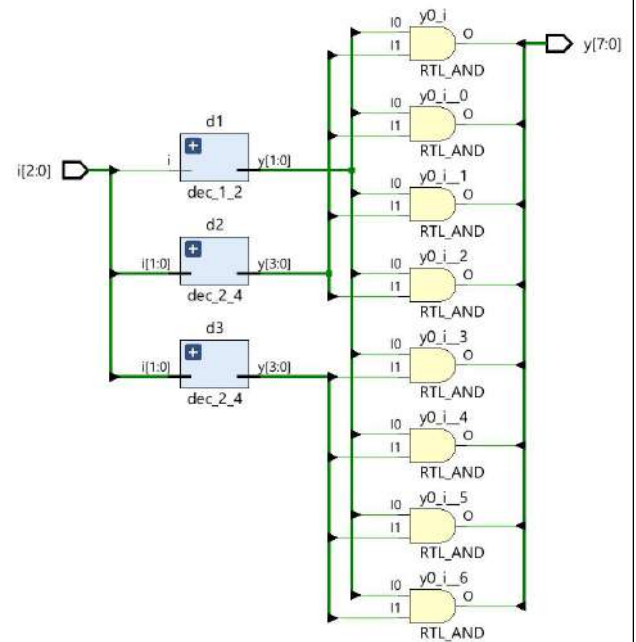
● Output

```

Time = 0 | i = 000 | y = 00000001
Time = 100000 | i = 001 | y = 00000010
Time = 200000 | i = 010 | y = 00000100
Time = 300000 | i = 011 | y = 00001000
Time = 400000 | i = 100 | y = 00010000
Time = 500000 | i = 101 | y = 00100000
Time = 600000 | i = 110 | y = 01000000
Time = 700000 | i = 111 | y = 10000000
Time = 800000 | i = 000 | y = 00000001

```

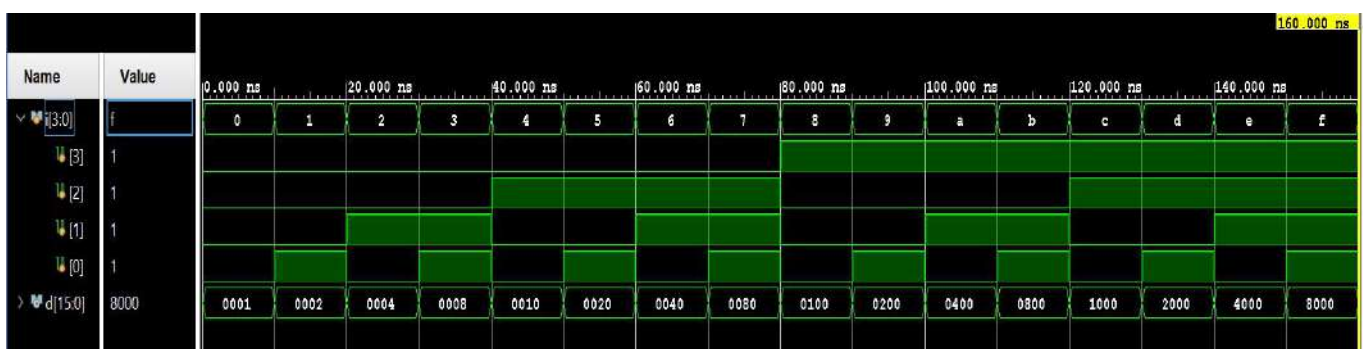
● Schematic



72. 4:16 Decoder using 2:4 Decoder

● Design code of 2:4 Decoder	● Design code of 4:16 Decoder	● Testbench
<pre> module dec_2_4(en,i,d); input en; input [1:0]i; output [3:0]d; assign d[0]= ~i[1]&~i[0]&en; assign d[1]= ~i[1]&i[0]&en; assign d[2]= i[1]&~i[0]&en; assign d[3]= i[1]&i[0]&en; endmodule </pre>	<pre> `include "dec_1_2.v" module dec_4_16(i,d); input [3:0]i; output [15:0]d; wire [3:0]w; dec_2_4 d1(w[0],i[1:0],d[3:0]); dec_2_4 d2(w[1],i[1:0],d[7:4]); dec_2_4 d3(w[2],i[1:0],d[11:8]); dec_2_4 d4(w[3],i[1:0],d[15:12]); dec_2_4 d5(1'b1,i[3:2],w); endmodule </pre>	<pre> module dec_4_16_tb; reg [3:0] i; wire [15:0] d; dec_4_16 uut (i,d); initial begin \$display("Time\tInput i\tOutput d"); \$monitor("%0t\t%4b\t%016b", \$time, i, d); i = 4'b0000; #10; i = 4'b0001; #10; i = 4'b0010; #10; i = 4'b0011; #10; i = 4'b0100; #10; i = 4'b0101; #10; i = 4'b0110; #10; i = 4'b0111; #10; i = 4'b1000; #10; i = 4'b1001; #10; i = 4'b1010; #10; i = 4'b1011; #10; i = 4'b1100; #10; i = 4'b1101; #10; i = 4'b1110; #10; i = 4'b1111; #10; \$finish; end initial begin \$dumpfile("dec_2_4.vcd"); \$dumpvars(0, dec_2_4_tb); end endmodule </pre>

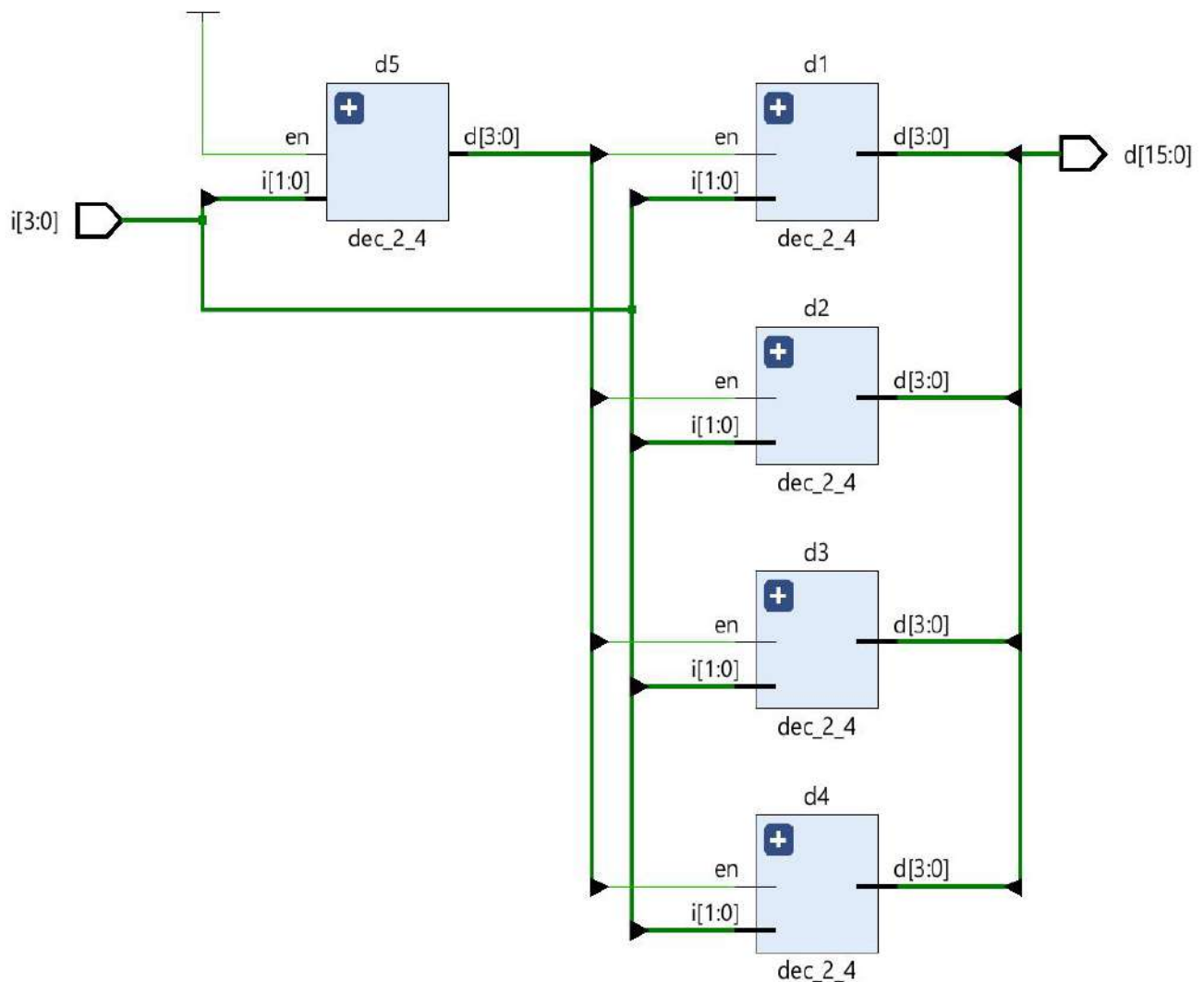
● Waveform



• Output

Time	Input i	Output d
0	0000	0000000000000001
10000	0001	0000000000000010
20000	0010	0000000000000100
30000	0011	0000000000001000
40000	0100	0000000000010000
50000	0101	0000000000100000
60000	0110	0000000001000000
70000	0111	0000000010000000
80000	1000	0000000100000000
90000	1001	0000001000000000
100000	1010	0000010000000000
110000	1011	0000100000000000
120000	1100	0001000000000000
130000	1101	0010000000000000
140000	1110	0100000000000000
150000	1111	1000000000000000

• Schematic



73. BCD to 7-Segment Display Decoder Module

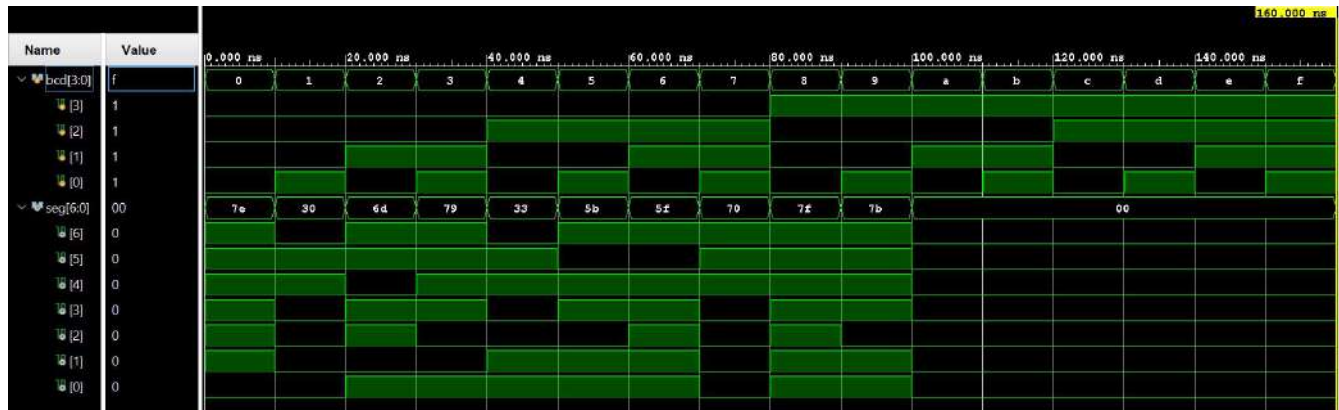
● Design Code	● Testbench Code																																				
<pre>module bcd_to_7segment(bcd, seg); input [3:0] bcd; output reg [6:0] seg; // seg[6:0] = {a,b,c,d,e,f,g} always @(bcd) begin case (bcd) 4'd0: seg = 7'b1111110; 4'd1: seg = 7'b0110000; 4'd2: seg = 7'b1101101; 4'd3: seg = 7'b1111001; 4'd4: seg = 7'b0110011; 4'd5: seg = 7'b1011011; 4'd6: seg = 7'b1011111; 4'd7: seg = 7'b1110000; 4'd8: seg = 7'b1111111; 4'd9: seg = 7'b1111011; default: seg = 7'b0000000; // Blank or OFF for invalid BCD endcase end endmodule</pre>	<pre>module bcd_to_7segment_tb; reg [3:0] bcd; wire [6:0] seg; bcd_to_7segment uut (bcd, seg); initial begin \$display("Time\tBCD\t7-Segment"); \$monitor("%0t\t%b\t%b", \$time, bcd, seg); bcd = 4'd0; #10 bcd = 4'd1; #10 bcd = 4'd2; #10 bcd = 4'd3; #10 bcd = 4'd4; #10 bcd = 4'd5; #10 bcd = 4'd6; #10 bcd = 4'd7; #10 bcd = 4'd8; #10 bcd = 4'd9; // Test invalid inputs #10 bcd = 4'd10; #10 bcd = 4'd11; #10 bcd = 4'd12; #10 bcd = 4'd13; #10 bcd = 4'd14; #10 bcd = 4'd15; #10 \$finish; end endmodule</pre>																																				
● Output																																					
<table><tr><th>Time</th><th>BCD</th><th>7-Segment</th></tr><tr><td>0</td><td>0000</td><td>1111110</td></tr><tr><td>10000</td><td>0001</td><td>0110000</td></tr><tr><td>20000</td><td>0010</td><td>1101101</td></tr><tr><td>30000</td><td>0011</td><td>1111001</td></tr><tr><td>40000</td><td>0100</td><td>0110011</td></tr><tr><td>50000</td><td>0101</td><td>1011011</td></tr><tr><td>60000</td><td>0110</td><td>1011111</td></tr><tr><td>70000</td><td>0111</td><td>1110000</td></tr><tr><td>80000</td><td>1000</td><td>1111111</td></tr><tr><td>90000</td><td>1001</td><td>1111011</td></tr><tr><td>100000</td><td>1010</td><td>0000000</td></tr></table>		Time	BCD	7-Segment	0	0000	1111110	10000	0001	0110000	20000	0010	1101101	30000	0011	1111001	40000	0100	0110011	50000	0101	1011011	60000	0110	1011111	70000	0111	1110000	80000	1000	1111111	90000	1001	1111011	100000	1010	0000000
Time	BCD	7-Segment																																			
0	0000	1111110																																			
10000	0001	0110000																																			
20000	0010	1101101																																			
30000	0011	1111001																																			
40000	0100	0110011																																			
50000	0101	1011011																																			
60000	0110	1011111																																			
70000	0111	1110000																																			
80000	1000	1111111																																			
90000	1001	1111011																																			
100000	1010	0000000																																			

```

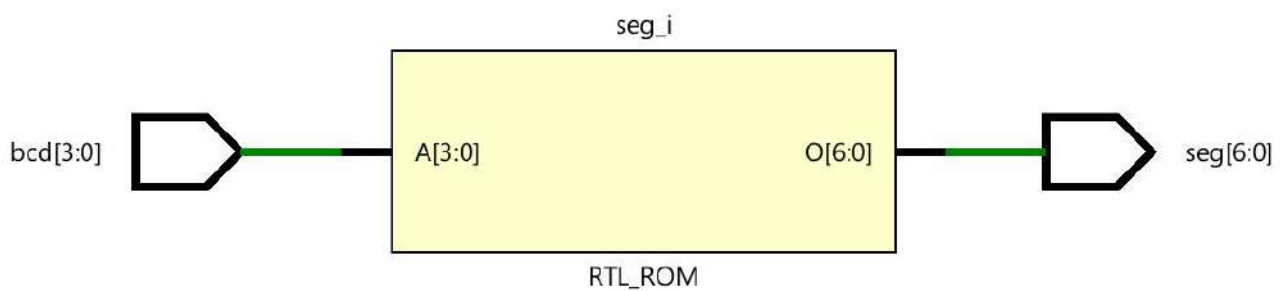
110000 1011 0000000
120000 1100 0000000
130000 1101 0000000
140000 1110 0000000
150000 1111 0000000

```

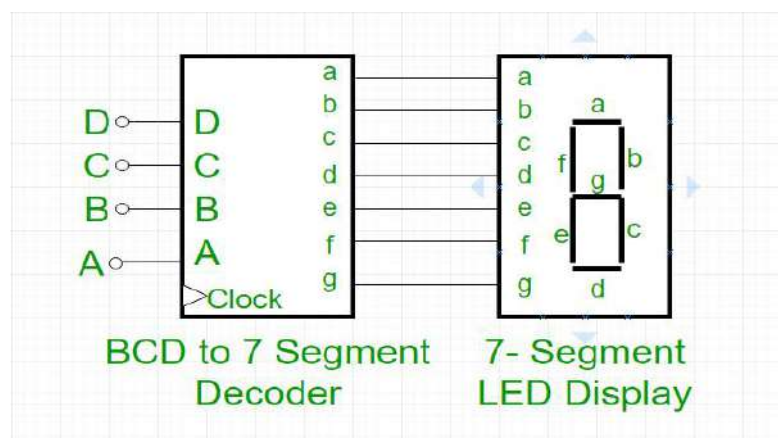
● Waveform



● Schematic



● Block Diagram

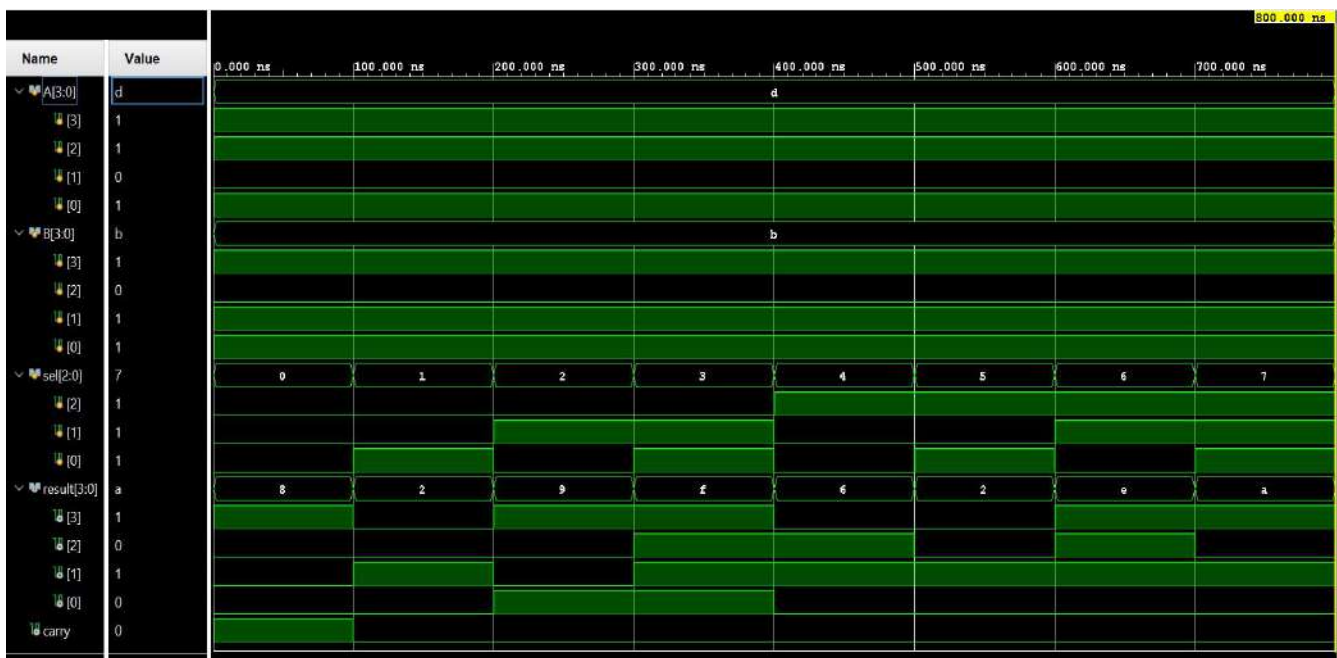


Note: This code assumes a common cathode 7-segment display (segment ON = 1).

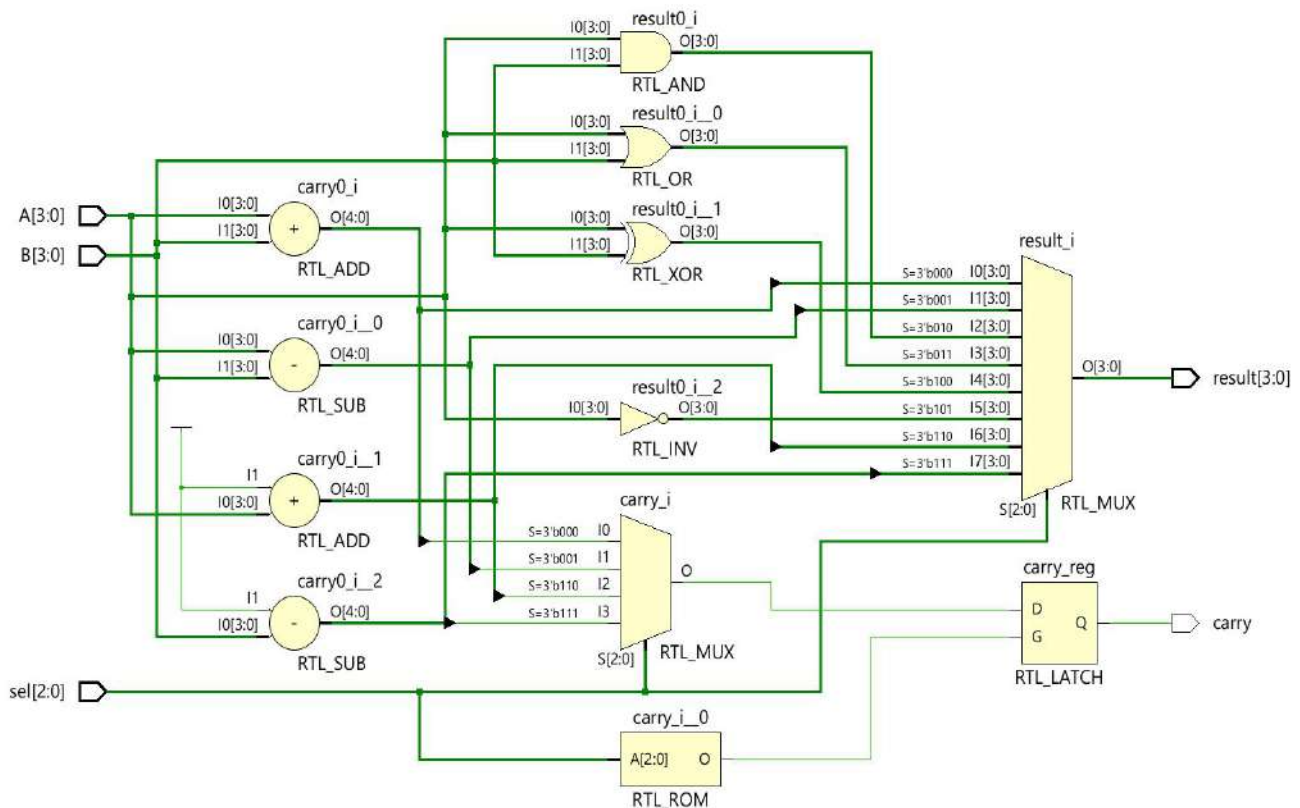
74. ALU

● Design Code	● Testbench Code																																																						
<pre>module alu (A, B, sel, result, carry); input [3:0] A, B; input [2:0] sel; output reg [3:0] result; output reg carry; initial carry = 0; always @(A, B, sel) begin case (sel) 3'b000: {carry, result} = A + B; // Addition 3'b001: {carry, result} = A - B; // Subtraction 3'b010: result = A & B; // AND 3'b011: result = A B; // OR 3'b100: result = A ^ B; // XOR 3'b101: result = ~A; // NOT A 3'b110: {carry, result} = A + 1; // Increment A 3'b111: {carry, result} = B - 1; // Decrement B default: result = 4'b0000; endcase end endmodule</pre>	<pre>module alu_tb; reg [3:0] A, B; reg [2:0] sel; wire [3:0] result; wire carry; alu uut (A, B, sel, result, carry); initial begin \$display("Time\tSel\tA\tB\tResult\tCarry"); \$monitor("%0t\t%b\t%h\t%h\t%h\t%b", \$time, sel, A, B, result, carry); A = 4'b1101; B = 4'b1011; sel = 3'b000; // Add #100 sel = 3'b001; // Sub #100 sel = 3'b010; // AND #100 sel = 3'b011; // OR #100 sel = 3'b100; // XOR #100 sel = 3'b101; // NOT A #100 sel = 3'b110; // INC A #100 sel = 3'b111; // DEC B #100 \$finish; end endmodule</pre>																																																						
● Output																																																							
<table><tr><th>Time</th><th>Sel</th><th>A</th><th>B</th><th>Result</th><th>Carry</th></tr><tr><td>0</td><td>000</td><td>d</td><td>b</td><td>8</td><td>1</td></tr><tr><td>100000</td><td>001</td><td>d</td><td>b</td><td>2</td><td>0</td></tr><tr><td>200000</td><td>010</td><td>d</td><td>b</td><td>9</td><td>0</td></tr><tr><td>300000</td><td>011</td><td>d</td><td>b</td><td>f</td><td>0</td></tr><tr><td>400000</td><td>100</td><td>d</td><td>b</td><td>6</td><td>0</td></tr><tr><td>500000</td><td>101</td><td>d</td><td>b</td><td>2</td><td>0</td></tr><tr><td>600000</td><td>110</td><td>d</td><td>b</td><td>e</td><td>0</td></tr><tr><td>700000</td><td>111</td><td>d</td><td>b</td><td>a</td><td>0</td></tr></table>		Time	Sel	A	B	Result	Carry	0	000	d	b	8	1	100000	001	d	b	2	0	200000	010	d	b	9	0	300000	011	d	b	f	0	400000	100	d	b	6	0	500000	101	d	b	2	0	600000	110	d	b	e	0	700000	111	d	b	a	0
Time	Sel	A	B	Result	Carry																																																		
0	000	d	b	8	1																																																		
100000	001	d	b	2	0																																																		
200000	010	d	b	9	0																																																		
300000	011	d	b	f	0																																																		
400000	100	d	b	6	0																																																		
500000	101	d	b	2	0																																																		
600000	110	d	b	e	0																																																		
700000	111	d	b	a	0																																																		

● Waveform



● Schematic



- **Design Code**

- **Testbench Code**

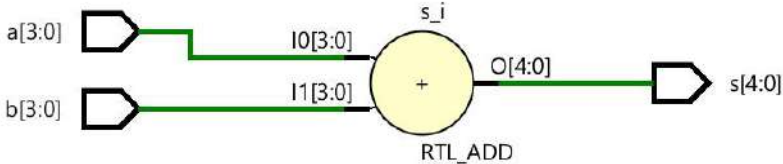
```

module generic_adder_tb;
reg [dut.N-1:0]a,b;
wire [dut.N:0]s;
//defparam dut.N=6; //2nd method modification of N
integer i;
generic_adder dut(a,b,s);//generic_adder#8 dut(a,b,s); ///3rd
method modification of N
initial
begin
for(i=0; i<(2**(2*dut.N)); i=i+1)
begin
{a,b}=i; #10;
end
end
initial
begin
$monitor("a=%d, b=%d, sum=%d",a,b,s);
end
endmodule

```

- **Waveform**



● Output	● Schematic
<pre> a= 0, b=12, sum=12 a= 0, b=13, sum=13 a= 0, b=14, sum=14 a= 0, b=15, sum=15 a= 1, b= 0, sum= 1 a= 1, b= 1, sum= 2 a= 1, b= 2, sum= 3 a= 1, b= 3, sum= 4 a= 1, b= 4, sum= 5 a= 1, b= 5, sum= 6 a= 1, b= 6, sum= 7 a= 1, b= 7, sum= 8 a= 1, b= 8, sum= 9 a= 1, b= 9, sum=10 a= 1, b=10, sum=11 a= 1, b=11, sum=12 a= 1, b=12, sum=13 a= 1, b=13, sum=14 a= 1, b=14, sum=15 a= 1, b=15, sum=16 a= 2, b= 0, sum= 2 a= 2, b= 1, sum= 3 a= 2, b= 2, sum= 4 </pre>	

NOTE: The three ways to override the value of ‘N’ are as follows

- i. Change the value of N in the generic adder design code.

Example: `module generic_adder #(parameter N=4)(a,b,s); //for N=4 bits`

- ii. Change the value of N in the generic adder testbench code by defining “defparam” function (next, after the declaration of output as wire type).

Example: `defparam dut.N=6; //for N=6 bits`

- iii. Declare, while instantiating the design code in the testbench

Example: `generic_adder#8 dut(a,b,s); //for N=8 bits`

76. Generic MUX

● Design Code	● Testbench Code
<pre> module generic_mux #(parameter N = 8)(I,s,y); input [N-1:0] I; input [\$clog2(N)-1:0] s; output y; assign y = I[s]; endmodule </pre>	<pre> module generic_mux_tb; // Parameter to control number of inputs parameter N = 8; reg [N-1:0] I; reg [\$clog2(N)-1:0] s; wire y; // Instantiate the MUX generic_mux #(N) dut (I, s, y); integer i; initial begin // Iterate over all select line values for (i = 0; i < N; i = i + 1) begin I = 8'b10101010; // Example pattern s = i; #100; end end initial begin \$monitor("Time=%0t I=%b s=%0d y=%b", \$time, I, s, y); end initial begin \$dumpfile("generic_mux.vcd"); \$dumpvars(0, generic_mux_tb); end endmodule </pre>

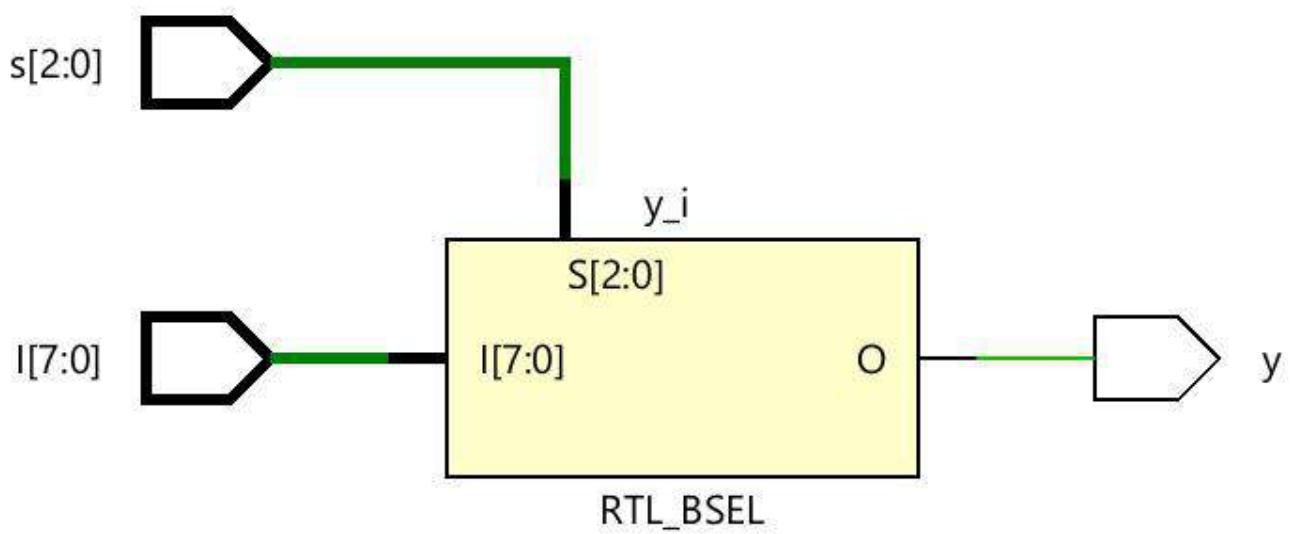
● Waveform



- **Output**

```
Time=0 | I=10101010 | s=0 | y=0
Time=100000 | I=10101010 | s=1 | y=1
Time=200000 | I=10101010 | s=2 | y=0
Time=300000 | I=10101010 | s=3 | y=1
Time=400000 | I=10101010 | s=4 | y=0
Time=500000 | I=10101010 | s=5 | y=1
Time=600000 | I=10101010 | s=6 | y=0
Time=700000 | I=10101010 | s=7 | y=1
```

- **Schematic**



77. D flip-flop

● Design Code	● Testbench Code
<pre> module d_ff(d, clk, rst, q); input d, clk, rst; output reg q; initial q=0; always @(posedge clk or posedge rst) if (rst) q <= 0; else q <= d; endmodule </pre>	<pre> module d_ff_tb; reg d, clk, rst; wire q; d_ff uut (d,clk,rst,q); initial clk = 0; always #5 clk = ~clk; initial begin \$display("Time\tD\tClk\tRst\tQ"); \$monitor("%0t\t%b\t%b\t%b\t%b", \$time, d, clk, rst, q); rst = 1; d = 0; #10; rst = 0; d = 0; #12; d = 1; #10; d = 0; #10; d = 1; #10; d = 1; #10; d = 0; #10; d = 1; #10; d = 1; #10; d = 0; #10; d = 1; #10; \$finish; end endmodule </pre>

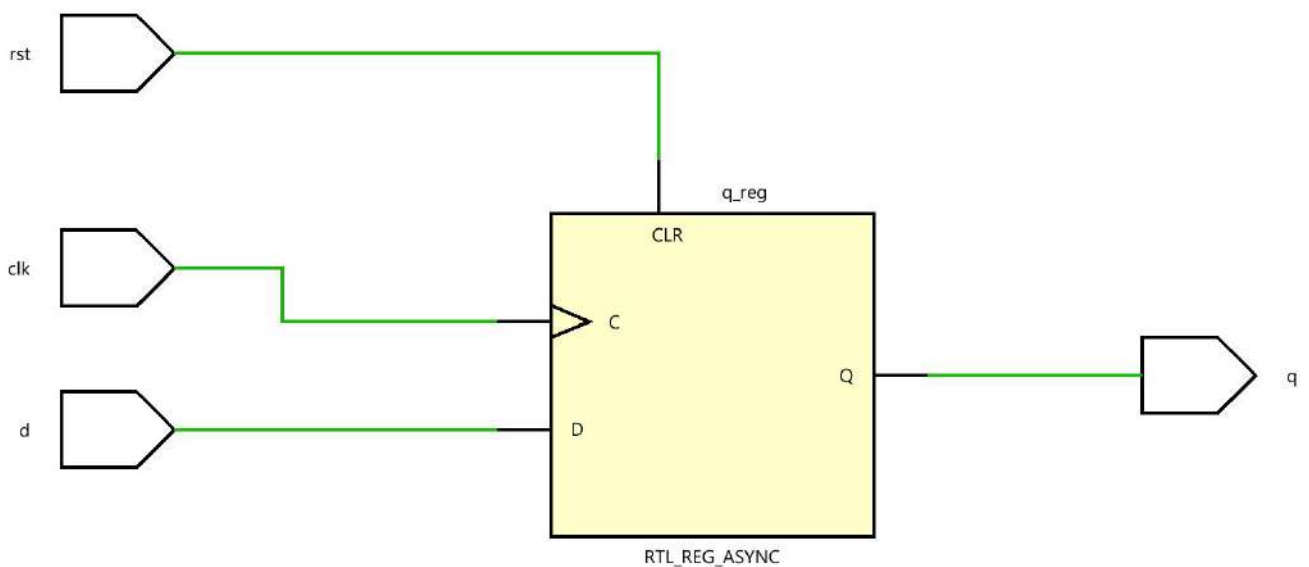
● Waveform



● Output

Time	D	Clk	Rst	Q
0	0	0	1	0
5000	0	1	1	0
10000	0	0	0	0
15000	0	1	0	0
20000	0	0	0	0
22000	1	0	0	0
25000	1	1	0	1
30000	1	0	0	1
32000	0	0	0	1
35000	0	1	0	0
40000	0	0	0	0
42000	1	0	0	0
45000	1	1	0	1
50000	1	0	0	1
55000	1	1	0	1
60000	1	0	0	1
62000	0	0	0	1
65000	0	1	0	0
70000	0	0	0	0
72000	1	0	0	0
75000	1	1	0	1
80000	1	0	0	1
85000	1	1	0	1
90000	1	0	0	1
92000	0	0	0	1
95000	0	1	0	0
100000	0	0	0	0
102000	1	0	0	0
105000	1	1	0	1
110000	1	0	0	1

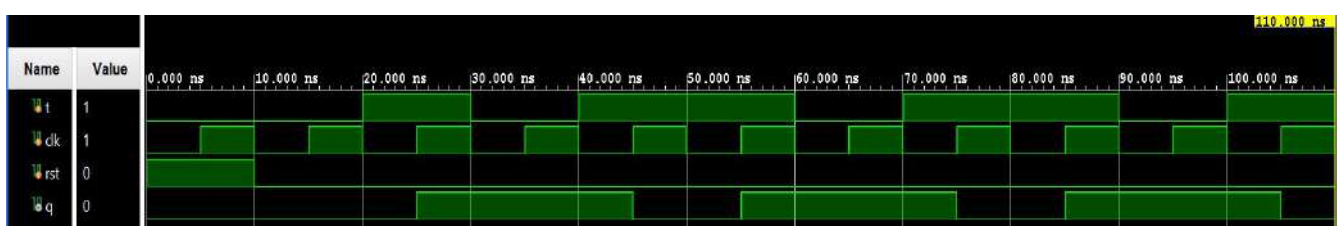
● Schematic



78. T flip-flop

● Design Code	● Testbench Code
<pre> module t_ff(t, clk, rst, q); input t, clk, rst; output reg q; initial q=0; always @(posedge clk or posedge rst) if (rst) q <= 0; else if (t) q <= ~q; endmodule </pre>	<pre> module t_ff_tb; reg t, clk, rst; wire q; t_ff uut (t, clk, rst, q); initial clk = 0; always #5 clk = ~clk; initial begin \$display("Time\tT\tClk\tRst\tQ"); \$monitor("%0t\t%b\t%b\t%b\t%b", \$time, t, clk, rst, q); rst = 1; t = 0; #10; rst = 0; t = 0; #10; t = 1; #10; t = 0; #10; t = 1; #10; t = 1; #10; t = 0; #10; t = 1; #20; t = 0; #10; t = 1; #10; \$finish; end endmodule </pre>

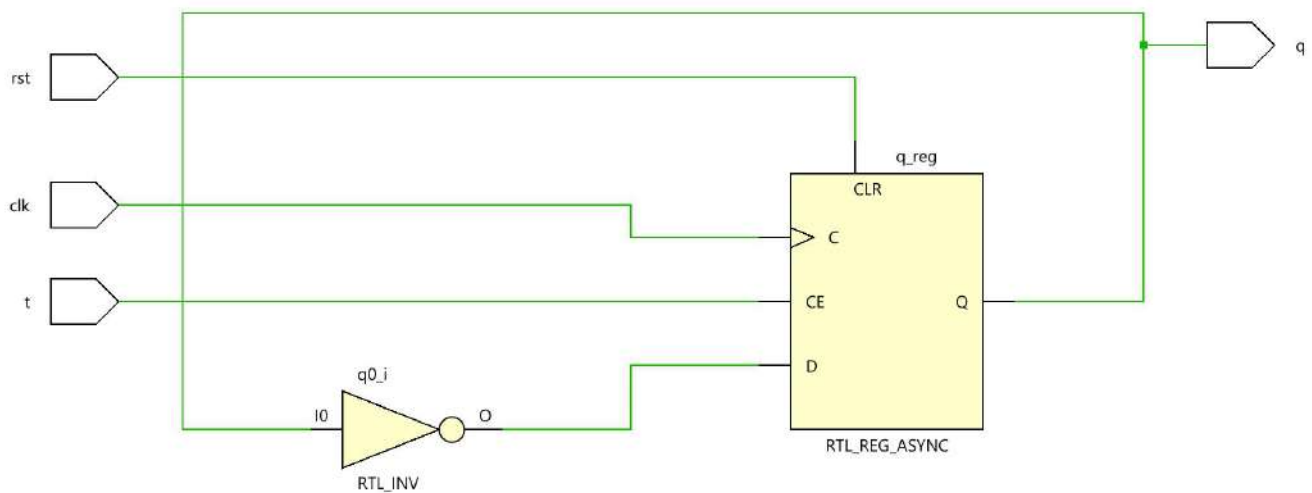
● Waveform



● Output

Time	T	Clk	Rst	Q
0	0	0	1	0
5000	0	1	1	0
10000	0	0	0	0
15000	0	1	0	0
20000	1	0	0	0
25000	1	1	0	1
30000	0	0	0	1
35000	0	1	0	1
40000	1	0	0	1
45000	1	1	0	0
50000	1	0	0	0
55000	1	1	0	1
60000	0	0	0	1
65000	0	1	0	1
70000	1	0	0	1
75000	1	1	0	0
80000	1	0	0	0
85000	1	1	0	1
90000	0	0	0	1
95000	0	1	0	1
100000	1	0	0	1
105000	1	1	0	0

● Schematic



79. SR flip-flop

● Design Code	● Testbench Code
<pre> module sr_ff(s,r,clk,rst,q); input s, r, clk, rst; output reg q; initial q=0; always @(posedge clk or posedge rst) if (rst) q <= 0; else if (s && !r) q <= 1; else if (!s && r) q <= 0; else if (!s && !r) q <= q; // s=1 and r=1 is invalid endmodule </pre>	<pre> module sr_ff_tb; reg s, r, clk, rst; wire q; sr_ff uut (s,r,clk,rst,q); initial clk = 0; always #5 clk = ~clk; initial begin \$display("Time\tS\tR\tClk\tRst\tQ"); \$monitor("%0t\t%b\t%b\t%b\t%b\t%b", \$time, s, r, clk, rst, q); rst = 1; s = 0; r = 0; #10; rst = 0; s = 1; r = 0; #10; s = 0; r = 0; #10; s = 0; r = 1; #10; s = 1; r = 1; #10; // invalid for SR FF s = 0; r = 0; #10; s = 1; r = 0; #10; s = 0; r = 0; #10; s = 0; r = 1; #10; s = 1; r = 1; #10; s = 0; r = 0; #10; \$finish; end endmodule </pre>

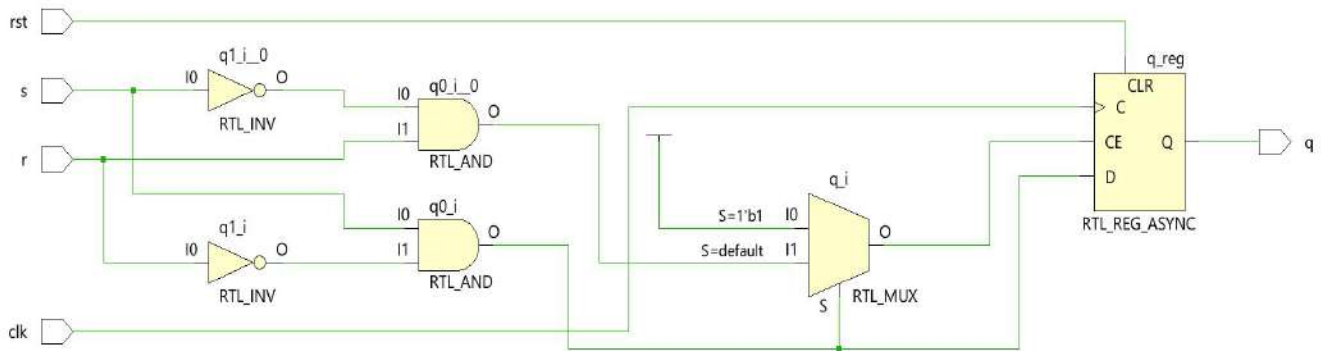
● Waveform



● Output

Time	S	R	Clk	Rst	Q
0	0	0	0	1	0
5000	0	0	1	1	0
10000	1	0	0	0	0
15000	1	0	1	0	1
20000	0	0	0	0	1
25000	0	0	1	0	1
30000	0	1	0	0	1
35000	0	1	1	0	0
40000	1	1	0	0	0
45000	1	1	1	0	0
50000	0	0	0	0	0
55000	0	0	1	0	0
60000	1	0	0	0	0
65000	1	0	1	0	1
70000	0	0	0	0	1
75000	0	0	1	0	1
80000	0	1	0	0	1
85000	0	1	1	0	0
90000	1	1	0	0	0
95000	1	1	1	0	0
100000	0	0	0	0	0
105000	0	0	1	0	0

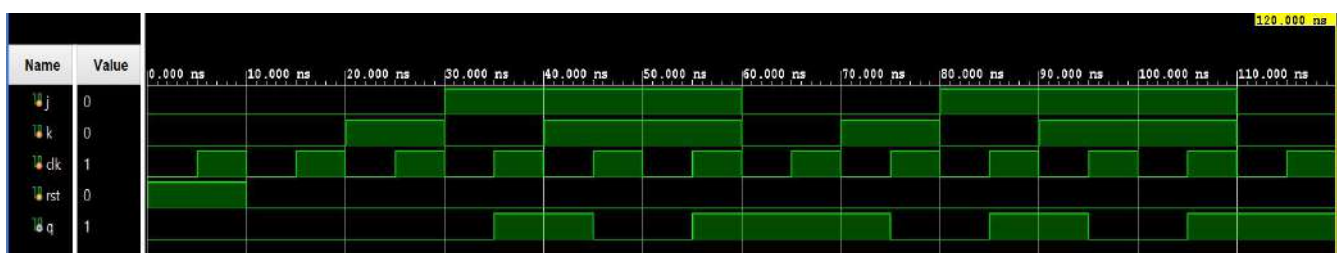
● Schematic



80. JK flip-flop

● Design Code	● Testbench Code
<pre> module jk_ff(j,k,clk,rst,q); input j, k, clk, rst; output reg q; initial q=0; always @(posedge clk or posedge rst) if (rst) q <= 0; else case ({j, k}) 2'b00: q <= q; 2'b01: q <= 0; 2'b10: q <= 1; 2'b11: q <= ~q; endcase endmodule </pre>	<pre> module jk_ff_tb; reg j, k, clk, rst; wire q; jk_ff uut (j,k,clk,rst,q); initial clk = 0; always #5 clk = ~clk; initial begin \$display("Time\tJ\tK\tClk\tRst\tQ"); \$monitor("%0t\t%b\t%b\t%b\t%b\t%b", \$time, j, k, clk, rst, q); rst = 1; j = 0; k = 0; #10; rst = 0; j = 0; k = 0; #10; j = 0; k = 1; #10; j = 1; k = 0; #10; j = 1; k = 1; #10; j = 1; k = 1; #10; j = 0; k = 0; #10; j = 0; k = 1; #10; j = 1; k = 0; #10; j = 1; k = 1; #10; j = 1; k = 1; #10; j = 0; k = 0; #10; \$finish; end endmodule </pre>

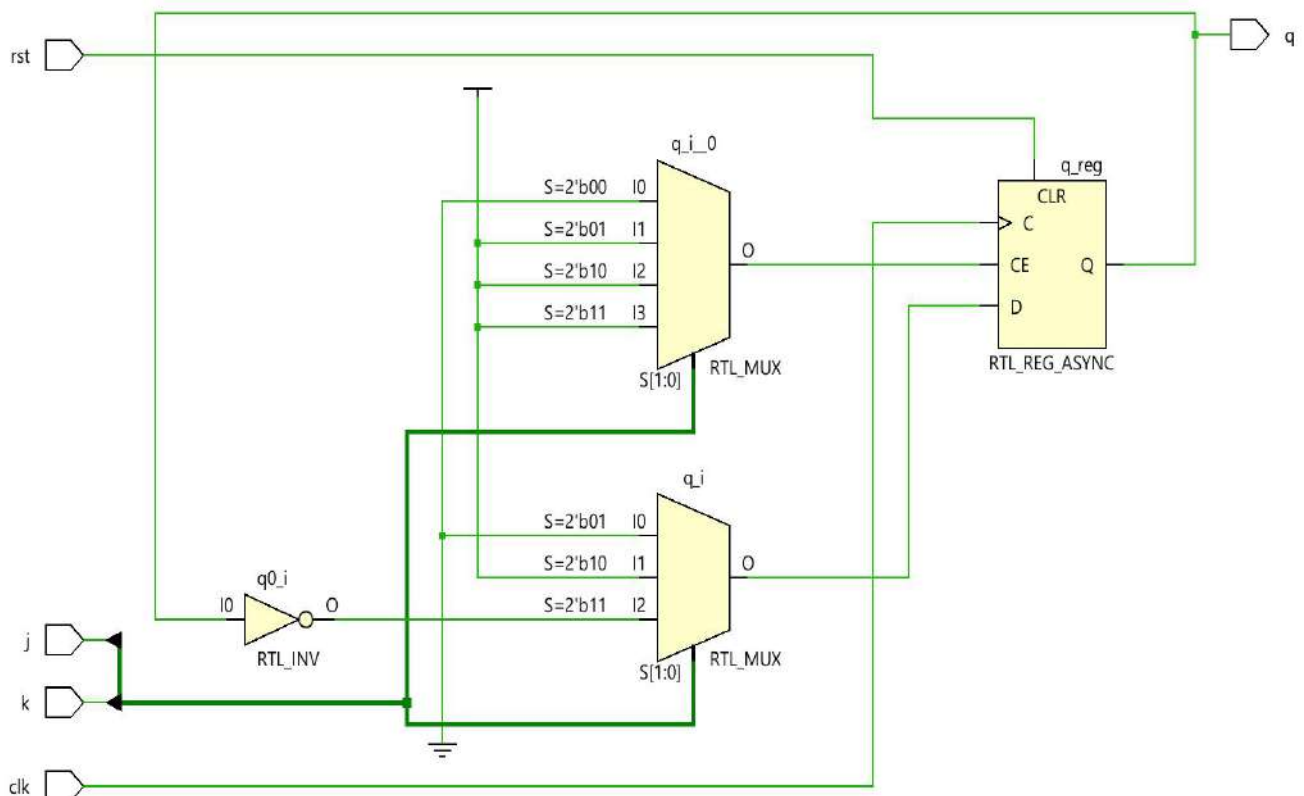
● Waveform



● Output

Time	J	K	Clk	Rst	Q
0	0	0	0	1	0
5000	0	0	1	1	0
10000	0	0	0	0	0
15000	0	0	1	0	0
20000	0	1	0	0	0
25000	0	1	1	0	0
30000	1	0	0	0	0
35000	1	0	1	0	1
40000	1	1	0	0	1
45000	1	1	1	0	0
50000	1	1	0	0	0
55000	1	1	1	0	1
60000	0	0	0	0	1
65000	0	0	1	0	1
70000	0	1	0	0	1
75000	0	1	1	0	0
80000	1	0	0	0	0
85000	1	0	1	0	1
90000	1	1	0	0	1
95000	1	1	1	0	0
100000	1	1	0	0	0
105000	1	1	1	0	1
110000	0	0	0	0	1
115000	0	0	1	0	1

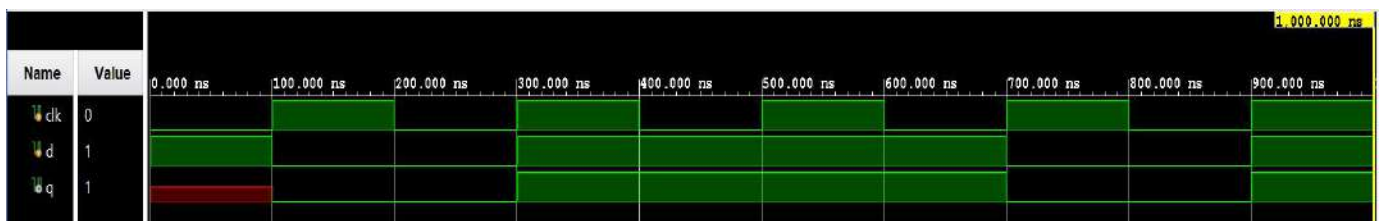
● Schematic

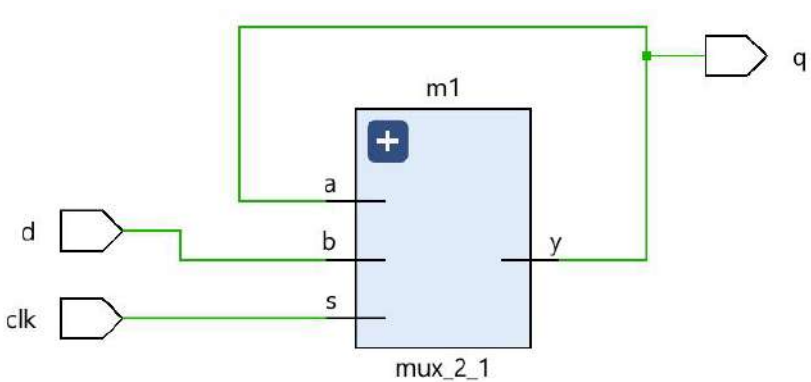


81. D latch using 2x1 MUX

● Design code of 2x1 mux	● Design code of D latch	● Testbench
<pre> module mux_2_1(a,b,s,y); input a,b,s; output y; assign y=((~s)&a) (s&b); endmodule </pre>	<pre> `include "mux_2_1.v" module d_latch(clk,d,q); input clk,d; output q; wire q_int; mux_2_1 m1(q_int,clk,d,q_int); assign q=q_int; endmodule </pre>	<pre> module d_latch_tb; reg clk,d; wire q; d_latch d1(clk,d,q); initial begin clk= 1'b0; d=1'b0; #100 clk= 1'b0; d=1'b1; #100 clk= 1'b1; d=1'b0; #100 clk= 1'b1; d=1'b1; #100 clk= 1'b0; d=1'b0; #100 clk= 1'b0; d=1'b1; #100 clk= 1'b1; d=1'b0; #100 clk= 1'b1; d=1'b1; #100 clk= 1'b0; d=1'b0; #100 clk= 1'b0; d=1'b1; #100 clk= 1'b1; d=1'b0; #100 clk= 1'b1; d=1'b1; end initial begin \$monitor("Simtime=%0t, clk=%b, d=%b, q=%b", \$time,clk,d,q); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,clk,d,q); end endmodule </pre>

● Waveform



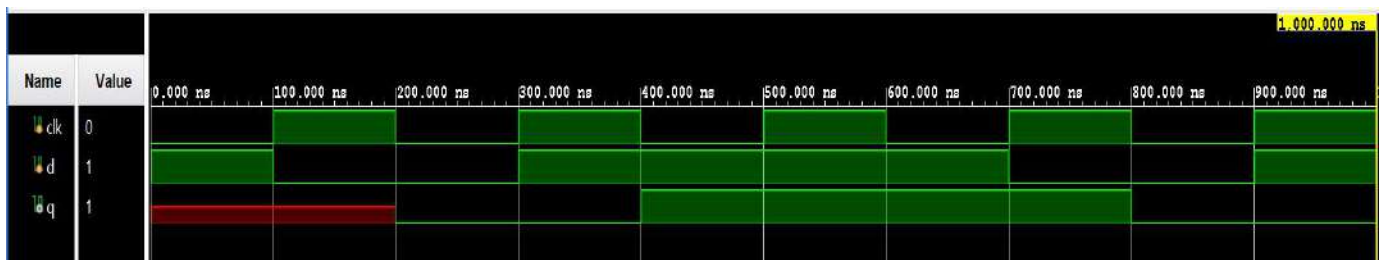
● Output	● Schematic
<pre>Simtime=0, clk=0, d=1, q=x Simtime=100000, clk=1, d=0, q=0 Simtime=200000, clk=0, d=0, q=0 Simtime=300000, clk=1, d=1, q=1 Simtime=400000, clk=0, d=1, q=1 Simtime=500000, clk=1, d=1, q=1 Simtime=600000, clk=0, d=1, q=1 Simtime=700000, clk=1, d=0, q=0 Simtime=800000, clk=0, d=0, q=0 Simtime=900000, clk=1, d=1, q=1 Simtime=1000000, clk=0, d=1, q=1</pre>	

NOTE: This is a positive level-triggered latch

82. D flip-flop using 2x1 MUX

● Design code of 2x1 mux	● Design code of D flip-flop	● Testbench
<pre> module mux_2_1(a,b,s,y); input a,b,s; output y; assign y=((~s)&a) (s&b); endmodule </pre>	<pre> `include "mux_2_1.v" module d_ff_using_2x1_mux(clk,d,q); input clk,d; output q; wire q1,q2; mux_2_1 m1(q1,d,clk,q1); mux_2_1 m2(q2,q1,~clk,q2); assign q=q2; endmodule </pre>	<pre> module d_ff_using_2x1_mux_tb; reg clk,d; wire q; d_ff_using_2x1_mux d1(clk,d,q); initial begin clk= 1'b0; d=1'b1; #100 clk= 1'b1; d=1'b0; #100 clk= 1'b0; d=1'b0; #100 clk= 1'b1; d=1'b1; #100 clk= 1'b0; d=1'b1; #100 clk= 1'b1; d=1'b1; #100 clk= 1'b0; d=1'b1; #100 clk= 1'b1; d=1'b0; #100 clk= 1'b0; d=1'b0; #100 clk= 1'b1; d=1'b1; #100 clk= 1'b0; d=1'b1; #100 clk= 1'b0; d=1'b1; end initial begin \$monitor("Simtime=%0t, clk=%b, d=%b, q=%b", \$time,clk,d,q); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,clk,d,q); end endmodule </pre>

● Waveform



● Output	● Schematic
<pre> Simtime=0, clk=0, d=1, q=x Simtime=100000, clk=1, d=0, q=x Simtime=200000, clk=0, d=0, q=0 Simtime=300000, clk=1, d=1, q=0 Simtime=400000, clk=0, d=1, q=1 Simtime=500000, clk=1, d=1, q=1 Simtime=600000, clk=0, d=1, q=1 Simtime=700000, clk=1, d=0, q=1 Simtime=800000, clk=0, d=0, q=0 Simtime=900000, clk=1, d=1, q=0 Simtime=1000000, clk=0, d=1, q=1 </pre>	

NOTE: This is a negative edge-triggered flip flop

83. D flip-flop using T flip-flop

● Design Code	● Testbench Code
<pre> module d_ff_using_t(D, clk, Q); input D, clk; output reg Q; wire T; // T = D XOR Q assign T = D ^ Q; // Initialize Q to avoid unknown state initial Q = 0; always @(posedge clk) begin if (T) Q <= ~Q; else Q <= Q; end endmodule </pre>	<pre> module d_ff_using_t_tb; reg D, clk; wire Q; // Instantiate DUT d_ff_using_t dut (D, clk, Q); // Clock generation initial clk = 0; always #5 clk = ~clk; // Stimulus initial begin \$display("Time\tclk\tD\tQ"); \$monitor("%0t\t%b\t%b\t%b", \$time, clk, D, Q); D = 0; #12; D = 1; #10; D = 0; #10; D = 1; #10; D = 1; #10; D = 0; #10; D = 1; #10; \$finish; end endmodule </pre>

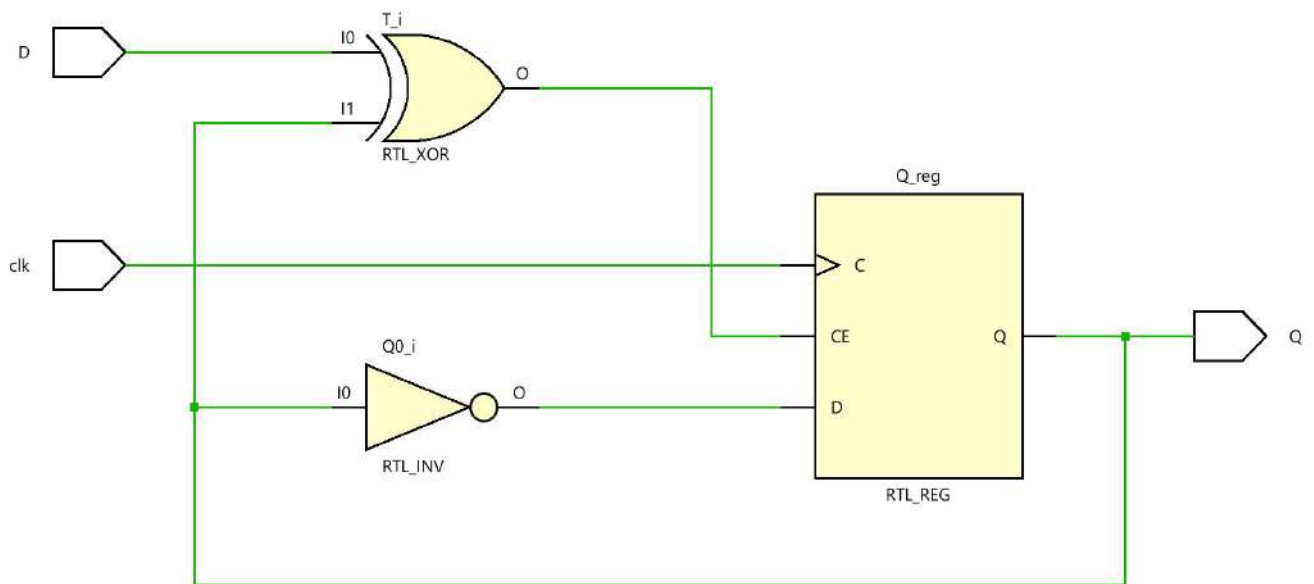
● Waveform



● Output

Time	clk	D	Q
0	0	0	0
5000	1	0	0
10000	0	0	0
12000	0	1	0
15000	1	1	1
20000	0	1	1
22000	0	0	1
25000	1	0	0
30000	0	0	0
32000	0	1	0
35000	1	1	1
40000	0	1	1
45000	1	1	1
50000	0	1	1
52000	0	0	1
55000	1	0	0
60000	0	0	0
62000	0	1	0
65000	1	1	1
70000	0	1	1

● Schematic

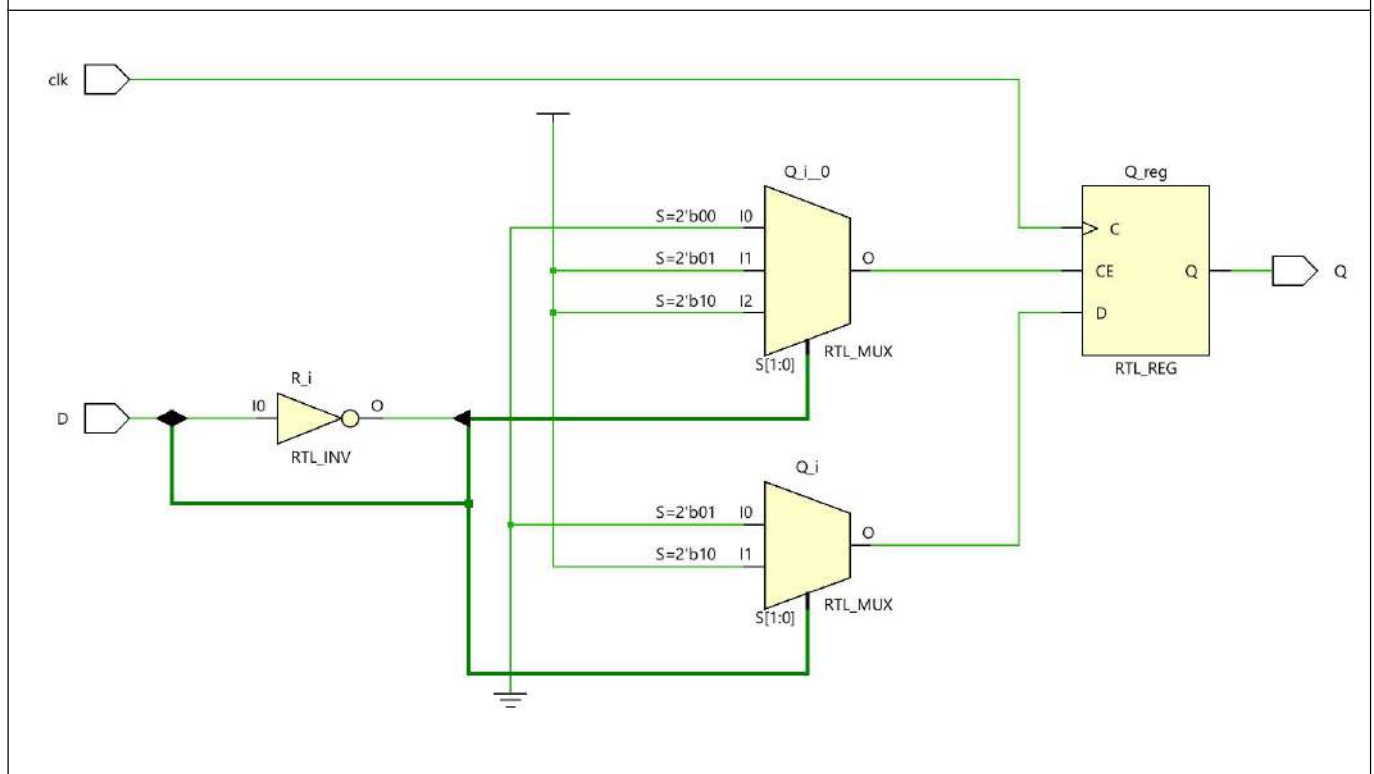


84. D flip-flop using SR flip-flop

● Design Code	● Testbench Code																												
<pre>module d_ff_using_sr(D, clk, Q); input D, clk; output reg Q; wire S, R; assign S = D; assign R = ~D; initial Q = 0; always @(posedge clk) begin case ({S, R}) 2'b00: Q <= Q; // No change 2'b01: Q <= 0; // Reset 2'b10: Q <= 1; // Set 2'b11: Q <= 1'bx; // Invalid for SR flip- flop endcase end endmodule</pre>	<pre>module d_ff_using_sr_tb; reg D, clk; wire Q; d_ff_using_sr dut(D, clk, Q); initial clk = 0; always #5 clk = ~clk; iInitial begin \$display("Time\tclk\tD\tQ"); \$monitor("%0t\t%b\t%b\t%b", \$time, clk, D, Q); D = 0; #12; D = 1; #10; D = 0; #10; D = 1; #10; D = 1; #10; D = 0; #10; D = 1; #10; \$finish; end endmodule</pre>																												
● Waveform																													
																													
● Output																													
<table><tr><th>Time</th><th>clk</th><th>D</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>5000</td><td>1</td><td>0</td><td>0</td></tr><tr><td>10000</td><td>0</td><td>0</td><td>0</td></tr><tr><td>12000</td><td>0</td><td>1</td><td>0</td></tr><tr><td>15000</td><td>1</td><td>1</td><td>1</td></tr><tr><td>20000</td><td>0</td><td>1</td><td>1</td></tr></table>		Time	clk	D	Q	0	0	0	0	5000	1	0	0	10000	0	0	0	12000	0	1	0	15000	1	1	1	20000	0	1	1
Time	clk	D	Q																										
0	0	0	0																										
5000	1	0	0																										
10000	0	0	0																										
12000	0	1	0																										
15000	1	1	1																										
20000	0	1	1																										

22000	0	0	1
25000	1	0	0
30000	0	0	0
32000	0	1	0
35000	1	1	1
40000	0	1	1
45000	1	1	1
50000	0	1	1
52000	0	0	1
55000	1	0	0
60000	0	0	0
62000	0	1	0
65000	1	1	1
70000	0	1	1

● Schematic

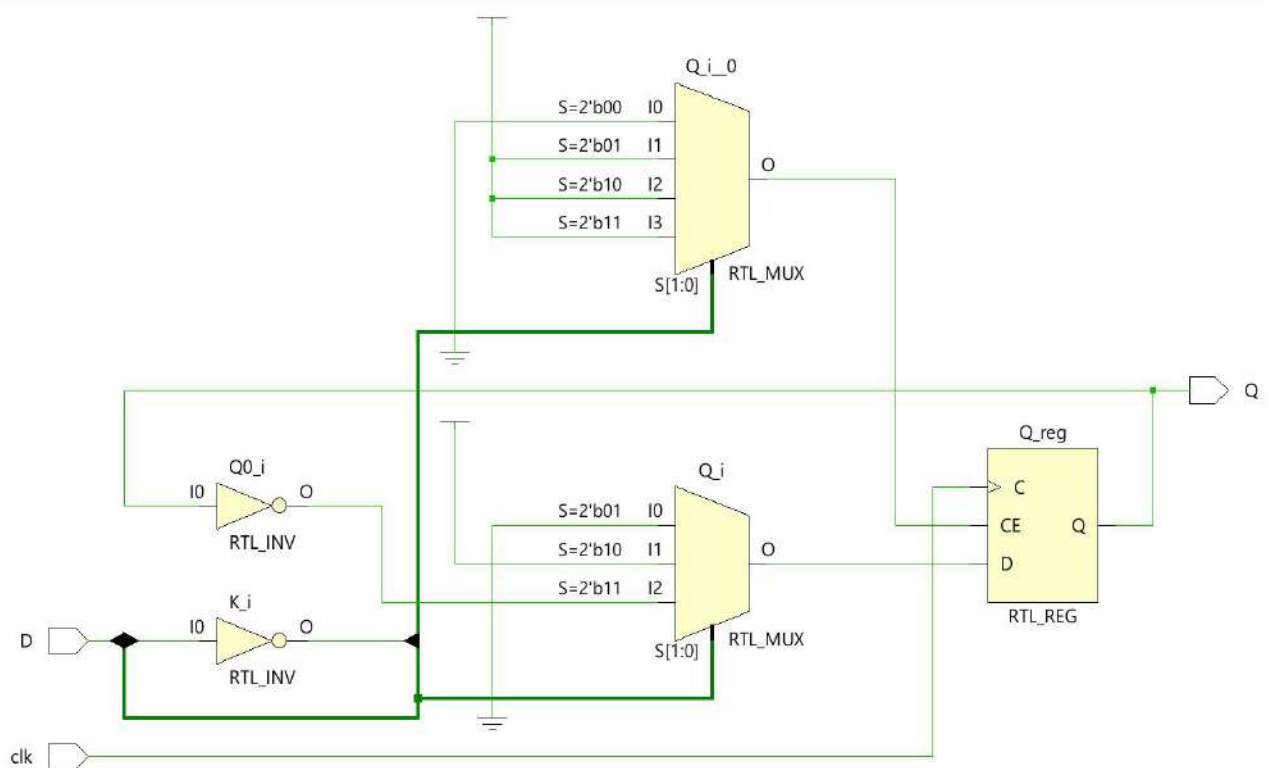


85. D flip-flop using JK flip-flop

● Design Code	● Testbench Code																												
<pre>module d_ff_using_jk(D, clk, Q); input D, clk; output reg Q; wire J, K; assign J = D; assign K = ~D; initial Q = 0; always @(posedge clk) begin case ({J, K}) 2'b00: Q <= Q; // No change 2'b01: Q <= 0; // Reset 2'b10: Q <= 1; // Set 2'b11: Q <= ~Q; // Toggle endcase end endmodule</pre>	<pre>module d_ff_using_jk_tb; reg D, clk; wire Q; d_ff_using_jk dut(D, clk, Q); initial clk = 0; always #5 clk = ~clk; initial begin \$display("Time\tclk\tD\tQ"); \$monitor("%0t\t%b\t%b\t%b", \$time, clk, D, Q); D = 0; #12; D = 1; #10; D = 0; #10; D = 1; #10; D = 1; #10; D = 0; #10; D = 1; #10; \$finish; end endmodule</pre>																												
● Waveform																													
																													
● Output																													
<table><tr><th>Time</th><th>clk</th><th>D</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>5000</td><td>1</td><td>0</td><td>0</td></tr><tr><td>10000</td><td>0</td><td>0</td><td>0</td></tr><tr><td>12000</td><td>0</td><td>1</td><td>0</td></tr><tr><td>15000</td><td>1</td><td>1</td><td>1</td></tr><tr><td>20000</td><td>0</td><td>1</td><td>1</td></tr></table>		Time	clk	D	Q	0	0	0	0	5000	1	0	0	10000	0	0	0	12000	0	1	0	15000	1	1	1	20000	0	1	1
Time	clk	D	Q																										
0	0	0	0																										
5000	1	0	0																										
10000	0	0	0																										
12000	0	1	0																										
15000	1	1	1																										
20000	0	1	1																										

22000	0	0	1
25000	1	0	0
30000	0	0	0
32000	0	1	0
35000	1	1	1
40000	0	1	1
45000	1	1	1
50000	0	1	1
52000	0	0	1
55000	1	0	0
60000	0	0	0
62000	0	1	0
65000	1	1	1
70000	0	1	1

● Schematic



86. T flip-flop using D flip-flop

● Design Code

```
module t_ff_using_d(T, clk, Q);
  input T, clk;
  output reg Q;
  wire D;

  assign D = T ^ Q;
  initial Q = 0;

  always @(posedge clk)
    Q <= D;
endmodule
```

● Testbench Code

```
module t_ff_using_d_tb;
  reg T, clk;
  wire Q;

  t_ff_using_d dut(T, clk, Q);

  initial clk = 0;
  always #5 clk = ~clk;

  initial
    begin
      $display("Time\tT\tQ");
      $monitor("%0t\t%b\t%b", $time, T, Q);

      T = 0; #10;
      T = 1; #10;
      T = 0; #10;
      T = 1; #10;
      T = 1; #10;
      T = 0; #10;

      $finish;
    end
endmodule
```

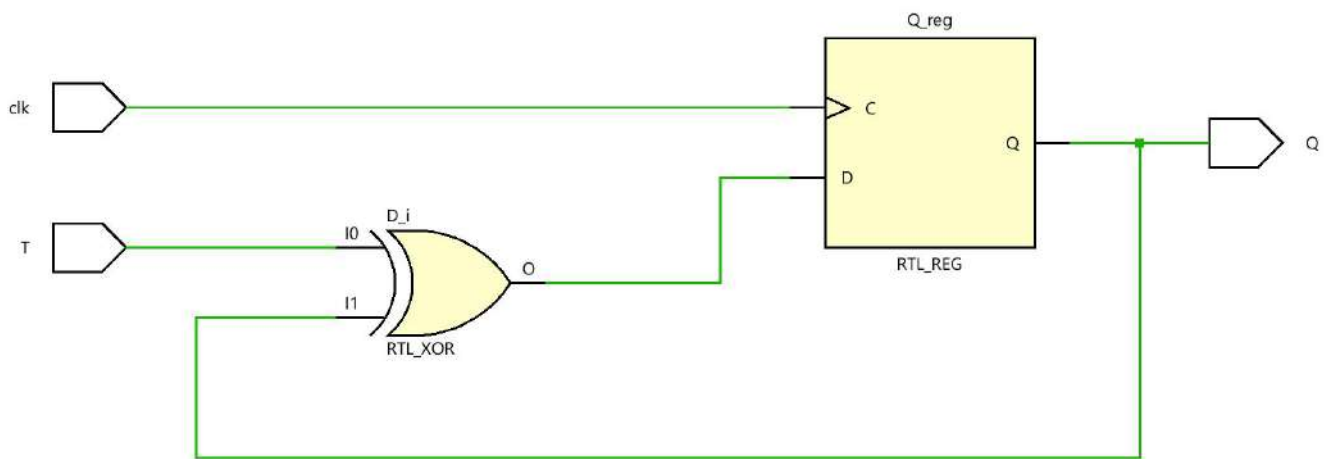
● Waveform



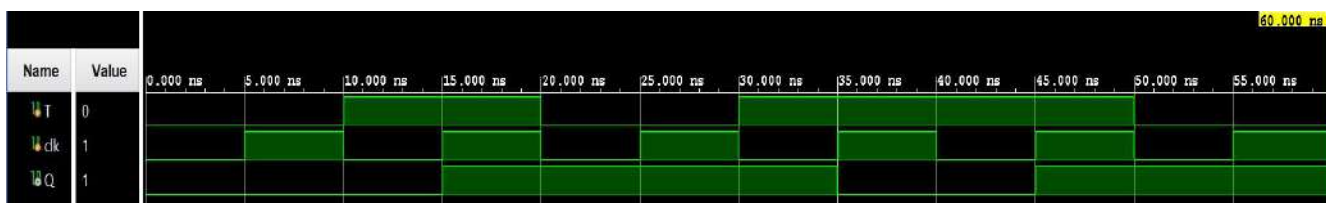
● Output

Time	T	Q
0	0	0
10000	1	0
15000	1	1
20000	0	1
30000	1	1
35000	1	0
45000	1	1
50000	0	1

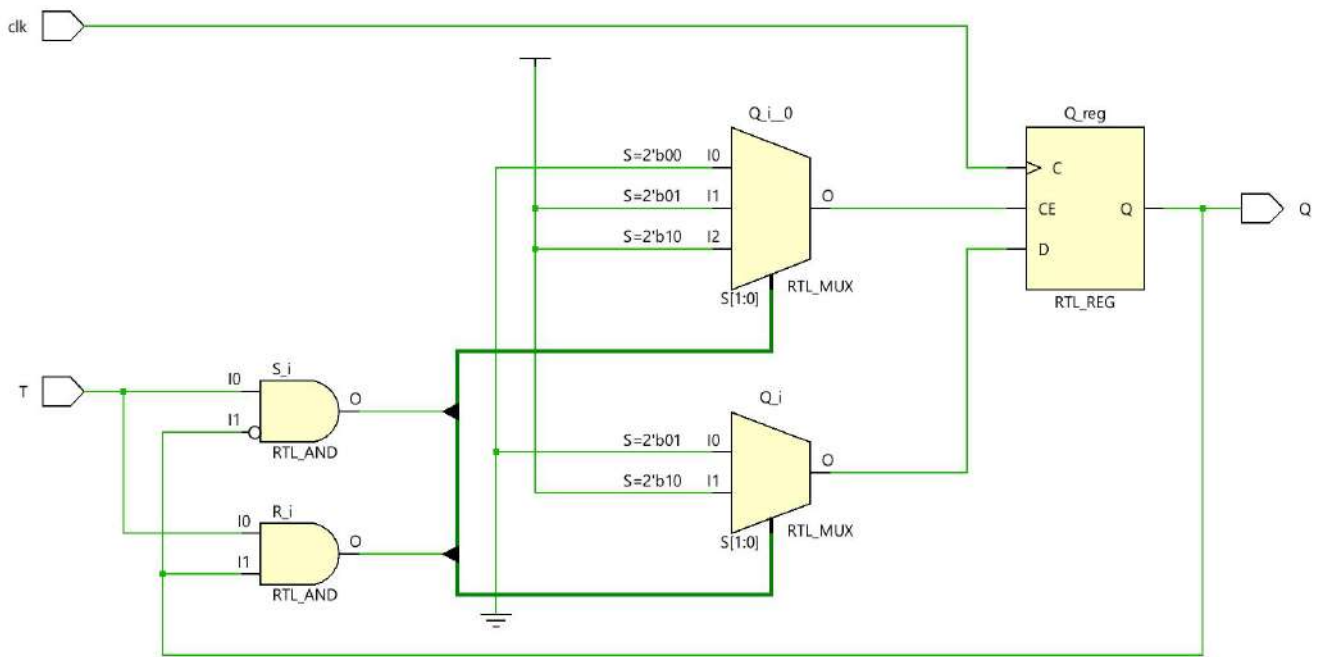
- Schematic



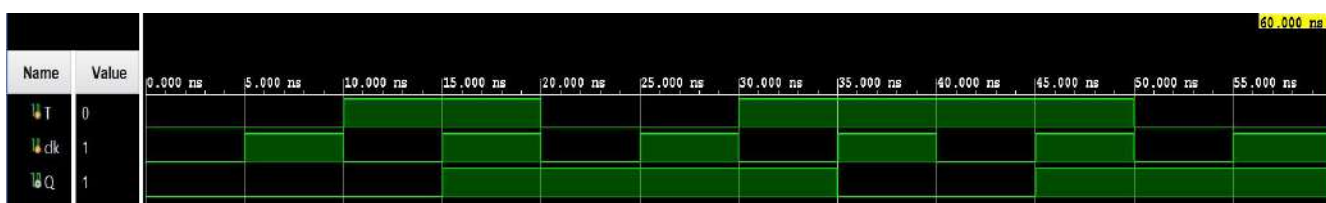
87. T flip-flop using SR flip-flop

● Design Code	● Testbench Code																											
<pre>module t_ff_using_sr(T, clk, Q); input T, clk; output reg Q; wire S, R; assign S = T & ~Q; assign R = T & Q; initial Q=0; always @(posedge clk) begin case ({S, R}) 2'b00: Q <= Q; 2'b01: Q <= 0; 2'b10: Q <= 1; 2'b11: Q <= 1'bx; // Invalid for SR FF endcase end endmodule</pre>	<pre>module t_ff_using_sr_tb; reg T, clk; wire Q; t_ff_using_sr dut(T, clk, Q); initial clk = 0; always #5 clk = ~clk; initial begin \$display("Time\tT\tQ"); \$monitor("%0t\t%b\t%b", \$time, T, Q); T = 0; #10; T = 1; #10; T = 0; #10; T = 1; #10; T = 1; #10; T = 0; #10; \$finish; end endmodule</pre>																											
● Waveform																												
																												
● Output																												
<table><tr><th>Time</th><th>T</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>10000</td><td>1</td><td>0</td></tr><tr><td>15000</td><td>1</td><td>1</td></tr><tr><td>20000</td><td>0</td><td>1</td></tr><tr><td>30000</td><td>1</td><td>1</td></tr><tr><td>35000</td><td>1</td><td>0</td></tr><tr><td>45000</td><td>1</td><td>1</td></tr><tr><td>50000</td><td>0</td><td>1</td></tr></table>		Time	T	Q	0	0	0	10000	1	0	15000	1	1	20000	0	1	30000	1	1	35000	1	0	45000	1	1	50000	0	1
Time	T	Q																										
0	0	0																										
10000	1	0																										
15000	1	1																										
20000	0	1																										
30000	1	1																										
35000	1	0																										
45000	1	1																										
50000	0	1																										

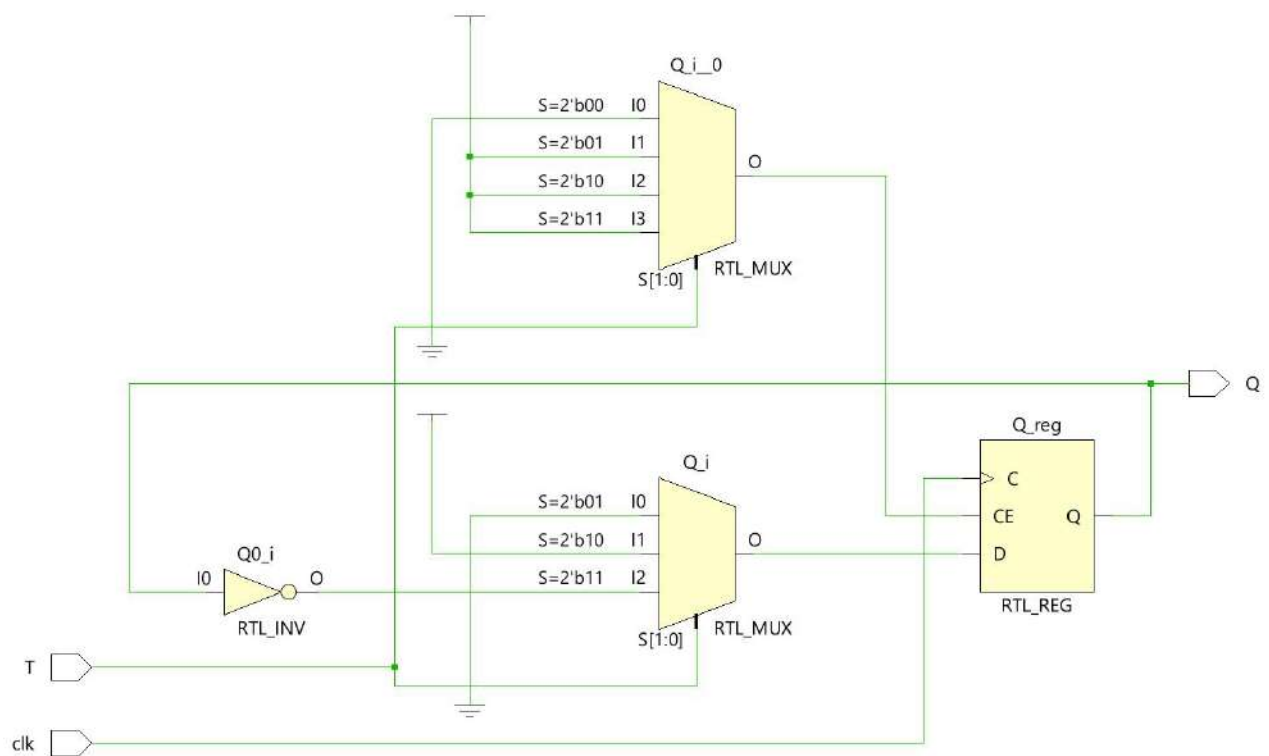
● Schematic



88. T flip-flop using JK flip-flop

● Design Code	● Testbench Code																											
<pre>module t_ff_using_jk(T, clk, Q); input T, clk; output reg Q; wire J, K; assign J = T; assign K = T; initial Q=0; always @(posedge clk) begin case ({J, K}) 2'b00: Q <= Q; // No change 2'b01: Q <= 0; 2'b10: Q <= 1; 2'b11: Q <= ~Q; // Toggle endcase end endmodule</pre>	<pre>module t_ff_using_jk_tb; reg T, clk; wire Q; t_ff_using_jk dut(T, clk, Q); initial clk = 0; always #5 clk = ~clk; initial begin \$display("Time\tT\tQ"); \$monitor("%0t\t%b\t%b", \$time, T, Q); T = 0; #10; T = 1; #10; T = 0; #10; T = 1; #10; T = 1; #10; T = 0; #10; \$finish; end endmodule</pre>																											
● Waveform																												
																												
● Output																												
<table><tr><th>Time</th><th>T</th><th>Q</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>10000</td><td>1</td><td>0</td></tr><tr><td>15000</td><td>1</td><td>1</td></tr><tr><td>20000</td><td>0</td><td>1</td></tr><tr><td>30000</td><td>1</td><td>1</td></tr><tr><td>35000</td><td>1</td><td>0</td></tr><tr><td>45000</td><td>1</td><td>1</td></tr><tr><td>50000</td><td>0</td><td>1</td></tr></table>		Time	T	Q	0	0	0	10000	1	0	15000	1	1	20000	0	1	30000	1	1	35000	1	0	45000	1	1	50000	0	1
Time	T	Q																										
0	0	0																										
10000	1	0																										
15000	1	1																										
20000	0	1																										
30000	1	1																										
35000	1	0																										
45000	1	1																										
50000	0	1																										

- Schematic



89. SR flip-flop using D flip-flop

● Design Code

```
module sr_ff_using_d(S, R, clk, Q);
  input S, R, clk;
  output reg Q;
  wire D;

  assign D = S | (~R & Q); // Derived logic
  initial Q=0;

  always @(posedge clk)
    Q <= D;
endmodule
```

● Testbench Code

```
module sr_ff_using_d_tb;
  reg S, R, clk;
  wire Q;

  sr_ff_using_d uut(S, R, clk, Q);

  initial clk = 0;
  always #5 clk = ~clk;

  initial
  begin
    $display("Time\tS R Q");
    $monitor("%0t\t%b %b %b", $time, S, R, Q);

    S = 0; R = 0; #10;
    S = 1; R = 0; #10;
    S = 0; R = 0; #10;
    S = 0; R = 1; #10;
    S = 1; R = 1; #10; // invalid for SR FF
    S = 0; R = 0; #10;
    S = 1; R = 0; #10;
    S = 0; R = 0; #10;
    S = 0; R = 1; #10;
    S = 1; R = 1; #10;
    S = 0; R = 0; #10;

    $finish;
  end
endmodule
```

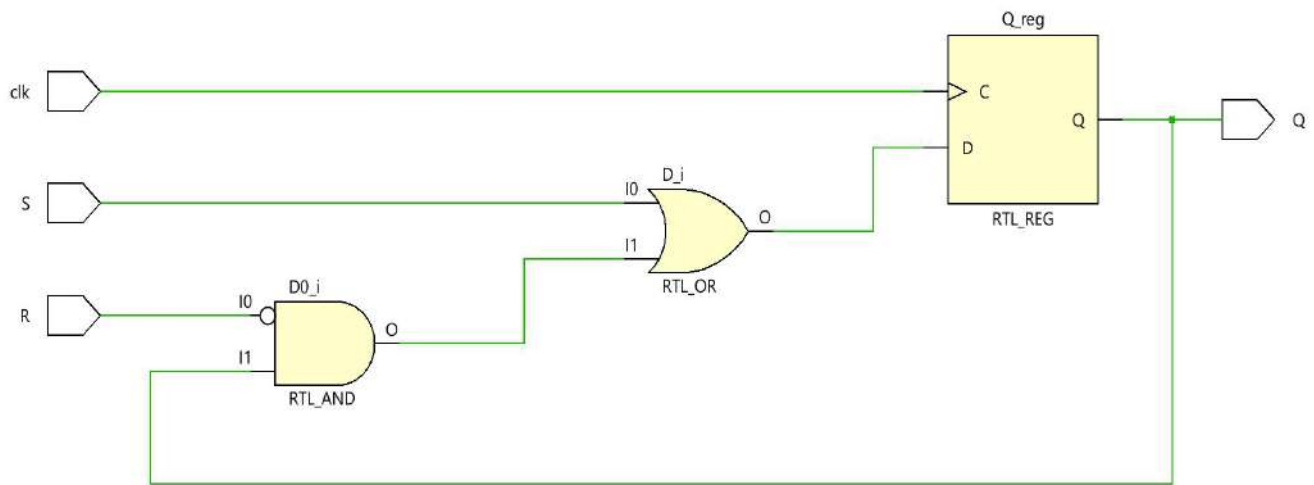
● Waveform



● Output

Time	S	R	Q
0	0	0	0
10000	1	0	0
15000	1	0	1
20000	0	0	1
30000	0	1	1
35000	0	1	0
40000	1	1	0
45000	1	1	1
50000	0	0	1
60000	1	0	1
70000	0	0	1
80000	0	1	1
85000	0	1	0
90000	1	1	0
95000	1	1	1
100000	0	0	1

● Schematic



90. SR flip-flop using T flip-flop

● Design Code

```
module sr_ff_using_t(S, R, clk, Q);
  input S, R, clk;
  output reg Q;
  wire T;

  assign T = S ^ R;
  initial Q=0;

  always @(posedge clk) begin
    if (S && R)
      Q <= 1'bx; // undefined for SR=11
    else if (T)
      Q <= ~Q;
    else
      Q <= Q;
  end
endmodule
```

● Testbench Code

```
module sr_ff_using_t_tb;
  reg S, R, clk;
  wire Q;

  sr_ff_using_t uut(S, R, clk, Q);

  initial clk = 0;
  always #5 clk = ~clk;

  initial
  begin
    $display("Time\tS R Q");
    $monitor("%0t\t%b %b %b", $time, S, R, Q);

    S = 0; R = 0; #10;
    S = 1; R = 0; #10;
    S = 0; R = 0; #10;
    S = 0; R = 1; #10;
    S = 1; R = 1; #10; // invalid for SR FF
    S = 0; R = 0; #10;
    S = 1; R = 0; #10;
    S = 0; R = 0; #10;
    S = 0; R = 1; #10;
    S = 1; R = 1; #10;
    S = 0; R = 0; #10;

    $finish;
  end
endmodule
```

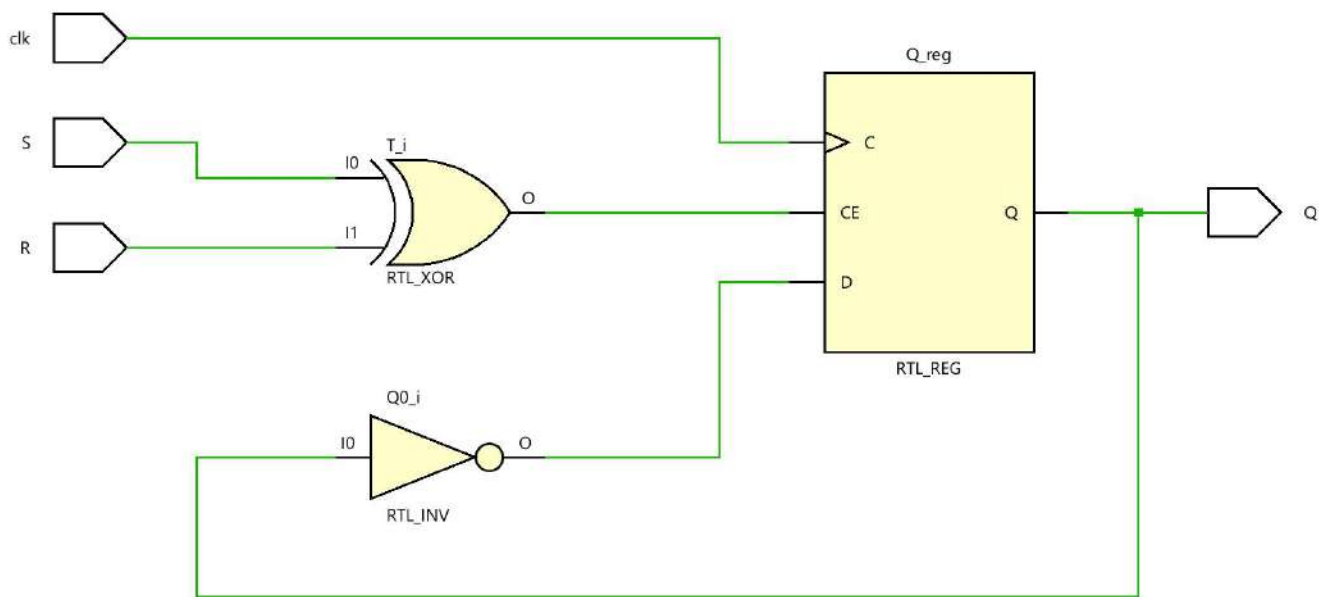
● Waveform



● Output

Time	S	R	Q
0	0	0	0
10000	1	0	0
15000	1	0	1
20000	0	0	1
30000	0	1	1
35000	0	1	0
40000	1	1	0
45000	1	1	x
50000	0	0	x
60000	1	0	x
70000	0	0	x
80000	0	1	x
90000	1	1	x
100000	0	0	x

● Schematic



91. SR flip-flop using JK flip-flop

● Design Code

```
module sr_ff_using_jk(S, R, clk, Q);
  input S, R, clk;
  output reg Q;

  wire J = S;
  wire K = R;
  initial Q=0;

  always @(posedge clk)
  begin
    case ({J,K})
      2'b00: Q <= Q;
      2'b01: Q <= 0;
      2'b10: Q <= 1;
      2'b11: Q <= ~Q; // Invalid for SR
    endcase
  end
endmodule
```

● Testbench Code

```
module sr_ff_using_jk_tb;
  reg S, R, clk;
  wire Q;

  sr_ff_using_jk uut(S, R, clk, Q);

  initial clk = 0;
  always #5 clk = ~clk;

  initial
  begin
    $display("Time\tS R Q");
    $monitor("%0t\t%b %b %b", $time, S, R, Q);

    S = 0; R = 0; #10;
    S = 1; R = 0; #10;
    S = 0; R = 0; #10;
    S = 0; R = 1; #10;
    S = 1; R = 1; #10; // invalid for SR FF
    S = 0; R = 0; #10;
    S = 1; R = 0; #10;
    S = 0; R = 0; #10;
    S = 0; R = 1; #10;
    S = 1; R = 1; #10;
    S = 0; R = 0; #10;

    $finish;
  end
endmodule
```

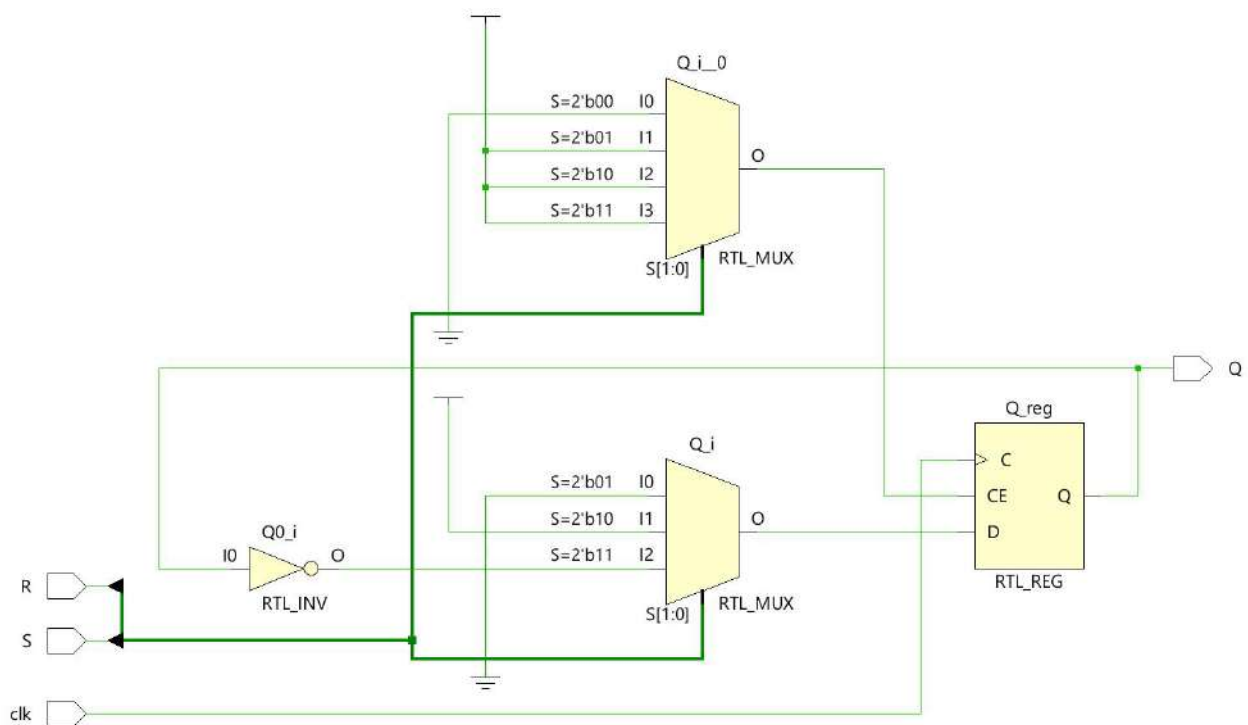
● Waveform



● Output

Time	S	R	Q
0	0	0	0
10000	1	0	0
15000	1	0	1
20000	0	0	1
30000	0	1	1
35000	0	1	0
40000	1	1	0
45000	1	1	1
50000	0	0	1
60000	1	0	1
70000	0	0	1
80000	0	1	1
85000	0	1	0
90000	1	1	0
95000	1	1	1
100000	0	0	1

● Schematic



92. JK flip-flop using D flip-flop

● Design Code

```
module jk_ff_using_d(J, K, clk, Q);
  input J, K, clk;
  output reg Q;
  wire D;

  assign D = (J & ~Q) | (~K & Q); // D = JQ'
  + K'Q
  initial Q=0;

  always @(posedge clk)
    Q <= D;
endmodule
```

● Testbench Code

```
module jk_ff_using_d_tb;
  reg J, K, clk;
  wire Q;

  jk_ff_using_d uut(J, K, clk, Q);

  initial clk = 0;
  always #5 clk = ~clk;

  initial begin
    $display("Time\tJ K\tQ");
    $monitor("%0t\t%b %b\t%b", $time, J, K, Q);

    J = 0; K = 0; #10;
    J = 0; K = 1; #10;
    J = 1; K = 0; #10;
    J = 1; K = 1; #10;
    J = 1; K = 1; #10;
    J = 0; K = 0; #10;
    J = 0; K = 1; #10;
    J = 1; K = 0; #10;
    J = 1; K = 1; #10;
    J = 1; K = 1; #10;
    J = 0; K = 0; #10;

    $finish;
  end
endmodule
```

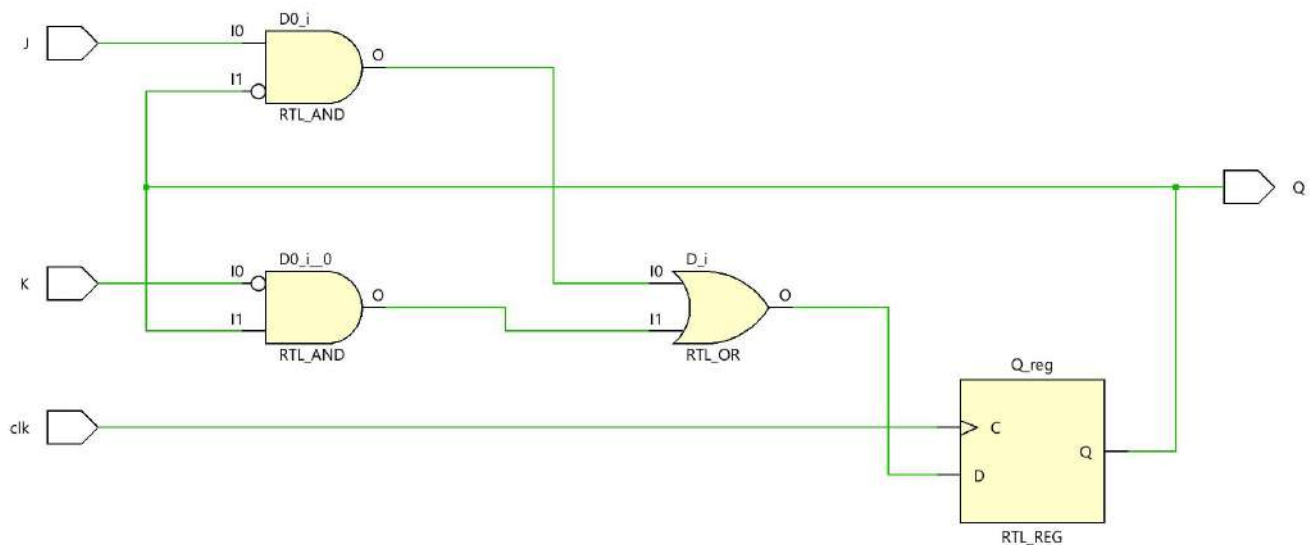
● Waveform



● Output

Time	J K	Q
0	0 0	0
10000	0 1	0
20000	1 0	0
25000	1 0	1
30000	1 1	1
35000	1 1	0
45000	1 1	1
50000	0 0	1
60000	0 1	1
65000	0 1	0
70000	1 0	0
75000	1 0	1
80000	1 1	1
85000	1 1	0
95000	1 1	1
100000	0 0	1

● Schematic



93. JK flip-flop using T flip-flop

● Design Code

```
module jk_ff_using_t(J, K, clk, Q);
  input J, K, clk;
  output reg Q;
  wire T;

  assign T = (J&(~Q)) | (K & Q); // Toggle
  condition
  initial Q=0;

  always @(posedge clk)
    if (T)
      Q <= ~Q;
    else
      Q <= Q;
endmodule
```

● Testbench Code

```
module jk_ff_using_t_tb;
  reg J, K, clk;
  wire Q;

  jk_ff_using_t uut(J, K, clk, Q);

  initial clk = 0;
  always #5 clk = ~clk;

  initial
    begin
      $display("Time\tJ K\tQ");
      $monitor("%0t\t%b %b\t%b", $time, J, K, Q);

      J = 0; K = 0; #10;
      J = 0; K = 1; #10;
      J = 1; K = 0; #10;
      J = 1; K = 1; #10;
      J = 1; K = 1; #10;
      J = 0; K = 0; #10;
      J = 0; K = 1; #10;
      J = 1; K = 0; #10;
      J = 1; K = 1; #10;
      J = 1; K = 1; #10;
      J = 0; K = 0; #10;
      $finish;
    end
endmodule
```

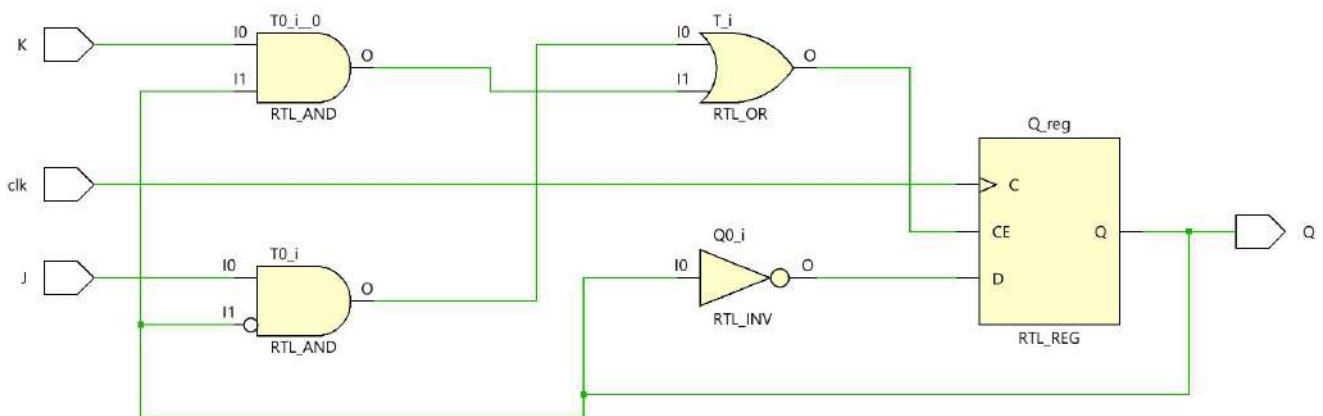
● Waveform



● Output

Time	J K	Q
0	0 0	0
10000	0 1	0
20000	1 0	0
25000	1 0	1
30000	1 1	1
35000	1 1	0
45000	1 1	1
50000	0 0	1
60000	0 1	1
65000	0 1	0
70000	1 0	0
75000	1 0	1
80000	1 1	1
85000	1 1	0
95000	1 1	1
100000	0 0	1

● Schematic



94. JK flip-flop using SR flip-flop

● Design Code

```
module jk_ff_using_sr(J, K, clk, Q);
  input J, K, clk;
  output reg Q;
  reg S, R;

  always @(J,K,clk)
  begin
    S = J & ~Q;
    R = K & Q;
  end

  initial Q=0;

  always @(posedge clk)
  begin
    case ({S, R})
      2'b00: Q <= Q;
      2'b01: Q <= 0;
      2'b10: Q <= 1;
      2'b11: Q <= 1'bx; // Invalid state
    endcase
  end
endmodule
```

● Testbench Code

```
module jk_ff_using_sr_tb;
  reg J, K, clk;
  wire Q;

  jk_ff_using_sr uut(J, K, clk, Q);

  initial clk = 0;
  always #5 clk = ~clk;

  initial
  begin
    $display("Time\tJ K\tQ");
    $monitor("%0t\t%b %b\t%b", $time, J, K, Q);

    J = 0; K = 0; #10;
    J = 0; K = 1; #10;
    J = 1; K = 0; #10;
    J = 1; K = 1; #10;
    J = 1; K = 1; #10;
    J = 0; K = 0; #10;
    J = 0; K = 1; #10;
    J = 1; K = 0; #10;
    J = 1; K = 1; #10;
    J = 1; K = 1; #10;
    J = 0; K = 0; #10;

    $finish;
  end
endmodule
```

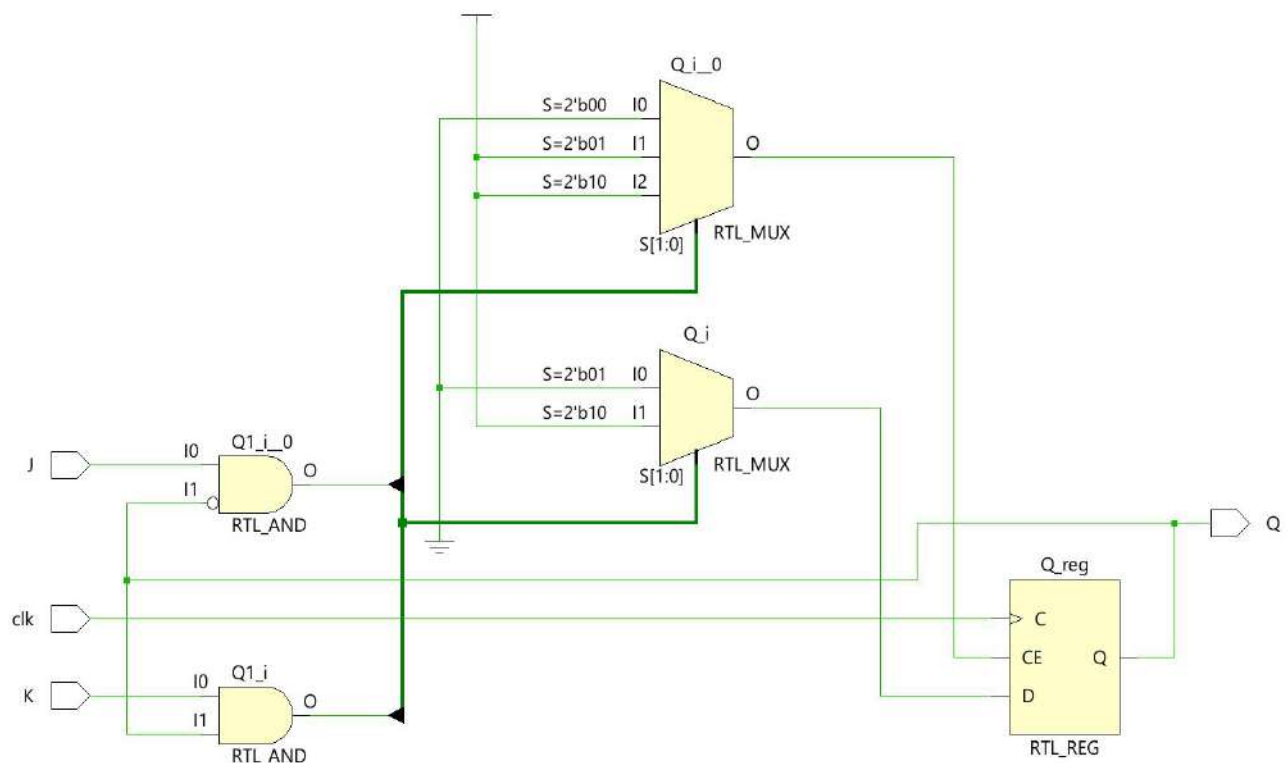
● Waveform



● Output

Time	J K	Q
0	0 0	0
10000	0 1	0
20000	1 0	0
25000	1 0	1
30000	1 1	1
35000	1 1	0
45000	1 1	1
50000	0 0	1
60000	0 1	1
65000	0 1	0
70000	1 0	0
75000	1 0	1
80000	1 1	1
85000	1 1	0
95000	1 1	1
100000	0 0	1

● Schematic



95. Serial-In Serial-Out

● Design Code	● Testbench Code
<pre> module siso (clk, rst, sin, sout); input clk, rst, sin; output reg sout; reg [3:0] shift; always @(posedge clk or posedge rst) if (rst) shift <= 0; else shift <= {shift[2:0], sin}; always @(clk, rst, sin) sout = shift[3]; // Output MSB endmodule </pre>	<pre> module siso_tb; reg clk, rst, sin; wire sout; siso uut (clk, rst, sin,sout); initial clk = 0; always #5 clk = ~clk; initial begin \$display("Time\tSin\tSout"); \$monitor("%0t\t%b\t%b", \$time, sin, sout); rst = 1; sin = 0; #10; rst = 0; sin = 1; #10; sin = 1; #10; sin = 0; #10; sin = 1; #10; sin = 0; #10; sin = 1; #10; sin = 0; #10; sin = 0; #10; sin = 1; #10; sin = 0; #10; \$finish; end endmodule </pre>

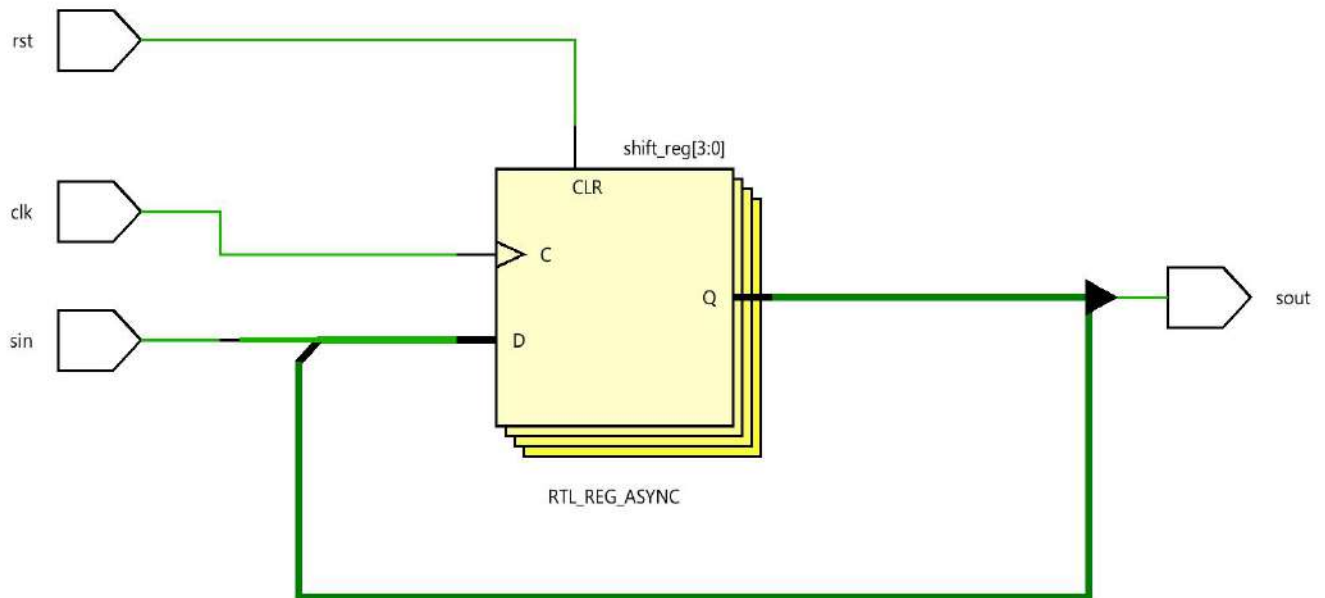
● Waveform



● Output

Time	Sin	Sout
0	0	x
5000	0	0
10000	1	0
30000	0	0
40000	1	0
50000	0	1
60000	1	1
70000	0	0
80000	0	1
90000	1	0
100000	0	1

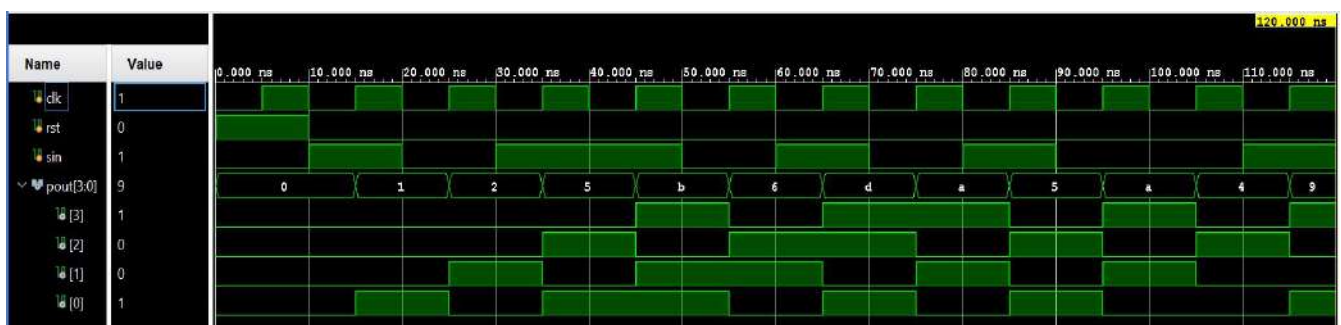
● Schematic



96. Serial-In Parallel-Out

● Design Code	● Testbench Code
<pre> module sipo (clk, rst, sin, pout); input clk, rst, sin; output reg [3:0] pout; always @(posedge clk or posedge rst) if (rst) pout <= 0; else pout <= {pout[2:0], sin}; // Shift left endmodule </pre>	<pre> module sipo_tb; reg clk, rst, sin; wire [3:0] pout; sipo uut (clk, rst, sin, pout); initial clk = 0; always #5 clk = ~clk; initial begin \$display("Time\tSin\tPout"); \$monitor("%0t\t%b\t%b", \$time, sin, pout); rst = 1; sin = 0; #10; rst = 0; sin = 1; #10; sin = 0; #10; sin = 1; #10; sin = 1; #10; sin = 0; #10; sin = 1; #10; sin = 0; #10; sin = 1; #10; sin = 0; #10; sin = 1; #10; \$finish; end endmodule </pre>

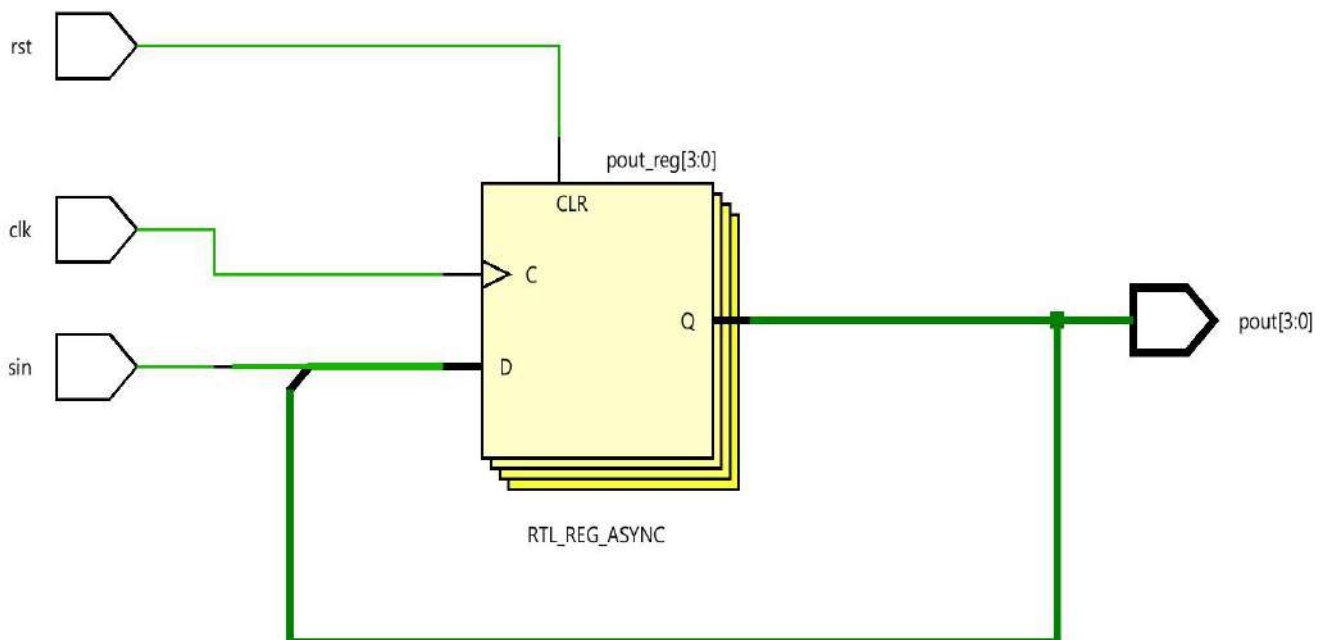
● Waveform



● Output

Time	Sin	Pout
0	0	0000
10000	1	0000
15000	1	0001
20000	0	0001
25000	0	0010
30000	1	0010
35000	1	0101
45000	1	1011
50000	0	1011
55000	0	0110
60000	1	0110
65000	1	1101
70000	0	1101
75000	0	1010
80000	1	1010
85000	1	0101
90000	0	0101
95000	0	1010
105000	0	0100
110000	1	0100
115000	1	1001

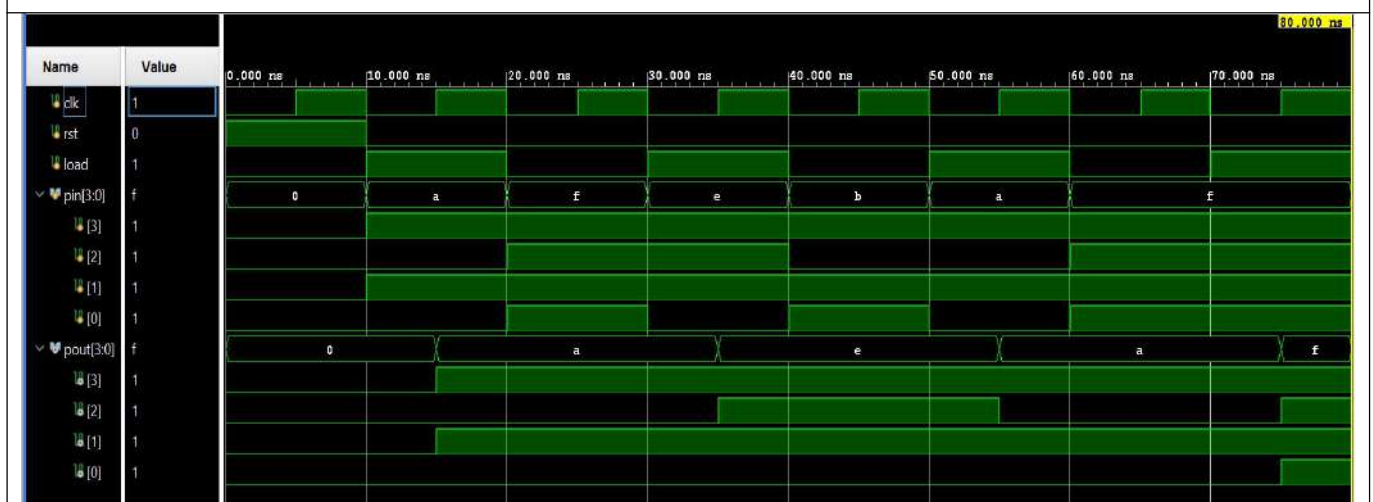
● Schematic



97. Parallel-In Parallel-Out

● Design Code	● Testbench Code
<pre> module pipo (clk, rst, load, pin, pout); input clk, rst, load; input [3:0] pin; output reg [3:0] pout; always @(posedge clk or posedge rst) if (rst) pout <= 0; else if (load) pout <= pin; endmodule </pre>	<pre> module pipo_tb; reg clk, rst, load; reg [3:0] pin; wire [3:0] pout; pipo uut (clk, rst, load, pin, pout); initial clk = 0; always #5 clk = ~clk; initial begin \$display("Time\tPin\tLoad\tPout"); \$monitor("%0t\t%b\t%b\t%b", \$time, pin, load, pout); rst = 1; pin = 4'b0000; load = 0; #10; rst = 0; pin = 4'b1010; load = 1; #10; load = 0; pin = 4'b1111; #10; pin = 4'b1110; load = 1; #10; load = 0; pin = 4'b1011; #10; pin = 4'b1010; load = 1; #10; load = 0; pin = 4'b1111; #10; load = 1; pin = 4'b1111; #10; \$finish; end endmodule </pre>

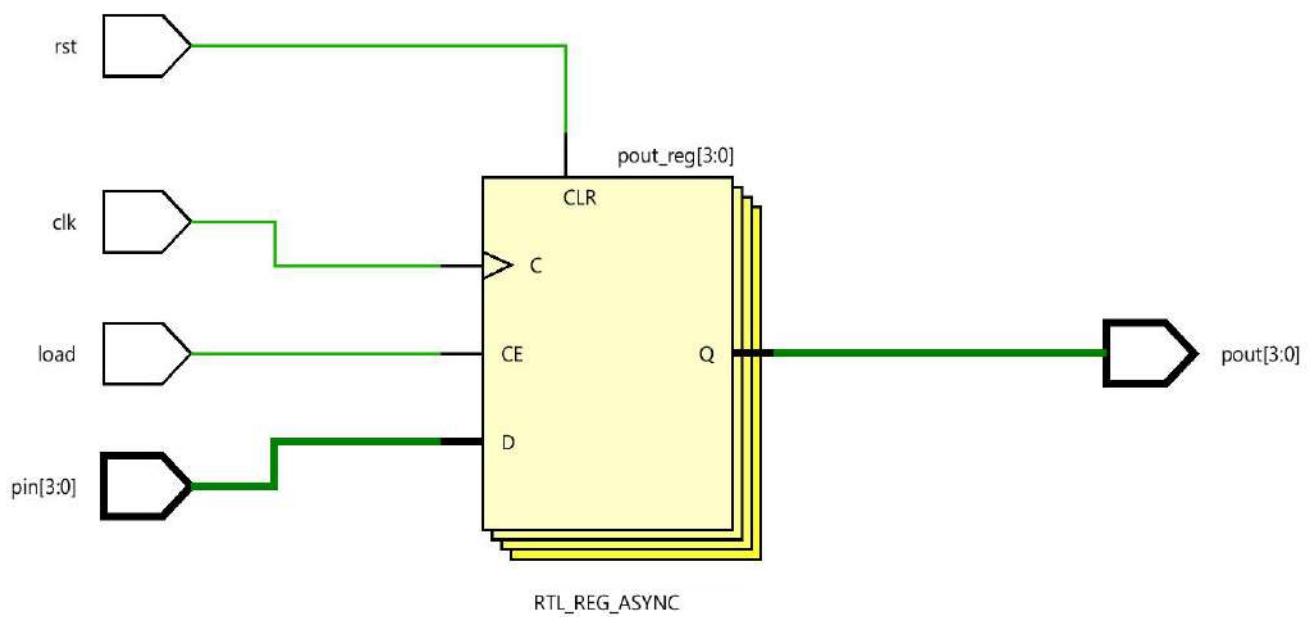
● Waveform



● Output

Time	Pin	Load	Pout
0	0000	0	0000
10000	1010	1	0000
15000	1010	1	1010
20000	1111	0	1010
30000	1110	1	1010
35000	1110	1	1110
40000	1011	0	1110
50000	1010	1	1110
55000	1010	1	1010
60000	1111	0	1010
70000	1111	1	1010
75000	1111	1	1111

● Schematic



98. Parallel-In Serial-Out

● Design Code

```
module piso (clk, rst, load, pin, sout);
  input clk, rst, load;
  input [3:0] pin;
  output reg sout;
  reg [3:0] shift;

  always @(posedge clk or posedge rst)
    if (rst)
      shift <= 0;
    else if (load)
      shift <= pin;
    else
      shift <= {shift[2:0], 1'b0};

  always @(clk, rst, load, pin)
    sout = shift[3]; // Output MSB
endmodule
```

● Testbench Code

```
module piso_tb;
  reg clk, rst, load;
  reg [3:0] pin;
  wire sout;

  piso uut (clk, rst, load, pin, sout);

  initial clk = 0;
  always #5 clk = ~clk;

  initial
    begin
      $display("Time\tPin\tLoad\tSout");
      $monitor("%0t\t%b\t%b\t%b", $time, pin, load, sout);

      rst = 1;
      load = 0; pin = 4'b0000; #10;

      rst = 0;
      load = 1; pin = 4'b1001; #10;
      load = 0;

      #40; // Allow shifting out
      $finish;
    end
endmodule
```

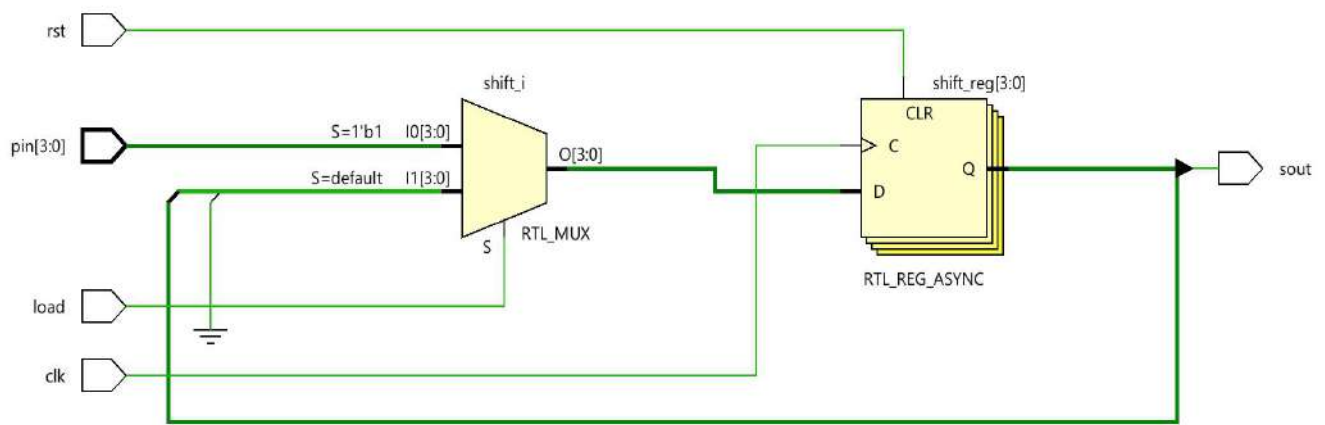
● Waveform



● Output

Time	Pin	Load	Sout
0	0000	0	x
5000	0000	0	0
10000	1001	1	1
20000	1001	0	0
30000	1001	0	0
50000	1001	0	1

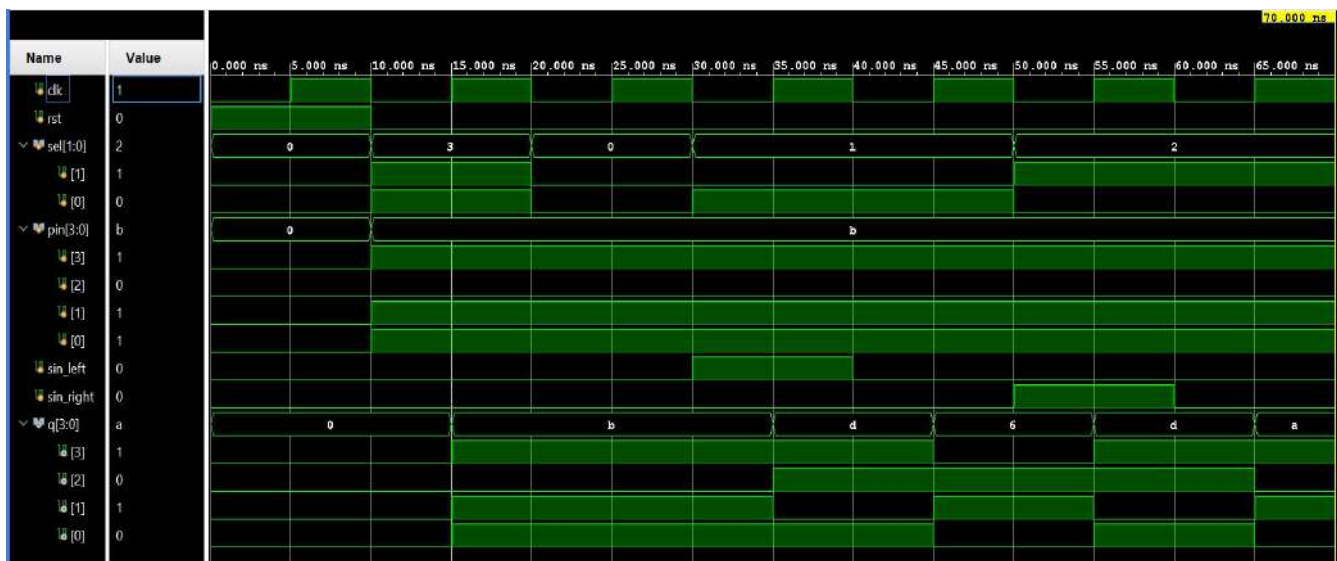
● Schematic



99. Universal Shift Register

● Design Code	● Testbench Code
<pre> module universal_shift_register (clk, rst, sel, pin, sin_left, sin_right, q); input clk; input rst; input [1:0] sel; // Mode select input [3:0] pin; // Parallel input input sin_left, sin_right; // Serial inputs for shift output reg [3:0] q; // Register output always @(posedge clk or posedge rst) if (rst) q <= 4'b0000; else begin case (sel) 2'b00: q <= q; // Hold 2'b01: q <= {sin_left, q[3:1]}; // Right Shift 2'b10: q <= {q[2:0], sin_right}; // Left Shift 2'b11: q <= pin; // Parallel Load endcase end endmodule </pre> ● 00: No change (Hold) ● 01: Right Shift ● 10: Left Shift ● 11: Parallel Load	<pre> module universal_shift_register_tb; reg clk, rst; reg [1:0] sel; reg [3:0] pin; reg sin_left, sin_right; wire [3:0] q; universal_shift_register uut (clk, rst, sel, pin, sin_left, sin_right, q); initial clk = 0; always #5 clk = ~clk; initial begin \$display("Time\tSel\tPin\tSL\tSR\tQ"); \$monitor("%0t\t%b\t%b\t%b\t%b\t%b", \$time, sel, pin, sin_left, sin_right, q); // Reset rst = 1; sel = 2'b00; pin = 4'b0000; sin_left = 0; sin_right = 0; #10; rst = 0; // Parallel load sel = 2'b11; pin = 4'b1011; #10; // Hold sel = 2'b00; #10; // Right shift sel = 2'b01; sin_left = 1; #10; sin_left = 0; #10; // Left shift sel = 2'b10; sin_right = 1; #10; sin_right = 0; #10; \$finish; end endmodule </pre>

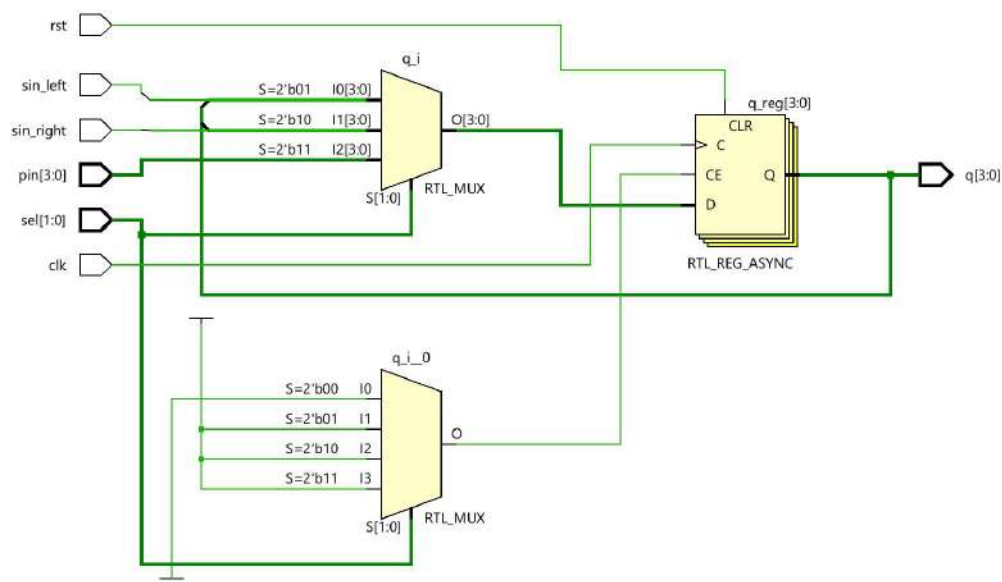
● Waveform



● Output

Time	Sel	Pin	SL	SR	Q
0	00	0000	0	0	0000
10000	11	1011	0	0	0000
15000	11	1011	0	0	1011
20000	00	1011	0	0	1011
30000	01	1011	1	0	1011
35000	01	1011	1	0	1101
40000	01	1011	0	0	1101
45000	01	1011	0	0	0110
50000	10	1011	0	1	0110
55000	10	1011	0	1	1101
60000	10	1011	0	0	1101
65000	10	1011	0	0	1010

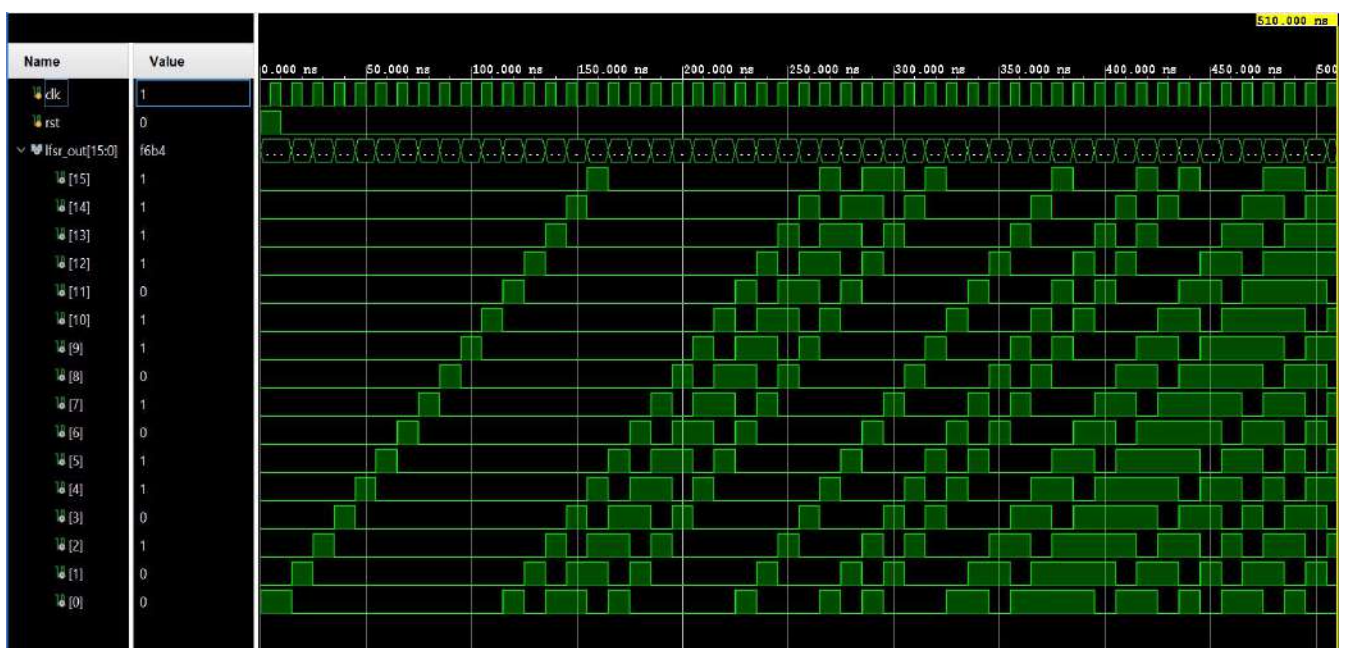
● Schematic



100. 16 bit LFSR

● Design Code	● Testbench Code
<pre> module lfsr(clk, rst, lfsr_out); input wire clk, rst; output reg [15:0] lfsr_out; wire feedback; assign feedback = lfsr_out[15] ^ lfsr_out[13] ^ lfsr_out[12] ^ lfsr_out[10]; always @(posedge clk or posedge rst) begin if (rst) lfsr_out <= 16'd1; else lfsr_out <= {lfsr_out[14:0], feedback}; // Shift left and insert feedback end endmodule </pre>	<pre> module lfsr_tb; reg clk, rst; wire [15:0] lfsr_out; lfsr uut (clk, rst, lfsr_out); // Clock generation initial begin clk = 0; forever #5 clk = ~clk; end initial begin \$display("Time\tReset\tLFSR_Out"); \$monitor("%0t\t%b\t%h", \$time, rst, lfsr_out); rst = 1; #10; rst = 0; #500 \$finish; end endmodule </pre>

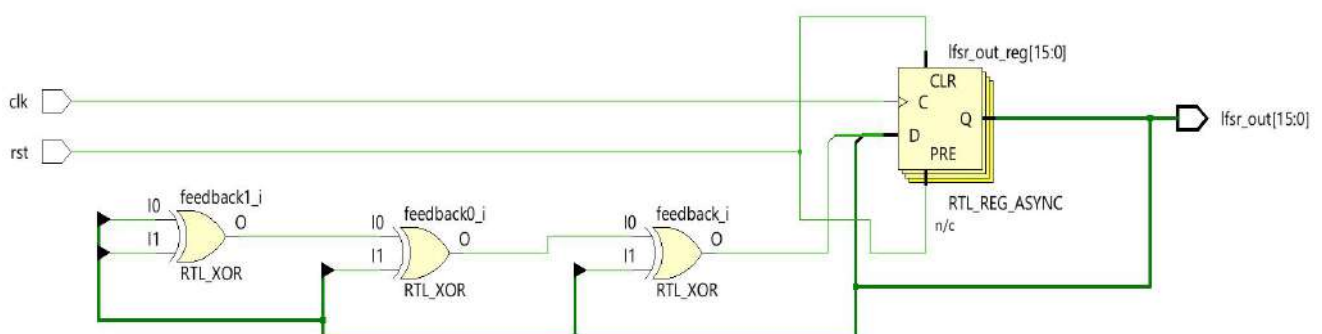
● Waveform



● Output

Time	Reset	LFSR_Out
0	1	0001
10000	0	0001
15000	0	0002
25000	0	0004
35000	0	0008
45000	0	0010
55000	0	0020
65000	0	0040
75000	0	0080
85000	0	0100
95000	0	0200
105000	0	0400
115000	0	0801
125000	0	1002
135000	0	2005
145000	0	400b
155000	0	8016
165000	0	002d
175000	0	005a
185000	0	00b4
195000	0	0168
205000	0	02d0
215000	0	05a0
225000	0	0b41
235000	0	1682
245000	0	2d04
255000	0	5a08
265000	0	b411
275000	0	6822
285000	0	d045
295000	0	a08a
305000	0	4114
315000	0	8228
325000	0	0451
335000	0	08a3
345000	0	1146
355000	0	228d
365000	0	451b
375000	0	8a37
385000	0	146f
395000	0	28de
405000	0	51bd
415000	0	a37b
425000	0	46f6
435000	0	8ded
445000	0	1bda
455000	0	37b5
465000	0	6f6b
475000	0	ded6
485000	0	bdad
495000	0	7b5a
505000	0	f6b4

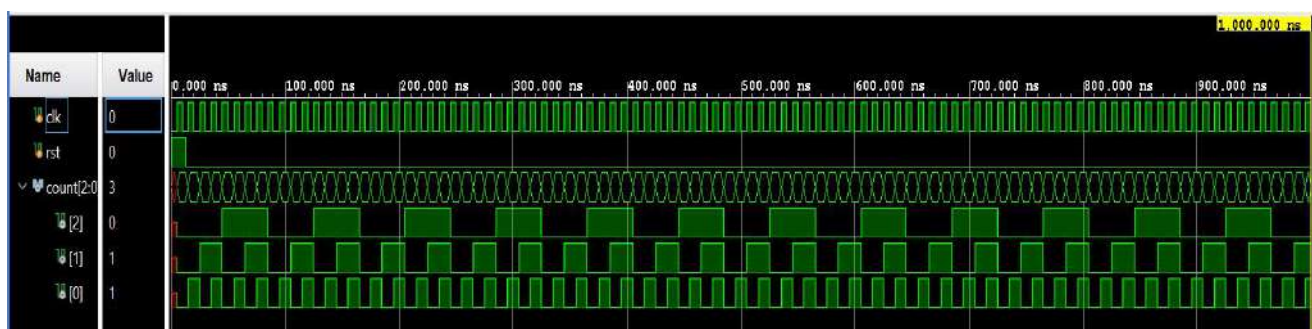
● Schematic



101. 3 bit up counter

● Design Code	● Testbench Code
<pre> module up_count_3bit(clk,rst,count); input clk,rst; output [2:0]count; reg [2:0]temp; always@(posedge clk) begin if(rst) temp<=1'b0; else temp<=temp+1; end assign count=temp; endmodule </pre>	<pre> module up_count_3bit_tb; reg clk,rst; wire [2:0]count; up_count_3bit dut(clk,rst,count); initial begin clk=0; rst=1; #13 rst=0; #1000 \$finish; end always #5 clk=~clk; initial begin \$monitor("Simtime=%0t, clk=%b, rst=%b, count=%b", \$time, clk, rst, count); end initial begin \$dumppfile("dump.vcd"); \$dumpvars(0,clk,rst,count); end endmodule </pre>

● Waveform



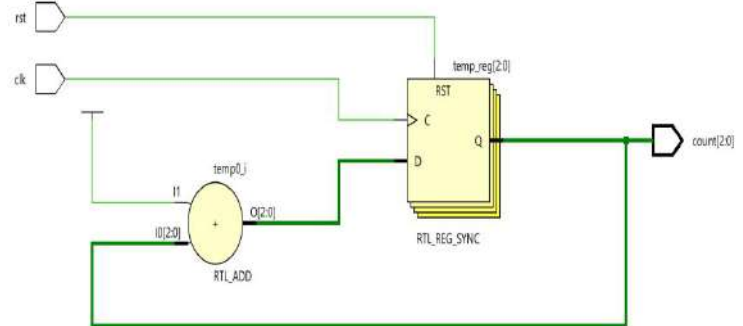
● Output

```

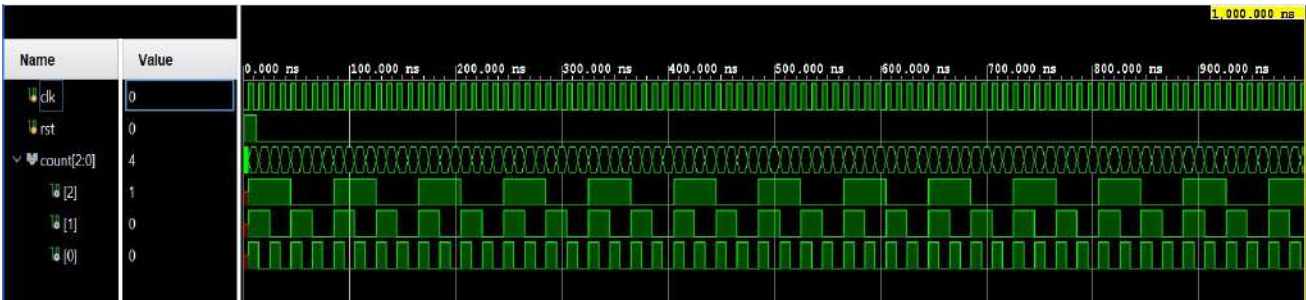
Simtime=0, clk=0, rst=1, count=xxx
Simtime=5000, clk=1, rst=1, count=000
Simtime=10000, clk=0, rst=1, count=000
Simtime=13000, clk=0, rst=0, count=000
Simtime=15000, clk=1, rst=0, count=001
Simtime=20000, clk=0, rst=0, count=001
Simtime=25000, clk=1, rst=0, count=010
Simtime=30000, clk=0, rst=0, count=010
Simtime=35000, clk=1, rst=0, count=011
Simtime=40000, clk=0, rst=0, count=011
Simtime=45000, clk=1, rst=0, count=100
Simtime=50000, clk=0, rst=0, count=100
Simtime=55000, clk=1, rst=0, count=101
Simtime=60000, clk=0, rst=0, count=101
Simtime=65000, clk=1, rst=0, count=110
Simtime=70000, clk=0, rst=0, count=110
Simtime=75000, clk=1, rst=0, count=111
Simtime=80000, clk=0, rst=0, count=111
Simtime=85000, clk=1, rst=0, count=000
Simtime=90000, clk=0, rst=0, count=000
Simtime=95000, clk=1, rst=0, count=001
Simtime=100000, clk=0, rst=0, count=001
Simtime=105000, clk=1, rst=0, count=010
Simtime=110000, clk=0, rst=0, count=010
Simtime=115000, clk=1, rst=0, count=011
Simtime=120000, clk=0, rst=0, count=011
Simtime=125000, clk=1, rst=0, count=100
Simtime=130000, clk=0, rst=0, count=100
Simtime=135000, clk=1, rst=0, count=101
Simtime=140000, clk=0, rst=0, count=101
Simtime=145000, clk=1, rst=0, count=110
Simtime=150000, clk=0, rst=0, count=110
Simtime=155000, clk=1, rst=0, count=111
Simtime=160000, clk=0, rst=0, count=111
Simtime=165000, clk=1, rst=0, count=000
Simtime=170000, clk=0, rst=0, count=000
Simtime=175000, clk=1, rst=0, count=001
Simtime=180000, clk=0, rst=0, count=001
Simtime=185000, clk=1, rst=0, count=010
Simtime=190000, clk=0, rst=0, count=010
Simtime=195000, clk=1, rst=0, count=011
Simtime=200000, clk=0, rst=0, count=011
Simtime=205000, clk=1, rst=0, count=100
Simtime=210000, clk=0, rst=0, count=100
Simtime=215000, clk=1, rst=0, count=101
Simtime=220000, clk=0, rst=0, count=101
Simtime=225000, clk=1, rst=0, count=110
Simtime=230000, clk=0, rst=0, count=110

```

● Schematic



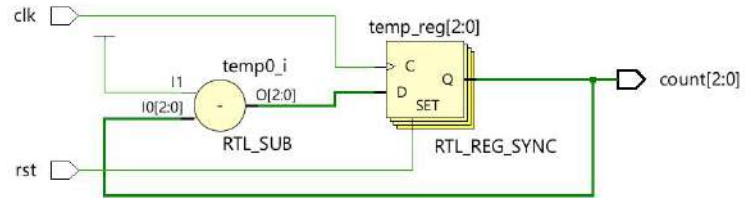
102. 3 bit down counter

● Design Code	● Testbench Code
<pre> module down_count_3bit(clk, rst, count); input clk, rst; output [2:0] count; reg [2:0] temp; always @(posedge clk) begin if (rst) temp <= 3'b111; else temp <= temp - 1; end assign count = temp; endmodule </pre>	<pre> module down_count_3bit_tb; reg clk; reg rst; wire [2:0] count; down_count_3bit uut (clk, rst, count); initial begin clk=0; rst=1; #13 rst=0; #1000 \$finish; end always #5 clk=~clk; initial begin \$monitor("Simtime=%0t, clk=%b, rst=%b, count=%b", \$time, clk, rst, count); end initial begin \$dumpfile("dump.vcd"); \$dumpvars(0,clk,rst,count); end endmodule </pre>
● Waveform	
	

● Output

Simtime=0, clk=0, rst=1, count=xxx
 Simtime=5000, clk=1, rst=1, count=111
 Simtime=10000, clk=0, rst=1, count=111
 Simtime=13000, clk=0, rst=0, count=111
 Simtime=15000, clk=1, rst=0, count=110
 Simtime=20000, clk=0, rst=0, count=110
 Simtime=25000, clk=1, rst=0, count=101
 Simtime=30000, clk=0, rst=0, count=101
 Simtime=35000, clk=1, rst=0, count=100
 Simtime=40000, clk=0, rst=0, count=100
 Simtime=45000, clk=1, rst=0, count=011
 Simtime=50000, clk=0, rst=0, count=011
 Simtime=55000, clk=1, rst=0, count=010
 Simtime=60000, clk=0, rst=0, count=010
 Simtime=65000, clk=1, rst=0, count=001
 Simtime=70000, clk=0, rst=0, count=001
 Simtime=75000, clk=1, rst=0, count=000
 Simtime=80000, clk=0, rst=0, count=000
 Simtime=85000, clk=1, rst=0, count=111
 Simtime=90000, clk=0, rst=0, count=111
 Simtime=95000, clk=1, rst=0, count=110
 Simtime=100000, clk=0, rst=0, count=110
 Simtime=105000, clk=1, rst=0, count=101
 Simtime=110000, clk=0, rst=0, count=101
 Simtime=115000, clk=1, rst=0, count=100
 Simtime=120000, clk=0, rst=0, count=100
 Simtime=125000, clk=1, rst=0, count=011
 Simtime=130000, clk=0, rst=0, count=011
 Simtime=135000, clk=1, rst=0, count=010
 Simtime=140000, clk=0, rst=0, count=010
 Simtime=145000, clk=1, rst=0, count=001
 Simtime=150000, clk=0, rst=0, count=001
 Simtime=155000, clk=1, rst=0, count=000
 Simtime=160000, clk=0, rst=0, count=000
 Simtime=165000, clk=1, rst=0, count=111
 Simtime=170000, clk=0, rst=0, count=111
 Simtime=175000, clk=1, rst=0, count=110
 Simtime=180000, clk=0, rst=0, count=110
 Simtime=185000, clk=1, rst=0, count=101
 Simtime=190000, clk=0, rst=0, count=101
 Simtime=195000, clk=1, rst=0, count=100
 Simtime=200000, clk=0, rst=0, count=100
 Simtime=205000, clk=1, rst=0, count=011
 Simtime=210000, clk=0, rst=0, count=011
 Simtime=215000, clk=1, rst=0, count=010
 Simtime=220000, clk=0, rst=0, count=010
 Simtime=225000, clk=1, rst=0, count=001
 Simtime=230000, clk=0, rst=0, count=001
 Simtime=235000, clk=1, rst=0, count=000
 Simtime=240000, clk=0, rst=0, count=000
 Simtime=245000, clk=1, rst=0, count=111
 Simtime=250000, clk=0, rst=0, count=111

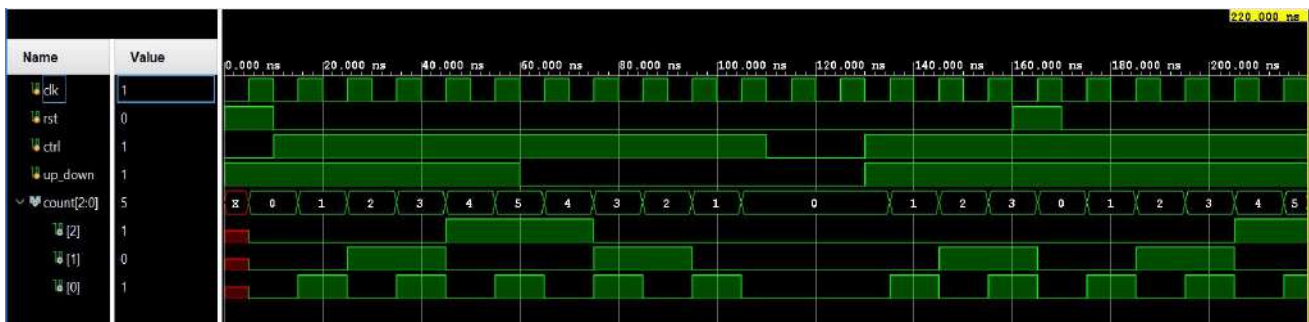
● Schematic



103. 3 bit up-down counter

● Design Code	● Testbench Code
<pre> module up_down_count_3bit(clk, rst, ctrl, up_down, count); input clk, rst, ctrl, up_down; output [2:0] count; reg [2:0] temp; always @(posedge clk) begin if (rst) temp <= 3'b000; else if (ctrl) begin if (up_down) temp <= temp + 1; else temp <= temp - 1; end end assign count = temp; endmodule </pre>	<pre> module up_down_count_3bit_tb; reg clk, rst, ctrl, up_down; wire [2:0] count; up_down_count_3bit uut (clk, rst, ctrl, up_down, count); always #5 clk = ~clk; initial begin // Initialize inputs clk = 0; rst = 1; ctrl = 0; up_down = 1; // Apply reset #10 rst = 0; // Count UP ctrl = 1; up_down = 1; #50; // Count DOWN up_down = 0; #50; // Stop counting ctrl = 0; #20; // Count UP again ctrl = 1; up_down = 1; #30; // Reset again rst = 1; #10 rst = 0; // Final UP count #50; \$finish; end // Monitor outputs initial begin \$monitor("Time = %0t clk = %b rst = %b ctrl = %b up_down = %b count = %b", \$time, clk, rst, ctrl, up_down, count); end endmodule </pre>

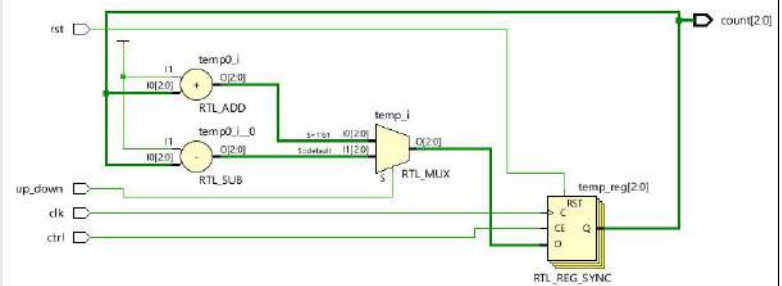
● Waveform



● Output

Time = 0 | clk = 0 | rst = 1 | ctrl = 0 | up_down = 1 | count = xxx
 Time = 5000 | clk = 1 | rst = 1 | ctrl = 0 | up_down = 1 | count = 000
 Time = 10000 | clk = 0 | rst = 0 | ctrl = 1 | up_down = 1 | count = 000
 Time = 15000 | clk = 1 | rst = 0 | ctrl = 1 | up_down = 1 | count = 001
 Time = 20000 | clk = 0 | rst = 0 | ctrl = 1 | up_down = 1 | count = 001
 Time = 25000 | clk = 1 | rst = 0 | ctrl = 1 | up_down = 1 | count = 010
 Time = 30000 | clk = 0 | rst = 0 | ctrl = 1 | up_down = 1 | count = 010
 Time = 35000 | clk = 1 | rst = 0 | ctrl = 1 | up_down = 1 | count = 011
 Time = 40000 | clk = 0 | rst = 0 | ctrl = 1 | up_down = 1 | count = 011
 Time = 45000 | clk = 1 | rst = 0 | ctrl = 1 | up_down = 1 | count = 100
 Time = 50000 | clk = 0 | rst = 0 | ctrl = 1 | up_down = 1 | count = 100
 Time = 55000 | clk = 1 | rst = 0 | ctrl = 1 | up_down = 1 | count = 101
 Time = 60000 | clk = 0 | rst = 0 | ctrl = 1 | up_down = 0 | count = 101
 Time = 65000 | clk = 1 | rst = 0 | ctrl = 1 | up_down = 0 | count = 100
 Time = 70000 | clk = 0 | rst = 0 | ctrl = 1 | up_down = 0 | count = 100
 Time = 75000 | clk = 1 | rst = 0 | ctrl = 1 | up_down = 0 | count = 011
 Time = 80000 | clk = 0 | rst = 0 | ctrl = 1 | up_down = 0 | count = 011
 Time = 85000 | clk = 1 | rst = 0 | ctrl = 1 | up_down = 0 | count = 010
 Time = 90000 | clk = 0 | rst = 0 | ctrl = 1 | up_down = 0 | count = 010
 Time = 95000 | clk = 1 | rst = 0 | ctrl = 1 | up_down = 0 | count = 001
 Time = 100000 | clk = 0 | rst = 0 | ctrl = 1 | up_down = 0 | count = 001
 Time = 105000 | clk = 1 | rst = 0 | ctrl = 1 | up_down = 0 | count = 000
 Time = 110000 | clk = 0 | rst = 0 | ctrl = 0 | up_down = 0 | count = 000
 Time = 115000 | clk = 1 | rst = 0 | ctrl = 0 | up_down = 0 | count = 000
 Time = 120000 | clk = 0 | rst = 0 | ctrl = 0 | up_down = 0 | count = 000
 Time = 125000 | clk = 1 | rst = 0 | ctrl = 0 | up_down = 0 | count = 000

● Schematic



104. 3 bit gradual increment-decrement counter

● Design Code

```

module gradual_counter(clk, rst, count);
  input clk, rst;
  output reg [2:0] count;
  reg dir; // 1 for up, 0 for down

  always @(posedge clk or posedge rst)
  begin
    if (rst)
      begin
        count <= 3'b000;
        dir <= 1'b1;
      end
    else begin
      if (dir)
        begin
          if (count == 3'b111)
            dir <= 0;
          else
            count <= count + 1;
          end
        else
          begin
            if (count == 3'b000)
              dir <= 1;
            else
              count <= count - 1;
            end
          end
        end
      end
    end
  end
endmodule

```

● Testbench Code

```

module gradual_counter_tb;
  reg clk, rst;
  wire [2:0] count;

  gradual_counter uut (clk, rst, count);

  initial
  begin
    $monitor("Time: %0d | count=%b", $time,
count);
    clk = 0; rst = 1;
    #5 rst = 0;

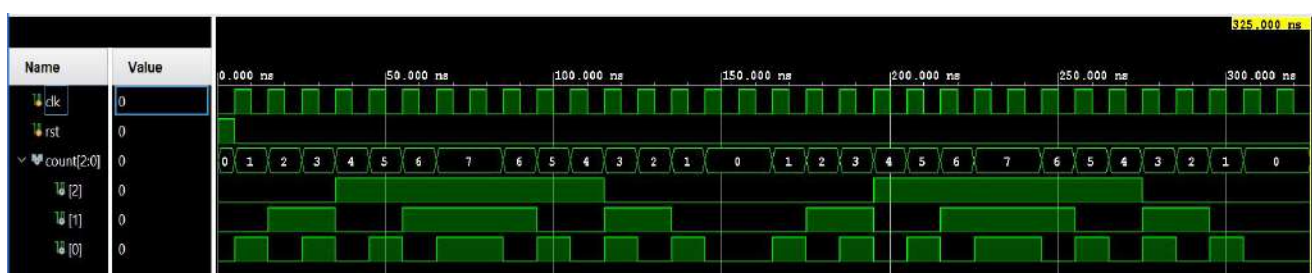
    repeat (32) #10;

    $finish;
  end

  always #5 clk = ~clk;
endmodule

```

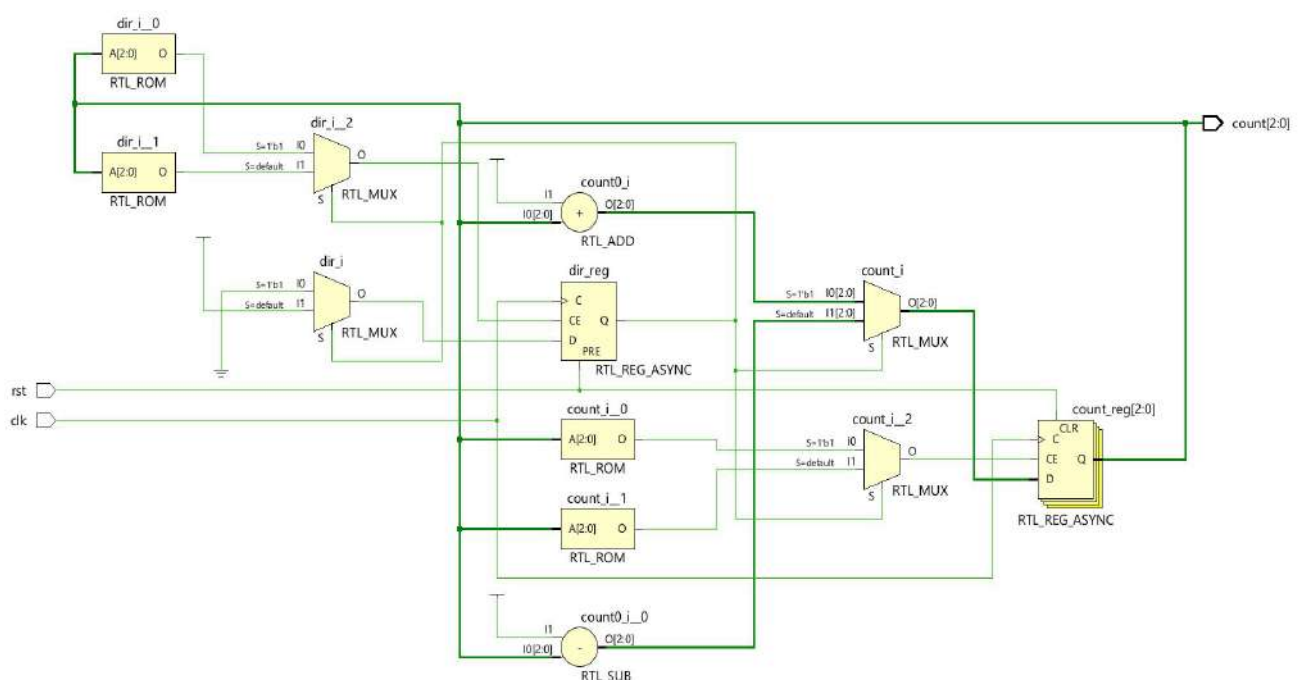
● Waveform



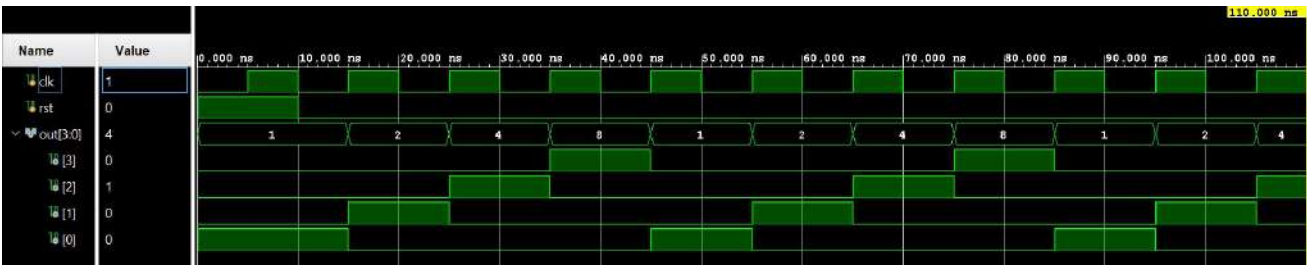
● Output

Time: 0 | count=000
 Time: 5 | count=001
 Time: 15 | count=010
 Time: 25 | count=011
 Time: 35 | count=100
 Time: 45 | count=101
 Time: 55 | count=110
 Time: 65 | count=111
 Time: 85 | count=110
 Time: 95 | count=101
 Time: 105 | count=100
 Time: 115 | count=011
 Time: 125 | count=010
 Time: 135 | count=001
 Time: 145 | count=000
 Time: 165 | count=001
 Time: 175 | count=010
 Time: 185 | count=011
 Time: 195 | count=100
 Time: 205 | count=101
 Time: 215 | count=110
 Time: 225 | count=111
 Time: 245 | count=110
 Time: 255 | count=101
 Time: 265 | count=100
 Time: 275 | count=011
 Time: 285 | count=010
 Time: 295 | count=001
 Time: 305 | count=000

● Schematic



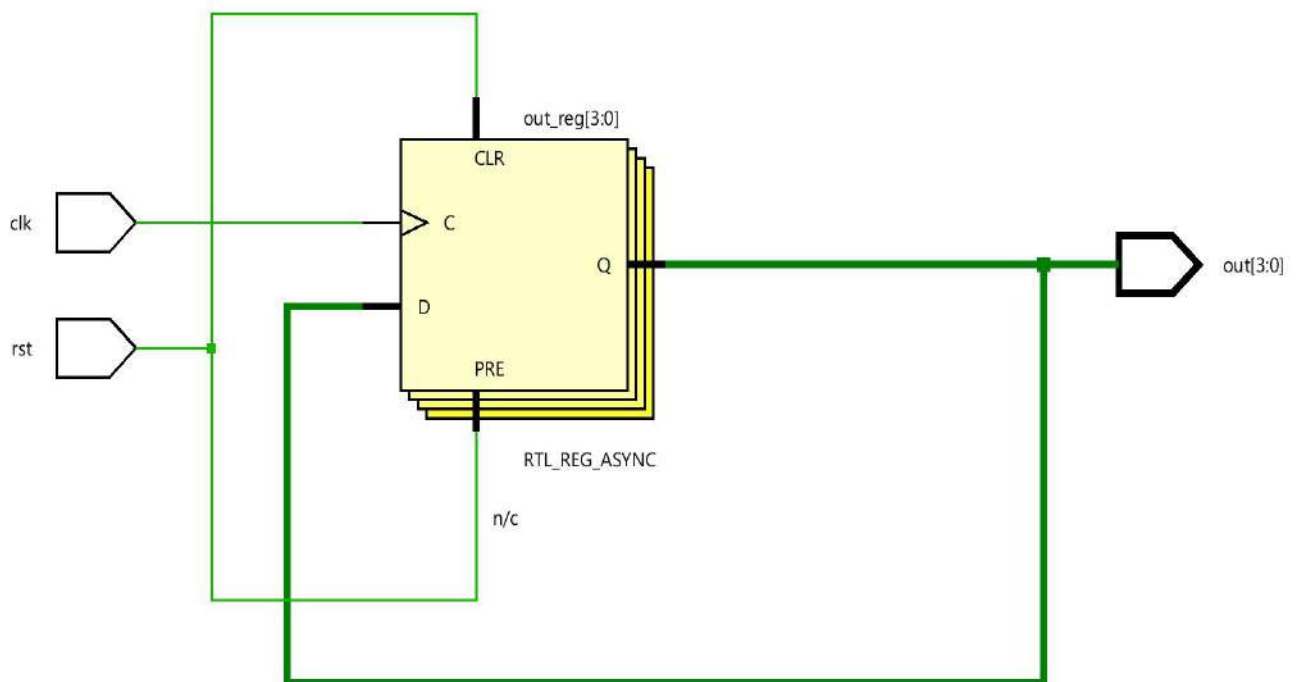
105. 4 bit Ring Counter

● Design Code	● Testbench Code
<pre> module ring_counter (clk, rst, out); input clk, rst; output reg [3:0] out; always @(posedge clk or posedge rst) begin if (rst) out <= 4'b0001; // initial state else out <= {out[2:0], out[3]}; // rotate left end endmodule </pre>	<pre> module ring_counter_tb; reg clk, rst; wire [3:0] out; ring_counter uut (clk, rst, out); initial begin clk = 0; forever #5 clk = ~clk; // 10 time units clock period end initial begin \$display("Time\tRST\tOUT"); \$monitor("%0t\t%b\t%b", \$time, rst, out); rst = 1; #10; rst = 0; #100; // run for 10 clock cycles \$finish; end endmodule </pre>
● Waveform	
	

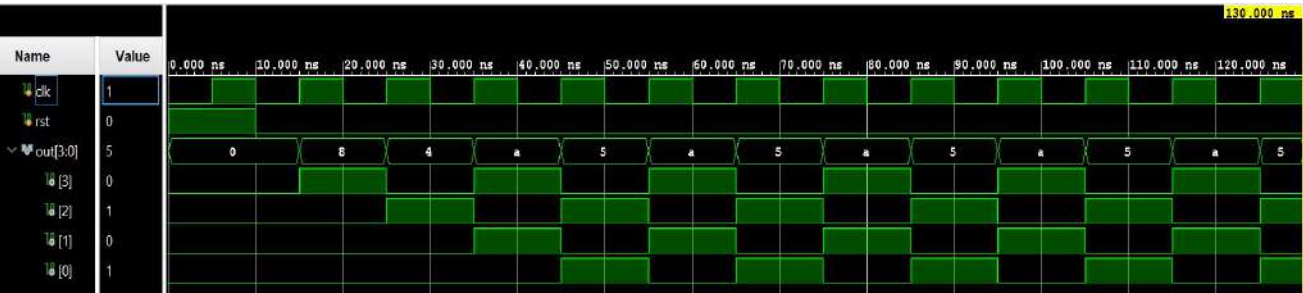
● Output

Time	RST	OUT
0	1	0001
10000	0	0001
15000	0	0010
25000	0	0100
35000	0	1000
45000	0	0001
55000	0	0010
65000	0	0100
75000	0	1000
85000	0	0001
95000	0	0010
105000	0	0100

● Schematic



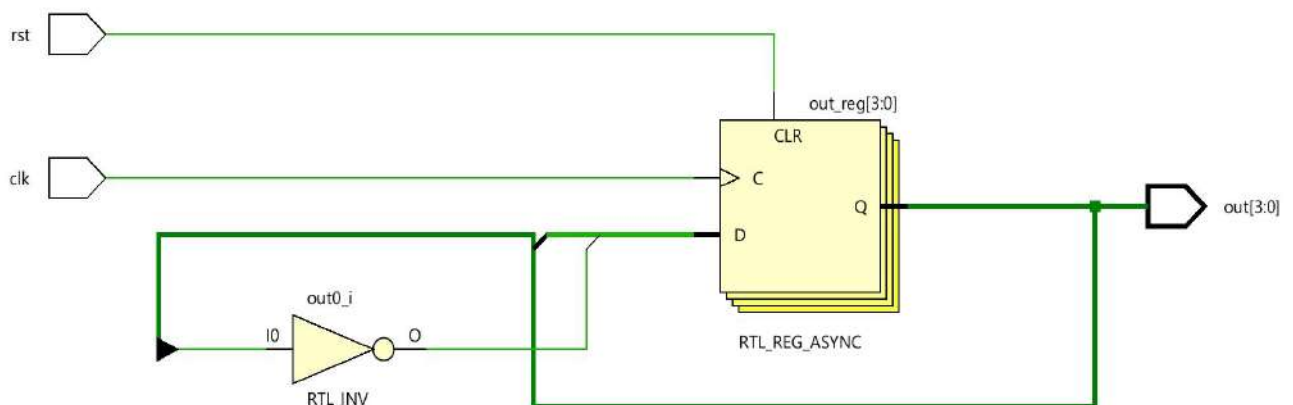
106. 4 bit Johnson Counter

● Design Code	● Testbench Code																
<pre> module johnson_counter (clk, rst, out); input clk, rst; output reg [3:0] out; always @(posedge clk or posedge rst) begin if (rst) out <= 4'b0000; else out <= {~out[3], out[3:1]}; // feedback inverted MSB end endmodule </pre>	<pre> module johnson_counter_tb; reg clk, rst; wire [3:0] out; johnson_counter uut (clk, rst, out); initial begin clk = 0; forever #5 clk = ~clk; end initial begin \$display("Time\tRST\tOUT"); \$monitor("%0t\t%b\t%b", \$time, rst, out); rst = 1; #10; rst = 0; #120; // enough cycles to complete full sequence \$finish; end endmodule </pre>																
● Waveform																	
 <table border="1" data-bbox="151 1825 311 2078"> <thead> <tr> <th>Name</th> <th>Value</th> </tr> </thead> <tbody> <tr> <td>clk</td> <td>1</td> </tr> <tr> <td>rst</td> <td>0</td> </tr> <tr> <td>out[3:0]</td> <td>5</td> </tr> <tr> <td>out[3]</td> <td>0</td> </tr> <tr> <td>out[2]</td> <td>1</td> </tr> <tr> <td>out[1]</td> <td>0</td> </tr> <tr> <td>out[0]</td> <td>1</td> </tr> </tbody> </table>		Name	Value	clk	1	rst	0	out[3:0]	5	out[3]	0	out[2]	1	out[1]	0	out[0]	1
Name	Value																
clk	1																
rst	0																
out[3:0]	5																
out[3]	0																
out[2]	1																
out[1]	0																
out[0]	1																

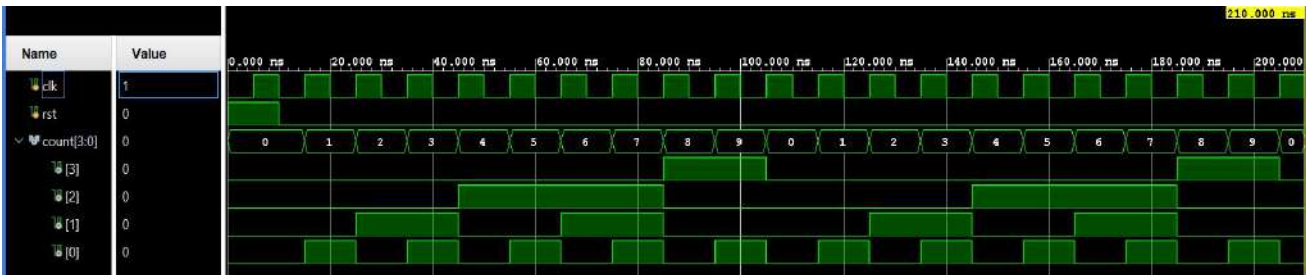
- **Output**

Time	RST	OUT
0	1	0000
10000	0	0000
15000	0	1000
25000	0	0100
35000	0	1010
45000	0	0101
55000	0	1010
65000	0	0101
75000	0	1010
85000	0	0101
95000	0	1010
105000	0	0101
115000	0	1010
125000	0	0101

- **Schematic**



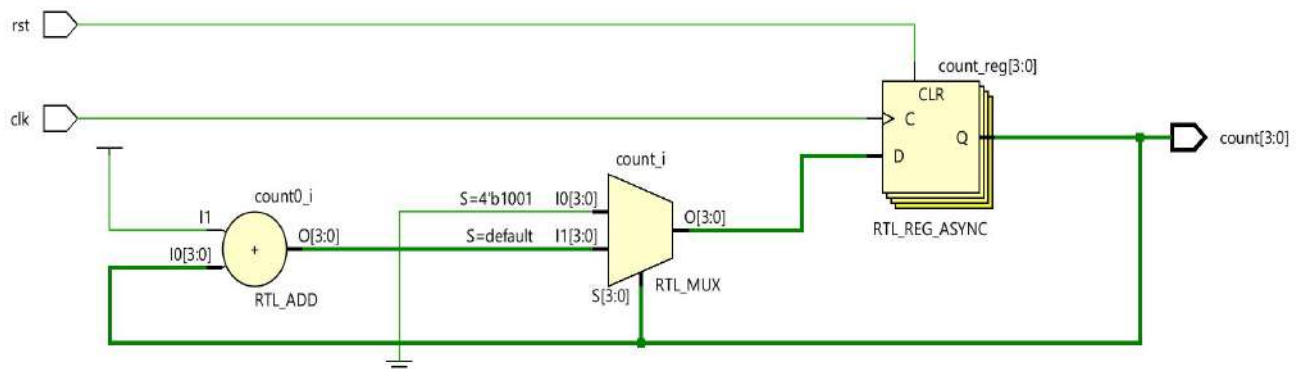
107. MOD-10 Upcounter

● Design Code	● Testbench Code
<pre> module mod10_upcounter (clk, rst, count); input clk, rst; output reg [3:0] count; always @(posedge clk or posedge rst) begin if (rst) count <= 4'd0; else if (count == 4'd9) count <= 4'd0; else count <= count + 1; end endmodule </pre>	<pre> module mod10_upcounter_tb; reg clk, rst; wire [3:0] count; mod10_upcounter uut (clk, rst, count); initial begin clk = 0; forever #5 clk = ~clk; end initial begin \$display("Time\tReset\tCount"); \$monitor("%0t\t%b\t%0d", \$time, rst, count); rst = 1; #10; rst = 0; #200; \$finish; end endmodule </pre>
● Waveform	
	

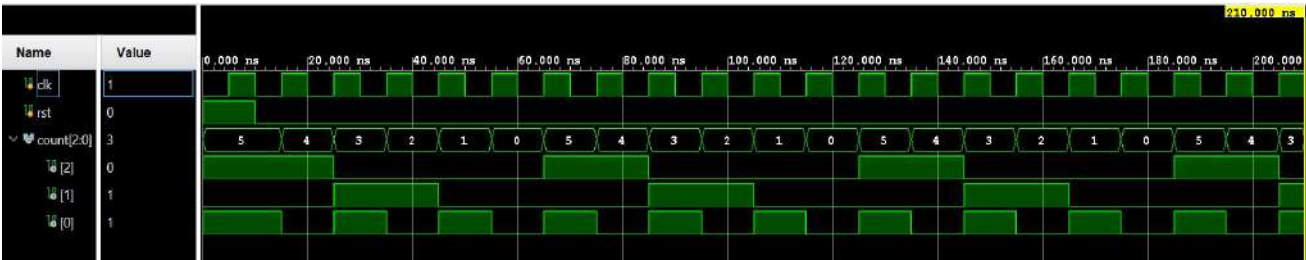
● Output

Time	Reset	Count
0	1	0
10000	0	0
15000	0	1
25000	0	2
35000	0	3
45000	0	4
55000	0	5
65000	0	6
75000	0	7
85000	0	8
95000	0	9
105000	0	0
115000	0	1
125000	0	2
135000	0	3
145000	0	4
155000	0	5
165000	0	6
175000	0	7
185000	0	8
195000	0	9
205000	0	0

● Schematic



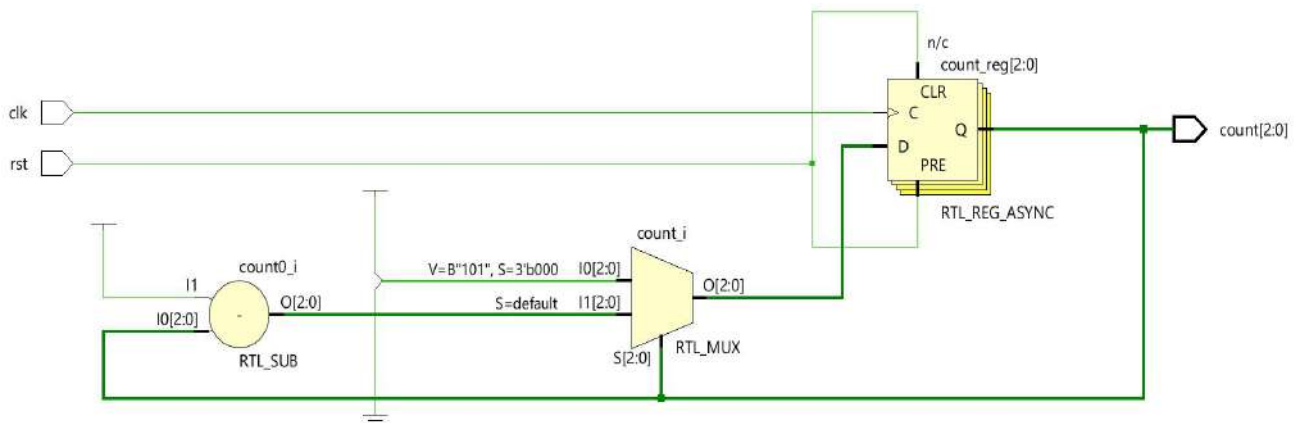
108. MOD-6 Downcounter

● Design Code	● Testbench Code														
<pre> module mod6_downcounter (clk, rst, count); input clk, rst; output reg [2:0] count; always @(posedge clk or posedge rst) begin if (rst) count <= 3'd5; else if (count == 3'd0) count <= 3'd5; else count <= count - 1; end endmodule </pre>	<pre> module mod6_downcounter_tb; reg clk, rst; wire [2:0] count; mod6_downcounter uut (clk, rst, count); initial begin clk = 0; forever #5 clk = ~clk; end initial begin \$display("Time\tReset\tCount"); \$monitor("%0t\t%b\t%0d", \$time, rst, count); rst = 1; #10; rst = 0; #200; \$finish; end endmodule </pre>														
● Waveform															
 <table border="1" data-bbox="148 1825 343 2083"> <thead> <tr> <th>Name</th> <th>Value</th> </tr> </thead> <tbody> <tr> <td>clk</td> <td>1</td> </tr> <tr> <td>rst</td> <td>0</td> </tr> <tr> <td>count[2:0]</td> <td>3</td> </tr> <tr> <td>count[2]</td> <td>0</td> </tr> <tr> <td>count[1]</td> <td>1</td> </tr> <tr> <td>count[0]</td> <td>1</td> </tr> </tbody> </table>		Name	Value	clk	1	rst	0	count[2:0]	3	count[2]	0	count[1]	1	count[0]	1
Name	Value														
clk	1														
rst	0														
count[2:0]	3														
count[2]	0														
count[1]	1														
count[0]	1														

● Output

Time	Reset	Count
0	1	5
10000	0	5
15000	0	4
25000	0	3
35000	0	2
45000	0	1
55000	0	0
65000	0	5
75000	0	4
85000	0	3
95000	0	2
105000	0	1
115000	0	0
125000	0	5
135000	0	4
145000	0	3
155000	0	2
165000	0	1
175000	0	0
185000	0	5
195000	0	4
205000	0	3

● Schematic



109. State Skipping Counter

● Design Code

```
module state_skipping_counter(clk, rst,
count);
input clk, rst;
output reg [2:0] count;

always @(posedge clk or posedge rst)
begin
if (rst)
count <= 3'd0;
else
begin
case (count)
3'd0: count <= 3'd2;
3'd2: count <= 3'd5;
3'd5: count <= 3'd6;
3'd6: count <= 3'd3;
3'd3: count <= 3'd4;
3'd4: count <= 3'd0;
default: count <= 3'd0;
endcase
end
end

endmodule
```

● Testbench Code

```
module state_skipping_counter_tb;
reg clk, rst;
wire [2:0] count;

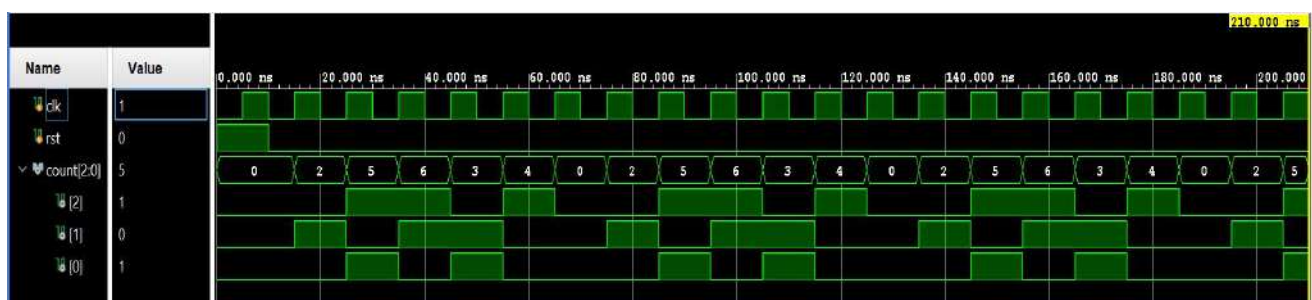
state_skipping_counter uut (clk, rst, count);

initial
begin
clk = 0;
forever #5 clk = ~clk;
end

initial
begin
$display("Time\tReset\tCount");
$monitor("%0t\t%b\t%0d", $time, rst, count);

rst = 1; #10;
rst = 0;
#200;
$finish;
end
endmodule
```

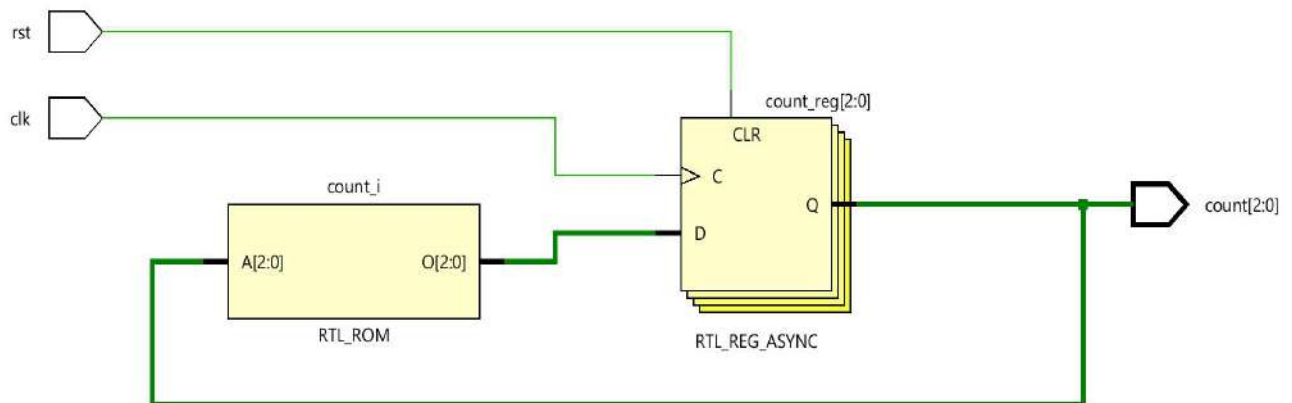
● Waveform



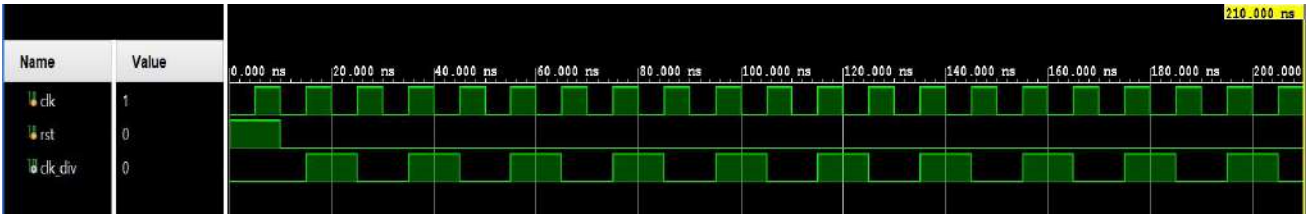
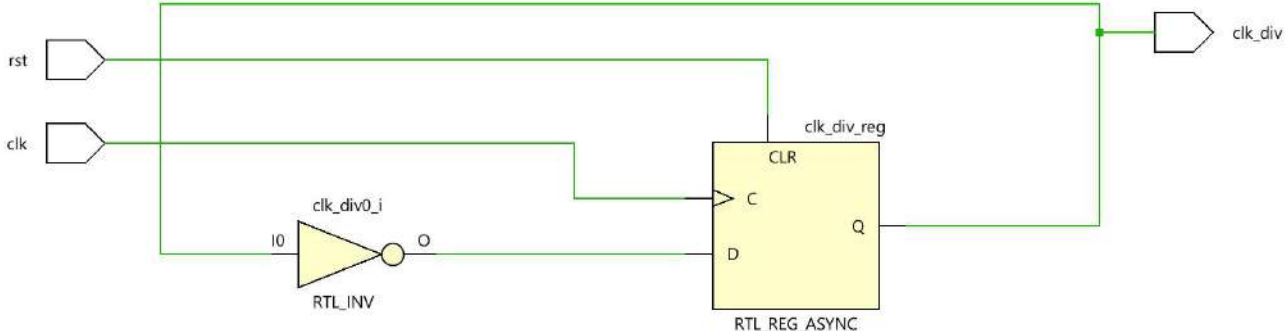
● Output

Time	Reset	Count
0	1	0
10000	0	0
15000	0	2
25000	0	5
35000	0	6
45000	0	3
55000	0	4
65000	0	0
75000	0	2
85000	0	5
95000	0	6
105000	0	3
115000	0	4
125000	0	0
135000	0	2
145000	0	5
155000	0	6
165000	0	3
175000	0	4
185000	0	0
195000	0	2
205000	0	5

● Schematic



110. Time period division by 2

● Design Code	● Testbench Code								
<pre> module clk_div2(clk, rst, clk_div); input clk, rst; output reg clk_div; always @(posedge clk or posedge rst) begin if (rst) clk_div <= 0; else clk_div <= ~clk_div; end endmodule </pre>	<pre> module clk_div2_tb; reg clk = 0, rst; wire clk_div; clk_div2 uut (clk, rst, clk_div); always #5 clk = ~clk; initial begin rst = 1; #10; rst = 0; #200; \$finish; end endmodule </pre>								
● Waveform									
 <table border="1" data-bbox="148 1317 371 1485"> <thead> <tr> <th>Name</th> <th>Value</th> </tr> </thead> <tbody> <tr> <td>clk</td> <td>1</td> </tr> <tr> <td>rst</td> <td>0</td> </tr> <tr> <td>clk_div</td> <td>0</td> </tr> </tbody> </table>		Name	Value	clk	1	rst	0	clk_div	0
Name	Value								
clk	1								
rst	0								
clk_div	0								
● Schematic									
									

111. Time period division by 4

● Design Code

```

module clk_div4(clk, rst, clk_div);
input clk, rst;
output reg clk_div;
reg [1:0] count;

always @(posedge clk or posedge rst)
begin
if (rst)
begin
count <= 0;
clk_div <= 0;
end
else
begin
count <= count + 1;
if (count == 2'd3) clk_div <= ~clk_div;
end
end
endmodule

```

● Testbench Code

```

module clk_div4_tb;
reg clk = 0, rst;
wire clk_div;

clk_div4 uut (clk, rst, clk_div);

always #5 clk = ~clk;

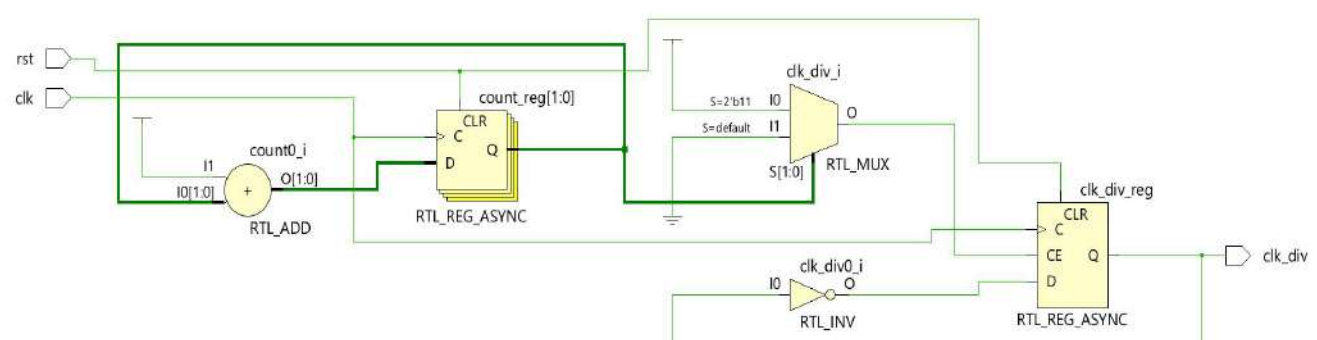
initial
begin
rst = 1; #10;
rst = 0; #400;
$finish;
end
endmodule

```

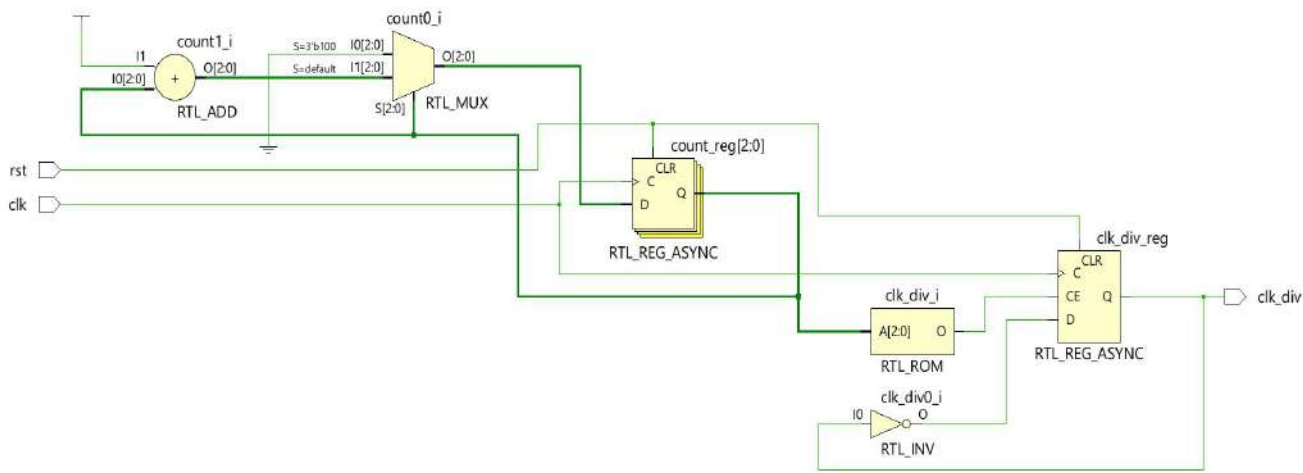
● Waveform



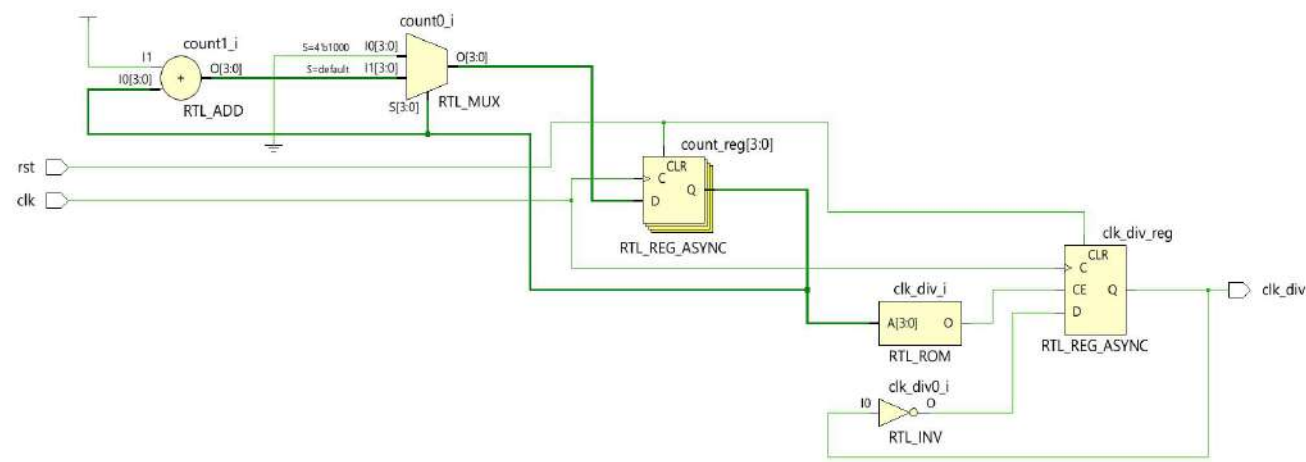
● Schematic



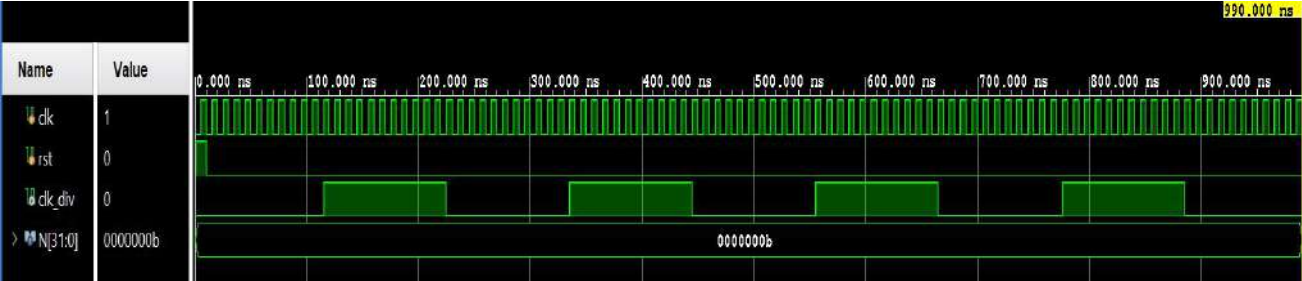
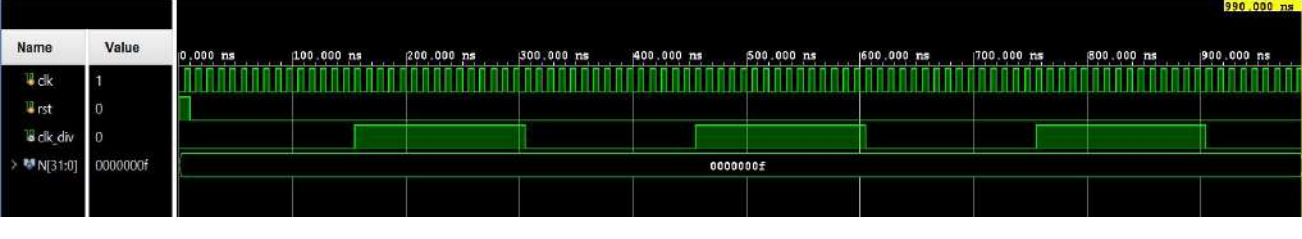
112. Time period division by 5

● Design Code	● Testbench Code								
<pre> module clk_div5(clk, rst, clk_div); input clk, rst; output reg clk_div; reg [2:0] count; always @(posedge clk or posedge rst) begin if (rst) begin count <= 0; clk_div <= 0; end else begin count <= (count == 4) ? 0 : count + 1; if (count == 2) clk_div <= ~clk_div; end end endmodule </pre>	<pre> module clk_div5_tb; reg clk = 0, rst; wire clk_div; clk_div5 uut (clk, rst, clk_div); always #5 clk = ~clk; initial begin rst = 1; #10; rst = 0; #600; \$finish; end endmodule </pre>								
● Waveform									
 <table border="1" data-bbox="151 1232 311 1422"> <thead> <tr> <th>Name</th> <th>Value</th> </tr> </thead> <tbody> <tr> <td>clk</td> <td>1</td> </tr> <tr> <td>rst</td> <td>0</td> </tr> <tr> <td>clk_div</td> <td>0</td> </tr> </tbody> </table>		Name	Value	clk	1	rst	0	clk_div	0
Name	Value								
clk	1								
rst	0								
clk_div	0								
● Schematic									
									

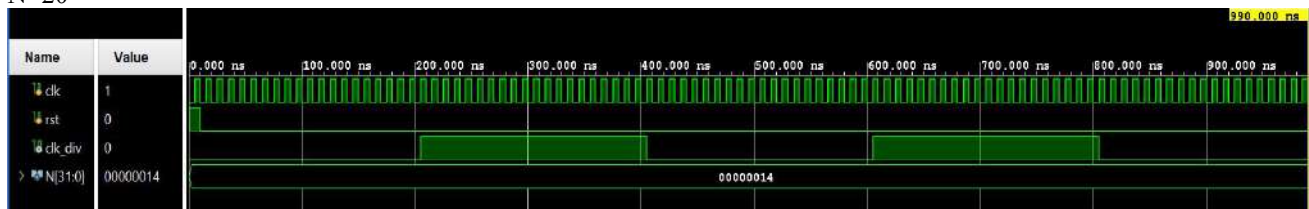
113. Time period division by 9

● Design Code	● Testbench Code
<pre> module clk_div9(clk, rst, clk_div); input clk, rst; output reg clk_div; reg [3:0] count; always @(posedge clk or posedge rst) begin if (rst) begin count <= 0; clk_div <= 0; end else begin count <= (count == 8) ? 0 : count + 1; if (count == 4) clk_div <= ~clk_div; end end endmodule </pre>	<pre> module clk_div9_tb; reg clk = 0, rst; wire clk_div; clk_div9 uut (clk, rst, clk_div); always #5 clk = ~clk; initial begin rst = 1; #10; rst = 0; #950; \$finish; end endmodule </pre>
● Waveform	
 The waveform shows three signals over a 960,000 ns period. The 'clk' signal is a periodic square wave. The 'rst' signal is high for the first 10 ns and then low. The 'clk_div' signal is a square wave that toggles at count 4, resulting in a period that is 9 times the input clock period.	
● Schematic	
 The schematic diagram illustrates the implementation of the clk_div9 module. It features a 4-bit counter (count0_i) implemented using an RTL_ADD block and an RTL_MUX block. The counter is reset by the rst signal. The output of the counter is connected to a 4-bit register (count_reg[3:0]) via an RTL_REG_ASYNC block. The register is also reset by rst. The output of the register is connected to a 4-bit register (clk_div_reg) via an RTL_REG_ASYNC block. The register is also reset by rst. The output of the register is connected to the clk_div output via an RTL_ROM block and an RTL_INV block.	

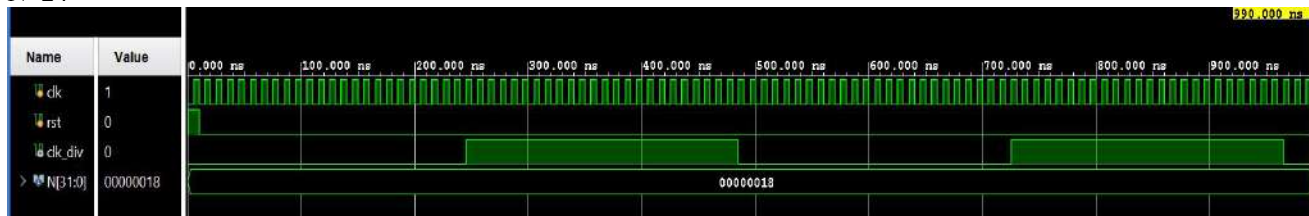
114. Generic N-bit Clock Divider

● Design Code	● Testbench Code
<pre> module clk_div_n #(parameter N = 11)(clk, rst, clk_div); input clk, rst; output reg clk_div; reg [\$clog2(N)-1:0] count; always @(posedge clk or posedge rst) begin if (rst) begin count <= 0; clk_div <= 0; end else begin if (count == N-1) begin count <= 0; clk_div <= ~clk_div; end else begin count <= count + 1; end end end endmodule </pre>	<pre> module clk_div_n_tb; reg clk = 0, rst; wire clk_div; parameter N = 11; clk_div_n #(N) uut (clk, rst, clk_div); always #5 clk = ~clk; initial begin rst = 1; #10; rst = 0; #980; \$finish; end endmodule </pre>
● Waveform	
N=11 	
N=15 	

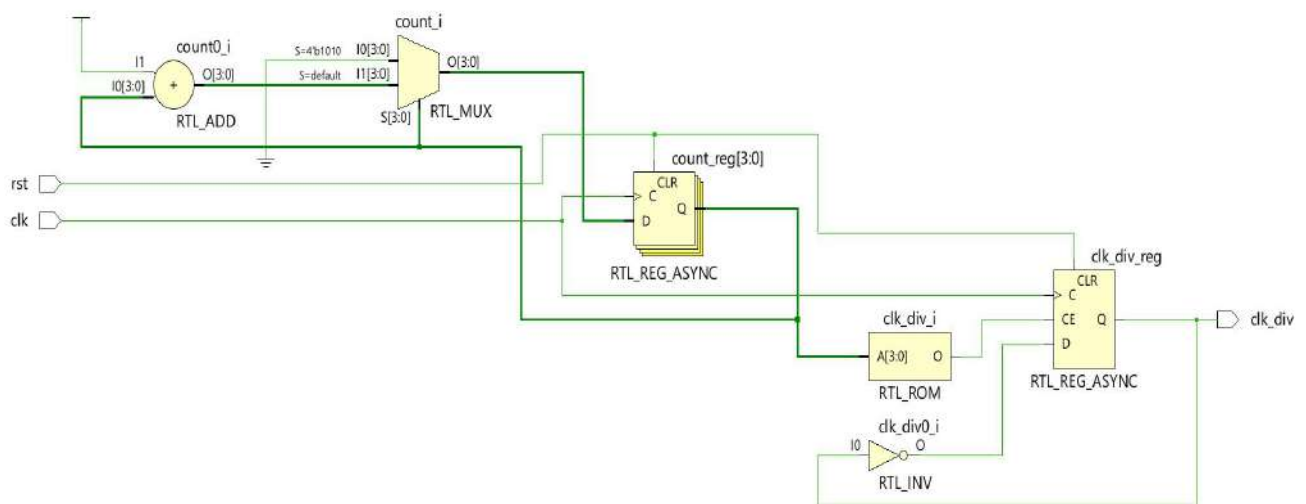
N=20



N=24



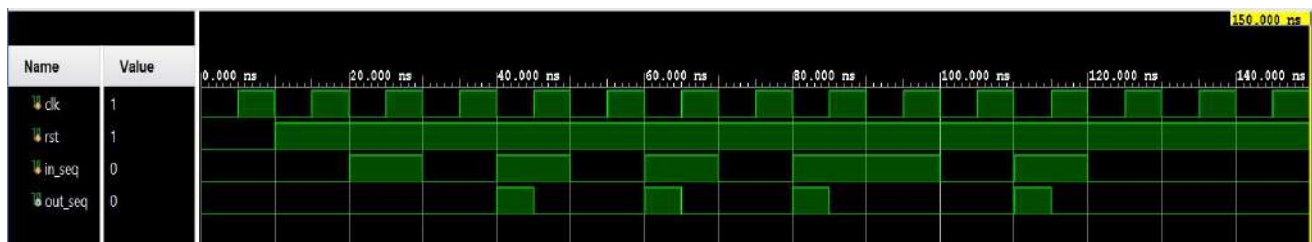
● Schematic



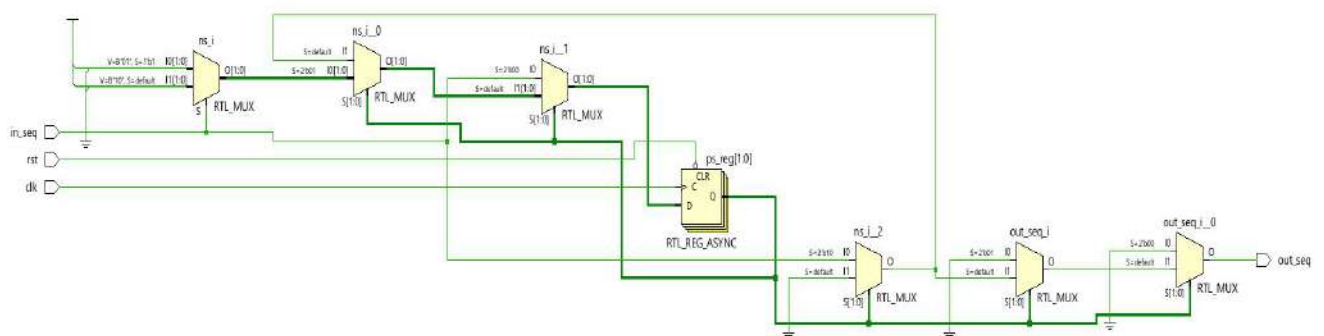
115. Mealy Overlapping (for sequence 101)

● Design Code	● Testbench Code
<pre> module mealy_101_overlap(clk, rst, in_seq, out_seq); input clk, rst, in_seq; output reg out_seq; reg [1:0] ps, ns; parameter S0 = 2'b00; parameter S1 = 2'b01; parameter S10 = 2'b10; always @(posedge clk or negedge rst) if (!rst) ps <= S0; else ps <= ns; always @(*) begin out_seq = 0; if (ps == S0) begin if (in_seq == 1) ns = S1; else ns = S0; end else if (ps == S1) begin if (in_seq == 1) ns = S1; else ns = S10; end else if (ps == S10) begin if (in_seq == 1) begin ns = S1; out_seq = 1; end else ns = S0; end else ns = S0; end endmodule </pre>	<pre> module mealy_101_overlap_tb; reg clk, rst, in_seq; wire out_seq; mealy_101_overlap dut(clk, rst, in_seq, out_seq); initial begin clk = 0; forever #5 clk = ~clk; end initial begin rst = 0; in_seq = 0; #10 rst = 1; #10 in_seq = 1; #10 in_seq = 0; #10 in_seq = 1; // 101 -> detect #10 in_seq = 0; #10 in_seq = 1; // 101 -> detect again (overlap) #10 in_seq = 0; #10 in_seq = 1; // 101 #10 in_seq = 0; #10 in_seq = 1; // 101 #10 in_seq = 0; #10 in_seq = 0; #20 \$finish; end endmodule </pre>

● Waveform



● Schematic

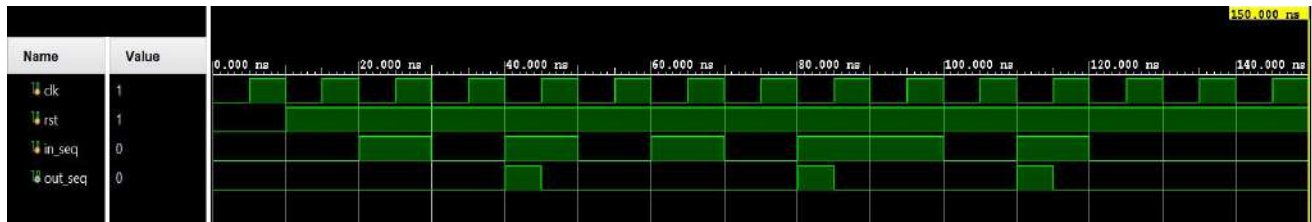


116. Mealy Non-Overlapping (for sequence 101)

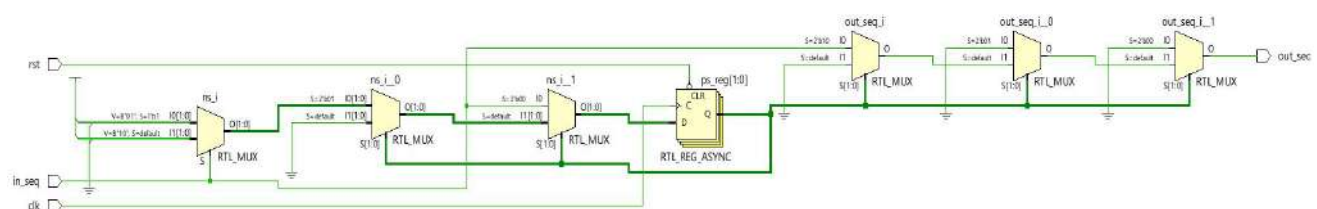
● Design Code	● Testbench Code
<pre> module mealy_101_nonoverlap(clk, rst, in_seq, out_seq); input clk, rst, in_seq; output reg out_seq; reg [1:0] ps, ns; parameter S0 = 2'b00; parameter S1 = 2'b01; parameter S10 = 2'b10; always @(posedge clk or negedge rst) if (!rst) ps <= S0; else ps <= ns; always @(*) begin out_seq = 0; if (ps == S0) begin if (in_seq == 1) ns = S1; else ns = S0; end else if (ps == S1) begin if (in_seq == 1) ns = S1; else ns = S10; end else if (ps == S10) begin if (in_seq == 1) begin ns = S0; out_seq = 1; end else ns = S0; end else ns = S0; end end </pre>	<pre> module mealy_101_nonoverlap_tb; reg clk, rst, in_seq; wire out_seq; mealy_101_nonoverlap dut(clk, rst, in_seq, out_seq); initial begin clk = 0; forever #5 clk = ~clk; end initial begin rst = 0; in_seq = 0; #10 rst = 1; #10 in_seq = 1; #10 in_seq = 0; #10 in_seq = 1; // detect (non-overlap: reset to S0) #10 in_seq = 0; #10 in_seq = 1; #10 in_seq = 0; #10 in_seq = 1; #10 in_seq = 0; #10 in_seq = 1; #10 in_seq = 0; #10 in_seq = 0; #20 \$finish; end endmodule </pre>

endmodule

● Waveform



● Schematic



117. Moore Overlapping (for sequence 101)

● Design Code	● Testbench Code
<pre> module moore_101_overlap(clk, rst, in_seq, out_seq); input clk, rst, in_seq; output reg out_seq; reg [2:0] ps, ns; parameter S0 = 3'b000; parameter S1 = 3'b001; parameter S10 = 3'b010; parameter S101 = 3'b011; always @(posedge clk or negedge rst) if (!rst) ps <= S0; else ps <= ns; always @(*) begin if (ps == S0) begin if (in_seq == 1) ns = S1; else ns = S0; end else if (ps == S1) begin if (in_seq == 1) ns = S1; else ns = S10; end else if (ps == S10) begin if (in_seq == 1) ns = S101; else ns = S0; end else if (ps == S101) begin if (in_seq == 1) ns = S1; else </pre>	<pre> module moore_101_overlap_tb; reg clk, rst, in_seq; wire out_seq; moore_101_overlap dut(clk, rst, in_seq, out_seq); initial begin clk = 0; forever #5 clk = ~clk; end initial begin rst = 0; in_seq = 0; #10 rst = 1; #10 in_seq = 1; #10 in_seq = 0; #10 in_seq = 1; // detect #10 in_seq = 0; #10 in_seq = 1; // detect again #10 in_seq = 0; #10 in_seq = 1; #10 in_seq = 1; #10 in_seq = 0; #10 in_seq = 1; // detect again #10 in_seq = 0; #10 in_seq = 0; #20 \$finish; end endmodule </pre>

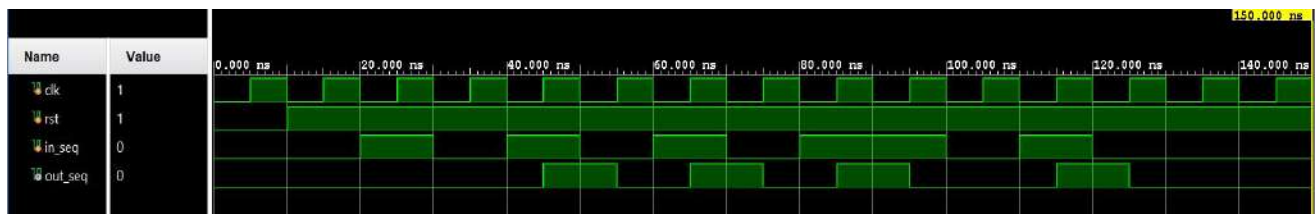
```

    ns = S10;
end
else
    ns = S0;
end

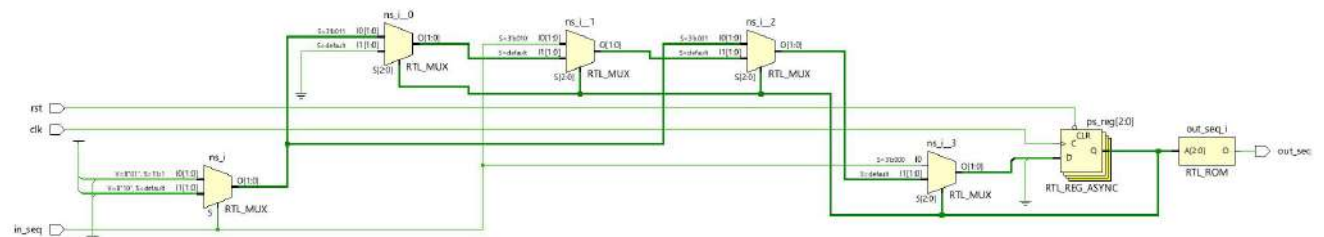
always @(*)
begin
    if (ps == S101)
        out_seq = 1;
    else
        out_seq = 0;
    end
end
endmodule

```

● Waveform



● Schematic



118. Moore Non-Overlapping (for sequence 101)

● Design Code	● Testbench Code
<pre> module moore_101_nonoverlap(clk, rst, in_seq, out_seq); input clk, rst, in_seq; output reg out_seq; reg [2:0] ps, ns; parameter S0 = 3'b000; parameter S1 = 3'b001; parameter S10 = 3'b010; parameter S101 = 3'b011; always @(posedge clk or negedge rst) if (!rst) ps <= S0; else ps <= ns; always @(*) begin if (ps == S0) begin if (in_seq == 1) ns = S1; else ns = S0; end else if (ps == S1) begin if (in_seq == 1) ns = S1; else ns = S10; end else if (ps == S10) begin if (in_seq == 1) ns = S101; else ns = S0; end else if (ps == S101) begin ns = S0; // Reset for non-overlapping end else </pre>	<pre> module moore_101_nonoverlap_tb; reg clk, rst, in_seq; wire out_seq; moore_101_nonoverlap dut(clk, rst, in_seq, out_seq); initial begin clk = 0; forever #5 clk = ~clk; end initial begin rst = 0; in_seq = 0; #10 rst = 1; #10 in_seq = 1; #10 in_seq = 0; #10 in_seq = 1; // detect #10 in_seq = 0; #10 in_seq = 1; // won't overlap #10 in_seq = 0; #10 in_seq = 1; #10 in_seq = 1; #10 in_seq = 0; #10 in_seq = 1; #10 in_seq = 0; #10 in_seq = 0; #20 \$finish; end endmodule </pre>

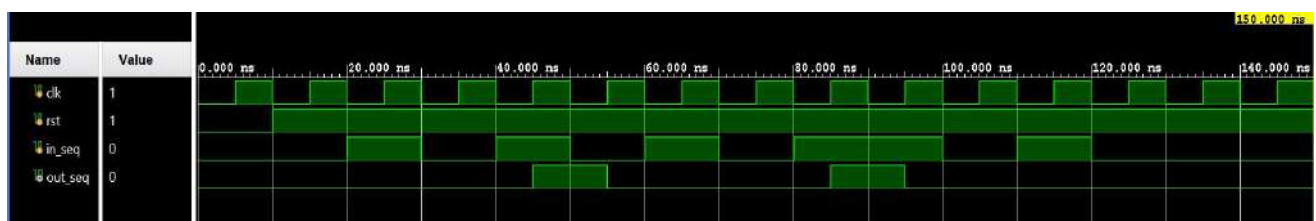
```

    ns = S0;
end

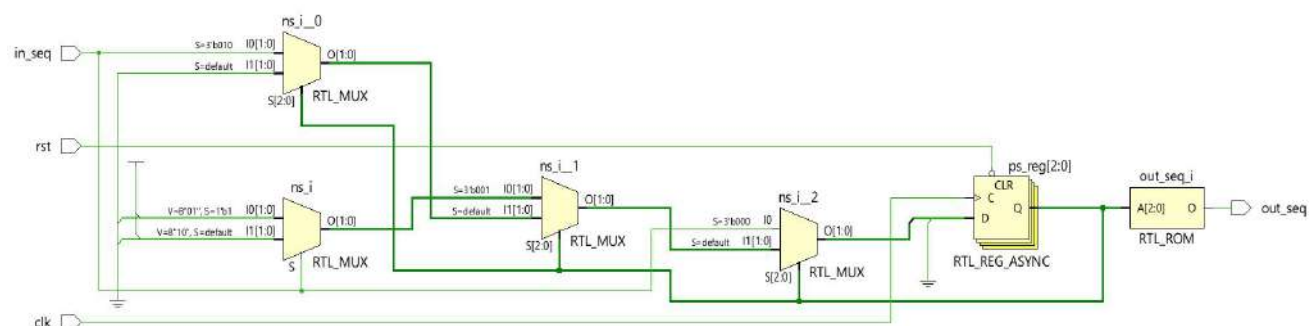
always @(*)
begin
    if (ps == S101)
        out_seq = 1;
    else
        out_seq = 0;
    end
end
endmodule

```

● Waveform



● Schematic

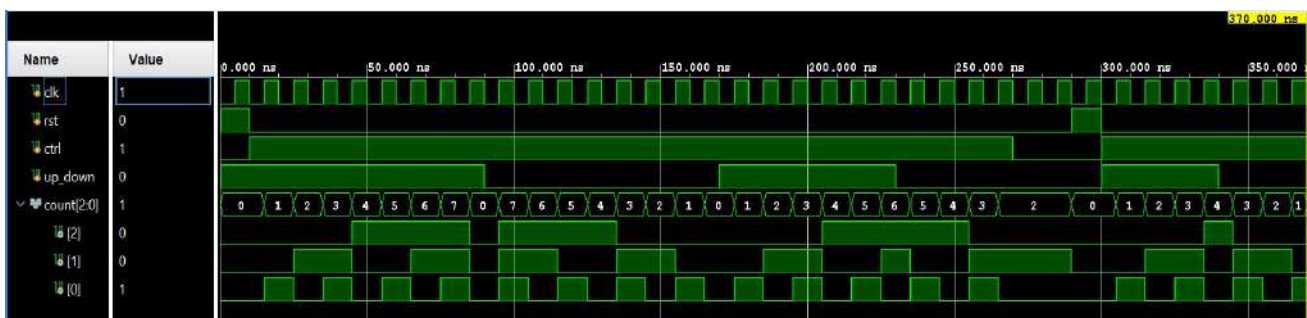


119. Up Down Counter using FSM approach

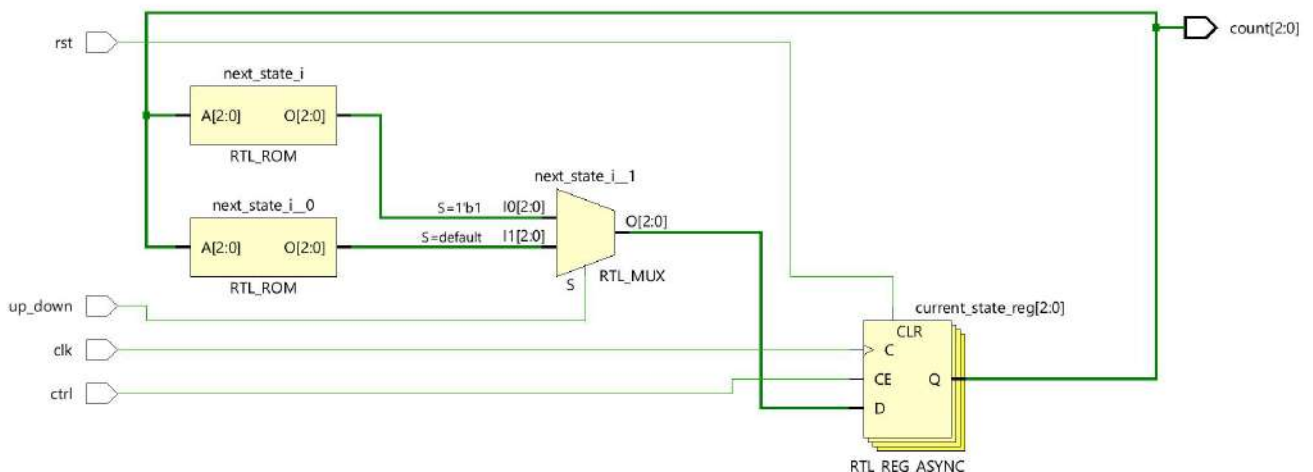
● Design Code	● Testbench Code																																										
<pre>module up_down_counter_fsm (clk, rst, ctrl, up_down, count); input clk, rst, ctrl, up_down; output reg [2:0] count; parameter S0 = 3'b000; parameter S1 = 3'b001; parameter S2 = 3'b010; parameter S3 = 3'b011; parameter S4 = 3'b100; parameter S5 = 3'b101; parameter S6 = 3'b110; parameter S7 = 3'b111; reg [2:0] current_state, next_state; always @(*) begin if (ctrl) begin if (up_down) begin // Count up case (current_state) S0: next_state = S1; S1: next_state = S2; S2: next_state = S3; S3: next_state = S4; S4: next_state = S5; S5: next_state = S6; S6: next_state = S7; S7: next_state = S0; default: next_state = S0; endcase end else begin // Count down case (current_state) S0: next_state = S7; S1: next_state = S0; S2: next_state = S1; S3: next_state = S2; S4: next_state = S3; S5: next_state = S4; S6: next_state = S5; S7: next_state = S6; default: next_state = S0; endcase end end end</pre>	<pre>module up_down_counter_fsm_tb; reg clk, rst, ctrl, up_down; wire [2:0] count; up_down_counter_fsm uut (clk, rst, ctrl, up_down, count); always #5 clk = ~clk; initial begin \$display("Time\tclk\trst\tctrl\tup_down\tcount"); \$monitor("%0t\t%b\t%b\t%b\t%b\t%b", \$time, clk, rst, ctrl, up_down, count); clk = 0; rst = 1; ctrl = 0; up_down = 1; #10 rst = 0; ctrl = 1; up_down = 1; #80; up_down = 0; #80; ctrl = 1; up_down = 1; #60; up_down = 0; #40; ctrl = 0; #20; rst = 1; #10 rst = 0; ctrl = 1; up_down = 1; #40; up_down = 0; #30; \$finish; end endmodule</pre> ● Output <table><tr><th>Time</th><th>clk</th><th>rst</th><th>ctrl</th><th>up_down</th><th>count</th></tr><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>000</td></tr><tr><td>5000</td><td>1</td><td>1</td><td>0</td><td>1</td><td>000</td></tr><tr><td>10000</td><td>0</td><td>0</td><td>1</td><td>1</td><td>000</td></tr><tr><td>15000</td><td>1</td><td>0</td><td>1</td><td>1</td><td>001</td></tr><tr><td>20000</td><td>0</td><td>0</td><td>1</td><td>1</td><td>001</td></tr><tr><td>25000</td><td>1</td><td>0</td><td>1</td><td>1</td><td>010</td></tr></table>	Time	clk	rst	ctrl	up_down	count	0	0	1	0	1	000	5000	1	1	0	1	000	10000	0	0	1	1	000	15000	1	0	1	1	001	20000	0	0	1	1	001	25000	1	0	1	1	010
Time	clk	rst	ctrl	up_down	count																																						
0	0	1	0	1	000																																						
5000	1	1	0	1	000																																						
10000	0	0	1	1	000																																						
15000	1	0	1	1	001																																						
20000	0	0	1	1	001																																						
25000	1	0	1	1	010																																						

end	30000	0	0	1	1	010
else	35000	1	0	1	1	011
begin	40000	0	0	1	1	011
next_state = current_state; // Hold	45000	1	0	1	1	100
end	50000	0	0	1	1	100
end	55000	1	0	1	1	101
	60000	0	0	1	1	101
// State register (synchronous reset)	65000	1	0	1	1	110
always @(posedge clk or posedge rst)	70000	0	0	1	1	110
begin	75000	1	0	1	1	111
if (rst)	80000	0	0	1	1	111
current_state <= S0;	85000	1	0	1	1	000
else	90000	0	0	1	0	000
current_state <= next_state;	95000	1	0	1	0	111
end	100000	0	0	1	0	111
	105000	1	0	1	0	110
// Output logic	110000	0	0	1	0	110
always @(*)	115000	1	0	1	0	101
begin	120000	0	0	1	0	101
count = current_state;	125000	1	0	1	0	100
end	130000	0	0	1	0	100
endmodule						

● Waveform



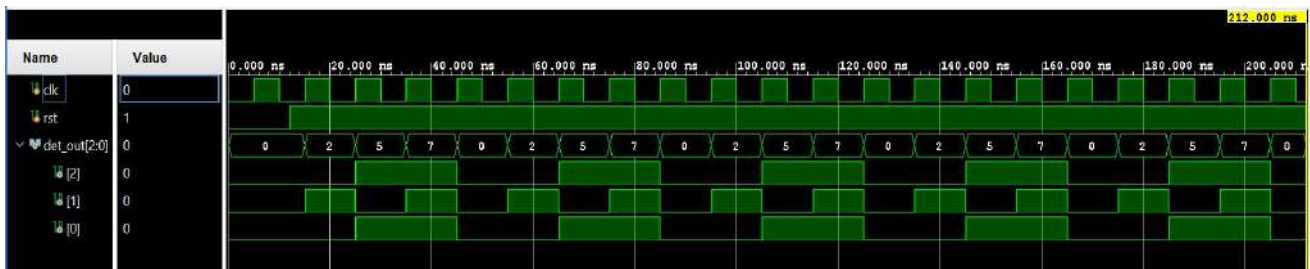
● Schematic



120. State Skipping Counter using FSM approach (0,2,5,7)

● Design Code	● Testbench Code
<pre> module state_skipping_counter_fsm(clk,rst,det_out); input clk,rst; output reg [2:0]det_out; reg [1:0] ps,ns; parameter s0=3'd0; parameter s2=3'd1; parameter s5=3'd2; parameter s7=3'd3; always@(posedge clk or negedge rst) begin if(!rst) ps<=s0; else ps<=ns; end always@(*) begin case(ps) s0: begin ns=s2; det_out=4'd0; end s2: begin ns=s5; det_out=4'd2; end s5: begin ns=s7; det_out=4'd5; end s7: begin ns=s0; det_out=4'd7; end default: begin ns=s0; det_out=4'd0; end endcase end endmodule </pre>	<pre> module state_skipping_counter_fsm_tb; reg clk,rst; wire [2:0]det_out; state_skipping_counter_fsm dut(clk,rst,det_out); initial begin clk=0; rst=0; #12 rst=1; #200 \$finish; end always #5 clk=~clk; initial begin \$monitor("simtime= %0t,clk=%b,rst=%b,count= %b",\$time,clk,rst,det_out); end initial begin \$dumppfile("dump.vcd"); \$dumppvars(0,clk,rst,det_out); end endmodule </pre>

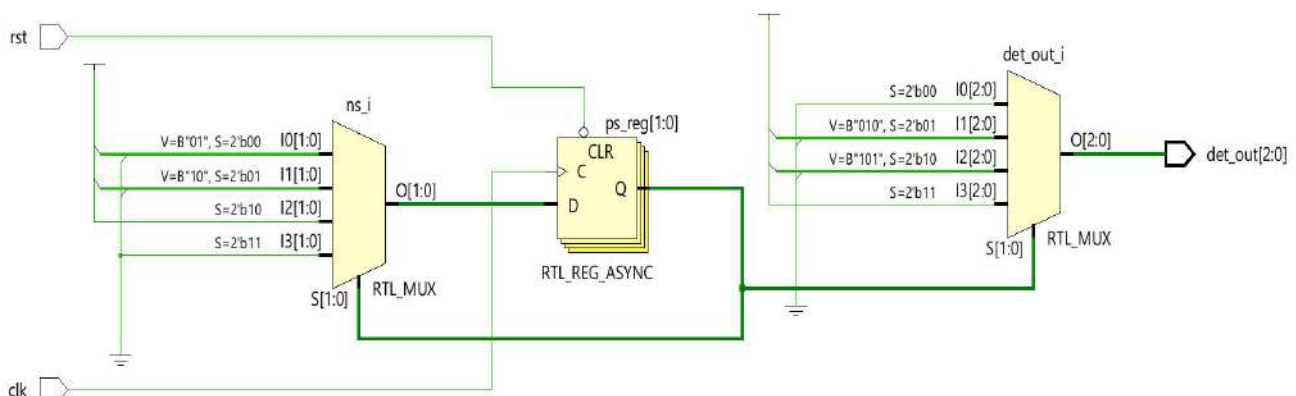
● Waveform



● Output

simtime= 0,clk=0,rst=0,count=000
 simtime= 5000,clk=1,rst=0,count=000
 simtime= 10000,clk=0,rst=0,count=000
 simtime= 12000,clk=0,rst=1,count=000
 simtime= 15000,clk=1,rst=1,count=010
 simtime= 20000,clk=0,rst=1,count=010
 simtime= 25000,clk=1,rst=1,count=101
 simtime= 30000,clk=0,rst=1,count=101
 simtime= 35000,clk=1,rst=1,count=111
 simtime= 40000,clk=0,rst=1,count=111
 simtime= 45000,clk=1,rst=1,count=000
 simtime= 50000,clk=0,rst=1,count=000
 simtime= 55000,clk=1,rst=1,count=010
 simtime= 60000,clk=0,rst=1,count=010
 simtime= 65000,clk=1,rst=1,count=101
 simtime= 70000,clk=0,rst=1,count=101
 simtime= 75000,clk=1,rst=1,count=111
 simtime= 80000,clk=0,rst=1,count=111
 simtime= 85000,clk=1,rst=1,count=000
 simtime= 90000,clk=0,rst=1,count=000
 simtime= 95000,clk=1,rst=1,count=010
 simtime= 100000,clk=0,rst=1,count=010
 simtime= 105000,clk=1,rst=1,count=101
 simtime= 110000,clk=0,rst=1,count=101

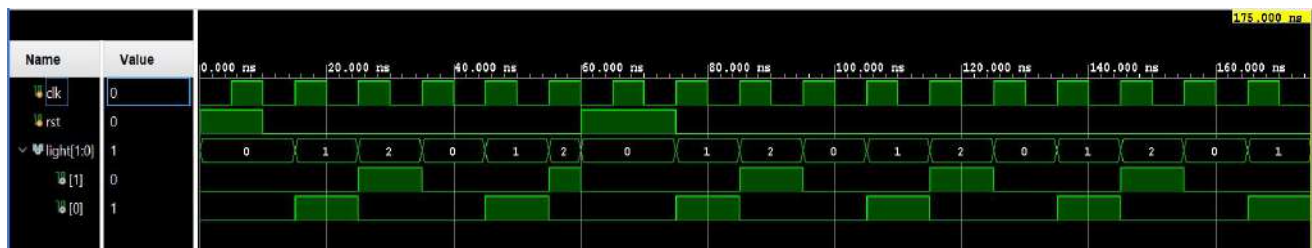
● Schematic



121. Traffic Light Controller (3-state FSM: RED → GREEN → YELLOW)

● Design Code	● Testbench Code
<pre> module traffic_light_controller (clk, rst, light); input clk, rst; output reg [1:0] light; // 2'b00=RED, 2'b01=GREEN, 2'b10=YELLOW parameter RED = 2'b00; parameter GREEN = 2'b01; parameter YELLOW = 2'b10; reg [1:0] state, next_state; always @(posedge clk or posedge rst) begin if (rst) state <= RED; else state <= next_state; end always @(*) begin case(state) RED: next_state = GREEN; GREEN: next_state = YELLOW; YELLOW: next_state = RED; default: next_state = RED; endcase end always @(*) begin light = state; end endmodule </pre>	<pre> module traffic_light_controller_tb; reg clk = 0, rst; wire [1:0] light; traffic_light_controller dut(clk, rst, light); always #5 clk = ~clk; initial begin \$monitor("Time=%0t, Light=%b", \$time, light); rst = 1; #10; rst = 0; #50 rst = 1; #15; rst = 0; #100 \$finish; end endmodule </pre>

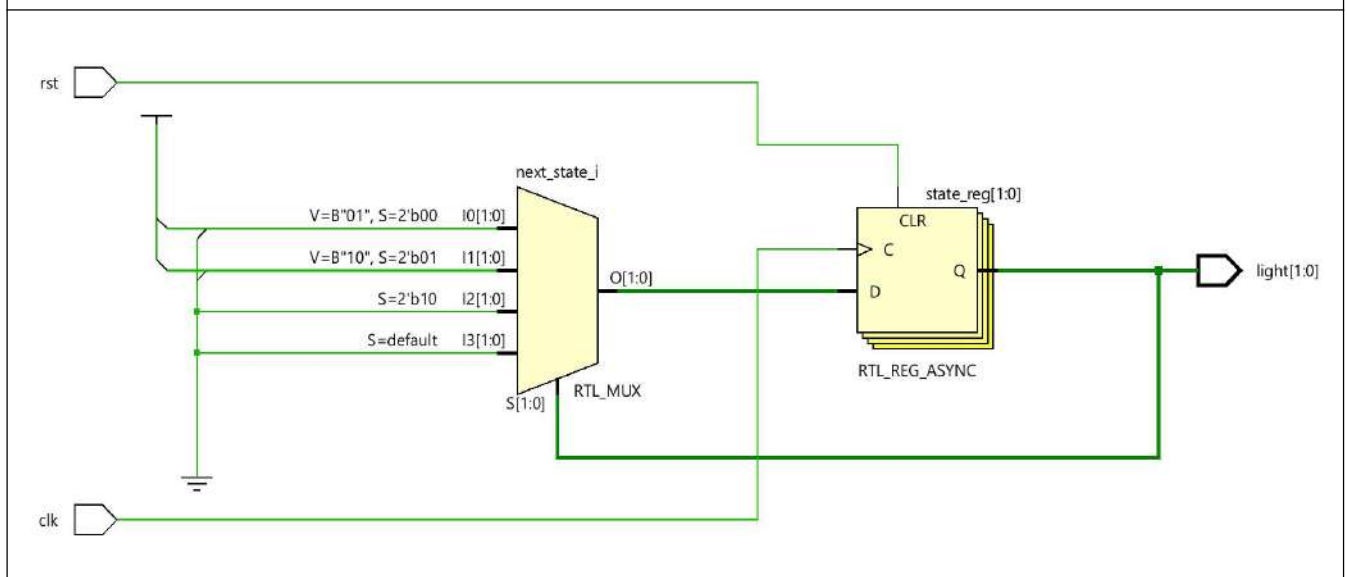
● Waveform



● Output

Time=0, Light=00
 Time=15000, Light=01
 Time=25000, Light=10
 Time=35000, Light=00
 Time=45000, Light=01
 Time=55000, Light=10
 Time=60000, Light=00
 Time=75000, Light=01
 Time=85000, Light=10
 Time=95000, Light=00
 Time=105000, Light=01
 Time=115000, Light=10
 Time=125000, Light=00
 Time=135000, Light=01
 Time=145000, Light=10
 Time=155000, Light=00
 Time=165000, Light=01

● Schematic



122. Lift Controller (2-Floor Lift: Floor 0 and 1)

● Design Code

```

module lift_controller (clk, rst,
floor_request, floor);
    input clk, rst, floor_request;
    output reg [1:0] floor;

    parameter FLOOR_0 = 2'b00;
    parameter FLOOR_1 = 2'b01;
    reg [1:0] state;

    always @(posedge clk or posedge rst)
    begin
        if (rst)
            state <= FLOOR_0;
        else if (floor_request)
            state <= FLOOR_1;
        else
            state <= FLOOR_0;
    end

    always @(*)
    begin
        floor = state;
    end
endmodule

```

● Testbench Code

```

module lift_controller_tb;
    reg clk = 0, rst, floor_request;
    wire [1:0] floor;

    lift_controller dut(clk, rst, floor_request, floor);

    always #5 clk = ~clk;

    initial
    begin
        $monitor("Time=%0t, Floor=%d", $time, floor);
        rst = 1; floor_request = 0; #10;
        rst = 0;
        floor_request = 1; #20;
        floor_request = 0; #20;
        floor_request = 1; #20;
        $finish;
    end
endmodule

```

● Waveform



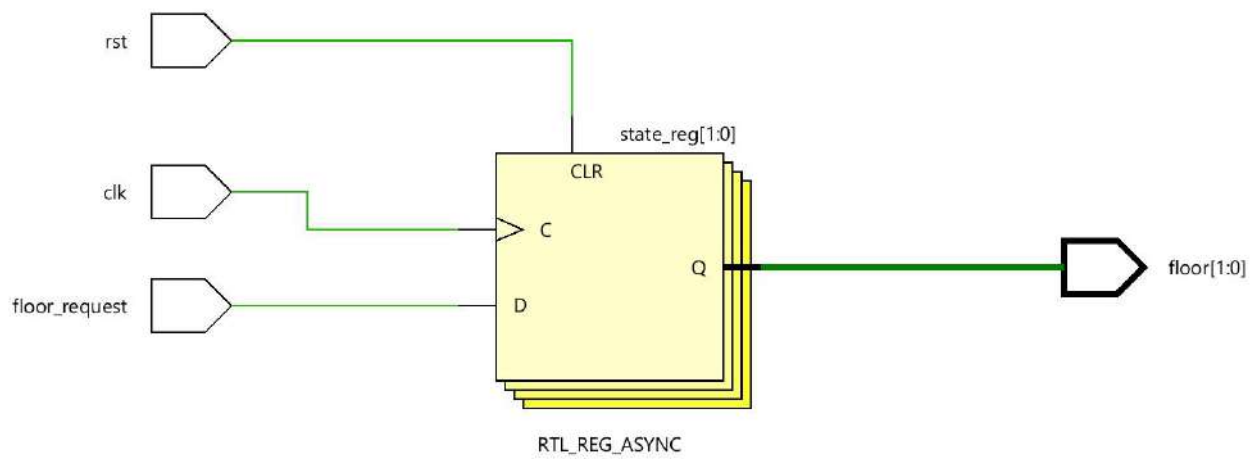
● Output

```

Time=0, Floor=0
Time=15000, Floor=1
Time=35000, Floor=0
Time=55000, Floor=1

```

- Schematic



123. Vending Machine (Dispense item when 2 coins are inserted)

● Design Code	● Testbench Code
<pre> module vending_machine (clk, rst, coin, item_dispensed); input clk, rst, coin; output reg item_dispensed; parameter S0 = 2'b00; parameter S1 = 2'b01; parameter S2 = 2'b10; reg [1:0] state; always @(posedge clk or posedge rst) begin if (rst) state <= S0; else begin case(state) S0: state <= coin ? S1 : S0; S1: state <= coin ? S2 : S1; S2: state <= S0; endcase end end end always @(*) begin item_dispensed = (state == S2); end endmodule </pre>	<pre> module vending_machine_tb; reg clk = 0, rst, coin; wire item_dispensed; vending_machine dut(clk, rst, coin, item_dispensed); always #5 clk = ~clk; initial begin \$monitor("Time=%0t, Coin=%b, Dispensed=%b", \$time, coin, item_dispensed); rst = 1; coin = 0; #10; rst = 0; coin = 1; #10; coin = 1; #10; coin = 0; #10; coin = 1; #10; coin = 0; #10; coin = 0; #10; coin = 1; #10; coin = 1; #10; coin = 1; #10; coin = 1; #50; \$finish; end endmodule </pre>

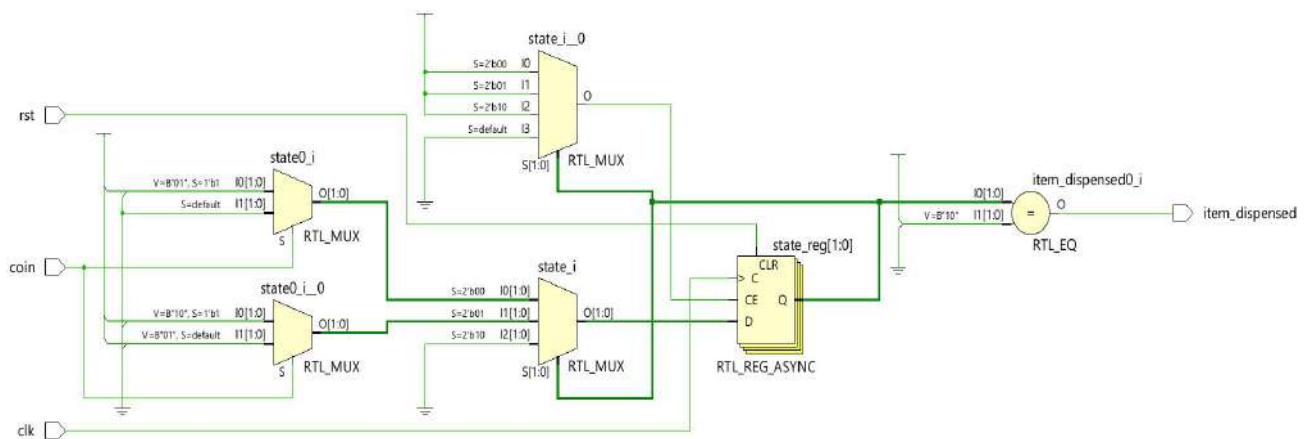
● Waveform



● Output

Time=0, Coin=0, Dispensed=0
 Time=10000, Coin=1, Dispensed=0
 Time=25000, Coin=1, Dispensed=1
 Time=30000, Coin=0, Dispensed=1
 Time=35000, Coin=0, Dispensed=0
 Time=40000, Coin=1, Dispensed=0
 Time=50000, Coin=0, Dispensed=0
 Time=70000, Coin=1, Dispensed=0
 Time=75000, Coin=1, Dispensed=1
 Time=85000, Coin=1, Dispensed=0
 Time=105000, Coin=1, Dispensed=1
 Time=115000, Coin=1, Dispensed=0
 Time=135000, Coin=1, Dispensed=1
 Time=145000, Coin=1, Dispensed=0

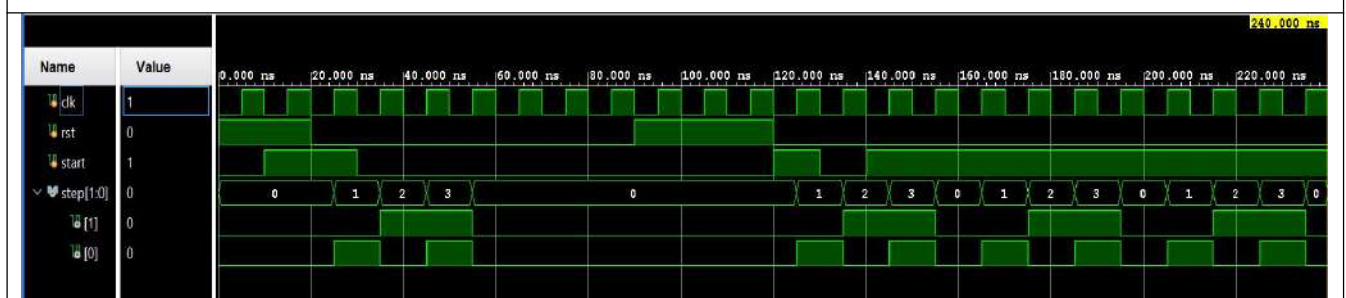
● Schematic



124. Washing Machine (Simple 4-step FSM: Idle, Wash, Rinse, Dry)

● Design Code	● Testbench Code
<pre> module washing_machine (clk, rst, start, step); input clk, rst, start; output reg [1:0] step; parameter IDLE = 2'b00; parameter WASH = 2'b01; parameter RINSE = 2'b10; parameter DRY = 2'b11; reg [1:0] state, next_state; always @(posedge clk or posedge rst) begin if (rst) state <= IDLE; else state <= next_state; end always @(*) begin case(state) IDLE: next_state = start ? WASH : IDLE; WASH: next_state = RINSE; RINSE: next_state = DRY; DRY: next_state = IDLE; default: next_state = IDLE; endcase end always @(*) begin step = state; end endmodule </pre>	<pre> module washing_machine_tb; reg clk = 0, rst, start; wire [1:0] step; washing_machine dut(clk, rst, start, step); always #5 clk = ~clk; initial begin \$monitor("Time=%0t, Step=%b", \$time, step); rst = 1; start = 0; #10; start = 1; #10; rst = 0; start = 1; #10; start = 0; #60; rst = 1; start = 0; #10; start = 0; #20; rst = 0; start = 1; #10; start = 0; #10; start = 1; #100; \$finish; end endmodule </pre>

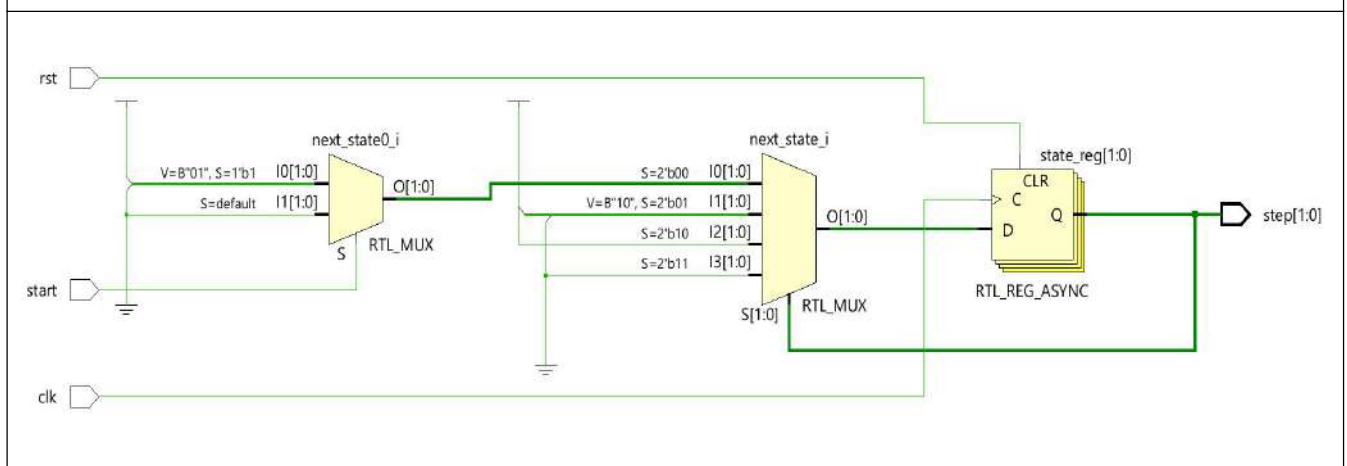
● Waveform



● Output

Time=0, Step=00
 Time=25000, Step=01
 Time=35000, Step=10
 Time=45000, Step=11
 Time=55000, Step=00
 Time=125000, Step=01
 Time=135000, Step=10
 Time=145000, Step=11
 Time=155000, Step=00
 Time=165000, Step=01
 Time=175000, Step=10
 Time=185000, Step=11
 Time=195000, Step=00
 Time=205000, Step=01
 Time=215000, Step=10
 Time=225000, Step=11
 Time=235000, Step=00

● Schematic



125. Digital Clock

● Design Code	● Testbench Code
<pre> module digital_clock(clk, rst, sec, min, hr); input clk, rst; output reg [5:0] sec, min; output reg [4:0] hr; always @(posedge clk) begin if (rst) begin sec <= 0; min <= 0; hr <= 0; end else begin if (sec == 59) begin sec <= 0; if (min == 59) begin min <= 0; if (hr == 23) hr <= 0; else hr <= hr + 1; end else min <= min + 1; end else sec <= sec + 1; end end end end endmodule </pre>	<pre> `timescale 1ns / 1ps module digital_clock_tb; reg clk, rst; wire [5:0] sec, min; wire [4:0] hr; digital_clock uut (clk, rst, sec, min, hr); initial clk = 0; always #5 clk = ~clk; initial begin \$display("Time\t Hr: Min: Sec"); \$monitor(" %0t\t %02d: %02d: %02d", \$time, hr, min, sec); rst = 1; #10; rst = 0; repeat(200) #10; \$finish; end endmodule </pre>

● Waveform



● Output

Time	Hr: Min: Sec
0	00: 00: 00
5000	00: 00: 01
15000	00: 00: 02
25000	00: 00: 03
35000	00: 00: 04
45000	00: 00: 05
55000	00: 00: 06
65000	00: 00: 07
75000	00: 00: 08
85000	00: 00: 09
95000	00: 00: 10
105000	00: 00: 11
115000	00: 00: 12
125000	00: 00: 13
135000	00: 00: 14
145000	00: 00: 15
155000	00: 00: 16
165000	00: 00: 17
175000	00: 00: 18
185000	00: 00: 19
195000	00: 00: 20
205000	00: 00: 21
215000	00: 00: 22
225000	00: 00: 23
235000	00: 00: 24
245000	00: 00: 25
255000	00: 00: 26
265000	00: 00: 27
275000	00: 00: 28
285000	00: 00: 29
295000	00: 00: 30
305000	00: 00: 31
315000	00: 00: 32
325000	00: 00: 33
335000	00: 00: 34
345000	00: 00: 35
355000	00: 00: 36
365000	00: 00: 37
375000	00: 00: 38
385000	00: 00: 39
395000	00: 00: 40
405000	00: 00: 41
415000	00: 00: 42
425000	00: 00: 43
435000	00: 00: 44
445000	00: 00: 45
455000	00: 00: 46
465000	00: 00: 47
475000	00: 00: 48
485000	00: 00: 49
495000	00: 00: 50
505000	00: 00: 51
515000	00: 00: 52
525000	00: 00: 53
535000	00: 00: 54
545000	00: 00: 55
555000	00: 00: 56
565000	00: 00: 57
575000	00: 00: 58
585000	00: 00: 59
595000	00: 01: 00
605000	00: 01: 01
615000	00: 01: 02
625000	00: 01: 03
635000	00: 01: 04
645000	00: 01: 05
655000	00: 01: 06
665000	00: 01: 07
675000	00: 01: 08
685000	00: 01: 09
695000	00: 01: 10
705000	00: 01: 11
715000	00: 01: 12
725000	00: 01: 13
735000	00: 01: 14
745000	00: 01: 15
755000	00: 01: 16
765000	00: 01: 17
775000	00: 01: 18
785000	00: 01: 19

● Schematic

